

XCPC

C++ 源码合集

共 253 个源文件 · 19,071 行代码

按原始目录与文件名排序 · A4 打印版

目录

001.01 - 基础算法 / 01A - 头文件 (完整版).cpp	008
002.01 - 基础算法 / 01B - 头文件 (极简版).cpp	010
003.01 - 基础算法 / 02 - 常用函数重载.cpp	011
004.01 - 基础算法 / 03 - 常用常量定义.cpp	013
005.01 - 基础算法 / 04 - fastio.cpp	014
006.01 - 基础算法 / 05 - i128函数重载.cpp	016
007.01 - 基础算法 / 06 - 手写随机哈希.cpp	018
008.01 - 基础算法 / 07 - 随机数.cpp	019
009.01 - 基础算法 / 08 - 对拍.cpp	020
010.01 - 基础算法 / 09 - 最长上升子序列 (LIS) .cpp	021
011.01 - 基础算法 / 10 - 单调栈.cpp	023
012.01 - 基础算法 / 11 - 坐标压缩.cpp	025
013.01 - 基础算法 / 12 - 二分查找 (边界) .cpp	026
014.01 - 基础算法 / 13 - 前缀和与差分.cpp	027
015.01 - 基础算法 / 14 - 逆序对 (归并排序) .cpp	029
016.01 - 基础算法 / 15 - 随机数据生成 (树与图) .cpp	030
017.02 - 树上问题 / 01 - 树的直径.cpp	032
018.02 - 树上问题 / 01A - 树论大封装 (直径+中心+重心) .cpp	033
019.02 - 树上问题 / 02 - 点分治 (重心树骨架) .cpp	036
020.02 - 树上问题 / 02A - 点分治 (距离询问) .cpp	038
021.02 - 树上问题 / 02B - 点分树 (动态点集最近距离) .cpp	040
022.02 - 树上问题 / 03A - 最近公共祖先 (Tree, 树链剖分) .cpp	043
023.02 - 树上问题 / 03B - 最近公共祖先 (Tree, 倍增) .cpp	045
024.02 - 树上问题 / 03C - 最近公共祖先 (Tree, 倍增, 有权图) .cpp	047
025.02 - 树上问题 / 04 - 树上启发式合并 (通用框架) .cpp	049
026.02 - 树上问题 / 04A - 树上启发式合并 (众数颜色和) .cpp	051
027.02 - 树上问题 / 05 - 树上路径交.cpp	053
028.02 - 树上问题 / 06 - 树链剖分 (路径与子树操作) .cpp	054
029.02 - 树上问题 / 07 - 树上差分.cpp	056
030.02 - 树上问题 / 08 - 换根 DP (全树 DP) .cpp	058
031.02 - 树上问题 / 09 - 虚树.cpp	060
032.02 - 树上问题 / 10 - Kruskal 重构树.cpp	061
033.02 - 树上问题 / 11 - Link-Cut Tree (动态树) .cpp	063
034.02 - 树上问题 / 12 - 树同构 (AHU) .cpp	066
035.02 - 树上问题 / 13 - Prüfer序列.cpp	068
036.03 - 数据结构 / 01A - 并查集 (DSU) .cpp	069
037.03 - 数据结构 / 01B - 可撤销并查集 (DSU) .cpp	070
038.03 - 数据结构 / 01C - 可删除并查集 (DSU) .cpp	072
039.03 - 数据结构 / 01D - 权值并查集 (DSU, with Int) .cpp	074
040.03 - 数据结构 / 01E - 异或并查集 (XorDSU) .cpp	075
041.03 - 数据结构 / 02A - 线段树 (SegTree, with Int) .cpp	076
042.03 - 数据结构 / 02B - 最值线段树 (SegTree, with Int) .cpp	079
043.03 - 数据结构 / 02C - 可持久化线段树 (SegTree) .cpp	082
044.03 - 数据结构 / 02D - 可合并信息线段树 (MergeSegTree) .cpp	084
045.03 - 数据结构 / 02E - 计数求和主席树 (CountSumSegTree) .cpp	086
046.03 - 数据结构 / 02F - 动态开点线段树.cpp	088
047.03 - 数据结构 / 02G - Segment Tree Beats.cpp	092

048.03 - 数据结构 / 02H - Li Chao Tree.cpp	096
049.03 - 数据结构 / 02I - 仿射懒标记线段树（自动取模）.cpp	098
050.03 - 数据结构 / 02J - 可合并信息懒标记线段树（LazyMergeSegTree）.cpp	100
051.03 - 数据结构 / 03 - 树状数组（BIT, with Int）.cpp	102
052.03 - 数据结构 / 03A - 区间加区间和树状数组.cpp	104
053.03 - 数据结构 / 03B - 二维树状数组.cpp	105
054.03 - 数据结构 / 04A - ST表（ST, with Int）.cpp	106
055.03 - 数据结构 / 04B - 可结合区间查询（Disjoint Sparse Table）.cpp	107
056.03 - 数据结构 / 04C - 线性RMQ（状压）.cpp	108
057.03 - 数据结构 / 05 - Bitset.cpp	110
058.03 - 数据结构 / 06 - 小波矩阵（WaveletMatrix）.cpp	115
059.03 - 数据结构 / 06A - 区间第k大（WaveletMatrix）.cpp	119
060.03 - 数据结构 / 07 - 线性基.cpp	122
061.03 - 数据结构 / 08A - 字典树.cpp	124
062.03 - 数据结构 / 08B - 二进制字典树（XOR Trie）.cpp	126
063.03 - 数据结构 / 08C - PBDS前缀字典树（GNU环境）.cpp	128
064.03 - 数据结构 / 09A - 可并堆（LeftistHeap, 可移植）.cpp	129
065.03 - 数据结构 / 09B - PBDS可并堆（GNU环境）.cpp	131
066.03 - 数据结构 / 10 - 可合并信息FHQTreap（MergeFHQTreap）.cpp	132
067.03 - 数据结构 / 10A - FHQTreap（有序多重集）.cpp	134
068.03 - 数据结构 / 10B - 隐式 FHQTreap（序列）.cpp	137
069.03 - 数据结构 / 10C - PBDS有序多重集（GNU环境）.cpp	140
070.03 - 数据结构 / 10D - vector有序多重集（小数据环境）.cpp	142
071.03 - 数据结构 / 10E - PBDS有序集合与映射（GNU环境）.cpp	143
072.03 - 数据结构 / 11A - KDTree（二维静态查询）.cpp	144
073.03 - 数据结构 / 11B - KDTree（二维动态插入与偏序计数）.cpp	146
074.03 - 数据结构 / 12 - 珂朵莉树（ODT, 随机区间赋值环境）.cpp	148
075.03 - 数据结构 / 13A - PBDS开放寻址哈希表（gp_hash_table, GNU环境）.cpp	150
076.03 - 数据结构 / 13B - PBDS拉链哈希表（cc_hash_table, GNU环境）.cpp	151
077.04 - 图论 / 00A - 图遍历（BFS、DFS、二分图判定）.cpp	152
078.04 - 图论 / 00B - 链式前向星（大图环境）.cpp	154
079.04 - 图论 / 01A - 单源最短路（Dijkstra）（无负权）.cpp	155
080.04 - 图论 / 01B - 单源最短路（Bellman-ford, 负权图）.cpp	156
081.04 - 图论 / 01C - 单源最短路（SPFA, 负权图）.cpp	158
082.04 - 图论 / 01D - 多源最短路（APSP）.cpp	159
083.04 - 图论 / 01E - 0-1 BFS.cpp	160
084.04 - 图论 / 01F - 差分约束.cpp	161
085.04 - 图论 / 01G - Johnson全源最短路（稀疏负权图）.cpp	162
086.04 - 图论 / 02A - 最小生成树（Kruskal）.cpp	163
087.04 - 图论 / 02B - 最小生成树（Prim）.cpp	165
088.04 - 图论 / 02C - 有向最小生成树（Chu-Liu-Edmonds）.cpp	167
089.04 - 图论 / 03A - 拓扑排序（Kahn）.cpp	169
090.04 - 图论 / 03B - 强连通分量（Tarjan）.cpp	170
091.04 - 图论 / 03C - 2-SAT.cpp	172
092.04 - 图论 / 04A - 边双连通分量（桥）.cpp	174
093.04 - 图论 / 04B - 点双连通分量（割点、圆方树）.cpp	176
094.04 - 图论 / 05A - 欧拉路（有向、无向）.cpp	178
095.04 - 图论 / 06A - 二分图最大匹配（Hopcroft-Karp）.cpp	180
096.04 - 图论 / 06B - 一般图最大匹配（带花树）.cpp	182

097.04 - 图论 / 06C - 二分图最优匹配 (Hungarian、KM) .cpp	184
098.04 - 图论 / 06D - 一般图最大权匹配 (带权带花树) .cpp	186
099.04 - 图论 / 07 - 全局最小割 (Stoer-Wagner) .cpp	190
100.04 - 图论 / 08 - 最大团 (位集分支限界) .cpp	191
101.04 - 图论 / 09 - 平面嵌入面与对偶图.cpp	192
102.05 - 网络流 / 01A - 最大流 (Dinic) .cpp	194
103.05 - 网络流 / 01B - 最大流 (HLPP) .cpp	197
104.05 - 网络流 / 01C - 最大流 (HLPP, 极致优化) .cpp	200
105.05 - 网络流 / 02 - 最小割.cpp	204
106.05 - 网络流 / 02A - 最大权闭合图 (最小割建模) .cpp	205
107.05 - 网络流 / 03 - 最小割树.cpp	206
108.05 - 网络流 / 04 - 最小费用最大流 (MinCostFlow) .cpp	208
109.05 - 网络流 / 05 - 上下界网络流.cpp	211
110.06 - 数学 / 01 - 素数 (线性筛) .cpp	213
111.06 - 数学 / 01A - 质因数分解 (最小质因子) .cpp	215
112.06 - 数学 / 01B - 欧拉函数 (单点试除) .cpp	216
113.06 - 数学 / 01C - 欧拉函数 (整表筛) .cpp	217
114.06 - 数学 / 01D - 莫比乌斯函数 (Mertens、杜教筛) .cpp	218
115.06 - 数学 / 01DA - 莫比乌斯反演 (约数和变换) .cpp	219
116.06 - 数学 / 01DB - 加权莫比乌斯函数 (d^2) .cpp	220
117.06 - 数学 / 01DC - 狄利克雷卷积与逆.cpp	221
118.06 - 数学 / 01EA - 检查是否是素数的幂 (试除) .cpp	222
119.06 - 数学 / 01EB - 检查是否是素数的幂 (最小质因子) .cpp	223
120.06 - 数学 / 01F - Miller-Rabin 素性检验.cpp	224
121.06 - 数学 / 01FA - Pollard-Rho 质因数分解.cpp	226
122.06 - 数学 / 01G - 素性检验 (试除) .cpp	227
123.06 - 数学 / 01H - 约数函数 (枚举、个数、和) .cpp	228
124.06 - 数学 / 01I - 质数计数 (Lehmer) .cpp	230
125.06 - 数学 / 01J - 质数和 (Min_25筛) .cpp	232
126.06 - 数学 / 02 - 快速幂 (ksm) .cpp	234
127.06 - 数学 / 02A - 自动取模 (MInt) .cpp	235
128.06 - 数学 / 02B - 裴蜀定理.cpp	238
129.06 - 数学 / 02BA - 扩展欧几里得 (exgcd) .cpp	239
130.06 - 数学 / 02BAA - 中国剩余定理.cpp	240
131.06 - 数学 / 02BAAB - 扩展 Lucas (组合数模合数) .cpp	241
132.06 - 数学 / 02C - BSGS.cpp	243
133.06 - 数学 / 02D - 逆元线性预处理.cpp	245
134.06 - 数学 / 02E - 模平方根 (Tonelli-Shanks) .cpp	246
135.06 - 数学 / 02F - 原根.cpp	247
136.06 - 数学 / 02G - 整除分块求和 (floor_sum) .cpp	249
137.06 - 数学 / 03 - 组合数学 (Comb. with Z) .cpp	250
138.06 - 数学 / 03A - Lucas 定理.cpp	251
139.06 - 数学 / 03B - 康托展开 (暴力, 小 n) .cpp	252
140.06 - 数学 / 03C - 康托展开 (树状数组, 大 n) .cpp	253
141.06 - 数学 / 03D - 卡特兰数.cpp	255
142.06 - 数学 / 03E - 斯特林数 (第一、二类) .cpp	256
143.06 - 数学 / 03F - 错排数与贝尔数.cpp	257
144.06 - 数学 / 03G - 拉格朗日插值 (任意横坐标) .cpp	258
145.06 - 数学 / 03H - 拉格朗日插值 (连续横坐标) .cpp	259

146.06 - 数学 / 03I - 牛顿插值 (差分级数) .cpp	260
147.06 - 数学 / 03J - 整数分拆 (五边形数) .cpp	261
148.06 - 数学 / 03K - Berlekamp-Massey (最短线性递推) .cpp	262
149.06 - 数学 / 03KA - 线性递推第 n 项 (Kitamasa) .cpp	263
150.06 - 数学 / 03L - Burnside 引理 (群作用计数) .cpp	264
151.06 - 数学 / 04A - 容斥 (bitmask, 小 m) .cpp	265
152.06 - 数学 / 04B - 容斥 (DFS 剪枝) .cpp	266
153.06 - 数学 / 05 - 矩阵 (Matrix, with Int) .cpp	267
154.06 - 数学 / 05A - Min-Plus矩阵 (MinPlusMatrix, with Int) .cpp	269
155.06 - 数学 / 05B - 高斯消元 (一般线性方程组) .cpp	271
156.06 - 数学 / 05C - 对称 LDLT 分解 (线性方程组) .cpp	272
157.06 - 数学 / 05D - 高斯消元 (模质数) .cpp	273
158.06 - 数学 / 05E - 行列式 (模质数) .cpp	274
159.06 - 数学 / 05F - 线性规划 (单纯形) .cpp	275
160.06 - 数学 / 05G - 高斯消元 (GF(2) 位集) .cpp	278
161.06 - 数学 / 05H - 矩阵树定理 (Kirchhoff) .cpp	279
162.06 - 数学 / 06 - 分数 (Frac, with Int) .cpp	280
163.06 - 数学 / 06A - 复数 (Complex, with Int) .cpp	282
164.06 - 数学 / 06B - 三次单位根复数 (Complex3) .cpp	284
165.06 - 数学 / 07 - 平面几何 (Point&Line, with Int) .cpp	285
166.06 - 数学 / 07A - 凸包与旋转卡壳 (依赖07) .cpp	291
167.06 - 数学 / 07AA - 闵可夫斯基和 (依赖07A) .cpp	293
168.06 - 数学 / 07B - 圆与最小覆盖圆 (依赖07) .cpp	294
169.06 - 数学 / 07C - 三维几何.cpp	296
170.06 - 数学 / 07D - 平面最近点对 (依赖07) .cpp	297
171.06 - 数学 / 08 - 快速数论变换 (NTT) .cpp	298
172.06 - 数学 / 08A - 多项式 (Poly, int) (NTT) .cpp	300
173.06 - 数学 / 08B - 多项式 (Poly, Z) (NTT) .cpp	303
174.06 - 数学 / 08C - 轻量多项式 (Poly, Z) (NTT, ln, exp) .cpp	307
175.06 - 数学 / 08D - 快速傅里叶变换 (FFT, with Complex) .cpp	309
176.06 - 数学 / 08E - 快速傅里叶变换 (ModFFT, with Complex) .cpp	311
177.06 - 数学 / 08F - 多模快速数论变换 (CRTNTT, with NTT) .cpp	313
178.06 - 数学 / 08G - FWT (OR、AND、XOR 卷积) .cpp	315
179.06 - 数学 / 08H - SOS DP (子集、超集 Zeta 变换) .cpp	316
180.06 - 数学 / 08HA - 子集卷积 (Subset Convolution) .cpp	317
181.06 - 数学 / 08I - 快速数论变换 (NTT, 极致优化) .cpp	318
182.06 - 数学 / 08IA - 任意模卷积 (CRTNTT, 极致优化) .cpp	323
183.06 - 数学 / 09 - 博弈论 (Nim、SG) .cpp	324
184.06 - 数学 / 09A - Wythoff Nim.cpp	327
185.06 - 数学 / 10A - 圆周率计算 (Machin, with FixedPoint) .cpp	328
186.06 - 数学 / 10B - 圆周率计算 (Chudnovsky, with FixedPoint) .cpp	329
187.06 - 数学 / 11A - 牛顿迭代 (方程、平方根、开 k 次方) .cpp	330
188.06 - 数学 / 11B - 自适应辛普森积分.cpp	331
189.06 - 数学 / 11C - 三分搜索 (实数、整数) .cpp	332
190.06 - 数学 / 11D - 数值求根 (二分、割线) .cpp	333
191.06 - 数学 / 12A - 位运算与子集枚举.cpp	334
192.06 - 数学 / 12B - 异或线性基 (XorBasis64) .cpp	335
193.06 - 数学 / 12C - 二进制 GCD 与 Gray 编码.cpp	336
194.06 - 数学 / 13A - Fibonacci (快速倍增) .cpp	337

195.06 - 数学 / 13B - 约瑟夫环.cpp	338
196.07 - 字符串 / 01A - KMP.cpp	339
197.07 - 字符串 / 01AA - KMP自动机.cpp	340
198.07 - 字符串 / 01B - Z函数 (扩展KMP) .cpp	342
199.07 - 字符串 / 02A - Manacher.cpp	343
200.07 - 字符串 / 02B - 回文自动机 (PalindromicTree) .cpp	345
201.07 - 字符串 / 03 - AC自动机.cpp	347
202.07 - 字符串 / 04A - 后缀自动机.cpp	349
203.07 - 字符串 / 04B - 后缀数组 (SuffixArray) .cpp	352
204.07 - 字符串 / 04C - 后缀数组 (SA-IS, 线性) .cpp	353
205.07 - 字符串 / 05 - 字符串哈希 (StringHash, with T) .cpp	355
206.07 - 字符串 / 06 - 最小表示法 (Booth) .cpp	357
207.07 - 字符串 / 07 - Lyndon 分解 (Duval) .cpp	358
208.07 - 字符串 / 08A - 子序列自动机 (位置表) .cpp	359
209.07 - 字符串 / 08B - 子序列自动机 (稠密转移) .cpp	360
210.08 - 高精度 / 01A - 整数 (BigInt, 常规) .cpp	361
211.08 - 高精度 / 01B - 整数 (BigInt, 极致优化) .cpp	368
212.08 - 高精度 / 02A - 定点数 (FixedPoint, 常规) .cpp	377
213.08 - 高精度 / 02B - 定点数 (FastFixedPoint, 极致优化) .cpp	382
214.08 - 高精度 / 03A - 浮点数 (Decimal, 常规) .cpp	385
215.08 - 高精度 / 03B - 浮点数 (FastDecimal, 极致优化) .cpp	392
216.08 - 高精度 / 04A - 2至62进制转换 (常规) .cpp	393
217.08 - 高精度 / 04B - 2至62进制转换 (极致优化) .cpp	394
218.09 - 杂类 / 01A - 普通莫队.cpp	396
219.09 - 杂类 / 01B - 带修改莫队.cpp	397
220.09 - 杂类 / 01C - 回滚莫队.cpp	398
221.09 - 杂类 / 01D - Hilbert序莫队.cpp	399
222.09 - 杂类 / 02 - 笛卡尔树.cpp	400
223.09 - 杂类 / 03 - CDQ分治 (三维偏序) .cpp	401
224.09 - 杂类 / 04 - 整体二分 (带修改区间第k小) .cpp	403
225.09 - 杂类 / 05 - WQS二分 (带权二分) .cpp	405
226.09 - 杂类 / 06 - 舞蹈链 (DLX, 精确覆盖) .cpp	406
227.09 - 杂类 / 07 - 模拟退火.cpp	408
228.09 - 杂类 / 08 - 公历日期换算.cpp	409
229.09 - 杂类 / 10 - 德州扑克五张牌比较.cpp	410
230.09 - 杂类 / 11A - 物品装箱 (First Fit线段树) .cpp	411
231.09 - 杂类 / 11B - 物品装箱 (Best Fit多重集) .cpp	412
232.09 - 杂类 / 12 - 奇阶幻方.cpp	413
233.09 - 杂类 / 13 - 阿达马矩阵.cpp	414
234.09 - 杂类 / 14A - 选数枚举 (DFS) .cpp	415
235.09 - 杂类 / 14B - 选数枚举 (Gosper位运算) .cpp	416
236.10 - 动态规划 / 01A - 01背包.cpp	417
237.10 - 动态规划 / 01B - 完全背包.cpp	418
238.10 - 动态规划 / 01C - 多重背包 (二进制分组) .cpp	419
239.10 - 动态规划 / 01D - 多重背包 (单调队列) .cpp	420
240.10 - 动态规划 / 01E - 分组背包.cpp	421
241.10 - 动态规划 / 01F - 二维费用背包.cpp	422
242.10 - 动态规划 / 01G - 01背包最优方案数 (自动取模) .cpp	423
243.10 - 动态规划 / 01H - 混合背包.cpp	424

244. 10 - 动态规划 / 01I - 有依赖的背包（树形）.cpp	425
245. 10 - 动态规划 / 01J - 01背包具体方案.cpp	426
246. 10 - 动态规划 / 02 - 数位DP（DFA）.cpp	427
247. 10 - 动态规划 / 03 - 状压DP（TSP）.cpp	428
248. 10 - 动态规划 / 04 - 轮廓线DP（多米诺骨牌）.cpp	429
249. 10 - 动态规划 / 05A - 分治DP优化.cpp	430
250. 10 - 动态规划 / 05B - Knuth优化.cpp	431
251. 10 - 动态规划 / 06A - 最长公共子序列（常规）.cpp	432
252. 10 - 动态规划 / 06B - 最长公共子序列（Bitset）.cpp	433
253. X - FastIO.cpp	434

01A - 头文件（完整版）.cpp

99 lines · C++ · 01 - 基础算法

```
#include <bits/stdc++.h>
using namespace std;

#ifdef ONLINE_JUDGE
    #include "linyue.hpp"
#else
    #define _(...)
#endif

namespace LYE {
#define int long long
#define pii array<int, 2>
#define endl "\n"
#define all(x) x.begin(), x.end()
#define rall(x) x.rbegin(), x.rend()
#define vi vector<int>
#define vvi vector<vector<int>>
#define pb push_back
#define pob pop_back
#define pf push_front
#define pof pop_front
    template <class T> istream &operator>>(istream &is, vector<T> &v) {
        for (auto &x : v) is >> x;
        return is;
    }
    template <typename T> T min(const vector<T> &v) {
        assert(!v.empty());
        return *std::min_element(v.begin(), v.end());
    }
    template <typename T> T max(const vector<T> &v) {
        assert(!v.empty());
        return *std::max_element(v.begin(), v.end());
    }
    template <typename T> T acc(const vector<T> &v) {
        return std::accumulate(v.begin(), v.end(), T{});
    }
    template <class T> void reverse(vector<T> &v) {
        std::reverse(v.begin(), v.end());
    }
    template <class T> void sort(vector<T> &v) {
        std::sort(v.begin(), v.end());
    }
    template <class T> void rsort(vector<T> &v) {
        std::sort(v.rbegin(), v.rend());
    }
    template <class T> void cmax(T &x, T y) {
        x = std::max(x, y);
    }
    template <class T> void cmin(T &x, T y) {
        x = std::min(x, y);
    }
    template <class T> T sign(const T &a) {
        return a == 0 ? 0 : (a < 0 ? -1 : 1);
    }
    template <class T> T floor(const T &a, const T &b) {
        assert(b != 0);
        assert(!(a == std::numeric_limits<T>::min() && b == -1));
        T q = a / b, r = a % b;
        if (r != 0 && ((r > 0) != (b > 0))) --q;
        return q;
    }
```



```
    }
    template <class T> T ceil(const T &a, const T &b) {
        assert(b != 0);
        assert(!(a == std::numeric_limits<T>::min() && b == -1));
        T q = a / b, r = a % b;
        if (r != 0 && ((r > 0) == (b > 0))) ++q;
        return q;
    }
    int mypow(int n, int k, int p) {
        using i128 = __int128_t;
        assert(k >= 0 && p > 0);
        n %= p;
        if (n < 0) n += p;
        int r = 1 % p;
        while (k > 0) {
            if (k & 1) r = i128(r) * n % p;
            n = i128(n) * n % p;
            k >>= 1;
        }
        return r;
    }
    constexpr int N = 1e6 + 7;
    template <typename T = int> constexpr T INF = std::numeric_limits<T>::max() / 2;
}; // namespace LYE
using namespace LYE;

void solve() {}

signed main() {
    ios::sync_with_stdio(0);
    cin.tie(0);
    cout << fixed << setprecision(20);
    int t = 1;
    cin >> t;
    for (int i = 0; i < t; ++i) {
        solve();
    }
    return 0;
}
```

01B - 头文件（极简版）.cpp

19 lines · C++ · 01 - 基础算法

```
#include <bits/stdc++.h>
using namespace std;

#define int long long
#define pii array<int, 2>
#define endl "\n"

void solve() {}

signed main() {
    ios::sync_with_stdio(0), cin.tie(0);
    cout << fixed << setprecision(20);
    int t = 1;
    cin >> t;
    for (int i = 0; i < t; ++i) {
        solve();
    }
    return 0;
}
```

02 - 常用函数重载.cpp

94 lines · C++ · 01 - 基础算法

```
using i128 = __int128_t;

// 常用整型辅助函数与无上下文的小工具。
// 根号
int mysqrt(int n) {
    assert(n >= 0);
    int ans = sqrtl(n);
    while (i128(ans + 1) * (ans + 1) <= n) ans++;
    while (i128(ans) * ans > n) ans--;
    return ans;
}
// 最小公倍数（约定返回非负值）。
int mylcm(int a, int b) {
    if (a == 0 || b == 0) return 0;
    int ans = a / gcd(a, b) * b;
    return ans < 0 ? -ans : ans;
}
// log2向下取整
template <class T> int log2f(T n) {
    assert(n > 0);
    int ans = 0;
    while (n > 1) n >>= 1, ++ans;
    return ans;
}
int log2f(int n) {
    assert(n > 0);
    return 63 - __builtin_clzll(n);
}
// log2向上取整
template <class T> int log2c(T n) {
    assert(n > 0);
    return log2f(n) + ((n & (n - 1)) != 0);
}
int log2c(int n) {
    assert(n > 0);
    return log2f(n) + ((n & (n - 1)) != 0);
}
template <class T> T sign(const T &a) {
    return a == 0 ? 0 : (a < 0 ? -1 : 1);
}
template <class T> T floor(const T &a, const T &b) {
    assert(b != 0);
    assert(!(a == std::numeric_limits<T>::min() && b == -1));
    T q = a / b, r = a % b;
    if (r != 0 && ((r > 0) != (b > 0))) --q;
    return q;
}
template <class T> T ceil(const T &a, const T &b) {
    assert(b != 0);
    assert(!(a == std::numeric_limits<T>::min() && b == -1));
    T q = a / b, r = a % b;
    if (r != 0 && ((r > 0) == (b > 0))) ++q;
    return q;
}
// a <= x && x <= b
template <typename T> bool In(T x, T a, T b) {
    return std::min(a, b) <= x && x <= std::max(a, b);
}
```

```
// 矩阵边界 (0-indexed) 。
inline bool inGrid(int x, int y, int n, int m) {
    return 0 <= x && x < n && 0 <= y && y < m;
}

inline bool inSq(int x, int y, int n) {
    return inGrid(x, y, n, n);
}

// 把若干位按 base 进制编码；调用者保证不会溢出。
template <class T, class... A> T encode(T bas, A... a) {
    T res = 0;
    ((res = res * bas + static_cast<T>(a)), ...);
    return res;
}

// ASCII 转大写/小写。
inline char tomaj(char c) {
    if ('a' <= c && c <= 'z') return c - 32;
    return c;
}

inline char tolow(char c) {
    if ('A' <= c && c <= 'Z') return c + 32;
    return c;
}

// 若干字符等于c
template <typename... A> bool sameto(char c, A... a) {
    return ((a == c) && ...);
}

// 若干字符存在等于c
template <typename... A> bool exi(char c, A... a) {
    return ((a == c) || ...);
}
```

03 - 常用常量定义.cpp

20 lines · C++ · 01 - 基础算法

```
// 最大值
using ld = long double;

constexpr int inf = 1'000'000'007;
constexpr int INF = 2'000'000'000'000'000'009LL;
// 若需要更大的上界，替换上一行：
// constexpr int INF = 4'000'000'000'000'000'000'000LL;
// 模数
constexpr int MOD = 1'000'000'007;
// constexpr int MOD = 998'244'353;
// 很小的正数常量
constexpr ld eps = 1e-10L;
// horse-左上起逆时针
int dx[8] = {-1, -2, -2, -1, 1, 2, 2, 1};
int dy[8] = {2, 1, -1, -2, -2, -1, 1, 2};
// 4-左起逆时针
int dx4[4] = {-1, 0, 1, 0}, dy4[4] = {0, -1, 0, 1};
// 8-左上起逆时针
int dx8[8] = {-1, -1, -1, 0, 1, 1, 1, 0};
int dy8[8] = {1, 0, -1, -1, -1, 0, 1, 1};
```

04 - fastio.cpp

114 lines · C++ · 01 - 基础算法

```
namespace FastIO {
    constexpr int BSZ = 1 << 21;

    char ib[BSZ];
    int p1 = 0, p2 = 0;

    inline int getc() {
        if (p1 >= p2) {
            p2 = fread(ib, 1, BSZ, stdin);
            p1 = 0;
            if (p2 == 0) return EOF;
        }
        return static_cast<unsigned char>(ib[p1++]);
    }

    template <typename T> bool Cin(T &a) {
        static_assert(std::is_integral_v<T> && !std::is_same_v<std::remove_cv_t<T>, bool>);

        int c = getc();
        while (c != EOF && (c < '0' || c > '9') && c != '-' && c != '+') c = getc();
        if (c == EOF) return false;

        bool y = false;
        if (c == '-') {
            if constexpr (!std::is_signed_v<T>) return false;
            y = true;
            c = getc();
        } else if (c == '+') {
            c = getc();
        }
        if (c < '0' || c > '9') return false;

        using U = std::make_unsigned_t<T>;
        const U lim = [&] {
            if constexpr (std::is_signed_v<T>) {
                return y ? static_cast<U>(std::numeric_limits<T>::max()) + U{1} :
                    static_cast<U>(std::numeric_limits<T>::max());
            } else {
                return std::numeric_limits<T>::max();
            }
        }();

        U val = 0;
        while (c >= '0' && c <= '9') {
            const U dig = c - '0';
            if (val > (lim - dig) / 10) return false;
            val = val * 10 + dig;
            c = getc();
        }

        if constexpr (std::is_signed_v<T>) {
            if (y) {
                a = (val == lim) ? std::numeric_limits<T>::min() : -static_cast<T>(val);
            } else {
                a = static_cast<T>(val);
            }
        } else {
            a = val;
        }
        return true;
    }
}
```



```

}

template <typename T, typename U, typename... A> bool Cin(T &a, U &b, A &...c) {
    return Cin(a) && Cin(b, c...);
}

char ob[BSZ];
int p3 = 0;

inline void flush() {
    if (p3 == 0) return;
    fwrite(ob, 1, p3, stdout);
    p3 = 0;
}

inline void putc(char c) {
    if (p3 == BSZ) flush();
    ob[p3++] = c;
}

template <typename T> void Cout(T x, char end = '\n') {
    static_assert(std::is_integral_v<T> && !std::is_same_v<std::remove_cv_t<T>, bool>);
    using U = std::make_unsigned_t<T>;

    U val;
    if constexpr (std::is_signed_v<T>) {
        if (x < 0) {
            putc('-');
            val = U{0} - static_cast<U>(x);
        } else {
            val = static_cast<U>(x);
        }
    } else {
        val = x;
    }

    char s[std::numeric_limits<U>::digits10 + 3];
    int n = 0;
    do {
        s[n++] = static_cast<char>('0' + val % 10);
        val /= 10;
    } while (val > 0);
    while (n > 0) putc(s[--n]);
    putc(end);
}

struct Flush {
    ~Flush() {
        flush();
    }
};
inline Flush io;
} // namespace FastIO

using namespace FastIO;

```

05 - i128函数重载.cpp

64 lines · C++ · 01 - 基础算法

```
using i128 = __int128_t;
using u128 = __uint128_t;

inline bool read128(const std::string &s, i128 &n) {
    if (s.empty()) return false;

    int pos = 0;
    bool y = false;
    if (s[pos] == '-' || s[pos] == '+') {
        y = (s[pos] == '-');
        ++pos;
    }
    if (pos == s.size()) return false;

    const u128 lim = y ? (u128{1} << 127) : ((u128{1} << 127) - 1);
    u128 val = 0;
    for (; pos < s.size(); ++pos) {
        const char c = s[pos];
        if (c < '0' || c > '9') return false;
        const u128 dig = static_cast<u128>(c - '0');
        if (val > (lim - dig) / 10) return false;
        val = val * 10 + dig;
    }

    if (y) {
        n = (val == lim) ? -static_cast<i128>(lim - 1) - 1 : -static_cast<i128>(val);
    } else {
        n = static_cast<i128>(val);
    }
    return true;
}

inline std::istream &operator>>(std::istream &is, i128 &n) {
    std::string s;
    if (!(is >> s)) return is;
    if (!read128(s, n)) is.setstate(std::ios::failbit);
    return is;
}

inline std::ostream &operator<<(std::ostream &os, i128 n) {
    if (n == 0) return os << '0';

    u128 val;
    if (n < 0) {
        os << '-';
        val = u128{0} - static_cast<u128>(n);
    } else {
        val = static_cast<u128>(n);
    }

    std::string s;
    while (val > 0) {
        s += static_cast<char>('0' + val % 10);
        val /= 10;
    }
    std::reverse(s.begin(), s.end());
    return os << s;
}

inline i128 toi128(const std::string &s) {
```

```
    i128 n = 0;  
    assert(read128(s, n));  
    return n;  
}
```

06 - 手写随机哈希.cpp

55 lines · C++ · 01 - 基础算法

```
using u64 = uint64_t;

// 抗 hack 哈希

struct Hash {

    static u64 hash(u64 x) {
        x += 0x9e3779b97f4a7c15;
        x = (x ^ (x >> 30)) * 0xbf58476d1ce4e5b9;
        x = (x ^ (x >> 27)) * 0x94d049bb133111eb;
        return x ^ (x >> 31);
    }

    static const u64 sd;

    static u64 hash(const string &x) {
        return std::hash<string>{}(x);
    }

    template <class T> size_t operator()(T x) const {
        return hash(u64(x) + sd);
    }

    size_t operator()(const string &s) const {
        u64 h = 0;
        for (char c : s) h = h * 131 + c;
        return hash(h + sd);
    }

    template <class T1, class T2> size_t operator()(const pair<T1, T2> &p) const {
        return operator()(p.first) ^ (operator()(p.second) >> 1);
    }

    template <class T> size_t operator()(const vector<T> &v) const {
        u64 h = 0;
        for (auto &x : v) {
            h = h * 131 + operator()(x);
        }
        return hash(h);
    }
};

const u64 Hash::sd = chrono::steady_clock::now().time_since_epoch().count();

// struct ，需要在结构体中重载等于号
struct Node {
    string x, y;
    int z;
    friend bool operator==(const Node &a, const Node &b) {
        return a.x == b.x && a.y == b.y && a.z == b.z;
    }
};

// 结构体版本；也可以把这个 operator() 作为重载加入上面的 Hash。
struct NodeHash {
    size_t operator()(const Node &p) const {
        Hash h;
        return h(p.x) ^ (h(p.y) << 1) ^ (h(p.z) << 2);
    }
};
```

07 - 随机数.cpp

11 lines · C++ · 01 - 基础算法

```
namespace Random {
    mt19937_64 rng(chrono::steady_clock::now().time_since_epoch().count());

    template <class T> T randint(T a, T b) {
        static_assert(std::is_integral_v<T> && !std::is_same_v<std::remove_cv_t<T>, bool>);
        assert(a <= b);
        return uniform_int_distribution<T>(a, b)(rng);
    }
} // namespace Random

using namespace Random;
```

08 - 对拍.cpp

37 lines · C++ · 01 - 基础算法

```
#include <bits/stdc++.h>
using namespace std;

#define int long long
#define pii array<int, 2>
#define endl "\n"

signed main(signed ac, char* av[]) {
    int n = 10000;
    if (ac >= 2) n = atoi(av[1]);
    for (int _ = 1; _ <= n; ++_) {
        if (system("data.exe > data.in") != 0) {
            cerr << "data.exe failed on test " << _ << "\n";
            return 1;
        }
        if (system("std.exe < data.in > std.out") != 0) {
            cerr << "std.exe failed on test " << _ << "\n";
            return 1;
        }
        if (system("solve.exe < data.in > solve.out") != 0) {
            cerr << "solve.exe failed on test " << _ << "\n";
            return 1;
        }

        int dif = system("fc std.out solve.out > diff.log");
        if (dif == 1) {
            cout << "WA on test " << _ << "\n";
            break;
        }
        if (dif != 0) {
            cerr << "fc failed on test " << _ << "\n";
            return 1;
        }
        cout << "AC " << _ << "\n";
    }
    return 0;
}
```

09 - 最长上升子序列 (LIS) .cpp

79 lines · C++ · 01 - 基础算法

```
// 严格递增子序列长度
template <class T> int LIS(const vector<T> &a) {
    vector<T> d;
    for (auto x : a) {
        auto it = lower_bound(d.begin(), d.end(), x);
        if (it == d.end()) d.push_back(x);
        else *it = x;
    }
    return d.size();
}

// 非降子序列长度
template <class T> int LNDS(const vector<T> &a) {
    vector<T> d;
    for (auto x : a) {
        auto it = upper_bound(d.begin(), d.end(), x);
        if (it == d.end()) d.push_back(x);
        else *it = x;
    }
    return d.size();
}

// 返回一组严格递增 LIS 的原数组下标 (0-indexed) 。
template <class T> vector<int> lis(const vector<T> &a) {
    int n = a.size();
    vector<T> d;
    vector<int> id, pre(n, -1);
    d.reserve(n);
    id.reserve(n);

    for (int i = 0; i < n; ++i) {
        int p = lower_bound(d.begin(), d.end(), a[i]) - d.begin();
        if (p > 0) pre[i] = id[p - 1];
        if (p == d.size()) {
            d.push_back(a[i]);
            id.push_back(i);
        } else {
            d[p] = a[i];
            id[p] = i;
        }
    }

    vector<int> ans;
    if (id.empty()) return ans;
    for (int cur = id.back(); cur != -1; cur = pre[cur]) {
        ans.push_back(cur);
    }
    reverse(ans.begin(), ans.end());
    return ans;
}

// 返回一组非降 LNDS 的原数组下标 (0-indexed) 。
template <class T> vector<int> lnds(const vector<T> &a) {
    int n = a.size();
    vector<T> d;
    vector<int> id, pre(n, -1);
    d.reserve(n);
    id.reserve(n);

    for (int i = 0; i < n; ++i) {
```

```
    int p = upper_bound(d.begin(), d.end(), a[i]) - d.begin();
    if (p > 0) pre[i] = id[p - 1];
    if (p == d.size()) {
        d.push_back(a[i]);
        id.push_back(i);
    } else {
        d[p] = a[i];
        id[p] = i;
    }
}

vector<int> ans;
if (id.empty()) return ans;
for (int cur = id.back(); cur != -1; cur = pre[cur]) {
    ans.push_back(cur);
}
reverse(ans.begin(), ans.end());
return ans;
}
```


10 - 单调栈.cpp

69 lines · C++ · 01 - 基础算法

/*

用途：求每个位置左侧 (prev) 或右侧 (next) 最近的满足大小关系的位置。

约定：

- 输入 vector 为 0-indexed，返回值也是下标；不存在时返回 -1。
- less / greater 为严格比较，less_equal / greater_equal 为非严格比较。
- 求所有子数组最小值/最大值贡献时，左右两侧应一边用严格比较、另一边用非严格比较，以保证重复元素只计算一次。

*/

```
template <class T> struct MonoStack {
private:
    template <class F> static vector<int> prev(const vector<T> &a, F f) {
        int n = a.size();
        vector<int> ans(n), st;
        st.reserve(n);
        for (int i = 0; i < n; ++i) {
            while (!st.empty() && !f(a[st.back()], a[i])) st.pop_back();
            ans[i] = st.empty() ? -1 : st.back();
            st.push_back(i);
        }
        return ans;
    }

    template <class F> static vector<int> next(const vector<T> &a, F f) {
        int n = a.size();
        vector<int> ans(n), st;
        st.reserve(n);
        for (int i = n - 1; i >= 0; --i) {
            while (!st.empty() && !f(a[st.back()], a[i])) st.pop_back();
            ans[i] = st.empty() ? -1 : st.back();
            st.push_back(i);
        }
        return ans;
    }

public:
    static vector<int> preL(const vector<T> &a) {
        return prev(a, [](const T &x, const T &y) { return x < y; });
    }

    static vector<int> preLE(const vector<T> &a) {
        return prev(a, [](const T &x, const T &y) { return x <= y; });
    }

    static vector<int> preG(const vector<T> &a) {
        return prev(a, [](const T &x, const T &y) { return x > y; });
    }

    static vector<int> preGE(const vector<T> &a) {
        return prev(a, [](const T &x, const T &y) { return x >= y; });
    }

    static vector<int> nxtL(const vector<T> &a) {
        return next(a, [](const T &x, const T &y) { return x < y; });
    }

    static vector<int> nxtLE(const vector<T> &a) {
        return next(a, [](const T &x, const T &y) { return x <= y; });
    }
};
```

```
static vector<int> nxtG(const vector<T> &a) {  
    return next(a, [](const T &x, const T &y) { return x > y; });  
}  
  
static vector<int> nxtGE(const vector<T> &a) {  
    return next(a, [](const T &x, const T &y) { return x >= y; });  
}  
};
```

11 - 坐标压缩.cpp

50 lines · C++ · 01 - 基础算法

```
/*
用途：离散化任意可比较值；下标从 0 开始。
接口：先 add / 构造，再 build；askId(x) 要求 x 已出现，ask / askR 可查询任意值。
复杂度：build  $O(k \log k)$ ，其余查询  $O(\log k)$ 。
*/
template <class T> struct Compress {
    vector<T> a;

    Compress() = default;

    explicit Compress(vector<T> a) : a(move(a)) {
        build();
    }

    void add(const T& x) {
        a.push_back(x);
    }

    template <class It> void add(It l, It r) {
        a.insert(a.end(), l, r);
    }

    void build() {
        sort(a.begin(), a.end());
        a.erase(unique(a.begin(), a.end()), a.end());
    }

    int askSz() const {
        return a.size();
    }

    int ask(const T& x) const {
        return lower_bound(a.begin(), a.end(), x) - a.begin();
    }

    int askR(const T& x) const {
        return upper_bound(a.begin(), a.end(), x) - a.begin();
    }

    int askId(const T& x) const {
        int p = ask(x);
        assert(p < askSz() && a[p] == x);
        return p;
    }

    const T& operator[](int p) const {
        assert(0 <= p && p < askSz());
        return a[p];
    }
};
```

12 - 二分查找（边界）.cpp

35 lines · C++ · 01 - 基础算法

```
/*
用途：单调谓词的边界二分。
约定：区间均为闭区间 [l, r]。first 要求 f 从 false 单调变为 true；
last 要求 f 从 true 单调变为 false。两者均不要求端点满足谓词，找不到时返回 nullopt。
复杂度：O(log(right - left + 1))。
*/
template <class I, class F> optional<I> first(I l, I r, F f) {
    static_assert(std::is_integral_v<I> && std::is_signed_v<I>);
    optional<I> a;
    while (l <= r) {
        I m = l + (r - l) / 2;
        if (f(m)) {
            a = m;
            r = m - 1;
        } else {
            l = m + 1;
        }
    }
    return a;
}

template <class I, class F> optional<I> last(I l, I r, F f) {
    static_assert(std::is_integral_v<I> && std::is_signed_v<I>);
    optional<I> a;
    while (l <= r) {
        I m = l + (r - l) / 2;
        if (f(m)) {
            a = m;
            l = m + 1;
        } else {
            r = m - 1;
        }
    }
    return a;
}
```

13 - 前缀和与差分.cpp

90 lines · C++ · 01 - 基础算法

```

/*
用途：静态一维/二维区间和与一维区间加差分。
约定：输入数组下标均为 0-indexed，ask / update 的区间均为闭区间 [l,r]。
复杂度：前缀和预处理  $O(n)$  或  $O(nm)$ 、查询  $O(1)$ ；差分每次更新  $O(1)$ 、build  $O(n)$ 。
*/
template <class T> struct PrefixSum {
    vector<T> pre;

    PrefixSum() = default;

    explicit PrefixSum(const vector<T>& a) {
        build(a);
    }

    void build(const vector<T>& a) {
        pre.assign(a.size() + 1, T{});
        for (int i = 0; i < a.size(); ++i) {
            pre[i + 1] = pre[i] + a[i];
        }
    }

    T ask(int l, int r) const {
        assert(0 <= l && l <= r && r + 1 < pre.size());
        return pre[r + 1] - pre[l];
    }
};

template <class T> struct Prefix2D {
    vector<vector<T>> pre;

    Prefix2D() = default;

    explicit Prefix2D(const vector<vector<T>>& a) {
        build(a);
    }

    void build(const vector<vector<T>>& a) {
        int n = a.size();
        int m = n == 0 ? 0 : a[0].size();
        for (const auto& v : a) assert(v.size() == m);

        pre.assign(n + 1, vector<T>(m + 1, T{}));
        for (int i = 0; i < n; ++i) {
            for (int j = 0; j < m; ++j) {
                pre[i + 1][j + 1] = a[i][j] + pre[i][j + 1] + pre[i + 1][j] - pre[i][j];
            }
        }
    }

    T ask(int x1, int y1, int x2, int y2) const {
        assert(0 <= x1 && x1 <= x2 && 0 <= y1 && y1 <= y2 &&
            x2 + 1 < pre.size() && y2 + 1 < pre[0].size());
        return pre[x2 + 1][y2 + 1] - pre[x1][y2 + 1] - pre[x2 + 1][y1] + pre[x1][y1];
    }
};

template <class T> struct Diff {
    vector<T> d;

    Diff() = default;

```

```
explicit Diff(const vector<T>& a) {
    build(a);
}

void build(const vector<T>& a) {
    d.assign(a.size() + 1, T{});
    for (int i = 0; i < a.size(); ++i) {
        d[i] += a[i];
        d[i + 1] -= a[i];
    }
}

void update(int l, int r, const T& x) {
    assert(0 <= l && l <= r && r + 1 < d.size());
    d[l] += x;
    d[r + 1] -= x;
}

vector<T> ask() const {
    assert(!d.empty());
    vector<T> res(d.size() - 1);
    T cur{};
    for (int i = 0; i < res.size(); ++i) {
        cur += d[i];
        res[i] = cur;
    }
    return res;
}
};
```

14 - 逆序对（归并排序）.cpp

29 lines · C++ · 01 - 基础算法

```
/*
用途：统计严格逆序对数，即  $i < j$  且  $values[i] > values[j]$  的对数。
约定：Compare 默认 less<T>；传入的比较器应满足严格弱序。
复杂度： $O(n \log n)$  时间、 $O(n)$  额外空间；输入数组不被修改。
*/

template <class T, class Cmp = less<T>> int invCnt(vector<T> a, Cmp cmp = Cmp{}) {
    vector<T> buf(a.size());

    auto mg = [&](auto&& go, int l, int r) -> int {
        if (r - l <= 1) return 0;
        int mid = l + (r - l) / 2;
        int ans = go(go, l, mid) + go(go, mid, r);

        int i = l, j = mid, p = l;
        while (i < mid || j < r) {
            if (j == r || (i < mid && !cmp(a[j], a[i]))) {
                buf[p++] = a[i++];
            } else {
                ans += mid - i;
                buf[p++] = a[j++];
            }
        }
        for (int id = l; id < r; ++id) a[id] = buf[id];
        return ans;
    };

    return mg(mg, 0, a.size());
}
```

15 - 随机数据生成（树与图）.cpp

74 lines · C++ · 01 - 基础算法

```

/*
用途：对拍时生成随机数、排列、树、简单连通无向图和字符串。
约定：点编号 1..n；图无自环、无重边。稠密图改为枚举补边，避免拒绝采样卡死。
性能：默认使用宏展开后的 int；若大规模数据因此超时或超空间，可将纯下标、状态或计数局部改为 i32。
*/
struct Gen {
    using E = array<int, 2>;
    mt19937_64 g;

    explicit Gen(int s = chrono::steady_clock::now().time_since_epoch().count()) : g(s) {}

    template <class T> T num(T l, T r) {
        assert(l <= r);
        return uniform_int_distribution<T>(l, r)(g);
    }

    template <class T> void mix(vector<T>& a) {
        shuffle(a.begin(), a.end(), g);
    }

    vector<int> perm(int n, int s = 1) {
        assert(n >= 0);
        vector<int> a(n);
        iota(a.begin(), a.end(), s);
        mix(a);
        return a;
    }

    vector<E> tree(int n) {
        assert(n >= 1);
        auto p = perm(n);
        vector<E> e;
        for (int i = 1; i < n; ++i) e.push_back({p[num<int>(0, i - 1)], p[i]});
        mix(e);
        return e;
    }

    vector<E> graph(int n, int m, bool con = true) {
        assert(n >= 1);
        int lim = n * (n - 1) / 2;
        assert(0 <= m && m <= lim && (!con || m >= n - 1));
        vector<E> e = con ? tree(n) : vector<E>{};
        unordered_set<int> st;
        auto key = [n](int u, int v) {
            if (u > v) swap(u, v);
            return u * (n + 1) + v;
        };
        for (auto [u, v] : e) st.insert(key(u, v));
        int nd = m - e.size(), lf = lim - st.size();
        if (nd * 2 > lf) {
            vector<E> a;
            for (int u = 1; u <= n; ++u) for (int v = u + 1; v <= n; ++v)
                if (!st.count(key(u, v))) a.push_back({u, v});
            mix(a);
            e.insert(e.end(), a.begin(), a.begin() + nd);
        } else {
            while (nd) {
                int u = num<int>(1, n), v = num<int>(1, n);
                if (u == v) continue;
                if (u > v) swap(u, v);
            }
        }
    }
}

```



```
        if (st.insert(key(u, v)).second) e.push_back({u, v}), --nd;
    }
}
mix(e);
return e;
}

string str(int n, string_view a = "abcdefghijklmnopqrstuvwxyz") {
    assert(n >= 0 && !a.empty());
    string s(n, ' ');
    for (char& c : s) c = a[num<int>(0, a.size() - 1)];
    return s;
}

};
```

01 - 树的直径.cpp

51 lines · C++ · 02 - 树上问题

```

/*
用途：无权树直径模板。两次 DFS/遍历即可求直径长度。

接口：
- Tree(n)：建立 1-indexed 的 n 点树。
- add(u, v)：加入无向边。
- askEnds(root = 1)：返回一组直径端点。
- askLen(root = 1)：返回直径长度（边数）。
*/
struct Tree {
    int n;
    vector<vector<int>> ver;

    explicit Tree(int n) : n(n), ver(n + 1) {}

    void add(int u, int v) {
        ver[u].push_back(v);
        ver[v].push_back(u);
    }

    pair<int, vector<int>> far(int src) const {
        vector<int> dep(n + 1, -1), st = {src};
        dep[src] = 0;
        for (int i = 0; i < st.size(); ++i) {
            int u = st[i];
            for (int v : ver[u]) {
                if (dep[v] != -1) continue;
                dep[v] = dep[u] + 1;
                st.push_back(v);
            }
        }

        int far = src;
        for (int u = 1; u <= n; ++u) {
            if (dep[u] > dep[far]) far = u;
        }
        return {far, dep};
    }

    pair<int, int> askEnds(int rt = 1) const {
        int u = far(rt).first;
        int v = far(u).first;
        return {u, v};
    }

    int askLen(int rt = 1) const {
        int u = far(rt).first;
        auto [v, dep] = far(u);
        return dep[v];
    }
};

```

01A - 树论大封装（直径+中心+重心）.cpp

154 lines · C++ · 02 - 树上问题

/*

用途：带非负边权树的基础树论封装。

支持：

- 直径端点与直径长度；
- 顶点树心（使到最远顶点距离最小的点）与半径；
- 树重心（删去该点后最大连通块最小）；
- 指定根下的 parent / dep / maxdep。

约定：

- 点编号为 $1..n$ ，树连通；边权必须非负。
- center 是“顶点树心”。若题目允许把中心放在边的内部，需要在直径上另行二分位置。

接口：

- add(u, v, w = 1)：加入无向边。
- work(root = 1)：一次性完成全部预处理。
- askEnds()：返回一组直径端点，同时更新 diam。

*/

```
struct Tree {

    int n;
    vector<vector<pair<int, int>>> e;

    // dep 为无权深度；maxdep 为该根树内最大的无权深度。
    vector<int> dep, fa, mxd;
    // d1 / d2：向下第一、第二长距离；up：经过父边到树外的最长距离。
    vector<int> d1, d2, up;
    vector<int> s1, s2;
    vector<bool> vis;
    int rem, cog;
    int rad, dia;
    int cen;

    explicit Tree(int n) : n(n), e(n + 1), dep(n + 1), fa(n + 1),
                           mxd(n + 1), d1(n + 1), d2(n + 1), up(n + 1),
                           s1(n + 1), s2(n + 1), vis(n + 1) {}

    void add(int u, int v, int w = 1) {
        e[u].push_back({v, w});
        e[v].push_back({u, w});
    }

    void dfs(int u, int _fa) {
        fa[u] = _fa;
        mxd[u] = dep[u];
        for (auto [v, w] : e[u]) {
            if (v == _fa) continue;
            dep[v] = dep[u] + 1;
            dfs(v, u);
            mxd[u] = max(mxd[u], mxd[v]);
        }
    }

    void dfs1(int u, int fa) {
        for (auto [v, w] : e[u]) {
            if (v == fa) continue;
            dfs1(v, u);
            int cur = d1[v] + w;
            if (cur > d1[u]) {
```

```

        d2[u] = d1[u], s2[u] = s1[u];
        d1[u] = cur, s1[u] = v;
    } else if (cur > d2[u]) {
        d2[u] = cur, s2[u] = v;
    }
}

}

void dfs2(int u, int fa) {
    for (auto [v, w] : e[u]) {
        if (v == fa) continue;
        up[v] = (s1[u] == v ? max(up[u], d2[u]) : max(up[u], d1[u])) + w;
        dfs2(v, u);
    }
}

void calCen() {
    cen = 1;
    for (int u = 1; u <= n; ++u) {
        if (max(d1[u], up[u]) < max(d1[cen], up[cen])) cen = u;
    }
    rad = max(d1[cen], up[cen]);

    dia = 0;
    for (int u = 1; u <= n; ++u) {
        dia = max(dia, d1[u] + max(d2[u], up[u]));
    }
}

int dfsCog(int u, int fa) {
    int siz = 1, mx = 0;
    for (auto [v, w] : e[u]) {
        if (v == fa) continue;
        int sub = dfsCog(v, u);
        siz += sub;
        mx = max(mx, sub);
    }
    mx = max(mx, n - siz);
    if (mx < rem) rem = mx, cog = u;
    return siz;
}

void calCog() {
    rem = numeric_limits<int>::max();
    cog = 1;
    dfsCog(1, 0);
}

pair<int, vector<int>> far(int src) const {
    vector<int> fa(n + 1, -1), st = {src};
    vector<int> dis(n + 1);
    fa[src] = 0;
    for (int i = 0; i < st.size(); ++i) {
        int u = st[i];
        for (auto [v, w] : e[u]) {
            if (v == fa[u]) continue;
            fa[v] = u;
            dis[v] = dis[u] + w;
            st.push_back(v);
        }
    }

    int far = src;
    for (int u = 1; u <= n; ++u) {

```

```
        if (dis[u] > dis[far]) far = u;
    }
    return {far, dis};
}

pair<int, int> askEnds() {
    auto [u, ban] = far(1);
    auto [v, dis] = far(u);
    dia = dis[v];
    return {u, v};
}

void work(int rt = 1) {
    dep.assign(n + 1, 0);
    fa.assign(n + 1, 0);
    mxd.assign(n + 1, 0);
    d1.assign(n + 1, 0);
    d2.assign(n + 1, 0);
    up.assign(n + 1, 0);
    s1.assign(n + 1, 0);
    s2.assign(n + 1, 0);

    dfs(rt, 0);
    dfs1(rt, 0);
    dfs2(rt, 0);
    calCen();
    calCog();
    askEnds();
}

};
```

02 - 点分治（重心树骨架）.cpp

75 lines · C++ · 02 - 树上问题

/*

用途：点分治（重心树）通用骨架。

适用：树上距离统计、动态点集距离查询、分治时按“经过重心”的路径统计等。

build 的回调会在每个重心被标记 removed 后调用：callback(centroid, centroidParent, level)。

回调可读取 e、removed、centroidParent、centroidDepth；随后每个未删除连通块会继续分治。

接口：

- add(u, v, w = 1)：加入无向边；w 仅供回调使用，分治本身不依赖边权。

- build(callback, root = 1)：建立重心树。

复杂度：建树 $O(n \log n)$ ，递归深度 $O(\log n)$ ；原树 DFS 仍可能有 $O(n)$ 调用深度。

*/

```
struct CenDec {

    struct Edge {
        int to;
        int w;
    };

    int n;
    vector<vector<Edge>> e;
    vector<int> siz, cfa, dep;
    vector<char> ban;

    explicit CenDec(int n) : n(n), e(n + 1), siz(n + 1),
                                cfa(n + 1), dep(n + 1),
                                ban(n + 1) {}

    void add(int u, int v, int w = 1) {
        e[u].push_back({v, w});
        e[v].push_back({u, w});
    }

    int dfsSz(int u, int fa) {
        siz[u] = 1;
        for (auto [v, w] : e[u]) {
            if (v == fa || ban[v]) continue;
            siz[u] += dfsSz(v, u);
        }
        return siz[u];
    }

    int cen(int u, int fa, int tot) {
        for (auto [v, w] : e[u]) {
            if (v == fa || ban[v]) continue;
            if (siz[v] > tot / 2) return cen(v, u, tot);
        }
        return u;
    }

    template <class F>
    void divide(int st, int fa, int d, F &f) {
        int tot = dfsSz(st, 0);
        int c = cen(st, 0, tot);
        cfa[c] = fa;
        dep[c] = d;
        ban[c] = true;
        f(c, fa, d);
    }
};
```

```
        for (auto [v, w] : e[c]) {
            if (!ban[v]) divide(v, c, d + 1, f);
        }
    }

    template <class F>
    void build(F f, int rt = 1) {
        fill(siz.begin(), siz.end(), 0);
        fill(cfa.begin(), cfa.end(), 0);
        fill(dep.begin(), dep.end(), 0);
        fill(ban.begin(), ban.end(), 0);
        divide(rt, 0, 0, f);
    }
};
```

02A - 点分治（距离询问）.cpp

85 lines · C++ · 02 - 树上问题

```

/*
用途：点分治模板，常用于统计/判断树上点对距离，当前写法用于回答是否存在距离为  $q[i]$  的点对。
依赖：
- 外部已有  $n, m$ 。
-  $ver[u]$  存边  $\{v, w\}$ 。
-  $q[1..m]$  存询问距离， $ans[1..m]$  会被置为 1 表示存在。
接口/流程：
-  $dfsSz(x, fa)$ ：计算当前未删除连通块大小。
-  $cen(x, fa, total)$ ：在大小为  $total$  的连通块内找重心  $root$ 。
-  $calc(x)$ ：处理所有经过重心  $x$  的路径贡献。
-  $dfz(root, 0)$ ：从重心  $root$  开始分治。
*/
int rt = 0, mt = 0;
vector<int> vis(n + 1), siz(n + 1);

auto fsz = [&](auto go, int x, int fa) -> int {
    siz[x] = 1;
    for (auto [y, w] : ver[x]) {
        if (y == fa || vis[y]) continue;
        siz[x] += go(go, y, x);
    }
    return siz[x];
};

auto cen = [&](auto go, int x, int fa, int tot) -> void {
    siz[x] = 1;
    int val = 0;
    for (auto [y, w] : ver[x]) {
        if (y == fa || vis[y]) continue;
        go(go, y, x, tot);
        siz[x] += siz[y];
        val = max(val, siz[y]);
    }
    val = max(val, tot - siz[x]);
    if (val < mt) {
        mt = val;
        rt = x;
    }
};

auto cal = [&](int x) -> void {
    set<int> pre = {0};
    vector<int> dis(n + 1);
    for (auto [y, w] : ver[x]) {
        if (vis[y]) continue;
        vector<int> son;
        auto dfs = [&](auto go, int u, int fa) -> void {
            son.push_back(dis[u]);
            for (auto [y, w] : ver[u]) {
                if (y == fa || vis[y]) continue;
                dis[y] = dis[u] + w;
                go(go, y, u);
            }
        };
        dis[y] = w;
        dfs(dfs, y, x);

        for (auto it : son) {
            for (int i = 1; i <= m; i++) {
                if (q[i] < it || !pre.count(q[i] - it)) continue;
            }
        }
    }
};

```



```
        ans[i] = 1;
    }
    }
    pre.insert(son.begin(), son.end());
}
};

auto dfz = [&](auto go, int x, int fa) -> void {
    vis[x] = 1;
    cal(x);
    for (auto [y, w] : ver[x]) {
        if (y == fa || vis[y]) continue;
        int tot = fsz(fsz, y, x);
        rt = 0;
        mt = tot;
        cen(cen, y, x, tot);
        go(go, rt, x);
    }
};

int tot = fsz(fsz, 1, 0);
rt = 0;
mt = tot;
cen(cen, 1, 0, tot);
dfz(dfz, rt, 0);
```

02B - 点分树（动态点集最近距离）.cpp

134 lines · C++ · 02 - 树上问题

/*

用途：点分树（重心树）维护动态点集到任意点的最近距离。

约定：点编号为 $1..n$ ，树连通，边权非负。

接口：

- add(u, v, w = 1) \ build(root = 1)。
- add(u) / del(u) / modify(u, on)。重复操作会被忽略并返回 false。
- askMin(u)：返回到最近激活点的距离；不存在激活点时返回 nullopt。
- ask(u)：askMin 的 int 版本；不存在激活点时返回 -1。

复杂度：建树 $O(n \log n)$ ，每次激活、撤销、查询均为 $O(\log^2 n)$ 。

path[u] 公开保存 u 到每层重心的 {centroid, distance}，可在此基础上扩展计数、距离和等信息。

*/

```
struct CenTree {

    struct Edge {
        int to;
        int w;
    };

    struct PathInfo {
        int cen;
        int dis;
    };

    int n;
    vector<vector<Edge>> e;
    vector<int> siz, cfa;
    vector<char> ban, on;
    vector<vector<PathInfo>> pth;
    vector<multiset<int>> q;

    explicit CenTree(int n)
        : n(n), e(n + 1), siz(n + 1), cfa(n + 1), ban(n + 1), on(n + 1),
          pth(n + 1), q(n + 1) {}

    void add(int u, int v, int w = 1) {
        e[u].push_back({v, w});
        e[v].push_back({u, w});
    }

    int dfsSz(int u, int fa) {
        siz[u] = 1;
        for (auto [v, w] : e[u]) {
            if (v == fa || ban[v]) continue;
            siz[u] += dfsSz(v, u);
        }
        return siz[u];
    }

    int cen(int u, int fa, int tot) {
        for (auto [v, w] : e[u]) {
            if (v == fa || ban[v]) continue;
            if (siz[v] > tot / 2) return cen(v, u, tot);
        }
        return u;
    }

    void dfsDis(int u, int fa, int dis, int cen) {
```

```

        pth[u].push_back({cen, dis});
        for (auto [v, w] : e[u]) {
            if (v == fa || ban[v]) continue;
            dfsDis(v, u, dis + w, cen);
        }
    }

void build(int st, int fa) {
    int tot = dfsSz(st, 0);
    int cen = this->cen(st, 0, tot);
    cfa[cen] = fa;
    ban[cen] = true;
    dfsDis(cen, 0, 0, cen);

    for (auto [v, w] : e[cen]) {
        if (!ban[v]) build(v, cen);
    }
}

void build(int rt = 1) {
    fill(siz.begin(), siz.end(), 0);
    fill(cfa.begin(), cfa.end(), 0);
    fill(ban.begin(), ban.end(), false);
    fill(on.begin(), on.end(), false);
    for (int u = 1; u <= n; ++u) {
        pth[u].clear();
        q[u].clear();
    }
    build(rt, 0);
}

bool modify(int u, bool _on) {
    if ((bool)on[u] == _on) return false;
    on[u] = _on;
    for (auto [cen, dis] : pth[u]) {
        if (_on) {
            q[cen].insert(dis);
        } else {
            auto it = q[cen].find(dis);
            assert(it != q[cen].end());
            q[cen].erase(it);
        }
    }
    return true;
}

bool add(int u) {
    return modify(u, true);
}

bool del(int u) {
    return modify(u, false);
}

void clear() {
    fill(on.begin(), on.end(), false);
    for (int u = 1; u <= n; ++u) q[u].clear();
}

optional<int> askMin(int u) const {
    optional<int> ans;
    for (auto [cen, dis] : pth[u]) {
        if (q[cen].empty()) continue;
        int cur = dis + *q[cen].begin();
    }
}

```

```
        if (!ans || cur < *ans) ans = cur;
    }
    return ans;
}

int ask(int u) const {
    auto ans = askMin(u);
    return ans ? *ans : -1;
}
};
```

03A - 最近公共祖先 (Tree, 树链剖分) .cpp

84 lines · C++ · 02 - 树上问题

/*

用途：树链剖分版 LCA，适合只需要最近公共祖先、无权距离或 DFS 序的场景。

接口：

- Tree(n)：建立 1-indexed 的 n 点树。
- add(u, v)：加入无向边。
- work(root = 1)：预处理 siz / dep / parent / son / top / dfn / out。
- askLCA(u, v)：返回最近公共祖先。
- askDis(u, v)：返回 u 到 v 的边数距离。
- askAnc(u, v)：判断 u 是否为 v 的祖先 (含 u == v)。

dfn 为重链优先 DFS 序，但子树仍是连续区间 [dfn[u], out[u]]，可直接用于虚树。

*/

```
struct Tree {
    int n, idx;
    vector<vector<int>> ver;
    vector<int> siz, dep, top, son, fa, dfn, out, rev;

    explicit Tree(int n) : n(n), idx(0), ver(n + 1), siz(n + 1), dep(n + 1),
        top(n + 1), son(n + 1), fa(n + 1), dfn(n + 1),
        out(n + 1), rev(n + 1) {}

    void add(int u, int v) {
        ver[u].push_back(v);
        ver[v].push_back(u);
    }

    void dfs1(int u) {
        siz[u] = 1;
        for (int v : ver[u]) {
            if (v == fa[u]) continue;
            fa[v] = u;
            dep[v] = dep[u] + 1;
            dfs1(v);
            siz[u] += siz[v];
            if (siz[v] > siz[son[u]]) son[u] = v;
        }
    }

    void dfs2(int u, int t) {
        top[u] = t;
        dfn[u] = ++idx;
        rev[idx] = u;
        if (son[u]) dfs2(son[u], t);
        for (int v : ver[u]) {
            if (v == fa[u] || v == son[u]) continue;
            dfs2(v, v);
        }
        out[u] = idx;
    }

    int askLCA(int u, int v) const {
        while (top[u] != top[v]) {
            if (dep[top[u]] > dep[top[v]]) {
                u = fa[top[u]];
            } else {
                v = fa[top[v]];
            }
        }
        return dep[u] < dep[v] ? u : v;
    }
};
```

```
    }

    int askDis(int u, int v) const {
        return dep[u] + dep[v] - 2 * dep[askLCA(u, v)];
    }

    bool askAnc(int u, int v) const {
        return dfn[u] <= dfn[v] && dfn[v] <= out[u];
    }

    void work(int rt = 1) {
        idx = 0;
        fill(siz.begin(), siz.end(), 0);
        fill(dep.begin(), dep.end(), 0);
        fill(top.begin(), top.end(), 0);
        fill(son.begin(), son.end(), 0);
        fill(fa.begin(), fa.end(), 0);
        fill(dfn.begin(), dfn.end(), 0);
        fill(out.begin(), out.end(), 0);
        fill(rev.begin(), rev.end(), 0);
        dfs1(rt);
        dfs2(rt, rt);
    }
};
```

03B - 最近公共祖先 (Tree, 倍增) .cpp

88 lines · C++ · 02 - 树上问题

/*
用途：倍增版 LCA，适合静态无权树上的最近公共祖先、距离和跳祖先查询。

接口：

- work(root = 1)：预处理 dep / val / tin / tout。
- askLCA(u, v)：最近公共祖先。
- askDis(u, v)：边数距离。
- askJmp(u, k)：u 的第 k 个祖先；越过根时返回 0。
- askKth(u, v, k)：路径 u → v 上第 k 条边到达的点；越界返回 0。

tin / tout 可直接配合“虚树”模板。点编号为 1..n。

性能：倍增祖先表默认使用 int；极限内存不足时可只将祖先编号改为 i32。

```
*/
struct Tree {
    int n, lg, tim;
    vector<vector<int>> ver, val;
    vector<int> dep, tin, out;

    explicit Tree(int n) : n(n), tim(0), ver(n + 1), dep(n + 1),
                          tin(n + 1), out(n + 1) {
        lg = 1;
        while ((1LL << lg) <= max<int>(1, n)) ++lg;
        val.assign(n + 1, vector<int>(lg));
    }

    void add(int u, int v) {
        ver[u].push_back(v);
        ver[v].push_back(u);
    }

    void dfs(int u, int fa) {
        val[u][0] = fa;
        for (int k = 1; k < lg; ++k) {
            val[u][k] = val[val[u][k - 1]][k - 1];
        }
        tin[u] = ++tim;
        for (int v : ver[u]) {
            if (v == fa) continue;
            dep[v] = dep[u] + 1;
            dfs(v, u);
        }
        out[u] = tim;
    }

    int askJmp(int u, int k) const {
        if (k < 0 || k > dep[u]) return 0;
        for (int bit = 0; bit < lg; ++bit) {
            if ((k >> bit) & 1) u = val[u][bit];
        }
        return u;
    }

    int askLCA(int u, int v) const {
        if (dep[u] < dep[v]) swap(u, v);
        u = askJmp(u, dep[u] - dep[v]);
        if (u == v) return u;
        for (int k = lg - 1; k >= 0; --k) {
            if (val[u][k] == val[v][k]) continue;
            u = val[u][k];
            v = val[v][k];
        }
    }
};
```

```
        }
        return val[u][0];
    }

    int askDis(int u, int v) const {
        return dep[u] + dep[v] - 2 * dep[askLCA(u, v)];
    }

    int askKth(int u, int v, int k) const {
        int p = askLCA(u, v);
        int l = dep[u] - dep[p], r = dep[v] - dep[p];
        if (k < 0 || k > l + r) return 0;
        return k <= l ? askJmp(u, k) : askJmp(v, l + r - k);
    }

    bool askAnc(int u, int v) const {
        return tin[u] <= tin[v] && out[v] <= out[u];
    }

    void work(int rt = 1) {
        tim = 0;
        fill(dep.begin(), dep.end(), 0);
        fill(tin.begin(), tin.end(), 0);
        fill(out.begin(), out.end(), 0);
        for (auto &row : val) fill(row.begin(), row.end(), 0);
        dfs(rt, 0);
    }
};
```


03C - 最近公共祖先 (Tree, 倍增, 有权图) .cpp

117 lines · C++ · 02 - 树上问题

/*

用途：倍增版有权树 LCA，支持路径距离、路径最大边权和跳祖先查询。

接口：

- add(u, v, w)：加入权值为 w 的无向边 (w 可为负)。
- work(root = 1)：预处理 dep / dis / val / mx / tin / out。
- askLCA(u, v)、askDis(u, v)、askMax(u, v)、askJmp(u, k)、askKth(u, v, k)。

askDis 返回路径权值和；askMax 返回路径最大边权。对空路径 (u == v)，askMax 返回 0。

性能：倍增祖先表默认使用 int；极限内存不足时可只将祖先编号改为 i32，边权与距离保持 int。

*/

```
struct Tree {
    static constexpr int NI = numeric_limits<int>::lowest() / 4;

    int n, lg, tim;
    vector<vector<int>> val;
    vector<vector<int>> mx;
    vector<vector<pair<int, int>>> ver;
    vector<int> dep, tin, out;
    vector<int> dis;

    explicit Tree(int n) : n(n), tim(0), ver(n + 1), dep(n + 1), tin(n + 1),
        out(n + 1), dis(n + 1) {
        lg = 1;
        while ((1LL <= lg) <= max<int>(1, n)) ++lg;
        val.assign(n + 1, vector<int>(lg));
        mx.assign(n + 1, vector<int>(lg, NI));
    }

    void add(int u, int v, int w) {
        ver[u].push_back({v, w});
        ver[v].push_back({u, w});
    }

    void dfs(int u, int fa) {
        val[u][0] = fa;
        for (int k = 1; k < lg; ++k) {
            val[u][k] = val[val[u][k - 1]][k - 1];
            mx[u][k] = max(mx[u][k - 1], mx[val[u][k - 1]][k - 1]);
        }
        tin[u] = ++tim;
        for (auto [v, w] : ver[u]) {
            if (v == fa) continue;
            dep[v] = dep[u] + 1;
            dis[v] = dis[u] + w;
            mx[v][0] = w;
            dfs(v, u);
        }
        out[u] = tim;
    }

    int askJmp(int u, int k) const {
        if (k < 0 || k > dep[u]) return 0;
        for (int bit = 0; bit < lg; ++bit) {
            if ((k >> bit) & 1) u = val[u][bit];
        }
        return u;
    }
}
```

```

int askLCA(int u, int v) const {
    if (dep[u] < dep[v]) swap(u, v);
    u = askJmp(u, dep[u] - dep[v]);
    if (u == v) return u;
    for (int k = lg - 1; k >= 0; --k) {
        if (val[u][k] == val[v][k]) continue;
        u = val[u][k];
        v = val[v][k];
    }
    return val[u][0];
}

int askDis(int u, int v) const {
    int p = askLCA(u, v);
    return dis[u] + dis[v] - 2 * dis[p];
}

int askMax(int u, int v) const {
    if (u == v) return 0;
    int ans = NI;
    auto up = [&](int x, int t) {
        for (int k = lg - 1; k >= 0; --k) {
            if (dep[x] - (1LL << k) >= dep[t]) {
                ans = max(ans, mx[x][k]);
                x = val[x][k];
            }
        }
    };
    up(u, v);
    up(v, u);
    return ans;
}

int p = askLCA(u, v);
up(u, p);
up(v, p);
return ans;
}

int askKth(int u, int v, int k) const {
    int p = askLCA(u, v);
    int l = dep[u] - dep[p], r = dep[v] - dep[p];
    if (k < 0 || k > l + r) return 0;
    return k <= l ? askJmp(u, k) : askJmp(v, l + r - k);
}

bool askAnc(int u, int v) const {
    return tin[u] <= tin[v] && out[v] <= out[u];
}

void work(int rt = 1) {
    tim = 0;
    fill(dep.begin(), dep.end(), 0);
    fill(tin.begin(), tin.end(), 0);
    fill(out.begin(), out.end(), 0);
    fill(dis.begin(), dis.end(), 0);
    for (auto &row : val) fill(row.begin(), row.end(), 0);
    for (auto &row : mx) fill(row.begin(), row.end(), NI);
    dfs(rt, 0);
}
};

```

04 - 树上启发式合并（通用框架）.cpp

76 lines · C++ · 02 - 树上问题

/*

用途：DSU on Tree / Sack 通用框架。

适用：对每个子树统计颜色、值域、出现次数等可“加入/删除一个点”的信息。

接口：

- add(u, v)：加入无向边。
- work(addVertex, answer, root = 1)：
 - * addVertex(vertex, delta)：delta 为 +1 或 -1，维护当前数据结构；
 - * answer(u)：此时当前数据结构恰为 u 的子树，记录答案。

内部维护 tin / tout / euler / siz / heavy，子树区间为 [tin[u], tout[u]]。

复杂度：每个点至多被加入/删除 $O(\log n)$ 次，总计 $O(n \log n)$ 次回调。

*/

```
struct DSUOnTree {
    int n, tim;
    vector<vector<int>>> e;
    vector<int> fa, siz, son, tin, out, ord;

    explicit DSUOnTree(int n) : n(n), tim(0), e(n + 1), fa(n + 1), siz(n + 1),
                                son(n + 1), tin(n + 1), out(n + 1), ord(n + 1) {}

    void add(int u, int v) {
        e[u].push_back(v);
        e[v].push_back(u);
    }

    void dfsSz(int u, int _fa) {
        fa[u] = _fa;
        siz[u] = 1;
        tin[u] = ++tim;
        ord[tim] = u;
        for (int v : e[u]) {
            if (v == _fa) continue;
            dfsSz(v, u);
            siz[u] += siz[v];
            if (siz[v] > siz[son[u]]) son[u] = v;
        }
        out[u] = tim;
    }

    template <class Add>
    void addSub(int u, int d, Add &add) const {
        for (int i = tin[u]; i <= out[u]; ++i) add(ord[i], d);
    }

    template <class A, class F>
    void solve(int u, bool kp, A &add, F &ans) {
        for (int v : e[u]) {
            if (v == fa[u] || v == son[u]) continue;
            solve(v, false, add, ans);
        }
        if (son[u]) solve(son[u], true, add, ans);

        for (int v : e[u]) {
            if (v == fa[u] || v == son[u]) continue;
            addSub(v, +1, add);
        }
        add(u, +1);
        ans(u);
    }
};
```

```
        if (!kp) addSub(u, -1, add);
    }

    template <class A, class F>
    void work(A add, F ans, int rt = 1) {
        tim = 0;
        fill(fa.begin(), fa.end(), 0);
        fill(siz.begin(), siz.end(), 0);
        fill(son.begin(), son.end(), 0);
        fill(tin.begin(), tin.end(), 0);
        fill(out.begin(), out.end(), 0);
        dfsSz(rt, 0);
        solve(rt, true, add, ans);
    }
};
```

04A - 树上启发式合并（众数颜色和）.cpp

78 lines · C++ · 02 - 树上问题

```

/*
用途：树上启发式合并（DSU on Tree），当前模板用于统计每个子树中出现次数最多的颜色编号之和。
依赖：
- 外部数组 color[u] 表示点 u 的颜色；若颜色值可能大于 n，需要先离散化或扩大 cnt。
接口：
- DSUOnTree(n)：建立 1-indexed 的 n 点树辅助结构。
- add(u, v)：加入无向边。
- dfs1(root, 0)：预处理子树大小和重儿子。
- dfs2(root, 0, 0)：计算 ans[u]。
- work(root = 1)：一次性完成 dfs1 和 dfs2。
- ans[u]：子树 u 中最高出现次数的颜色编号之和。
*/

struct DSUOnTree {
    vector<vector<int>> e;
    vector<int> siz, son, cnt;
    vector<int> ans;
    int sum, mx;
    int hs;
    DSUOnTree(int n) {
        e.resize(n + 1);
        siz.resize(n + 1);
        son.resize(n + 1);
        ans.resize(n + 1);
        cnt.resize(n + 1);
        hs = 0;
        sum = 0;
        mx = 0;
    }
    void add(int u, int v) {
        e[u].push_back(v);
        e[v].push_back(u);
    }
    void dfs1(int u, int fa) {
        siz[u] = 1;
        for (auto v : e[u]) {
            if (v == fa) continue;
            dfs1(v, u);
            siz[u] += siz[v];
            if (siz[v] > siz[son[u]]) son[u] = v;
        }
    }
    void calc(int u, int fa, int val) {
        cnt[col[u]] += val;
        if (cnt[col[u]] > mx) {
            mx = cnt[col[u]];
            sum = col[u];
        } else if (cnt[col[u]] == mx) {
            sum += col[u];
        }
        for (auto v : e[u]) {
            if (v == fa || v == hs) continue;
            calc(v, u, val);
        }
    }
    void dfs2(int u, int fa, int opt) {
        for (auto v : e[u]) {
            if (v == fa || v == son[u]) continue;
            dfs2(v, u, 0);
        }
    }
}

```

```
        if (son[u]) {
            dfs2(son[u], u, 1);
            hs = son[u];
        }
        calc(u, fa, 1);
        hs = 0;
        ans[u] = sum;
        if (!opt) {
            calc(u, fa, -1);
            sum = 0;
            mx = 0;
        }
    }
}
void work(int rt = 1) {
    dfs1(rt, 0);
    dfs2(rt, 0, 0);
}
};
```

05 - 树上路径交.cpp

38 lines · C++ · 02 - 树上问题

```

/*
用途：求两条树上简单路径的公共点数。

依赖：
- askLCA(u, v)：返回最近公共祖先。
- askDis(u, v)：返回 u 到 v 的边数距离。

接口：ask(a, b, c, d) 返回路径 a-b 与 c-d 的公共点个数，
不相交时返回 0。时间复杂度  $O(\log n)$ ，常数很小。
*/
bool ask0n(int a, int b, int x) {
    return askDis(a, x) + askDis(x, b) == askDis(a, b);
}

int ask(int a, int b, int c, int d) {
    vector<int> pnt = {a, b, c, d};
    for (int i = 0; i < 4; ++i) {
        for (int j = i + 1; j < 4; ++j) {
            pnt.push_back(askLCA(pnt[i], pnt[j]));
        }
    }
    sort(pnt.begin(), pnt.end());
    pnt.erase(unique(pnt.begin(), pnt.end()), pnt.end());

    vector<int> g;
    for (int x : pnt) {
        if (ask0n(a, b, x) && ask0n(c, d, x)) g.push_back(x);
    }
    if (g.empty()) return 0;

    int ans = 1;
    for (int x : g) {
        for (int y : g) {
            ans = max(ans, askDis(x, y) + 1);
        }
    }
    return ans;
}

```

06 - 树链剖分（路径与子树操作）.cpp

98 lines · C++ · 02 - 树上问题

```

/*
用途：树链剖分，把路径/子树转为 DFS 序区间。
约定：节点、DFS 序和重儿子数组使用 int；区间回调仍可接收默认 int。
性能：默认使用宏展开后的 int；若大规模数据因此超时或超空间，可将纯下标、状态或计数局部改为 i32。
*/
struct HLD {

    int n, tm = 0;
    vector<vector<int>> g;
    vector<int> fa, dep, sz, son, top, in, out, rid;

    explicit HLD(int n)
        : n(n), g(n + 1), fa(n + 1), dep(n + 1), sz(n + 1), son(n + 1), top(n + 1), in(n + 1),
          out(n + 1), rid(n + 1) {
        assert(n >= 0);
    }

    void add(int u, int v) {
        g[u].push_back(v);
        g[v].push_back(u);
    }

    void dfs1(int u, int f) {
        fa[u] = f;
        sz[u] = 1;
        for (int v : g[u]) {
            if (v == f) continue;
            dep[v] = dep[u] + 1;
            dfs1(v, u);
            sz[u] += sz[v];
            if (sz[v] > sz[son[u]]) son[u] = v;
        }
    }

    void dfs2(int u, int h) {
        top[u] = h;
        in[u] = ++tm;
        rid[tm] = u;
        if (son[u]) dfs2(son[u], h);
        for (int v : g[u]) {
            if (v == fa[u] || v == son[u]) continue;
            dfs2(v, v);
        }
        out[u] = tm;
    }

    void work(int rt = 1) {
        assert(1 <= rt && rt <= n);
        tm = 0;
        fill(fa.begin(), fa.end(), 0);
        fill(dep.begin(), dep.end(), 0);
        fill(sz.begin(), sz.end(), 0);
        fill(son.begin(), son.end(), 0);
        fill(top.begin(), top.end(), 0);
        fill(in.begin(), in.end(), 0);
        fill(out.begin(), out.end(), 0);
        fill(rid.begin(), rid.end(), 0);
        dfs1(rt, 0);
        dfs2(rt, rt);
    }
}

```



```
int askLCA(int u, int v) const {
    while (top[u] != top[v]) {
        if (dep[top[u]] > dep[top[v]]) u = fa[top[u]];
        else v = fa[top[v]];
    }
    return dep[u] < dep[v] ? u : v;
}

int askDis(int u, int v) const {
    int p = askLCA(u, v);
    return dep[u] + dep[v] - 2 * dep[p];
}

pair<int, int> askSub(int u) const {
    return {in[u], out[u]};
}

template <class F> void pathV(int u, int v, F&& f) const {
    while (top[u] != top[v]) {
        if (dep[top[u]] < dep[top[v]]) swap(u, v);
        f(in[top[u]], in[u]);
        u = fa[top[u]];
    }
    if (dep[u] > dep[v]) swap(u, v);
    f(in[u], in[v]);
}

template <class F> void pathE(int u, int v, F&& f) const {
    while (top[u] != top[v]) {
        if (dep[top[u]] < dep[top[v]]) swap(u, v);
        f(in[top[u]], in[u]);
        u = fa[top[u]];
    }
    if (dep[u] > dep[v]) swap(u, v);
    if (u != v) f(in[u] + 1, in[v]);
}

};
```

07 - 树上差分.cpp

115 lines · C++ · 02 - 树上问题

/*

用途：多次树上路径加法后的离线汇总，支持“加到路径点”与“加到路径边”。

接口：

- add(u, v)：加入无向边；work(root = 1) 预处理 LCA 和自顶向下 order。
- updateV(u, v, x)：给路径上所有点加 x。
- updateE(u, v, x)：给路径上所有边加 x。
- askV()：返回每个点的最终值。
- askE()：返回每个点到父亲边的最终值，root 位置恒为 0。

T 需要支持默认构造、+= 和 -=，通常直接使用 TreeDiff<int>。

所有路径更新为 $O(\log n)$ ，两种 collect 均为 $O(n)$ ，且不会清空已记录的更新。

性能：倍增祖先表默认使用 int；极限内存不足时可只将祖先编号改为 i32。

*/

```
template <class T = int>
struct TreeDiff {
    int n, lg;
    vector<vector<int>> e, up;
    vector<int> fa, dep, ord;
    vector<T> tv, te;
    int rt0 = 1;

    explicit TreeDiff(int n)
        : n(n), e(n + 1), fa(n + 1), dep(n + 1), tv(n + 1), te(n + 1) {
        lg = 1;
        while ((1LL <= lg) <= max<int>(1, n)) ++lg;
        up.assign(lg, vector<int>(n + 1));
    }

    void add(int u, int v) {
        e[u].push_back(v);
        e[v].push_back(u);
    }

    void dfs(int u, int _fa) {
        fa[u] = _fa;
        up[0][u] = _fa;
        for (int bit = 1; bit < lg; ++bit) up[bit][u] = up[bit - 1][up[bit - 1][u]];
        ord.push_back(u);
        for (int v : e[u]) {
            if (v == _fa) continue;
            dep[v] = dep[u] + 1;
            dfs(v, u);
        }
    }

    void work(int rt = 1) {
        rt0 = rt;
        fill(fa.begin(), fa.end(), 0);
        fill(dep.begin(), dep.end(), 0);
        for (auto &row : up) fill(row.begin(), row.end(), 0);
        ord.clear();
        dfs(rt0, 0);
    }

    int askJmp(int u, int k) const {
        if (k < 0 || k > dep[u]) return 0;
        for (int bit = 0; bit < lg; ++bit) {
            if ((k >> bit) & 1) u = up[bit][u];
        }
    }
};
```

```
    }
    return u;
}

int askLCA(int u, int v) const {
    if (dep[u] < dep[v]) swap(u, v);
    u = askJmp(u, dep[u] - dep[v]);
    if (u == v) return u;
    for (int bit = lg - 1; bit >= 0; --bit) {
        if (up[bit][u] == up[bit][v]) continue;
        u = up[bit][u];
        v = up[bit][v];
    }
    return fa[u];
}

void clear() {
    fill(tv.begin(), tv.end(), T{});
    fill(te.begin(), te.end(), T{});
}

void updateV(int u, int v, const T &d) {
    int p = askLCA(u, v);
    tv[u] += d;
    tv[v] += d;
    tv[p] -= d;
    if (fa[p]) tv[fa[p]] -= d;
}

void updateE(int u, int v, const T &d) {
    int p = askLCA(u, v);
    te[u] += d;
    te[v] += d;
    te[p] -= d;
    te[p] -= d;
}

vector<T> collect(vector<T> val) const {
    for (int i = ord.size(); i--;) {
        int u = ord[i];
        if (fa[u]) val[fa[u]] += val[u];
    }
    return val;
}

vector<T> askV() const {
    return collect(tv);
}

vector<T> askE() const {
    vector<T> res = collect(te);
    res[rt0] = T{};
    return res;
}
};
```

08 - 换根 DP (全树 DP) .cpp

89 lines · C++ · 02 - 树上问题

/*

用途：通用换根 DP (Rerooting)，在一次 $O(n)$ 遍历中计算每个点作为根时的全树答案。

DP 约定：

- `base[u]` 是节点 `u` 自身的初始贡献；
- `merge(a, b)` 合并同一节点收到的两份贡献，必须满足结合律，`identity` 为其单位元；
- `transfer(value, from, to, edgeId)` 将以 `from` 为根的一侧贡献跨边传给 `to`。

例：全树距离和可令 `T = pair<int, int> = {点数, 距离和}`，

`base[u] = {1, 0}`，`merge` 为逐项相加，`transfer({cnt, sum}) = {cnt, sum + cnt * w[edgeId]}`。

接口：

- `Rerooting(n, identity, merge, transfer)`，`add(u, v)` 返回边编号。
- 设置 `base[u]` 后调用 `work(root = 1)`，答案写入 `answer[u]`。

*/

```
template <class T>
struct Rerooting {
    struct Edge {
        int to, id;
    };
    using Merge = function<T(const T &, const T &)>;
    using Transfer = function<T(const T &, int, int, int)>;

    int n, m = 0;
    T id;
    Merge mg;
    Transfer trn;
    vector<vector<Edge>> e;
    vector<int> fa, pe, ord;
    vector<T> bas, dn, up, ans;

    Rerooting(int n, T id, Merge mg, Transfer trn)
        : n(n), id(id), mg(move(mg)), trn(move(trn)), e(n + 1),
          fa(n + 1), pe(n + 1), bas(n + 1, id), dn(n + 1, id),
          up(n + 1, id), ans(n + 1, id) {}

    int add(int u, int v) {
        int id = m++;
        e[u].push_back({v, id});
        e[v].push_back({u, id});
        return id;
    }

    void work(int rt = 1) {
        fill(fa.begin(), fa.end(), 0);
        fill(pe.begin(), pe.end(), -1);
        ord.clear();
        ord.push_back(rt);
        for (int i = 0; i < ord.size(); ++i) {
            int u = ord[i];
            for (auto [v, id] : e[u]) {
                if (v == fa[u]) continue;
                fa[v] = u;
                pe[v] = id;
                ord.push_back(v);
            }
        }

        fill(dn.begin(), dn.end(), id);
        for (int i = ord.size(); i--;) {
```

```
        int u = ord[i];
        dn[u] = bas[u];
        for (auto [v, id] : e[u]) {
            if (fa[v] != u) continue;
            dn[u] = mg(dn[u], trn(dn[v], v, u, id));
        }
    }

    fill(up.begin(), up.end(), id);
    for (int u : ord) {
        int deg = e[u].size();
        vector<T> get(deg), pre(deg + 1, id), suf(deg + 1, id);
        for (int i = 0; i < deg; ++i) {
            auto [v, id] = e[u][i];
            get[i] = (v == fa[u]) ? up[u] : trn(dn[v], v, u, id);
            pre[i + 1] = mg(pre[i], get[i]);
        }
        for (int i = deg - 1; i >= 0; --i) suf[i] = mg(get[i], suf[i + 1]);
        ans[u] = mg(bas[u], pre[deg]);

        for (int i = 0; i < deg; ++i) {
            auto [v, id] = e[u][i];
            if (fa[v] != u) continue;
            T oth = mg(bas[u], mg(pre[i], suf[i + 1]));
            up[v] = trn(oth, u, v, id);
        }
    }
};
```

09 - 虚树.cpp

51 lines · C++ · 02 - 树上问题

/*

用途：从若干关键点及其必要 LCA 构造虚树，压缩无关节点。

接口：

- build(points, lca, dfn):
 - * points 可重复；
 - * lca(u, v) 可传入原树的 askLCA；
 - * dfn(u) 返回原树的先序 DFS 编号（祖先必须在子孙之前）。
- vertex：虚树节点（原树编号，按 dfn 排序）。
- edge：有向边 parent → child（均为原树节点编号）。
- root：虚树根；空输入时为 0。

使用 03A 或 03B 的 askLCA + dfn/tin 即可。复杂度 $O(k \log k)$ （含 LCA 查询）。

*/

```
struct VTree {
    int rt = 0;
    vector<int> u;
    vector<pair<int, int>> e;

    template <class Lca, class Dfn>
    void build(vector<int> p, Lca lca, Dfn dfn) {
        u.clear();
        e.clear();
        rt = 0;
        if (p.empty()) return;

        auto cmp = [&](int u, int v) { return dfn(u) < dfn(v); };
        sort(p.begin(), p.end(), cmp);
        p.erase(unique(p.begin(), p.end()), p.end());

        int n0 = p.size();
        for (int i = 1; i < n0; ++i) p.push_back(lca(p[i - 1], p[i]));
        sort(p.begin(), p.end(), cmp);
        p.erase(unique(p.begin(), p.end()), p.end());
        u = p;

        vector<int> st;
        for (int u : u) {
            while (!st.empty() && lca(st.back(), u) != st.back()) st.pop_back();
            if (!st.empty()) e.push_back({st.back(), u});
            st.push_back(u);
        }
        rt = u.front();
    }

    vector<vector<int>> buildG(int n) const {
        vector<vector<int>> g(n + 1);
        for (auto [u, v] : e) g[u].push_back(v);
        return g;
    }
};
```

10 - Kruskal 重构树.cpp

112 lines · C++ · 02 - 树上问题

/*

用途：Kruskal 重构树（升序），把“边权不超过 w 时的连通性”转化为树上的 LCA。

build 后：

- askMin(u, v) 返回 u, v 连通所需的最小阈值，即原图 $u-v$ 路径中“最大边权最小”的值；
- askLCA(u, v) 对应两点首次被 Kruskal 合并时的重构树节点；
- sz[askLCA(u, v)] 是该时刻所在连通块大小；
- askDisc(u, v) 判断原图是否连通。

输入可为非连通图，点编号为 $1..n$ 。边按权值升序处理；相同权值的合并顺序不影响 askMin，但若题目需要“同权边同时合并”的连通块语义，需要将同权边额外缩点后再建树。

性能：倍增祖先表默认使用 int；极限内存不足时可只将祖先编号改为 i32，权值保持 int。

*/

```
struct KRT {

    struct Edge {
        int u, v;
        int w;
    };

    int n, tot = 0, lg = 1;
    vector<int> dsu, bel, sz, fa, dep;
    vector<vector<int>> son, up;
    vector<int> val;
    vector<int> rts;

    explicit KRT(int n) : n(n) {
        reset();
    }

    void reset() {
        int cap = 2 * n + 5;
        tot = n;
        dsu.resize(cap);
        bel.assign(n + 1, 0);
        sz.assign(cap, 0);
        fa.assign(cap, 0);
        dep.assign(cap, 0);
        son.assign(cap, {});
        val.assign(cap, 0);
        rts.clear();
        for (int u = 1; u <= n; ++u) dsu[u] = u, sz[u] = 1;
    }

    int find(int x) {
        return dsu[x] == x ? x : dsu[x] = find(dsu[x]);
    }

    void dfs(int u, int _fa) {
        fa[u] = _fa;
        up[0][u] = _fa;
        for (int bit = 1; bit < lg; ++bit) up[bit][u] = up[bit - 1][up[bit - 1][u]];
        for (int v : son[u]) {
            dep[v] = dep[u] + 1;
            dfs(v, u);
        }
    }

    void build(vector<Edge> es) {
```

```

    reset();
    sort(es.begin(), es.end(), [](const Edge &a, const Edge &b) { return a.w < b.w; });
    for (const auto &[u, v, w] : es) {
        int x = find(u), y = find(v);
        if (x == y) continue;
        int now = ++tot;
        val[now] = w;
        son[now] = {x, y};
        sz[now] = sz[x] + sz[y];
        dsu[x] = dsu[y] = now;
        dsu[now] = now;
    }

    for (int u = 1; u <= n; ++u) bel[u] = find(u);
    rts.assign(bel.begin() + 1, bel.end());
    sort(rts.begin(), rts.end());
    rts.erase(unique(rts.begin(), rts.end()), rts.end());

    lg = 1;
    while ((1LL << lg) <= max<int>(1, tot)) ++lg;
    up.assign(lg, vector<int>(tot + 1));
    fill(fa.begin(), fa.end(), 0);
    fill(dep.begin(), dep.end(), 0);
    for (int r : rts) dfs(r, 0);
}

bool askDisc(int u, int v) const {
    return bel[u] != bel[v];
}

int askLCA(int u, int v) const {
    if (askDisc(u, v)) return 0;
    if (dep[u] < dep[v]) swap(u, v);
    int dif = dep[u] - dep[v];
    for (int bit = 0; bit < lg; ++bit) {
        if ((dif >> bit) & 1) u = up[bit][u];
    }
    if (u == v) return u;
    for (int bit = lg - 1; bit >= 0; --bit) {
        if (up[bit][u] == up[bit][v]) continue;
        u = up[bit][u];
        v = up[bit][v];
    }
    return up[0][u];
}

optional<int> askMin(int u, int v) const {
    if (askDisc(u, v)) return nullopt;
    if (u == v) return 0;
    return val[askLCA(u, v)];
}
};

```


11 - Link-Cut Tree (动态树) .cpp

155 lines · C++ · 02 - 树上问题

/*

用途：Link-Cut Tree，维护动态森林上的路径和与路径最大值。

接口（点编号 1..n）：

- modify(u, v)：修改点权。
- ask(u, v)：判断是否连通。
- link(u, v)：连一条边；若会成环则返回 false。
- cut(u, v)：删除边 u-v；若该边不存在则返回 false。
- askSum(u, v) / askMax(u, v)：返回路径点权和/最大值，调用前保证两点连通。

T 应为支持加法、比较和 numeric_limits 的数值类型，默认 int。该版本维护点权，边权可通过“为每条边新建一个点并把权值放在该点”转换。

*/

```
template <class T = int>
struct LCT {
    int n;
    vector<array<int, 2>> son;
    vector<int> fa;
    vector<char> tag;
    vector<T> val, sum, mx;

    explicit LCT(int n, T v = T{})
        : n(n), son(n + 1), fa(n + 1), tag(n + 1), val(n + 1, v),
          sum(n + 1, v), mx(n + 1, v) {
        sum[0] = T{};
        mx[0] = numeric_limits<T>::lowest();
    }

    bool isRoot(int x) const {
        int p = fa[x];
        return p == 0 || (son[p][0] != x && son[p][1] != x);
    }

    void pull(int x) {
        sum[x] = sum[son[x][0]] + val[x] + sum[son[x][1]];
        mx[x] = max(val[x], max(mx[son[x][0]], mx[son[x][1]]));
    }

    void rev(int x) {
        if (!x) return;
        swap(son[x][0], son[x][1]);
        tag[x] ^= 1;
    }

    void push(int x) {
        if (!x || !tag[x]) return;
        rev(son[x][0]);
        rev(son[x][1]);
        tag[x] = 0;
    }

    void pushAll(int x) {
        static vector<int> pth;
        pth.clear();
        pth.push_back(x);
        for (int u = x; !isRoot(u); u = fa[u]) pth.push_back(fa[u]);
        for (int i = pth.size(); i--;) push(pth[i]);
    }
}
```

```
void rotate(int x) {
    int p = fa[x], g = fa[p];
    int sd = (son[p][1] == x);
    int mid = son[x][sd ^ 1];
    if (!isRoot(p)) son[g][son[g][1] == p] = x;
    fa[x] = g;
    son[x][sd ^ 1] = p;
    fa[p] = x;
    son[p][sd] = mid;
    if (mid) fa[mid] = p;
    pull(p);
    pull(x);
}

void splay(int x) {
    pushAll(x);
    while (!isRoot(x)) {
        int p = fa[x], g = fa[p];
        if (!isRoot(p)) {
            if ((son[p][0] == x) == (son[g][0] == p)) rotate(p);
            else rotate(x);
        }
        rotate(x);
    }
    pull(x);
}

void access(int x) {
    int lst = 0;
    for (int u = x; u; u = fa[u]) {
        splay(u);
        son[u][1] = lst;
        pull(u);
        lst = u;
    }
    splay(x);
}

void mkRoot(int x) {
    access(x);
    rev(x);
}

int root(int x) {
    access(x);
    while (son[x][0]) {
        push(x);
        x = son[x][0];
    }
    splay(x);
    return x;
}

bool ask(int u, int v) {
    return u == v || root(u) == root(v);
}

bool link(int u, int v) {
    mkRoot(u);
    if (root(v) == u) return false;
    fa[u] = v;
    return true;
}
```

```
bool cut(int u, int v) {
    mkRoot(u);
    access(v);
    if (son[v][0] != u || son[u][1] != 0) return false;
    son[v][0] = 0;
    fa[u] = 0;
    pull(v);
    return true;
}

void split(int u, int v) {
    mkRoot(u);
    access(v);
}

void modify(int u, T v) {
    access(u);
    val[u] = v;
    pull(u);
}

T askSum(int u, int v) {
    split(u, v);
    return sum[v];
}

T askMax(int u, int v) {
    split(u, v);
    return mx[v];
}
};
```

12 - 树同构 (AHU) .cpp

66 lines · C++ · 02 - 树上问题

/*

用途：无根树同构判定 (AHU canonical form, 无随机哈希碰撞)。

输入邻接表必须为 1-indexed: $g.size() == n + 1$, $g[1..n]$ 构成一棵树。

$ask(g, h)$ 返回两棵无标号树是否同构, 复杂度 $O(n \log n)$ (子树类型排序与 map)。

若题目要求保留点权/颜色, 可把它们加入 $rootId$ 的 key 中。

*/

```
struct TreeIso {
    static vector<int> centers(const vector<vector<int>> &g) {
        int n = g.size() - 1;
        if (n == 1) return {1};

        vector<int> deg(n + 1);
        vector<char> ban(n + 1);
        queue<int> q;
        for (int u = 1; u <= n; ++u) {
            deg[u] = g[u].size();
            if (deg[u] <= 1) q.push(u);
        }

        int rem = n;
        while (rem > 2) {
            int cnt = q.size();
            rem -= cnt;
            while (cnt-- > 0) {
                int u = q.front();
                q.pop();
                ban[u] = true;
                for (int v : g[u]) {
                    if (!ban[v] && --deg[v] == 1) q.push(v);
                }
            }
        }

        vector<int> res;
        for (int u = 1; u <= n; ++u) {
            if (!ban[u]) res.push_back(u);
        }
        return res;
    }

    static int rootId(const vector<vector<int>> &g, int u, int fa, map<vector<int>, int> &mp) {
        vector<int> key;
        for (int v : g[u]) {
            if (v != fa) key.push_back(rootId(g, v, u, mp));
        }
        sort(all(key));
        auto it = mp.find(key);
        if (it == mp.end()) {
            int id = mp.size() + 1;
            it = mp.emplace(move(key), id).first;
        }
        return it->second;
    }

    static bool ask(const vector<vector<int>> &g, const vector<vector<int>> &h) {
        if (g.size() != h.size()) return false;
        map<vector<int>, int> mp;
        set<int> id;
        for (int rt : centers(g)) id.insert(rootId(g, rt, 0, mp));
    }
};
```

```
        for (int rt : centers(h)) {
            if (id.count(rootId(h, rt, 0, mp))) return true;
        }
        return false;
    }
};
```

13 - Prüfer序列.cpp

57 lines · C++ · 02 - 树上问题

```

/*
用途：有标号树与 Prüfer 序列互转；点编号 1..n。
性质：序列长度  $n-2$ ，点  $v$  的度数等于其出现次数 + 1，因此有标号树共  $n^{n-2}$  棵。
复杂度：优先队列实现  $O(n \log n)$ 。
性能：默认使用宏展开后的 int；若大规模数据因此超时或超空间，可将纯下标、状态或计数局部改为 i32。
*/
struct PruferCode {

    static vector<int> encode(const vector<vector<int>>& g) {
        int n = g.size() - 1;
        if (n <= 2) return {};
        vector<int> d(n + 1);
        priority_queue<int, vector<int>, greater<int>> q;
        for (int i = 1; i <= n; ++i) {
            d[i] = g[i].size();
            if (d[i] == 1) q.push(i);
        }
        vector<int> p;
        p.reserve(n - 2);
        for (int k = 0; k < n - 2; ++k) {
            int u = q.top();
            q.pop();
            int v = 0;
            for (int x : g[u]) if (d[x]) {
                v = x;
                break;
            }
            assert(v);
            p.push_back(v);
            d[u] = 0;
            if (--d[v] == 1) q.push(v);
        }
        return p;
    }

    static vector<array<int, 2>> decode(int n, const vector<int>& p) {
        assert(n >= 1 && p.size() == max<int>(0, n - 2));
        if (n == 1) return {};
        vector<int> d(n + 1, 1);
        for (int x : p) assert(1 <= x && x <= n), ++d[x];
        priority_queue<int, vector<int>, greater<int>> q;
        for (int i = 1; i <= n; ++i) if (d[i] == 1) q.push(i);
        vector<array<int, 2>> e;
        e.reserve(n - 1);
        for (int v : p) {
            int u = q.top();
            q.pop();
            e.push_back({u, v});
            --d[u];
            if (--d[v] == 1) q.push(v);
        }
        int u = q.top(); q.pop();
        int v = q.top();
        e.push_back({u, v});
        return e;
    }
};

```

01A - 并查集 (DSU) .cpp

58 lines · C++ · 03 - 数据结构

```

/*
用途：普通并查集，维护连通块大小、数量和可选的连通块信息。

约定：点编号 1..n。T 需要支持默认构造和 +=，合并时 v[根] += v[子根]。
接口：find、merge、ask、askSz、operator[]、askCnt。
时间复杂度：均摊  $O(\alpha(n))$ 。
*/
template <class T = int>
struct DSU {
    vector<int> fa, sz;
    vector<T> v;
    int cc = 0;

    DSU() = default;
    explicit DSU(int n, const T &x = T()) {
        init(n, x);
    }

    void init(int n, const T &x = T()) {
        fa.resize(n + 1);
        iota(all(fa), 0);
        sz.assign(n + 1, 1);
        v.assign(n + 1, x);
        cc = n;
    }

    int find(int x) {
        return fa[x] == x ? x : fa[x] = find(fa[x]);
    }

    bool merge(int x, int y) {
        x = find(x);
        y = find(y);
        if (x == y) return false;
        if (sz[x] < sz[y]) swap(x, y);
        fa[y] = x;
        sz[x] += sz[y];
        v[x] += v[y];
        --cc;
        return true;
    }

    bool ask(int x, int y) {
        return find(x) == find(y);
    }

    int askSz(int x) {
        return sz[find(x)];
    }

    T &operator[](int x) {
        return v[find(x)];
    }

    int askCnt() const {
        return cc;
    }
};

```

01B - 可撤销并查集 (DSU) .cpp

72 lines · C++ · 03 - 数据结构

/*

用途：可撤销并查集。适合线段树分治、离线动态连通性等场景。

注意：find 不做路径压缩，否则无法回滚；按大小合并保证单次 find 为 $O(\log n)$ 。

接口：time、merge、undo、ask、askSz、askCnt；undo(t) 回到时刻 t。

*/

```
struct DSU {
    struct Ch {
        int x, y, sz;
    };

    vector<int> fa, sz;
    vector<Ch> his;
    int cc = 0;

    DSU() = default;
    explicit DSU(int n) {
        init(n);
    }

    void init(int n) {
        fa.resize(n + 1);
        iota(all(fa), 0);
        sz.assign(n + 1, 1);
        his.clear();
        cc = n;
    }

    int find(int x) const {
        while (fa[x] != x) x = fa[x];
        return x;
    }

    bool merge(int x, int y) {
        x = find(x);
        y = find(y);
        if (x == y) return false;
        if (sz[x] < sz[y]) swap(x, y);
        his.push_back({x, y, sz[x]});
        fa[y] = x;
        sz[x] += sz[y];
        --cc;
        return true;
    }

    int time() const {
        return his.size();
    }

    void undo(int t) {
        assert(0 <= t && t <= his.size());
        while (his.size() > t) {
            auto [x, y, s] = his.back();
            his.pop_back();
            fa[y] = y;
            sz[x] = s;
            ++cc;
        }
    }
}
```



```
bool ask(int x, int y) const {  
    return find(x) == find(y);  
}  
  
int askSz(int x) const {  
    return sz[find(x)];  
}  
  
int askCnt() const {  
    return cc;  
}  
};
```

01C - 可删除并查集 (DSU) .cpp

70 lines · C++ · 03 - 数据结构

/*

用途：可删除并查集（“替身节点”技巧）。

erase(x) 把原编号 x 从当前集合移出，并让 x 重新成为一个大小为 1 的单点集合；
此前的内部节点仍保留在旧集合中，但不再代表任何当前编号。适合“合并集合 + 删除元素”。

接口：merge(x,y)、erase(x)、ask(x,y)、askSz(x)。点编号为 1..n。

*/

```
struct DSU {
    vector<int> fa, sz, id;
    int tot = 0;

    DSU() = default;
    explicit DSU(int n) {
        init(n);
    }

    void init(int n) {
        tot = n;
        fa.resize(n + 1);
        iota(all(fa), 0);
        sz.assign(n + 1, 1);
        id.resize(n + 1);
        for (int i = 1; i <= n; ++i) id[i] = i;
    }

    int find(int x) {
        return fa[x] == x ? x : fa[x] = find(fa[x]);
    }

    int node() {
        ++tot;
        fa.push_back(tot);
        sz.push_back(1);
        return tot;
    }

    bool merge(int x, int y) {
        int a = find(id[x]);
        int b = find(id[y]);
        if (a == b) return false;
        if (sz[a] < sz[b]) swap(a, b);
        fa[b] = a;
        sz[a] += sz[b];
        return true;
    }

    bool link(int a, int b) {
        a = find(a);
        b = find(b);
        if (a == b) return false;
        if (sz[a] < sz[b]) swap(a, b);
        fa[b] = a;
        sz[a] += sz[b];
        return true;
    }

    void erase(int x) {
        --sz[find(id[x])];
        id[x] = node();
    }
};
```

```
    }

    bool ask(int x, int y) {
        return find(id[x]) == find(id[y]);
    }

    int askSz(int x) {
        return sz[find(id[x])];
    }
};
```

01D - 权值并查集 (DSU, with Int) .cpp

59 lines · C++ · 03 - 数据结构

```

/*
用途：带势能（权值）并查集，维护形如  $\text{potential}[x] - \text{potential}[y] = \text{diff}$  的约束。

 $w[x]$  表示  $\text{potential}[x] - \text{potential}[\text{fa}[x]]$ ；ask(x) 返回  $x$  相对所在根的势能。
merge(x,y,d)：加入  $\text{potential}[x] - \text{potential}[y] = d$ ，
若两点已连通则返回该约束是否与已有约束一致。

T 通常为 int 或 MInt，需支持 +、-、== 和默认构造。
*/
template <class T>
struct DSU {
    vector<int> fa, sz;
    vector<T> w;

    DSU() = default;
    explicit DSU(int n) {
        init(n);
    }

    void init(int n) {
        fa.resize(n + 1);
        iota(all(fa), 0);
        sz.assign(n + 1, 1);
        w.assign(n + 1, T{});
    }

    int find(int x) {
        if (fa[x] == x) return x;
        int p = fa[x];
        int r = find(p);
        w[x] = w[x] + w[p];
        return fa[x] = r;
    }

    T ask(int x) {
        find(x);
        return w[x];
    }

    bool merge(int x, int y, const T &d) {
        int fx = find(x), fy = find(y);
        if (fx == fy) return w[x] - w[y] == d;

        if (sz[fx] < sz[fy]) {
            fa[fx] = fy;
            w[fx] = d - w[x] + w[y];
            sz[fy] += sz[fx];
        } else {
            fa[fy] = fx;
            w[fy] = w[x] - w[y] - d;
            sz[fx] += sz[fy];
        }
        return true;
    }

    bool ask(int x, int y) {
        return find(x) == find(y);
    }
};

```

01E - 异或并查集 (XorDSU) .cpp

44 lines · C++ · 03 - 数据结构

```
/*
用途：维护形如  $\text{value}[x] \text{ xor } \text{value}[y] = d$  的二元异或约束。
约定：点编号  $0..n-1$ ， $d$  为 0 或 1； $w[x]$  表示  $\text{value}[x] \text{ xor } \text{value}[\text{fa}[x]]$ 。
接口：find、merge、ask；merge 返回新约束是否与已有约束相容。
复杂度：均摊  $O(\alpha(n))$ 。
*/
struct XorDSU {
    vector<int> fa, sz, w;

    XorDSU() = default;
    explicit XorDSU(int n) {
        init(n);
    }

    void init(int n) {
        fa.resize(n);
        iota(fa.begin(), fa.end(), 0);
        sz.assign(n, 1);
        w.assign(n, 0);
    }

    int find(int x) {
        if (fa[x] == x) return x;
        int p = fa[x];
        fa[x] = find(p);
        w[x] ^= w[p];
        return fa[x];
    }

    int ask(int x) {
        find(x);
        return w[x];
    }

    bool merge(int x, int y, int d) {
        int fx = find(x), fy = find(y);
        if (fx == fy) return (w[x] ^ w[y]) == d;
        if (sz[fx] < sz[fy]) swap(fx, fy);
        fa[fy] = fx;
        w[fy] = w[x] ^ w[y] ^ d;
        sz[fx] += sz[fy];
        return true;
    }
};
```

02A - 线段树 (SegTree, with Int) .cpp

151 lines · C++ · 03 - 数据结构

/*

用途：区间加、区间和、单点赋值的懒标记线段树。

约定：初始数组和所有区间均为 1-indexed 闭区间。

接口：build(a)、update(l, r, Tag<T>{delta})、modify(pos, value)、ask(l, r).val。

T 需要支持 +、*、-= 等常规数值运算，通常使用 int。

*/

```
template <class T>
```

```
struct Tag {
```

```
    T v = T{};
```

```
    Tag &operator+=(const Tag &o) {
```

```
        v += o.v;
```

```
        return *this;
```

```
    }
```

```
    bool has() const {
```

```
        return v != T{};
```

```
    }
```

```
};
```

```
template <class T>
```

```
struct Info {
```

```
    T val = T{};
```

```
    int l = 1, r = 0; // 空信息满足 l > r。
```

```
    bool askEmp() const {
```

```
        return l > r;
```

```
    }
```

```
    bool hasSon() const {
```

```
        return l < r;
```

```
    }
```

```
    Info operator+(const Info &o) const {
```

```
        if (askEmp()) return o;
```

```
        if (o.askEmp()) return *this;
```

```
        return {val + o.val, l, o.r};
```

```
    }
```

```
    void operator+=(const Tag<T> &tag) {
```

```
        if (!askEmp()) val += tag.v * (r - l + 1);
```

```
    }
```

```
};
```

```
template <class T>
```

```
class SegTree {
```

```
private:
```

```
    int n;
```

```
    vector<Info<T>> tr;
```

```
    vector<Tag<T>> tag;
```

```
    static int ls(int x) {
```

```
        return x << 1;
```

```
    }
```

```
    static int rs(int x) {
```

```
        return x << 1 | 1;
```

```
    }
```

```
    void apply(int x, const Tag<T> &v) {
```

```

        tr[x] += v;
        tag[x] += v;
    }

    void pull(int x) {
        tr[x] = tr[ls(x)] + tr[rs(x)];
    }

    void push(int x) {
        if (!tr[x].hasSon() || !tag[x].has()) return;
        apply(ls(x), tag[x]);
        apply(rs(x), tag[x]);
        tag[x] = Tag<T>{};
    }

    template <class A>
    void build(int x, int l, int r, const A &a) {
        tr[x].l = l;
        tr[x].r = r;
        tag[x] = Tag<T>{};
        if (l == r) {
            tr[x].val = a[l];
            return;
        }
        int mid = (l + r) >> 1;
        build(ls(x), l, mid, a);
        build(rs(x), mid + 1, r, a);
        pull(x);
    }

    void update(int x, int l, int r, int ql, int qr, const Tag<T> &v) {
        if (qr < l || r < ql) return;
        if (ql <= l && r <= qr) {
            apply(x, v);
            return;
        }
        push(x);
        int mid = (l + r) >> 1;
        update(ls(x), l, mid, ql, qr, v);
        update(rs(x), mid + 1, r, ql, qr, v);
        pull(x);
    }

    void modify(int x, int l, int r, int pos, const T &v) {
        if (l == r) {
            tr[x].val = v;
            tag[x] = Tag<T>{};
            return;
        }
        push(x);
        int mid = (l + r) >> 1;
        if (pos <= mid) modify(ls(x), l, mid, pos, v);
        else modify(rs(x), mid + 1, r, pos, v);
        pull(x);
    }

    Info<T> ask(int x, int l, int r, int ql, int qr) {
        if (qr < l || r < ql) return {};
        if (ql <= l && r <= qr) return tr[x];
        push(x);
        int mid = (l + r) >> 1;
        return ask(ls(x), l, mid, ql, qr) + ask(rs(x), mid + 1, r, ql, qr);
    }

```

```
public:
    explicit SegTree(int n) : n(n), tr(4 * n + 5), tag(4 * n + 5) {
        assert(n > 0);
    }

    template <class A>
    void build(const A &a) {
        build(1, 1, n, a);
    }

    void update(int l, int r, const Tag<T> &v) {
        if (l > r) return;
        assert(1 <= l && r <= n);
        update(1, 1, n, l, r, v);
    }

    void modify(int pos, const T &v) {
        assert(1 <= pos && pos <= n);
        modify(1, 1, n, pos, v);
    }

    Info<T> ask(int l, int r) {
        if (l > r) return {};
        assert(1 <= l && r <= n);
        return ask(1, 1, n, l, r);
    }
};
```


02B - 最值线段树 (SegTree, with Int) .cpp

152 lines · C++ · 03 - 数据结构

/*

用途：区间加、区间最小值/最大值、单点赋值的懒标记线段树。

约定：初始数组和所有区间均为 1-indexed 闭区间。

接口：build(a)、update(l, r, Tag<T>{delta})、modify(pos, value)、ask(l, r)。

查询结果的 mx / mn 分别为区间最大值 / 最小值。

*/

```
template <class T>
struct Tag {
    T v = T{};

    Tag &operator+=(const Tag &o) {
        v += o.v;
        return *this;
    }

    bool has() const {
        return v != T{};
    }
};

template <class T>
struct Info {
    T mx = numeric_limits<T>::lowest();
    T mn = numeric_limits<T>::max();
    int l = 1, r = 0;

    bool askEmp() const {
        return l > r;
    }

    bool hasSon() const {
        return l < r;
    }

    Info operator+(const Info &o) const {
        if (askEmp()) return o;
        if (o.askEmp()) return *this;
        return {max(mx, o.mx), min(mn, o.mn), l, o.r};
    }

    void operator+=(const Tag<T> &tag){
        if (!askEmp()) mx += tag.v, mn += tag.v;
    }
};
```

```
template <class T>
class SegTree {
private:
    int n;
    vector<Info<T>> val;
    vector<Tag<T>> tag;

    static int ls(int x) {
        return x << 1;
    }

    static int rs(int x) {
        return x << 1 | 1;
    }
};
```

```

void apply(int x, const Tag<T> &val) {
    val[x] += val;
    tag[x] += val;
}

void pull(int x) {
    val[x] = val[ls(x)] + val[rs(x)];
}

void push(int x) {
    if (!val[x].hasSon() || !tag[x].has()) return;
    apply(ls(x), tag[x]);
    apply(rs(x), tag[x]);
    tag[x] = Tag<T>{};
}

template <class A>
void build(int x, int l, int r, const A &a) {
    val[x].l = l;
    val[x].r = r;
    tag[x] = Tag<T>{};
    if (l == r) {
        val[x].mx = val[x].mn = a[l];
        return;
    }
    int mid = (l + r) >> 1;
    build(ls(x), l, mid, a);
    build(rs(x), mid + 1, r, a);
    pull(x);
}

void update(int x, int l, int r, int ql, int qr, const Tag<T> &val) {
    if (qr < l || r < ql) return;
    if (ql <= l && r <= qr) {
        apply(x, val);
        return;
    }
    push(x);
    int mid = (l + r) >> 1;
    update(ls(x), l, mid, ql, qr, val);
    update(rs(x), mid + 1, r, ql, qr, val);
    pull(x);
}

void modify(int x, int l, int r, int pos, const T &val) {
    if (l == r) {
        val[x].mx = val[x].mn = val;
        tag[x] = Tag<T>{};
        return;
    }
    push(x);
    int mid = (l + r) >> 1;
    if (pos <= mid) modify(ls(x), l, mid, pos, val);
    else modify(rs(x), mid + 1, r, pos, val);
    pull(x);
}

Info<T> ask(int x, int l, int r, int ql, int qr) {
    if (qr < l || r < ql) return {};
    if (ql <= l && r <= qr) return val[x];
    push(x);
    int mid = (l + r) >> 1;
    return ask(ls(x), l, mid, ql, qr) + ask(rs(x), mid + 1, r, ql, qr);
}

```

```
public:
    explicit SegTree(int n) : n(n), val(4 * n + 5), tag(4 * n + 5) {
        assert(n > 0);
    }

    template <class A>
    void build(const A &a) {
        build(1, 1, n, a);
    }

    void update(int l, int r, const Tag<T> &val) {
        if (l > r) return;
        assert(1 <= l && r <= n);
        update(1, 1, n, l, r, val);
    }

    void modify(int pos, const T &val) {
        assert(1 <= pos && pos <= n);
        modify(1, 1, n, pos, val);
    }

    Info<T> ask(int l, int r) {
        if (l > r) return {};
        assert(1 <= l && r <= n);
        return ask(1, 1, n, l, r);
    }
};
```

02C - 可持久化线段树 (SegTree) .cpp

110 lines · C++ · 03 - 数据结构

```

/*
用途：可持久化“计数 + 权值和”线段树（主席树）。

值域为半开区间 [L, R)，版本 0 为空树。
- modify(ver,p,v)：从旧版本克隆一个新版本，位置 p 的计数 +1、权值和 +v。
- ask(ver,l,r)：查询该版本在 [l,r) 的计数和权值和。
- askKth(x,y,k)：版本差 y-x 中第 k 小的位置 (k 从 1 开始)，非法返回 -1。
*/

struct SegTree {
    struct Info {
        int cnt = 0;
        int sum = 0;

        Info operator+(const Info &o) const {
            return {cnt + o.cnt, sum + o.sum};
        }
    };

    struct Node {
        int ch[2]{};
        Info val;
    };

    int lo = 0, hi = 0;
    vector<Node> tr;
    vector<int> rt;

    SegTree() = default;
    SegTree(int l, int r) {
        init(l, r);
    }

    void init(int l, int r) {
        assert(l < r);
        lo = l;
        hi = r;
        tr.assign(1, Node{});
        rt.assign(1, 0);
    }

    int clone(int p) {
        tr.push_back(tr[p]);
        return tr.size() - 1;
    }

    int modify0(int u, int l, int r, int p, int v) {
        int x = clone(u);
        ++tr[x].val.cnt;
        tr[x].val.sum += v;
        if (r - l == 1) return x;
        int mid = l + (r - l) / 2;
        int o = p >= mid;
        if (o) tr[x].ch[1] = modify0(tr[u].ch[1], mid, r, p, v);
        else tr[x].ch[0] = modify0(tr[u].ch[0], l, mid, p, v);
        return x;
    }

    Info ask0(int u, int l, int r, int ql, int qr) const {

```

```

        if (!u || qr <= l || r <= ql) return {};
        if (ql <= l && r <= qr) return tr[u].val;
        int mid = l + (r - l) / 2;
        return ask0(tr[u].ch[0], l, mid, ql, qr) + ask0(tr[u].ch[1], mid, r, ql, qr);
    }

    int askKth0(int x, int y, int l, int r, int k) const {
        int tot = tr[y].val.cnt - tr[x].val.cnt;
        if (k <= 0 || k > tot) return -1;
        while (r - l > 1) {
            int cnt = tr[tr[y].ch[0]].val.cnt - tr[tr[x].ch[0]].val.cnt;
            int mid = l + (r - l) / 2;
            if (k <= cnt) {
                x = tr[x].ch[0];
                y = tr[y].ch[0];
                r = mid;
            } else {
                k -= cnt;
                x = tr[x].ch[1];
                y = tr[y].ch[1];
                l = mid;
            }
        }
        return l;
    }

    int modify(int ver, int p, int v) {
        assert(0 <= ver && ver < rt.size());
        assert(lo <= p && p < hi);
        rt.push_back(modify0(rt[ver], lo, hi, p, v));
        return rt.size() - 1;
    }

    Info ask(int ver, int l, int r) const {
        assert(0 <= ver && ver < rt.size());
        l = max(l, lo);
        r = min(r, hi);
        return l >= r ? Info{} : ask0(rt[ver], lo, hi, l, r);
    }

    int askKth(int x, int y, int k) const {
        assert(0 <= x && x < rt.size());
        assert(0 <= y && y < rt.size());
        return askKth0(rt[x], rt[y], lo, hi, k);
    }

    int askCnt() const {
        return rt.size();
    }
};

```

02D - 可合并信息线段树 (MergeSegTree) .cpp

74 lines · C++ · 03 - 数据结构

/*

用途：不带懒标记的通用线段树。

Info 需要默认构造，并定义 Info operator+(const Info&)；数组与区间均为 1-indexed 闭区间。

支持 build、单点 modify、区间 ask 和整段 ask，均为 $O(\log n)$ 。

*/

```
template <class T> class SegTree {
private:
    vector<T> val;
    int n = 0;

    int ls(int x) {
        return x << 1;
    }
    int rs(int x) {
        return x << 1 | 1;
    }

    template <class A> void build(int x, int l, int r, const A &a) {
        if (l == r) {
            val[x] = a[l];
            return;
        }
        int mid = (l + r) >> 1;
        build(ls(x), l, mid, a);
        build(rs(x), mid + 1, r, a);
        val[x] = val[ls(x)] + val[rs(x)];
    }

    void modify(int x, int l, int r, int p, const T &v) {
        if (l == r) {
            val[x] = v;
            return;
        }
        int mid = (l + r) >> 1;
        if (p <= mid) modify(ls(x), l, mid, p, v);
        else modify(rs(x), mid + 1, r, p, v);
        val[x] = val[ls(x)] + val[rs(x)];
    }

    T ask(int x, int l, int r, int ql, int qr) {
        if (ql <= l && r <= qr) return val[x];
        int mid = (l + r) >> 1;
        if (qr <= mid) return ask(ls(x), l, mid, ql, qr);
        if (ql > mid) return ask(rs(x), mid + 1, r, ql, qr);
        return ask(ls(x), l, mid, ql, qr) + ask(rs(x), mid + 1, r, ql, qr);
    }

public:
    SegTree(int _n = 0) {
        if (_n) init(_n);
    }

    void init(int _n) {
        n = _n;
        val.assign(4 * n + 5, T());
    }

    template <class A> void build(const A &a) {
        build(1, 1, n, a);
    }
};
```

```
    }

    void modify(int p, const T &v) {
        modify(1, 1, n, p, v);
    }

    T ask(int l, int r) {
        return ask(1, 1, n, l, r);
    }

    T ask() {
        return val[1];
    }
};
```

02E - 计数求和主席树 (CountSumSegTree) .cpp

103 lines · C++ · 03 - 数据结构

/*

用途：前缀版本主席树，维护值域 $[L, R]$ 上的出现次数和附加权值和。

add(x,v) 追加一个前缀版本；ask(l,r,ql,qv) 查询第 $l..r$ 个插入元素
(l/r 均为 1-indexed 闭区间) 在值域 $[ql, qv]$ 的计数和权值和。

*/

```
struct SegTree {
    struct Info {
        int cnt = 0;
        int sum = 0;
        Info operator+(const Info &o) const {
            return {cnt + o.cnt, sum + o.sum};
        }
    };

    int lo, hi;
    vector<array<int, 2>> ch;
    vector<int> cnt, rt;
    vector<int> sum;

    SegTree() {}
    SegTree(int l, int r) {
        init(l, r);
    }

    void init(int l, int r) {
        lo = l;
        hi = r;
        ch.assign(1, {0, 0});
        cnt.assign(1, 0);
        sum.assign(1, 0);
        rt.assign(1, 0);
    }

    int clone(int p) {
        ch.push_back(ch[p]);
        cnt.push_back(cnt[p]);
        sum.push_back(sum[p]);
        return cnt.size() - 1;
    }

    int modify(int p, int l, int r, int x, int v) {
        int q = clone(p);
        ++cnt[q];
        sum[q] += v;
        if (l < r) {
            int mid = (l + r) >> 1;
            if (x <= mid) {
                ch[q][0] = modify(ch[p][0], l, mid, x, v);
            } else {
                ch[q][1] = modify(ch[p][1], mid + 1, r, x, v);
            }
        }
        return q;
    }

    int modify(int pre, int x, int v) {
        return modify(pre, lo, hi, x, v);
    }
}
```



```
int add(int x) {
    rt.push_back(modify(rt.back(), x, x));
    return rt.back();
}

int add(int x, int v) {
    rt.push_back(modify(rt.back(), x, v));
    return rt.back();
}

Info ask0(int u, int v, int l, int r, int ql, int qr) const {
    if (ql > qr || qr < l || r < ql) {
        return {};
    }
    if (ql <= l && r <= qr) {
        return {cnt[u] - cnt[v], sum[u] - sum[v]};
    }
    int mid = (l + r) >> 1;
    return ask0(ch[u][0], ch[v][0], l, mid, ql, qr) +
        ask0(ch[u][1], ch[v][1], mid + 1, r, ql, qr);
}

Info ask0(int u, int v, int ql, int qr) const {
    ql = max(ql, lo);
    qr = min(qr, hi);
    if (ql > qr) {
        return {};
    }
    return ask0(u, v, lo, hi, ql, qr);
}

Info ask(int l, int r, int ql, int qr) const {
    if (l > r) {
        return {};
    }
    return ask0(rt[r], rt[l - 1], ql, qr);
}

int askCnt() const {
    return rt.size() - 1;
}

};
```

02F - 动态开点线段树.cpp

194 lines · C++ · 03 - 数据结构

/*

用途：按需开点的区间加、区间和、单点赋值线段树。

接口与 02A 一致，区间和数组均为 1-indexed；适用于 n 很大、实际访问稀疏的场景。

注意查询非整段时也可能创建子节点，以保证懒标记语义正确。

*/

```
template <class T> struct Tag {
    T v = 0;

    void operator+=(const Tag<T> &o) {
        v += o.v;
    }

    bool has() const {
        return v != 0;
    }
};

template <class T> struct Info {
    T val = 0;
    int l = 1, r = 0;

    Info operator+(const Info<T> &o) const {
        if (l > r) return o;
        if (o.l > o.r) return *this;
        Info res;
        res.l = l;
        res.r = o.r;
        res.val = val + o.val;
        return res;
    }

    void operator+=(const Tag<T> &o) {
        if (l <= r) val += o.v * (r - l + 1);
    }

    bool hasSon() const {
        return l < r;
    }
};

template <class T> class SegTree {
private:
    struct Node {
        Info<T> in;
        Tag<T> tag;
        int ls = 0, rs = 0;
    };

    vector<Node> tr;
    int n, rt;

    int node(int l, int r) {
        tr.push_back(Node());
        int x = tr.size() - 1;
        tr[x].in.l = l;
        tr[x].in.r = r;
        return x;
    }
};
```

```
void apply(int x, const Tag<T> &v) {
    tr[x].in += v;
    tr[x].tag += v;
}

void push(int x) {
    if (!tr[x].in.hasSon()) return;

    int l = tr[x].in.l, r = tr[x].in.r;
    int mid = l + ((r - l) >> 1);

    if (!tr[x].ls) tr[x].ls = node(l, mid);
    if (!tr[x].rs) tr[x].rs = node(mid + 1, r);

    if (tr[x].tag.has()) {
        apply(tr[x].ls, tr[x].tag);
        apply(tr[x].rs, tr[x].tag);
        tr[x].tag = Tag<T>();
    }
}

void pull(int x) {
    tr[x].in = tr[tr[x].ls].in + tr[tr[x].rs].in;
}

void print(int x) {
    if (!x) return;
    cout << x << ":[" << tr[x].in.l << "," << tr[x].in.r << "],val:" << tr[x].in.val << ",tag:" <<
        tr[x].tag.v << endl;
    print(tr[x].ls);
    print(tr[x].rs);
}

template <class A> void build(int x, A &dat) {
    int l = tr[x].in.l, r = tr[x].in.r;
    tr[x].tag = Tag<T>();

    if (l == r) {
        tr[x].in.val = dat[l];
        return;
    }

    int mid = l + ((r - l) >> 1);
    tr[x].ls = node(l, mid);
    tr[x].rs = node(mid + 1, r);

    build(tr[x].ls, dat);
    build(tr[x].rs, dat);
    pull(x);
}

void update(int x, int lq, int rq, const Tag<T> &v) {
    int l = tr[x].in.l, r = tr[x].in.r;
    if (rq < l || r < lq) return;

    if (lq <= l && r <= rq) {
        apply(x, v);
        return;
    }

    push(x);
    int mid = l + ((r - l) >> 1);

    if (lq <= mid) update(tr[x].ls, lq, rq, v);
```

```

        if (rq > mid) update(tr[x].rs, lq, rq, v);

        pull(x);
    }

    void modify(int x, int pos, T v) {
        int l = tr[x].in.l, r = tr[x].in.r;

        if (l == r) {
            tr[x].in.val = v;
            tr[x].tag = Tag<T>();
            return;
        }

        push(x);
        int mid = l + ((r - l) >> 1);

        if (pos <= mid)
            modify(tr[x].ls, pos, v);
        else
            modify(tr[x].rs, pos, v);

        pull(x);
    }

    Info<T> ask(int x, int lq, int rq) {
        int l = tr[x].in.l, r = tr[x].in.r;
        if (rq < l || r < lq) return Info<T>();

        if (lq <= l && r <= rq) return tr[x].in;

        push(x);
        int mid = l + ((r - l) >> 1);

        if (rq <= mid) return ask(tr[x].ls, lq, rq);
        if (lq > mid) return ask(tr[x].rs, lq, rq);
        return ask(tr[x].ls, lq, rq) + ask(tr[x].rs, lq, rq);
    }

public:
    SegTree(int _n) : n(_n) {
        tr.push_back(Node());
        rt = node(1, n);
    }

    void print() {
        print(rt);
    }

    template <class A> void build(A &dat) {
        tr.clear();
        tr.push_back(Node());
        rt = node(1, n);
        build(rt, dat);
    }

    void update(int l, int r, Tag<T> v) {
        if (l > r) return;
        update(rt, l, r, v);
    }

    void modify(int pos, T v) {
        if (pos < 1 || pos > n) return;
        modify(rt, pos, v);
    }

```

```
    }  
  
    Info<T> ask(int l, int r) {  
        if (l > r) return Info<T>();  
        return ask(rt, l, r);  
    }  
};
```

02G - Segment Tree Beats.cpp

210 lines · C++ · 03 - 数据结构

```

/*
用途：Segment Tree Beats，支持区间 chmin、chmax、加法、和/最小/最大值查询。

约定：初始数组为 1-indexed。接口：build(a)、updMin、updMax、update、
askSum/askMin/askMax（均为闭区间）。均摊复杂度  $O(\log n)$ 。
*/

struct SegTree {
    static constexpr int NI = numeric_limits<int>::lowest() / 4;
    static constexpr int PI = numeric_limits<int>::max() / 4;

    struct Node {
        int sum = 0, mx1 = NI, mx2 = NI, mn1 = PI, mn2 = PI, add = 0;
        int cmx = 0, cmn = 0;
    };

    int n = 0;
    vector<Node> tr;

    SegTree() = default;
    explicit SegTree(int n) {
        init(n);
    }

    void init(int _n) {
        assert(_n > 0);
        n = _n;
        tr.assign(4 * n + 5, Node{});
    }

    static int ls(int p) {
        return p << 1;
    }
    static int rs(int p) {
        return p << 1 | 1;
    }

    void pull(int p) {
        const Node &a = tr[ls(p)], &b = tr[rs(p)];
        Node &x = tr[p];
        x.sum = a.sum + b.sum;

        x.mx1 = max(a.mx1, b.mx1);
        x.cmx = 0;
        x.mx2 = NI;
        for (const Node *son : {&a, &b}) {
            if (son->mx1 == x.mx1) {
                x.cmx += son->cmx;
                x.mx2 = max(x.mx2, son->mx2);
            } else {
                x.mx2 = max(x.mx2, son->mx1);
            }
        }

        x.mn1 = min(a.mn1, b.mn1);
        x.cmn = 0;
        x.mn2 = PI;
        for (const Node *son : {&a, &b}) {
            if (son->mn1 == x.mn1) {
                x.cmn += son->cmn;
            }
        }
    }
};

```

```

        x.mn2 = min(x.mn2, son->mn2);
    } else {
        x.mn2 = min(x.mn2, son->mn1);
    }
}
x.add = 0;
}

void setAdd(int p, int l, int r, int val) {
    Node &x = tr[p];
    x.sum += val * (r - l + 1);
    x.mx1 += val;
    if (x.mx2 != NI) x.mx2 += val;
    x.mn1 += val;
    if (x.mn2 != PI) x.mn2 += val;
    x.add += val;
}

void setMin(int p, int val) {
    Node &x = tr[p];
    assert(x.mx2 < val && val < x.mx1);
    x.sum += (val - x.mx1) * x.cmx;
    if (x.mn1 == x.mx1) x.mn1 = val;
    else if (x.mn2 == x.mx1) x.mn2 = val;
    x.mx1 = val;
}

void setMax(int p, int val) {
    Node &x = tr[p];
    assert(x.mn1 < val && val < x.mn2);
    x.sum += (val - x.mn1) * x.cmn;
    if (x.mx1 == x.mn1) x.mx1 = val;
    else if (x.mx2 == x.mn1) x.mx2 = val;
    x.mn1 = val;
}

void push(int p, int l, int r) {
    if (l == r) return;
    int mid = (l + r) >> 1;
    if (tr[p].add) {
        setAdd(ls(p), l, mid, tr[p].add);
        setAdd(rs(p), mid + 1, r, tr[p].add);
        tr[p].add = 0;
    }
    if (tr[ls(p)].mx1 > tr[p].mx1) setMin(ls(p), tr[p].mx1);
    if (tr[rs(p)].mx1 > tr[p].mx1) setMin(rs(p), tr[p].mx1);
    if (tr[ls(p)].mn1 < tr[p].mn1) setMax(ls(p), tr[p].mn1);
    if (tr[rs(p)].mn1 < tr[p].mn1) setMax(rs(p), tr[p].mn1);
}

template <class A>
void build(int p, int l, int r, const A &a) {
    if (l == r) {
        int val = a[l];
        tr[p] = {val, val, NI, val, PI, 0, 1, 1};
        return;
    }
    int mid = (l + r) >> 1;
    build(ls(p), l, mid, a);
    build(rs(p), mid + 1, r, a);
    pull(p);
}

template <class A>

```

```

void build(const A &a) {
    build(1, 1, n, a);
}

void updMin(int p, int l, int r, int ql, int qr, int val) {
    if (qr < l || r < ql || tr[p].mx1 <= val) return;
    if (ql <= l && r <= qr && tr[p].mx2 < val) {
        setMin(p, val);
        return;
    }
    push(p, l, r);
    int mid = (l + r) >> 1;
    updMin(ls(p), l, mid, ql, qr, val);
    updMin(rs(p), mid + 1, r, ql, qr, val);
    pull(p);
}

void updMax(int p, int l, int r, int ql, int qr, int val) {
    if (qr < l || r < ql || tr[p].mn1 >= val) return;
    if (ql <= l && r <= qr && tr[p].mn2 > val) {
        setMax(p, val);
        return;
    }
    push(p, l, r);
    int mid = (l + r) >> 1;
    updMax(ls(p), l, mid, ql, qr, val);
    updMax(rs(p), mid + 1, r, ql, qr, val);
    pull(p);
}

void update(int p, int l, int r, int ql, int qr, int val) {
    if (qr < l || r < ql) return;
    if (ql <= l && r <= qr) {
        setAdd(p, l, r, val);
        return;
    }
    push(p, l, r);
    int mid = (l + r) >> 1;
    update(ls(p), l, mid, ql, qr, val);
    update(rs(p), mid + 1, r, ql, qr, val);
    pull(p);
}

int askSum(int p, int l, int r, int ql, int qr) {
    if (qr < l || r < ql) return 0;
    if (ql <= l && r <= qr) return tr[p].sum;
    push(p, l, r);
    int mid = (l + r) >> 1;
    return askSum(ls(p), l, mid, ql, qr) + askSum(rs(p), mid + 1, r, ql, qr);
}

int askMin(int p, int l, int r, int ql, int qr) {
    if (qr < l || r < ql) return PI;
    if (ql <= l && r <= qr) return tr[p].mn1;
    push(p, l, r);
    int mid = (l + r) >> 1;
    return min(askMin(ls(p), l, mid, ql, qr), askMin(rs(p), mid + 1, r, ql, qr));
}

int askMax(int p, int l, int r, int ql, int qr) {
    if (qr < l || r < ql) return NI;
    if (ql <= l && r <= qr) return tr[p].mx1;
    push(p, l, r);
    int mid = (l + r) >> 1;

```



```
        return max(askMax(ls(p), l, mid, ql, qr), askMax(rs(p), mid + 1, r, ql, qr));
    }

    void updMin(int l, int r, int val) {
        if (l <= r) updMin(1, 1, n, l, r, val);
    }
    void updMax(int l, int r, int val) {
        if (l <= r) updMax(1, 1, n, l, r, val);
    }
    void update(int l, int r, int val) {
        if (l <= r) update(1, 1, n, l, r, val);
    }
    int askSum(int l, int r) {
        return askSum(1, 1, n, l, r);
    }
    int askMin(int l, int r) {
        return askMin(1, 1, n, l, r);
    }
    int askMax(int l, int r) {
        return askMax(1, 1, n, l, r);
    }
};
```

02H - Li Chao Tree.cpp

88 lines · C++ · 03 - 数据结构

```

/*
用途：动态 Li Chao Tree，维护直线  $y = m \cdot x + b$  的点集，查询指定  $x$  的最小值。

定义域为整数闭区间  $[L, R]$ 。add 添加整条直线，addSeg 添加只在  $[ql, qr]$  有效的线段。
ask(x) 返回最小值；若尚未添加直线，返回 INF。比较过程使用 i128，
但最终返回 int，需保证实际答案不溢出 int。
*/

using i128 = __int128_t;

struct LiChaoTree {
    static constexpr int INF = numeric_limits<int>::max();

    struct Line {
        int m = 0, b = INF;
        i128 eval(int x) const {
            return i128(m) * x + b;
        }
    };

    struct Node {
        Line f;
        int ch[2]{};
    };

    int lo, hi;
    vector<Node> tr;
    int rt = 0;

    LiChaoTree(int l, int r) : lo(l), hi(r), tr(1) {
        assert(lo <= hi);
    }

    int node() {
        tr.push_back(Node{});
        return tr.size() - 1;
    }

    int add0(int p, int l, int r, Line f) {
        if (!p) p = node();
        int mid = l + (r - l) / 2;
        bool bl = f.eval(l) < tr[p].f.eval(l);
        bool bm = f.eval(mid) < tr[p].f.eval(mid);
        if (bm) swap(f, tr[p].f);
        if (l == r) return p;
        if (bl != bm) tr[p].ch[0] = add0(tr[p].ch[0], l, mid, f);
        else tr[p].ch[1] = add0(tr[p].ch[1], mid + 1, r, f);
        return p;
    }

    int add1(int p, int l, int r, int ql, int qr, Line f) {
        if (qr < l || r < ql) return p;
        if (ql <= l && r <= qr) {
            return add0(p, l, r, f);
        }
        if (!p) p = node();
        int mid = l + (r - l) / 2;
        tr[p].ch[0] = add1(tr[p].ch[0], l, mid, ql, qr, f);
        tr[p].ch[1] = add1(tr[p].ch[1], mid + 1, r, ql, qr, f);
        return p;
    }
};

```

```
    }

    i128 ask0(int p, int l, int r, int x) const {
        if (!p) return i128(INF);
        i128 ans = tr[p].f.eval(x);
        if (l == r) return ans;
        int mid = l + (r - l) / 2;
        if (x <= mid) return min(ans, ask0(tr[p].ch[0], l, mid, x));
        return min(ans, ask0(tr[p].ch[1], mid + 1, r, x));
    }

    void add(int m, int b) {
        rt = add0(rt, lo, hi, {m, b});
    }

    void addSeg(int m, int b, int ql, int qr) {
        ql = max(ql, lo);
        qr = min(qr, hi);
        if (ql <= qr) rt = add1(rt, lo, hi, ql, qr, {m, b});
    }

    int ask(int x) const {
        assert(lo <= x && x <= hi);
        i128 ans = ask0(rt, lo, hi, x);
        assert(ans >= numeric_limits<int>::lowest() && ans <= numeric_limits<int>::max());
        return ans;
    }
};
```

02l - 仿射懒标记线段树（自动取模）.cpp

74 lines · C++ · 03 - 数据结构

```

/*
用途：模意义下维护区间和，支持区间乘、加、赋值；依赖自动取模类型 Z。
接口：build(a)、update(l,r,m,a) 表示  $x \leftarrow m * x + a$ ；updMul/updAdd/updSet/ask 均为闭区间。
复杂度：建树  $O(n)$ ，每次操作  $O(\log n)$ 。
*/
template <class Z> struct SegTree {
    int n;
    vector<Z> s, m, a;

    explicit SegTree(int _n) : n(_n), s(4 * _n + 5), m(4 * _n + 5, Z(1)), a(4 * _n + 5) {
        assert(n > 0);
    }

    template <class V> void build(const V& v) {
        assert(v.size() > n);
        build(1, 1, n, v);
    }

    void update(int l, int r, Z x, Z y) {
        if (l > r) return;
        assert(1 <= l && r <= n);
        upd(1, 1, n, l, r, x, y);
    }

    void updMul(int l, int r, Z x) { update(l, r, x, Z(0)); }
    void updAdd(int l, int r, Z x) { update(l, r, Z(1), x); }
    void updSet(int l, int r, Z x) { update(l, r, Z(0), x); }
    Z ask(int l, int r) {
        if (l > r) return Z(0);
        assert(1 <= l && r <= n);
        return ask(1, 1, n, l, r);
    }

private:
    template <class V> void build(int p, int l, int r, const V& v) {
        if (l == r) return void(s[p] = Z(v[l]));
        int q = (l + r) >> 1;
        build(p << 1, l, q, v), build(p << 1 | 1, q + 1, r, v);
        pull(p);
    }

    void apply(int p, int l, int r, Z x, Z y) {
        s[p] = s[p] * x + y * (r - l + 1);
        m[p] *= x;
        a[p] = a[p] * x + y;
    }

    void pull(int p) { s[p] = s[p << 1] + s[p << 1 | 1]; }

    void push(int p, int l, int r) {
        if (l == r) return;
        int q = (l + r) >> 1;
        apply(p << 1, l, q, m[p], a[p]);
        apply(p << 1 | 1, q + 1, r, m[p], a[p]);
        m[p] = Z(1), a[p] = Z(0);
    }

    void upd(int p, int l, int r, int x, int y, Z u, Z v) {
        if (x <= l && r <= y) return apply(p, l, r, u, v);
        push(p, l, r);
        int q = (l + r) >> 1;

```

```
    if (x <= q) upd(p << 1, l, q, x, y, u, v);
    if (y > q) upd(p << 1 | 1, q + 1, r, x, y, u, v);
    pull(p);
}

Z ask(int p, int l, int r, int x, int y) {
    if (x <= l && r <= y) return s[p];
    push(p, l, r);
    int q = (l + r) >> 1;
    if (y <= q) return ask(p << 1, l, q, x, y);
    if (x > q) return ask(p << 1 | 1, q + 1, r, x, y);
    return ask(p << 1, l, q, x, y) + ask(p << 1 | 1, q + 1, r, x, y);
}

};
```

02J - 可合并信息懒标记线段树 (LazyMergeSegTree) .cpp

121 lines · C++ · 03 - 数据结构

/*

用途：通用可合并信息懒标记线段树。

Info 需要默认构造，并定义 Info operator+(const Info&) 与 Info::operator+=(const Tag&);

Tag 需要默认构造为恒等操作，并定义 Tag::operator+=(const Tag&) 和 bool has() const。

tag += o 表示先作用当前 tag，再作用 o。数组与区间均为 1-indexed 闭区间。

支持 build、区间 update、单点 modify、区间 ask 和整段 ask，均为 $O(\log n)$ 。

*/

```
template <class Info, class Tag> class LazySegTree {
private:
    int n = 0;
    vector<Info> val;
    vector<Tag> tag;

    int ls(int x) {
        return x << 1;
    }
    int rs(int x) {
        return x << 1 | 1;
    }

    void apply(int x, const Tag &v) {
        val[x] += v;
        tag[x] += v;
    }

    void pull(int x) {
        val[x] = val[ls(x)] + val[rs(x)];
    }

    void push(int x) {
        if (!tag[x].has()) return;
        apply(ls(x), tag[x]);
        apply(rs(x), tag[x]);
        tag[x] = Tag();
    }

    template <class A> void build(int x, int l, int r, const A &a) {
        tag[x] = Tag();
        if (l == r) {
            val[x] = a[l];
            return;
        }
        int mid = (l + r) >> 1;
        build(ls(x), l, mid, a);
        build(rs(x), mid + 1, r, a);
        pull(x);
    }

    void update(int x, int l, int r, int ql, int qr, const Tag &v) {
        if (ql <= l && r <= qr) {
            apply(x, v);
            return;
        }
        push(x);
        int mid = (l + r) >> 1;
        if (ql <= mid) update(ls(x), l, mid, ql, qr, v);
        if (qr > mid) update(rs(x), mid + 1, r, ql, qr, v);
        pull(x);
    }
}
```

```

void modify(int x, int l, int r, int p, const Info &v) {
    if (l == r) {
        val[x] = v;
        tag[x] = Tag();
        return;
    }
    push(x);
    int mid = (l + r) >> 1;
    if (p <= mid) modify(ls(x), l, mid, p, v);
    else modify(rs(x), mid + 1, r, p, v);
    pull(x);
}

Info ask(int x, int l, int r, int ql, int qr) {
    if (ql <= l && r <= qr) return val[x];
    push(x);
    int mid = (l + r) >> 1;
    if (qr <= mid) return ask(ls(x), l, mid, ql, qr);
    if (ql > mid) return ask(rs(x), mid + 1, r, ql, qr);
    return ask(ls(x), l, mid, ql, qr) + ask(rs(x), mid + 1, r, ql, qr);
}

public:
    explicit LazySegTree(int _n = 0) {
        if (_n) init(_n);
    }

    void init(int _n) {
        assert(_n > 0);
        n = _n;
        val.assign(4 * n + 5, Info());
        tag.assign(4 * n + 5, Tag());
    }

    template <class A> void build(const A &a) {
        assert(n > 0);
        build(1, 1, n, a);
    }

    void update(int l, int r, const Tag &v) {
        if (l > r) return;
        assert(1 <= l && r <= n);
        update(1, 1, n, l, r, v);
    }

    void modify(int p, const Info &v) {
        assert(1 <= p && p <= n);
        modify(1, 1, n, p, v);
    }

    Info ask(int l, int r) {
        if (l > r) return Info();
        assert(1 <= l && r <= n);
        return ask(1, 1, n, l, r);
    }

    Info ask() const {
        return val[1];
    }
};

```

03 - 树状数组 (BIT, with Int) .cpp

73 lines · C++ · 03 - 数据结构

/*

用途：一维树状数组，默认维护单点加、前缀和与区间和。

接口（均为 1-indexed）：modify(p,x)、ask(p)、ask(l,r)、askKth(k)。

update(l,r,x) 是差分用法：区间加、单点查询；不要与区间和语义混用。

askKth(k) 返回最小的 p 使前缀和 $\geq k$ ；要求各点权非负，非法 k 返回 $n + 1$ 。

*/

```
template <class T>
struct BIT {
    int n = 0;
    vector<T> bit;

    BIT() = default;
    explicit BIT(int n) {
        init(n);
    }

    void init(int _n) {
        n = _n;
        bit.assign(n + 1, T{});
    }

    template <class A>
    void build(const A &a) {
        fill(bit.begin(), bit.end(), T{});
        for (int i = 1; i <= n; ++i) {
            bit[i] += a[i];
            int j = i + (i & -i);
            if (j <= n) bit[j] += bit[i];
        }
    }

    void modify(int x, const T &v) {
        assert(1 <= x && x <= n);
        for (; x <= n; x += x & -x) bit[x] += v;
    }

    void update(int l, int r, const T &v) {
        if (l > r) return;
        assert(1 <= l && r <= n);
        modify(l, v);
        if (r < n) modify(r + 1, -v);
    }

    T ask(int x) const {
        x = min(x, n);
        T res{};
        for (; x > 0; x -= x & -x) res += bit[x];
        return res;
    }

    T ask(int l, int r) const {
        if (l > r) return T{};
        assert(1 <= l && r <= n);
        return ask(r) - ask(l - 1);
    }

    int askKth(const T &k) const {
        if (!(k > T{})) return 1;
        int pos = 0;
```



```
    T pre{};
    int stp = 1;
    while ((stp << 1) <= n) stp <<= 1;
    for (; stp; stp >>= 1) {
        int nxt = pos + stp;
        if (nxt <= n && pre + bit[nxt] < k) {
            pre += bit[nxt];
            pos = nxt;
        }
    }
    return pos + 1;
}
};
```

03A - 区间加区间和树状数组.cpp

53 lines · C++ · 03 - 数据结构

```

/*
用途：双树状数组，支持区间加和区间求和。

下标和区间均为 1-indexed 闭区间。
接口：update(l,r,v)、ask(x)、askSum(l,r)。每次  $O(\log n)$ 。
*/
template <class T> struct BIT {
    int n = 0;
    vector<T> b1, b2;

    BIT() = default;
    explicit BIT(int n) {
        init(n);
    }

    void init(int _n) {
        n = _n;
        b1.assign(n + 1, T{});
        b2.assign(n + 1, T{});
    }

    void mod0(vector<T> &bit, int x, const T &v) {
        for (; x <= n; x += x & -x) bit[x] += v;
    }

    T ask0(const vector<T> &bit, int x) const {
        T res{};
        for (; x > 0; x -= x & -x) res += bit[x];
        return res;
    }

    void update(int l, int r, const T &v) {
        if (l > r) return;
        assert(1 <= l && r <= n);
        mod0(b1, l, v);
        mod0(b2, l, v * (l - 1));
        if (r < n) {
            mod0(b1, r + 1, -v);
            mod0(b2, r + 1, -v * r);
        }
    }

    T ask(int x) const {
        x = min(x, n);
        return ask0(b1, x) * x - ask0(b2, x);
    }

    T askSum(int l, int r) const {
        if (l > r) return T{};
        assert(1 <= l && r <= n);
        return ask(r) - ask(l - 1);
    }
};

```

03B - 二维树状数组.cpp

39 lines · C++ · 03 - 数据结构

```

/*
用途：稠密二维树状数组，支持单点加与矩形和。
约定：坐标均为 1-indexed；ask(x1,y1,x2,y2) 为闭矩形。
复杂度：修改/查询  $O(\log n \log m)$ ，空间  $O(nm)$ 。坐标很大且操作稀疏时应离线压缩或改动态结构。
性能：默认使用宏展开后的 int；若大规模数据因此超时或超空间，可将纯下标、状态或计数局部改为 i32。
*/
template <class T> struct BIT {
    int n = 0, m = 0;
    vector<vector<T>> t;

    BIT() = default;
    BIT(int n, int m) { init(n, m); }

    void init(int _n, int _m) {
        assert(_n >= 0 && _m >= 0);
        n = _n, m = _m;
        t.assign(n + 1, vector<T>(m + 1));
    }

    void modify(int x, int y, const T& v) {
        assert(1 <= x && x <= n && 1 <= y && y <= m);
        for (int i = x; i <= n; i += i & -i)
            for (int j = y; j <= m; j += j & -j) t[i][j] += v;
    }

    T ask(int x, int y) const {
        x = min(x, n), y = min(y, m);
        T r{};
        for (int i = x; i > 0; i -= i & -i)
            for (int j = y; j > 0; j -= j & -j) r += t[i][j];
        return r;
    }

    T ask(int x1, int y1, int x2, int y2) const {
        if (x1 > x2 || y1 > y2) return T{};
        assert(1 <= x1 && x2 <= n && 1 <= y1 && y2 <= m);
        return ask(x2, y2) - ask(x1 - 1, y2) - ask(x2, y1 - 1) + ask(x1 - 1, y1 - 1);
    }
};

```

04A - ST表 (ST, with Int) .cpp

58 lines · C++ · 03 - 数据结构

/*

用途：静态区间最值 ST 表，查询最大/最小值及其最左位置。

约定：数组和查询均为 1-indexed 闭区间；askMax/askMin 自动交换 l、r。

仅适用于 max/min/gcd 等幂等操作，区间和请用“04B - 可结合区间查询 (Disjoint Sparse Table)”。

预处理 $O(n \log n)$ ，每次查询 $O(1)$ 。

性能：为返回最左位置，表项额外保存 int 下标；极限内存不足时可只将下标改为 i32。

*/

```
template <class T>
struct ST {
    int n = 0, lg = 0;
    vector<vector<pair<T, int>>> mx, mn;

    ST() = default;
    explicit ST(int n) {
        init(n);
    }

    void init(int _n) {
        assert(_n > 0);
        n = _n;
        lg = __lg(n);
        mx.assign(lg + 1, vector<pair<T, int>>(n + 1));
        mn.assign(lg + 1, vector<pair<T, int>>(n + 1));
    }

    template <class A>
    void build(const A &a) {
        assert(n > 0);
        for (int i = 1; i <= n; ++i) {
            mx[0][i] = {a[i], -i}; // 值相同时优先较小下标。
            mn[0][i] = {a[i], i};
        }
        for (int k = 1; k <= lg; ++k) {
            int len = 1 << k;
            int h = len >> 1;
            for (int l = 1; l + len - 1 <= n; ++l) {
                mx[k][l] = max(mx[k - 1][l], mx[k - 1][l + h]);
                mn[k][l] = min(mn[k - 1][l], mn[k - 1][l + h]);
            }
        }
    }

    pair<T, int> askMax(int l, int r) const {
        if (l > r) swap(l, r);
        assert(1 <= l && r <= n);
        int k = __lg(r - l + 1);
        auto res = max(mx[k][l], mx[k][r - (1 << k) + 1]);
        return {res.first, -res.second};
    }

    pair<T, int> askMin(int l, int r) const {
        if (l > r) swap(l, r);
        assert(1 <= l && r <= n);
        int k = __lg(r - l + 1);
        return min(mn[k][l], mn[k][r - (1 << k) + 1]);
    }
};
```

04B - 可结合区间查询 (Disjoint Sparse Table) .cpp

53 lines · C++ · 03 - 数据结构

/*

用途：Disjoint Sparse Table，支持任意结合操作的静态区间查询，例如区间和、gcd、矩阵乘法。

数组与查询为 0-indexed；ask(l,r) 的 r 为闭端点，要求 $l \leq r$ 。

Op 必须满足结合律，不需要幂等，也不需要单位元。预处理 $O(n \log n)$ ，查询 $O(1)$ 。

*/

```
template <class T, class Op>
struct DST {
    int n = 0, lg = 0;
    Op op;
    vector<T> val;
    vector<vector<T>> tab;

    DST() = default;
    DST(const vector<T> &a, Op op = Op()) : op(move(op)) {
        build(a);
    }

    void build(const vector<T> &a) {
        val = a;
        n = val.size();
        if (n == 0) {
            lg = 0;
            tab.clear();
            return;
        }
        lg = __lg(max<int>(1, n - 1)) + 1;
        tab.assign(lg, vector<T>(n));
        for (int lev = 0; lev < lg; ++lev) {
            int h = 1 << lev;
            int blk = h << 1;
            for (int beg = 0; beg < n; beg += blk) {
                int mid = min(beg + h, n);
                int end = min(beg + blk, n);
                if (mid > beg) {
                    tab[lev][mid - 1] = val[mid - 1];
                    for (int i = mid - 2; i >= beg; --i) tab[lev][i] = op(val[i], tab[lev][i + 1]);
                }
                if (mid < end) {
                    tab[lev][mid] = val[mid];
                    for (int i = mid + 1; i < end; ++i) tab[lev][i] = op(tab[lev][i - 1], val[i]);
                }
            }
        }
    }

    T ask(int l, int r) const {
        assert(0 <= l && l <= r && r < n);
        if (l == r) return val[l];
        int lev = __lg(l ^ r);
        return op(tab[lev][l], tab[lev][r]);
    }
};
```

04C - 线性RMQ (状压) .cpp

82 lines · C++ · 03 - 数据结构

```

/*
用途：静态幂等最值 RMQ；64 位块内单调栈状压，块间 Sparse Table。
约定：数组与查询均为 0-indexed，ask/askPos 使用闭区间 [l,r]；相等时返回最左位置。
复杂度：字长模型下 O(n) 预处理、O(1) 查询，空间 O(n)。Cmp 默认为求最小值。
性能：默认使用宏展开后的 int；若大规模数据因此超时或超空间，可将纯下标、状态或计数局部改为 i32。
*/

using u64 = uint64_t;

template <class T, class Cmp = less<T>> struct LinearRMQ {
    static constexpr int B = 64;

    int n = 0, nb = 0;
    Cmp cmp;
    vector<T> a;
    vector<int> pre, suf;
    vector<u64> stk;
    vector<vector<int>> st;

    LinearRMQ() = default;
    explicit LinearRMQ(const vector<T>& a, Cmp cmp = Cmp()) : cmp(move(cmp)) { build(a); }

    int best(int x, int y) const {
        if (x < 0) return y;
        if (y < 0) return x;
        if (cmp(a[y], a[x])) return y;
        if (cmp(a[x], a[y])) return x;
        return min(x, y);
    }

    void build(const vector<T>& v) {
        a = v;
        n = a.size();
        if (!n) {
            nb = 0, pre.clear(), suf.clear(), stk.clear(), st.clear();
            return;
        }
        nb = (n + B - 1) / B;
        pre.resize(n), suf.resize(n), stk.resize(n);
        vector<int> b(nb);
        for (int q = 0; q < nb; ++q) {
            int l = q * B, r = min<int>(n, l + B);
            u64 s = 0;
            for (int i = l; i < r; ++i) {
                pre[i] = i == l ? i : best(pre[i - 1], i);
                while (s) {
                    int k = 63 - __builtin_clzll(s);
                    if (!cmp(a[i], a[l + k])) break;
                    s ^= u64(1) << k;
                }
                s |= u64(1) << (i - l);
                stk[i] = s;
            }
            for (int i = r; i-- > l;) suf[i] = i + 1 == r ? i : best(i, suf[i + 1]);
            b[q] = pre[r - 1];
        }
        int lg = nb == 1 ? 0 : 31 - __builtin_clz(nb);
        st.assign(lg + 1, vector<int>(nb));
        st[0] = move(b);
        for (int k = 1; k <= lg; ++k)

```

```
        for (int i = 0; i + (1LL << k) <= nb; ++i)
            st[k][i] = best(st[k - 1][i], st[k - 1][i + (1LL << (k - 1))]);
    }

    int askPos(int l, int r) const {
        assert(0 <= l && l <= r && r < n);
        int x = l / B, y = r / B;
        if (x == y) {
            u64 s = stk[r] >> (l - x * B);
            return l + __builtin_ctzll(s);
        }
        int z = best(suf[l], pre[r]);
        int ql = x + 1, qr = y - 1;
        if (ql <= qr) {
            int k = 31 - __builtin_clz(qr - ql + 1);
            z = best(z, best(st[k][ql], st[k][qr - (1LL << k) + 1]));
        }
        return z;
    }

    const T& ask(int l, int r) const { return a[askPos(l, r)]; }
};
```

05 - Bitset.cpp

280 lines · C++ · 03 - 数据结构

```

/*
用途：运行时长度 Bitset，支持位运算、移位和枚举置位。

下标为 0..askSz()-1；askNxt/askPre 找不到时返回 askSz()。
resize(n, true) 会将“新增的位”置 1，行为与 vector 的扩容语义一致。
*/

using u64 = uint64_t;

class Bitset {
public:
    static constexpr int W = 64;

private:
    vector<u64> a;
    int nb = 0;
    u64 msk = ~u64(0);

    static int wds(int bit) {
        if (bit < 0) throw invalid_argument("negative bitset size");
        return (bit + W - 1) / W;
    }

    void trim() {
        int rem = nb % W;
        msk = rem == 0 ? ~u64(0) : (u64(1) << rem) - 1;
        if (!a.empty()) a.back() &= msk;
    }

    void check(int pos) const {
        if (pos < 0 || pos >= nb) throw out_of_range("bit index out of range");
    }

public:
    Bitset() = default;
    explicit Bitset(int n, bool val = false) {
        resize(n, val);
    }

    int askSz() const noexcept {
        return nb;
    }

    bool askEmp() const noexcept {
        return nb == 0;
    }

    void reset() noexcept {
        fill(all(a), 0);
    }

    void set() noexcept {
        fill(all(a), ~u64(0));
        if (!a.empty()) a.back() &= msk;
    }

    void resize(int n, bool val = false) {
        int nw = wds(n);
        int ob = nb;
        if (n == ob) return;

```



```

    if (n > ob) {
        a.resize(nw, val ? ~u64(0) : 0);
        if (val) {
            int fw = ob / W;
            int fb = ob % W;
            if (fw < nw) {
                if (fb) a[fw] |= (~u64(0) << fb);
                else a[fw] = ~u64(0);
                for (int i = fw + 1; i < nw; ++i) a[i] = ~u64(0);
            }
        }
    } else {
        a.resize(nw);
    }
    nb = n;
    trim();
}

void modify(int pos, bool v) {
    check(pos);
    u64 b = u64(1) << (pos % W);
    if (v) a[pos / W] |= b;
    else a[pos / W] &= ~b;
}

void flip(int pos) {
    check(pos);
    a[pos / W] ^= u64(1) << (pos % W);
}

bool ask(int pos) const {
    check(pos);
    return (a[pos / W] >> (pos % W)) & 1;
}

bool askAny() const noexcept {
    for (u64 wd : a) {
        if (wd) return true;
    }
    return false;
}

bool askNone() const noexcept {
    return !askAny();
}

bool askAll() const noexcept {
    if (a.empty()) return true;
    for (int i = 0; i + 1 < a.size(); ++i) {
        if (a[i] != ~u64(0)) return false;
    }
    return a.back() == msk;
}

int askCnt() const noexcept {
    int ans = 0;
    for (u64 wd : a) ans += __builtin_popcountll(wd);
    return ans;
}

Bitset &operator&=(const Bitset &o) {
    assert(nb == o.nb);
    for (int i = 0; i < a.size(); ++i) a[i] &= o.a[i];
}

```

```

        return *this;
    }

    Bitset &operator|=(const Bitset &o) {
        assert(nb == o.nb);
        for (int i = 0; i < a.size(); ++i) a[i] |= o.a[i];
        if (!a.empty()) a.back() &= msk;
        return *this;
    }

    Bitset &operator^=(const Bitset &o) {
        assert(nb == o.nb);
        for (int i = 0; i < a.size(); ++i) a[i] ^= o.a[i];
        if (!a.empty()) a.back() &= msk;
        return *this;
    }

    Bitset operator~() const {
        Bitset res(*this);
        for (u64 &wd : res.a) wd = ~wd;
        if (!res.a.empty()) res.a.back() &= res.msk;
        return res;
    }

    friend Bitset operator&(Bitset lhs, const Bitset &rhs) {
        return lhs &= rhs;
    }

    friend Bitset operator|(Bitset lhs, const Bitset &rhs) {
        return lhs |= rhs;
    }

    friend Bitset operator^(Bitset lhs, const Bitset &rhs) {
        return lhs ^= rhs;
    }

    Bitset &operator<<=(int sh) {
        if (sh < 0) throw invalid_argument("negative bit shift");
        if (sh == 0 || nb == 0) return *this;
        int sw = sh / W;
        int sb = sh % W;
        int wds = a.size();
        if (sw >= wds) {
            reset();
            return *this;
        }
        if (sb == 0) {
            for (int i = wds - 1; i >= sw; --i) a[i] = a[i - sw];
        } else {
            for (int i = wds - 1; i >= sw; --i) {
                u64 hi = a[i - sw] << sb;
                u64 low = i - sw > 0 ? a[i - sw - 1] >> (W - sb) : 0;
                a[i] = hi | low;
            }
        }
        for (int i = 0; i < sw; ++i) a[i] = 0;
        a.back() &= msk;
        return *this;
    }

    Bitset &operator>>=(int sh) {
        if (sh < 0) throw invalid_argument("negative bit shift");
        if (sh == 0 || nb == 0) return *this;
        int sw = sh / W;

```

```

    int sb = sh % W;
    int wds = a.size();
    if (sw >= wds) {
        reset();
        return *this;
    }
    if (sb == 0) {
        for (int i = 0; i + sw < wds; ++i) a[i] = a[i + sw];
    } else {
        for (int i = 0; i + sw < wds; ++i) {
            u64 low = a[i + sw] >> sb;
            u64 hi = i + sw + 1 < wds ? a[i + sw + 1] << (W - sb) : 0;
            a[i] = low | hi;
        }
    }
    for (int i = wds - sw; i < wds; ++i) a[i] = 0;
    a.back() &= msk;
    return *this;
}

friend Bitset operator<<(Bitset lhs, int sh) {
    return lhs <=< sh;
}

friend Bitset operator>>(Bitset lhs, int sh) {
    return lhs >=> sh;
}

int askNxt(int pos) const noexcept {
    if (pos < 0) pos = 0;
    if (pos >= nb) return nb;
    int id = pos / W;
    u64 wd = a[id] & (~u64(0) << (pos % W));
    if (wd) return id * W + __builtin_ctzll(wd);
    for (++id; id < a.size(); ++id) {
        if (a[id]) return id * W + __builtin_ctzll(a[id]);
    }
    return nb;
}

int askPre(int pos) const noexcept {
    if (pos < 0 || nb == 0) return nb;
    if (pos >= nb) pos = nb - 1;
    int id = pos / W;
    int off = pos % W;
    u64 msk = off == 63 ? ~u64(0) : (u64(1) << (off + 1)) - 1;
    u64 wd = a[id] & msk;
    if (wd) return id * W + 63 - __builtin_clzll(wd);
    while (id > 0) {
        --id;
        if (a[id]) return id * W + 63 - __builtin_clzll(a[id]);
    }
    return nb;
}

template <class F>
void each(F f) const {
    for (int i = 0; i < a.size(); ++i) {
        u64 wd = a[i];
        while (wd) {
            int bit = __builtin_ctzll(wd);
            f(i * W + bit);
            wd &= wd - 1;
        }
    }
}

```

```
    }  
}  
  
string str() const {  
    string res;  
    res.reserve(nb);  
    for (int i = nb - 1; i >= 0; --i) res.push_back(ask(i) ? '1' : '0');  
    return res;  
}  
  
const u64 *askData() const noexcept {  
    return a.empty() ? nullptr : a.data();  
}  
  
u64 *askData() noexcept {  
    return a.empty() ? nullptr : a.data();  
}  
  
static Bitset read(const string &s) {  
    Bitset res(s.size());  
    for (int i = 0; i < s.size(); ++i) {  
        char c = s[s.size() - 1 - i];  
        if (c == '1') res.modify(i, true);  
        else if (c != '0') throw invalid_argument("invalid char in bitstring");  
    }  
    return res;  
}  
};
```

06 - 小波矩阵 (WaveletMatrix) .cpp

225 lines · C++ · 03 - 数据结构

```

/*
用途：压缩值域 Wavelet Matrix，支持 0-indexed 半开区间 [l, r) 的访问、k 小、计数和频率查询。
askLT/askLE 分别表示  $x / \leq x$ ；askCnt(l,r,L,R) 统计 [L,R] 内元素个数。
预处理  $O(n \log \sigma)$ ，单次查询  $O(\log \sigma)$ 。
性能：层前缀、位置和压缩编号默认使用 int；极限内存不足时可只将这些纯计数与下标改为 i32。
*/

using u64 = uint64_t;

template <class T> struct Wavelet {
    struct BitVec {
        int n;
        vector<u64> bit;
        vector<int> pre;

        BitVec() : n(0) {}
        BitVec(int _n) {
            init(_n);
        }

        void init(int _n) {
            n = _n;
            int blk = (n >> 6) + 1;
            bit.assign(blk, 0);
            pre.assign(blk + 1, 0);
        }

        inline void modify(int i) {
            bit[i >> 6] |= (1ull << (i & 63));
        }

        void build() {
            for (int i = 0; i < bit.size(); ++i) {
                pre[i + 1] = pre[i] + __builtin_popcountll(bit[i]);
            }
        }

        inline int ask1(int pos) const {
            return pre[pos >> 6] + __builtin_popcountll(bit[pos >> 6] & ((1ull << (pos & 63)) - 1));
        }

        inline int ask0(int pos) const {
            return pos - ask1(pos);
        }

        inline int ask(int i) const {
            return (bit[i >> 6] >> (i & 63)) & 1;
        }
    };

    int n, len;
    vector<BitVec> mat;
    vector<int> mid;
    vector<T> val;

    Wavelet() : n(0), len(0) {}

    Wavelet(vector<T> dat) {
        build(dat);
    }

```

```

void build(vector<T> dat) {
    n = dat.size();
    val = dat;
    sort(val.begin(), val.end());
    val.erase(unique(val.begin(), val.end()), val.end());
    int sig = val.size();
    if (sig == 0) {
        len = 0;
        mat.clear();
        mid.clear();
        return;
    }

    vector<int> cod(n);
    for (int i = 0; i < n; ++i) {
        cod[i] = lower_bound(val.begin(), val.end(), dat[i]) - val.begin();
    }

    len = 0;
    while ((1ll << len) < sig) ++len;
    if (len == 0) len = 1;

    mat.assign(len, BitVec());
    mid.assign(len, 0);

    for (int lev = 0; lev < len; ++lev) {
        mat[lev].init(n);
    }

    vector<int> nxt(n);
    for (int lev = len - 1; lev >= 0; --lev) {
        auto &bv = mat[lev];
        for (int i = 0; i < n; ++i) {
            if ((cod[i] >> lev) & 1) bv.modify(i);
        }
        bv.build();

        int cnt = 0;
        for (int i = 0; i < n; ++i) {
            if (!((cod[i] >> lev) & 1)) {
                nxt[cnt++] = cod[i];
            }
        }
        int idx = cnt;
        for (int i = 0; i < n; ++i) {
            if ((cod[i] >> lev) & 1) {
                nxt[idx++] = cod[i];
            }
        }
        mid[lev] = cnt;
        cod.swap(nxt);
    }
}

inline int askSz() const {
    return n;
}

inline bool askEmp() const {
    return n == 0;
}

T ask(int pos) const {
    assert(pos >= 0 && pos < n);
}

```

```

    int cod = 0;
    for (int lev = len - 1; lev >= 0; --lev) {
        int b = mat[lev].ask(pos);
        if (b) {
            cod |= (1 << lev);
            pos = mid[lev] + mat[lev].ask1(pos);
        } else {
            pos = mat[lev].ask0(pos);
        }
    }
    return val[cod];
}

T askKth(int l, int r, int k) const {
    assert(l >= 0 && l <= r && r <= n && k >= 0 && k < r - l);
    int cod = 0;
    for (int lev = len - 1; lev >= 0; --lev) {
        const BitVec &bv = mat[lev];
        int zl = bv.ask0(l);
        int zr = bv.ask0(r);
        int zs = zr - zl;
        if (k < zs) {
            l = zl;
            r = zr;
        } else {
            k -= zs;
            cod |= (1 << lev);
            l = mid[lev] + bv.ask1(l);
            r = mid[lev] + bv.ask1(r);
        }
    }
    return val[cod];
}

int askLT(int l, int r, T x) const {
    if (l >= r || val.empty()) return 0;
    auto it = lower_bound(val.begin(), val.end(), x);
    if (it == val.end()) return r - l;
    int xc = it - val.begin();
    if (xc == 0) return 0;
    int res = 0;
    for (int lev = len - 1; lev >= 0; --lev) {
        const BitVec &bv = mat[lev];
        int zl = bv.ask0(l), zr = bv.ask0(r);
        if ((xc >> lev) & 1) {
            res += zr - zl;
            l = mid[lev] + bv.ask1(l);
            r = mid[lev] + bv.ask1(r);
        } else {
            l = zl;
            r = zr;
        }
    }
    return res;
}

int askLE(int l, int r, T x) const {
    if (l >= r || val.empty()) return 0;
    auto it = upper_bound(val.begin(), val.end(), x);
    if (it == val.end()) return r - l;
    int xc = it - val.begin();
    if (xc == 0) return 0;
    int res = 0;
    for (int lev = len - 1; lev >= 0; --lev) {

```

```
        const BitVec &bv = mat[lev];
        int zl = bv.ask0(l), zr = bv.ask0(r);
        if ((xc >> lev) & 1) {
            res += zr - zl;
            l = mid[lev] + bv.ask1(l);
            r = mid[lev] + bv.ask1(r);
        } else {
            l = zl;
            r = zr;
        }
    }
    return res;
}

int askCnt(int l, int r, T mn, T mx) const {
    if (mn > mx) return 0;
    return askLE(l, r, mx) - askLT(l, r, mn);
}

int askFreq(int l, int r, T x) const {
    if (l >= r || val.empty()) return 0;
    auto it = lower_bound(val.begin(), val.end(), x);
    if (it == val.end() || *it != x) return 0;
    int cod = it - val.begin();
    for (int lev = len - 1; lev >= 0; --lev) {
        const BitVec &bv = mat[lev];
        if (!((cod >> lev) & 1)) {
            l = bv.ask0(l);
            r = bv.ask0(r);
        } else {
            l = mid[lev] + bv.ask1(l);
            r = mid[lev] + bv.ask1(r);
        }
    }
    return r - l;
}
};
```


06A - 区间第k大 (WaveletMatrix) .cpp

142 lines · C++ · 03 - 数据结构

/*

用途：只关注区间第 k 小/第 k 大的轻量 Wavelet Matrix 封装。

区间和 k 均为 0-indexed 闭区间：askKth(l,r,k) 是第 k 小，askKth<true>(l,r,k) 是第 k 大。

构造后静态查询，预处理 $O(n \log \sigma)$ 、查询 $O(\log \sigma)$ 。

性能：默认使用宏展开后的 int；若大规模数据因此超时或超空间，可将纯下标、状态或计数局部改为 i32。

*/

using u64 = uint64_t;

```
template <typename T> class Wavelet {
private:
    struct Core {
        struct BitVec {
            static constexpr int W = 64;
            static int ask64(u64 x, int i) {
                return __builtin_popcountll(x & ((1ull << i) - 1));
            }
        };
#pragma pack(4)
        struct Block {
            u64 bit;
            int sum;
        };
#pragma pack()
        int n, z;
        vector<Block> b;
        BitVec(int _n = 0) : n(_n), b(n / W + 1) {}
        int operator[](int i) const {
            return b[i / W].bit >> i % W & 1;
        }
        void modify(int i) {
            b[i / W].bit |= u64(1) << i % W;
        }
        void build() {
            for (int j = 0; j < n / W; ++j) b[j + 1].sum = b[j].sum +
                __builtin_popcountll(b[j].bit);
            z = ask0(n);
        }
        int ask0(int i) const {
            return i - ask1(i);
        }
        int ask1(int i) const {
            auto &&e = b[i / W];
            return e.sum + ask64(e.bit, i % W);
        }
    };

    int n = 0, lg = 0;
    vector<T> a;
    vector<BitVec> bv;
    Core(int _n = 0) : n(_n), a(n) {}
    Core(const vector<T> &a) : n(a.size()), a(a) {
        build();
    }
    T &operator[](int i) {
        return a[i];
    }
    void build() {
        if (n == 0) {
            lg = 0;

```

```

        bv.clear();
        return;
    }
    lg = __lg(max<T>(*max_element(begin(a), end(a)), 1)) + 1;
    bv.assign(lg, n);
    vector<T> cur = a, nxt(n);
    for (auto h = lg; h--;) {
        for (int i = 0; i < n; ++i)
            if (cur[i] >> h & 1) bv[h].modify(i);
        bv[h].build();
        array<typename vector<T>::iterator, 2> it{begin(nxt), begin(nxt) + bv[h].z};
        for (int i = 0; i < n; ++i) *it[bv[h][i]]++ = cur[i];
        swap(cur, nxt);
    }
}

T askKth(int l, int r, int k) const {
    T res = 0;
    for (auto h = lg; h--;) {
        auto l0 = bv[h].ask0(l), r0 = bv[h].ask0(r);
        if (k < r0 - l0)
            l = l0, r = r0;
        else {
            k -= r0 - l0;
            res |= T{1} << h;
            l += bv[h].z - l0;
            r += bv[h].z - r0;
        }
    }
    return res;
}

int askCnt(int l, int r, T ub) const {
    if (ub >= T{1} << lg) return r - l;
    int res = 0;
    for (auto h = lg; h--;) {
        auto l0 = bv[h].ask0(l), r0 = bv[h].ask0(r);
        if (~ub >> h & 1)
            l = l0, r = r0;
        else {
            res += r0 - l0;
            l += bv[h].z - l0;
            r += bv[h].z - r0;
        }
    }
    return res;
}

int askCnt(int l, int r, T lb, T ub) const {
    return askCnt(l, r, ub) - askCnt(l, r, lb);
}

};

pair<vector<int>, vector<T>> zip(const vector<T> &a) {
    int n = a.size();
    vector<pair<T, int>> p(n);
    for (int i = 0; i < n; ++i) p[i] = {a[i], i};
    sort(begin(p), end(p));
    vector<int> na(n);
    vector<T> v;
    for (int k = 0, rnk = -1; k < n; ++k) {
        if (k == 0 or p[k - 1].first < p[k].first) v.push_back(p[k].first), ++rnk;
        na[p[k].second] = rnk;
    }
    return make_pair(na, v);
}

void check(int l, int r, int k) const {
    assert(l >= 0 && l <= r && r < n && k <= r - l && k >= 0);
}

```

```
    }
    vector<T> vec;
    Core wm;
    int n = 0;

public:
    Wavelet() {}
    Wavelet(const vector<T> &v) : n(v.size()) {
        auto [a, _v] = zip(v);
        wm = Core(a);
        vec = _v;
    }
    template <bool Rev = false> T askKth(int l, int r, int k) const {
        check(l, r, k);
        if constexpr (Rev) k = r - l - k;
        r++;
        return vec[wm.askKth(l, r, k)];
    }
};
```

07 - 线性基.cpp

103 lines · C++ · 03 - 数据结构

/*

用途：异或线性基，支持可表示性、最大异或、最小非空异或和第 k 小非空子集异或值。

T 必须是整数类型；内部按 T 的全部二进制位工作，不会遗漏最高位。

- insert(x)：插入 x ，返回是否线性无关。
- ask(x)：是否能由已插入数异或得到 x 。
- askMax($seed = 0$)：max($seed$ xor subsetXor)。
- askMin()：最小非空子集异或值；若存在非空子集异或为 0 则返回 0。
- askKth(k)：第 k 小“非空子集”异或结果 (1-indexed，重复结果只计一次)，不存在返回 -1。

调用 askKth/askMin 前会自动化为最简基，复杂度 $O(B^2)$ ， B 为位数（最多 64）。

*/

using u64 = uint64_t;

template <class T = u64>

```
struct LB {
    static_assert(is_integral_v<T>);
    using U = make_unsigned_t<T>;
    static constexpr int B = numeric_limits<U>::digits;

    array<U, B> bas{};
    vector<U> ord;
    int cnt = 0;
    bool dep = false;

    bool insert(T val) {
        U x = static_cast<U>(val);
        for (int bit = B - 1; bit >= 0; --bit) {
            if (!((x >> bit) & U(1))) continue;
            if (!bas[bit]) {
                bas[bit] = x;
                ++cnt;
                ord.clear();
                return true;
            }
            x ^= bas[bit];
        }
        dep = true;
        return false;
    }

    bool ask(T val) const {
        U x = static_cast<U>(val);
        for (int bit = B - 1; bit >= 0; --bit) {
            if (!((x >> bit) & U(1))) continue;
            if (!bas[bit]) return false;
            x ^= bas[bit];
        }
        return true;
    }

    T askMax(T sd = T{}) const {
        U ans = static_cast<U>(sd);
        for (int bit = B - 1; bit >= 0; --bit) {
            if ((ans ^ bas[bit]) > ans) ans ^= bas[bit];
        }
        return static_cast<T>(ans);
    }
}
```

```

void rebuild() {
    for (int hi = B - 1; hi >= 0; --hi) {
        if (!bas[hi]) continue;
        for (int low = hi - 1; low >= 0; --low) {
            if ((bas[hi] >> low) & U(1)) bas[hi] ^= bas[low];
        }
    }
    ord.clear();
    for (int bit = 0; bit < B; ++bit) {
        if (bas[bit]) ord.push_back(bas[bit]);
    }
}

T askMin() {
    if (dep) return T{};
    rebuild();
    return ord.empty() ? T{} : static_cast<T>(ord.front());
}

T askKth(u64 k) {
    if (k == 0) return T(-1);
    rebuild();
    u64 msk;
    if (dep) {
        msk = k - 1; // 非空子集也可得到 0。
    } else {
        msk = k; // 0 对应空集，非空结果从 mask = 1 开始。
    }
    if (cnt < 64 && msk >= (1ull << cnt)) return T(-1);
    U ans = 0;
    for (int i = 0; i < cnt; ++i) {
        if ((msk >> i) & 1ull) ans ^= ord[i];
    }
    return static_cast<T>(ans);
}

void merge(const LB &o) {
    for (int bit = B - 1; bit >= 0; --bit) {
        if (o.bas[bit]) insert(static_cast<T>(o.bas[bit]));
    }
    dep = dep || o.dep;
}
};

```

08A - 字典树.cpp

91 lines · C++ · 03 - 数据结构

```

/*
用途：定长连续字符集 Trie，支持插入、删除、单词/前缀计数与最长前缀词链。
用法：Trie<26, 'a'>; 节点转移、编号和计数使用 int。
性能：默认使用宏展开后的 int；若大规模数据因此超时或超空间，可将纯下标、状态或计数局部改为 i32。
*/
template <int A = 26, char F = 'a'> struct Trie {

    struct Node {
        array<int, A> to{};
        int ps = 0, ed = 0;
    };

    vector<Node> tr;

    Trie() {
        clear();
    }

    void clear() {
        tr.assign(1, Node());
    }

    int id(char c) const {
        int x = c - F;
        assert(0 <= x && x < A);
        return x;
    }

    void insert(const string& s) {
        int u = 0;
        ++tr[u].ps;
        for (char c : s) {
            int v = id(c);
            if (!tr[u].to[v]) {
                tr[u].to[v] = tr.size();
                tr.emplace_back();
            }
            u = tr[u].to[v];
            ++tr[u].ps;
        }
        ++tr[u].ed;
    }

    bool erase(const string& s) {
        int u = 0;
        vector<int> p{0};
        for (char c : s) {
            int v = id(c);
            if (!tr[u].to[v]) return false;
            u = tr[u].to[v];
            p.push_back(u);
        }
        if (!tr[u].ed) return false;
        --tr[u].ed;
        for (int v : p) --tr[v].ps;
        return true;
    }

    int ask(const string& s) const {
        int u = 0;

```

```
        for (char c : s) {
            int v = id(c);
            if (!tr[u].to[v]) return 0;
            u = tr[u].to[v];
        }
        return tr[u].ed;
    }

    int askPre(const string& s) const {
        int u = 0;
        for (char c : s) {
            int v = id(c);
            if (!tr[u].to[v]) return 0;
            u = tr[u].to[v];
        }
        return tr[u].ps;
    }

    int askMax() const {
        int r = 0;
        auto dfs = [&](auto&& go, int u, int d) -> void {
            d += tr[u].ed > 0;
            r = max(r, d);
            for (int v : tr[u].to)
                if (v && tr[v].ps) go(go, v, d);
        };
        dfs(dfs, 0, 0);
        return r;
    }

};
```

08B - 二进制字典树 (XOR Trie) .cpp

107 lines · C++ · 03 - 数据结构

/*

用途：二进制 Trie (XOR Trie)，维护可重集合并查询异或最值。

U 必须为无符号整数类型，B 默认为其完整位数。

接口：insert(x)、erase(x)、askMax(x)、askMin(x)、ask(x,lim)。

askMax/askMin 在空 Trie 时返回 nullopt；ask 统计所有 y 使 (x xor y) < lim。

*/

using u64 = uint64_t;

template <class U = u64, int B = numeric_limits<U>::digits>

struct XorTrie {

static_assert(is_unsigned_v<U>);

struct Node {

array<int, 2> nxt{};

int cnt = 0;

};

vector<Node> tr = {Node{}};

int askSz() const {

return tr[0].cnt;

}

bool askEmp() const {

return askSz() == 0;

}

void insert(U val) {

int u = 0;

++tr[u].cnt;

for (int bit = B - 1; bit >= 0; --bit) {

int b = (val >> bit) & U(1);

if (!tr[u].nxt[b]) {

tr[u].nxt[b] = tr.size();

tr.emplace_back();

}

u = tr[u].nxt[b];

++tr[u].cnt;

}

}

bool erase(U val) {

int u = 0;

vector<int> pth = {0};

for (int bit = B - 1; bit >= 0; --bit) {

int b = (val >> bit) & U(1);

if (!tr[u].nxt[b] || tr[tr[u].nxt[b]].cnt == 0) return false;

u = tr[u].nxt[b];

pth.push_back(u);

}

for (int p : pth) --tr[p].cnt;

return true;

}

optional<U> askMax(U val) const {

if (askEmp()) return nullopt;

U ans = 0;

int u = 0;


```

        for (int bit = B - 1; bit >= 0; --bit) {
            int b = (val >> bit) & U(1);
            int pre = b ^ 1;
            if (tr[u].nxt[pre] && tr[tr[u].nxt[pre]].cnt) {
                ans |= U(1) << bit;
                u = tr[u].nxt[pre];
            } else {
                u = tr[u].nxt[b];
            }
        }
        return ans;
    }

optional<U> askMin(U val) const {
    if (askEmp()) return nullopt;
    U ans = 0;
    int u = 0;
    for (int bit = B - 1; bit >= 0; --bit) {
        int b = (val >> bit) & U(1);
        if (tr[u].nxt[b] && tr[tr[u].nxt[b]].cnt) {
            u = tr[u].nxt[b];
        } else {
            ans |= U(1) << bit;
            u = tr[u].nxt[b ^ 1];
        }
    }
    return ans;
}

int ask(U val, U lim) const {
    int u = 0;
    int ans = 0;
    for (int bit = B - 1; bit >= 0; --bit) {
        int xb = (val >> bit) & U(1);
        int b = (lim >> bit) & U(1);
        if (b) {
            int sam = tr[u].nxt[xb];
            if (sam) ans += tr[sam].cnt;
            u = tr[u].nxt[xb ^ 1];
        } else {
            u = tr[u].nxt[xb];
        }
        if (!u) break;
    }
    return ans;
}
};

```

08C - PBDS前缀字典树（GNU环境）.cpp

24 lines · C++ · 03 - 数据结构

```
/*
用途：GNU PBDS Patricia trie；维护唯一字符串，并直接枚举具有给定前缀的字符串。
限制：仅 GCC/libstdc++ 可用，节点开销较大；只需插入/查询时优先使用更轻的 08A。

insert/erase/find/size 与集合相同；prefix_range(s) 返回 [l,r)，其中均以 s 为前缀。
键不可重复；若需要出现次数，把次数另存在哈希表中或改用 08A。
*/
#ifdef int
#pragma push_macro("int")
#undef int
#define LYE_RESTORE_INT
#endif
#include <ext/pb_ds/assoc_container.hpp>
#include <ext/pb_ds/trie_policy.hpp>
#ifdef LYE_RESTORE_INT
#pragma pop_macro("int")
#undef LYE_RESTORE_INT
#endif

using PTrie = __gnu_pbds::trie<
    string, __gnu_pbds::null_type,
    __gnu_pbds::trie_string_access_traits<>,
    __gnu_pbds::pat_trie_tag,
    __gnu_pbds::trie_prefix_search_node_update>;
```

09A - 可并堆 (LeftistHeap, 可移植) .cpp

116 lines · C++ · 03 - 数据结构

/*

用途：可移植的左偏可并堆。默认 `Cmp = less<T>` 时为大根堆；使用 `greater<T>` 可得到小根堆。

初始化 `Heap(n)` 后，`h[i]` 代表第 `i` 个堆，支持 `push/pop/ask/merge/clear`。

`merge(x, y)` 是破坏性合并，合并后 `y` 变为空堆。所有操作 $O(\log n)$ 。

与 09B 解决同一问题；非 GCC/libstdc++ 环境使用本模板。

*/

```
template <class T, class Cmp = less<T>> struct Heap {
    struct Node {
        T v;
        int l, r, d;
    };

    vector<Node> tr;
    vector<int> rt, siz;
    Cmp cmp;
    int tot;

    Heap() {}
    Heap(int n) {
        init(n);
    }

    void init(int n) {
        tr.assign(1, {});
        tr.reserve(n + 1);
        rt.assign(n + 1, 0);
        siz.assign(n + 1, 0);
        tot = 0;
    }

    int merge0(int x, int y) {
        if (!x || !y) return x | y;
        if (cmp(tr[x].v, tr[y].v)) swap(x, y);
        tr[x].r = merge0(tr[x].r, y);
        if (tr[tr[x].l].d < tr[tr[x].r].d) swap(tr[x].l, tr[x].r);
        tr[x].d = tr[tr[x].r].d + 1;
        return x;
    }

    void merge(int x, int y) {
        if (x == y) return;
        rt[x] = merge0(rt[x], rt[y]);
        siz[x] += siz[y];
        rt[y] = siz[y] = 0;
    }

    void push(int x, const T &v) {
        tr.push_back({v, 0, 0, 1});
        rt[x] = merge0(rt[x], ++tot);
        ++siz[x];
    }

    T ask(int x) {
        return tr[rt[x]].v;
    }

    T pop(int x) {
        int u = rt[x];
        T res = tr[u].v;
```

```
        rt[x] = merge0(tr[u].l, tr[u].r);
        --siz[x];
        return res;
    }

    void clear(int x) {
        rt[x] = siz[x] = 0;
    }

    bool askEmp(int x) {
        return siz[x] == 0;
    }

    int askSz(int x) {
        return siz[x];
    }

    struct Ref {
        Heap *h;
        int x;

        Ref &operator+=(Ref y) {
            h->merge(x, y.x);
            return *this;
        }

        Ref &operator<<(const T &v) {
            h->push(x, v);
            return *this;
        }

        T ask() {
            return h->ask(x);
        }

        T pop() {
            return h->pop(x);
        }

        void clear() {
            h->clear(x);
        }

        bool askEmp() {
            return h->askEmp(x);
        }

        int askSz() {
            return h->askSz(x);
        }
    };

    Ref operator[](int x) {
        return {this, x};
    }
};
```

09B - PBDS可并堆 (GNU环境) .cpp

33 lines · C++ · 03 - 数据结构

```

/*
用途：GNU PBDS pairing heap；支持普通优先队列、修改/删除任意节点及破坏性合并。
限制：仅 GCC/libstdc++ 可用。默认 Cmp = less<T> 为大根堆，greater<T> 为小根堆。

push 返回的句柄可交给 modify/erase；join 后另一个堆清空。除 empty/size 外，操作空堆未定义。
pairing heap 的 push/join 为  $O(1)$ ，pop 为均摊  $O(\log n)$ ，modify/erase 的实际复杂度依赖修改方向。
*/
#ifdef int
#pragma push_macro("int")
#undef int
#define LYE_RESTORE_INT
#endif
#include <ext/pb_ds/priority_queue.hpp>
#ifdef LYE_RESTORE_INT
#pragma pop_macro("int")
#undef LYE_RESTORE_INT
#endif

template <class T, class Cmp = less<T>> struct Heap {
    using H = __gnu_pbds::priority_queue<T, Cmp, __gnu_pbds::pairing_heap_tag>;
    using It = typename H::point_iterator;
    H h;

    It push(const T &x) { return h.push(x); }
    void pop() { h.pop(); }
    const T &ask() const { return h.top(); }
    void modify(It p, const T &x) { h.modify(p, x); }
    void erase(It p) { h.erase(p); }
    void merge(Heap &o) { h.join(o.h); }
    bool askEmp() const { return h.empty(); }
    int askSz() const { return h.size(); }
    void clear() { h.clear(); }
};

```

10 - 可合并信息FHQTreap (MergeFHQTreap) .cpp

105 lines · C++ · 03 - 数据结构

/*

用途：带自定义聚合信息的键值 FHQ Treap 多重集。

Info 需支持默认构造与 operator+; MakeInfo(key, count) 生成单键贡献。

当前接口聚焦 insert/erase/ask (全局聚合)；若需要 rank/kth/前驱后继，使用 10A。

*/

```
template <class T, class F> class FHQTreap {
private:
    struct Node {
        int l = 0, r = 0;
        int key = 0, cnt = 0;
        unsigned pri = 0;
        T val;
    };

    vector<Node> tr;
    int rt = 0;
    F mk;
    mt19937 rng;

    int node(int key, int cnt = 1) {
        tr.push_back(Node());
        int p = tr.size() - 1;
        tr[p].key = key;
        tr[p].cnt = cnt;
        tr[p].pri = rng();
        tr[p].val = mk(key, cnt);
        return p;
    }

    void pull(int p) {
        if (!p) return;
        tr[p].val = tr[tr[p].l].val + mk(tr[p].key, tr[p].cnt) + tr[tr[p].r].val;
    }

    void split(int p, int key, int &x, int &y) {
        if (!p) {
            x = y = 0;
            return;
        }
        if (tr[p].key <= key) {
            x = p;
            split(tr[p].r, key, tr[p].r, y);
            pull(x);
        } else {
            y = p;
            split(tr[p].l, key, x, tr[p].l);
            pull(y);
        }
    }

    int merge(int x, int y) {
        if (!x || !y) return x | y;
        if (tr[x].pri < tr[y].pri) {
            tr[x].r = merge(tr[x].r, y);
            pull(x);
            return x;
        } else {
            tr[y].l = merge(x, tr[y].l);
            pull(y);
        }
    }
};
```

```
        return y;
    }
}

public:
    FHQTreap(F f)
        : tr(1), mk(f),
          rng(chrono::steady_clock::now().time_since_epoch().count()) {}

    bool askEmp() const {
        return rt == 0;
    }

    void insert(int key, int cnt = 1) {
        int a, b, c;
        split(rt, key - 1, a, b);
        split(b, key, b, c);
        if (b) {
            tr[b].cnt += cnt;
            pull(b);
        } else {
            b = node(key, cnt);
        }
        rt = merge(merge(a, b), c);
    }

    void erase(int key, int cnt = 1) {
        int a, b, c;
        split(rt, key - 1, a, b);
        split(b, key, b, c);
        if (b) {
            if (tr[b].cnt > cnt) {
                tr[b].cnt -= cnt;
                pull(b);
            } else {
                b = merge(tr[b].l, tr[b].r);
            }
        }
        rt = merge(merge(a, b), c);
    }

    T ask() const {
        return tr[rt].val;
    }
};
```

10A - FHQTreap (有序多重集) .cpp

170 lines · C++ · 03 - 数据结构

```

/*
用途：FHQ Treap 有序多重集，支持插入、删除、计数、排名、第 k 小、前驱和后继。

Key 需支持 <；askRank/askKth 为 1-indexed。erase(x,c) 删除至多 c 个。
所有操作期望  $O(\log n)$ ，askKth/askPre/askNxt 不存在时返回 nullopt。
*/
template <class Key = int>
struct OSet {
    struct Node {
        int l = 0, r = 0;
        int sz = 0, cnt = 0;
        int pri = 0;
        Key key{};
    };

    vector<Node> tr = {Node{}};
    int rt = 0;
    mt19937 rng{chrono::steady_clock::now().time_since_epoch().count()};

    int sz(int p) const {
        return p ? tr[p].sz : 0;
    }

    void pull(int p) {
        if (p) tr[p].sz = tr[tr[p].l].sz + tr[p].cnt + tr[tr[p].r].sz;
    }

    int node(const Key &key, int cnt) {
        tr.push_back(Node{});
        int p = tr.size() - 1;
        tr[p].key = key;
        tr[p].cnt = tr[p].sz = cnt;
        tr[p].pri = rng();
        return p;
    }

    // a 中键 < key, b 中键 >= key。
    void split(int p, const Key &key, int &a, int &b) {
        if (!p) {
            a = b = 0;
        } else if (tr[p].key < key) {
            a = p;
            split(tr[p].r, key, tr[p].r, b);
            pull(a);
        } else {
            b = p;
            split(tr[p].l, key, a, tr[p].l);
            pull(b);
        }
    }

    // a 中键 <= key, b 中键 > key。
    void splitR(int p, const Key &key, int &a, int &b) {
        if (!p) {
            a = b = 0;
        } else if (!(key < tr[p].key)) {
            a = p;
            splitR(tr[p].r, key, tr[p].r, b);
            pull(a);
        } else {
    
```



```

        b = p;
        splitR(tr[p].l, key, a, tr[p].l);
        pull(b);
    }
}

int merge(int a, int b) {
    if (!a || !b) return a | b;
    if (tr[a].pri < tr[b].pri) {
        tr[a].r = merge(tr[a].r, b);
        pull(a);
        return a;
    }
    tr[b].l = merge(a, tr[b].l);
    pull(b);
    return b;
}

int askSz() const {
    return sz(rt);
}

bool askEmp() const {
    return rt == 0;
}

void clear() {
    tr.assign(1, Node{});
    rt = 0;
}

void insert(const Key &key, int cnt = 1) {
    if (cnt <= 0) return;
    int a, b, c;
    split(rt, key, a, b);
    splitR(b, key, b, c);
    if (b) tr[b].cnt += cnt, pull(b);
    else b = node(key, cnt);
    rt = merge(merge(a, b), c);
}

bool erase(const Key &key, int cnt = 1) {
    if (cnt <= 0) return false;
    int a, b, c;
    split(rt, key, a, b);
    splitR(b, key, b, c);
    bool ok = b;
    if (b) {
        if (tr[b].cnt > cnt) tr[b].cnt -= cnt, pull(b);
        else b = merge(tr[b].l, tr[b].r);
    }
    rt = merge(merge(a, b), c);
    return ok;
}

int askCnt(const Key &key) const {
    int p = rt;
    while (p) {
        if (key < tr[p].key) p = tr[p].l;
        else if (tr[p].key < key) p = tr[p].r;
        else return tr[p].cnt;
    }
    return 0;
}

```

```
int askRank(const Key &key) const {
    int p = rt, ans = 1;
    while (p) {
        if (!(tr[p].key < key)) {
            p = tr[p].l;
        } else {
            ans += sz(tr[p].l) + tr[p].cnt;
            p = tr[p].r;
        }
    }
    return ans;
}

optional<Key> askKth(int k) const {
    if (k <= 0 || k > askSz()) return nullopt;
    int p = rt;
    while (p) {
        int nl = sz(tr[p].l);
        if (k <= nl) p = tr[p].l;
        else if (k <= nl + tr[p].cnt) return tr[p].key;
        else k -= nl + tr[p].cnt, p = tr[p].r;
    }
    return nullopt;
}

optional<Key> askPre(const Key &key) const {
    int p = rt, ans = 0;
    bool ok = false;
    while (p) {
        if (tr[p].key < key) ans = p, ok = true, p = tr[p].r;
        else p = tr[p].l;
    }
    return ok ? optional<Key>(tr[ans].key) : nullopt;
}

optional<Key> askNxt(const Key &key) const {
    int p = rt, ans = 0;
    bool ok = false;
    while (p) {
        if (key < tr[p].key) ans = p, ok = true, p = tr[p].l;
        else p = tr[p].r;
    }
    return ok ? optional<Key>(tr[ans].key) : nullopt;
}
};
```

10B - 隐式 FHQTreap (序列) .cpp

173 lines · C++ · 03 - 数据结构

/*

用途：隐式 FHQ Treap，维护可修改序列。

下标均为 0-indexed，区间为半开 $[l, r)$ 。支持插入、删除、区间加、区间翻转、区间和与访问单点。

T 通常为 int；所有操作期望 $O(\log n)$ 。删除节点不回收，适合竞赛一次性使用。

*/

```
template <class T = int>
struct FHQTreap {
    struct Node {
        int l = 0, r = 0;
        int sz = 0;
        int pri = 0;
        T v{}, sum{}, add{};
        bool rev = false;
    };

    vector<Node> tr = {Node{}};
    int rt = 0;
    mt19937 rng{chrono::steady_clock::now().time_since_epoch().count()};

    int sz(int p) const {
        return p ? tr[p].sz : 0;
    }
    T sum(int p) const {
        return p ? tr[p].sum : T{};
    }

    int node(const T &v) {
        tr.push_back(Node{});
        int p = tr.size() - 1;
        tr[p].v = tr[p].sum = v;
        tr[p].sz = 1;
        tr[p].pri = rng();
        return p;
    }

    void pull(int p) {
        if (!p) return;
        tr[p].sz = sz(tr[p].l) + 1 + sz(tr[p].r);
        tr[p].sum = sum(tr[p].l) + tr[p].v + sum(tr[p].r);
    }

    void setAdd(int p, const T &v) {
        if (!p) return;
        tr[p].v += v;
        tr[p].sum += v * tr[p].sz;
        tr[p].add += v;
    }

    void rev(int p) {
        if (!p) return;
        swap(tr[p].l, tr[p].r);
        tr[p].rev = !tr[p].rev;
    }

    void push(int p) {
        if (!p) return;
        if (tr[p].add != T{}) {
            setAdd(tr[p].l, tr[p].add);

```

```

        setAdd(tr[p].r, tr[p].add);
        tr[p].add = T{};
    }
    if (tr[p].rev) {
        rev(tr[p].l);
        rev(tr[p].r);
        tr[p].rev = false;
    }
}

// a 为前 k 个元素, b 为其余元素。
void split(int p, int k, int &a, int &b) {
    if (!p) {
        a = b = 0;
        return;
    }
    push(p);
    if (sz(tr[p].l) >= k) {
        b = p;
        split(tr[p].l, k, a, tr[p].l);
        pull(b);
    } else {
        a = p;
        split(tr[p].r, k - sz(tr[p].l) - 1, tr[p].r, b);
        pull(a);
    }
}

int merge(int a, int b) {
    if (!a || !b) return a | b;
    if (tr[a].pri < tr[b].pri) {
        push(a);
        tr[a].r = merge(tr[a].r, b);
        pull(a);
        return a;
    }
    push(b);
    tr[b].l = merge(a, tr[b].l);
    pull(b);
    return b;
}

int askSz() const {
    return sz(rt);
}

void insert(int pos, const T &v) {
    assert(0 <= pos && pos <= askSz());
    int a, b;
    split(rt, pos, a, b);
    rt = merge(merge(a, node(v)), b);
}

void erase(int l, int r) {
    assert(0 <= l && l <= r && r <= askSz());
    int a, b, c;
    split(rt, r, a, c);
    split(a, l, a, b);
    rt = merge(a, c);
}

void update(int l, int r, const T &v) {
    assert(0 <= l && l <= r && r <= askSz());
    int a, b, c;

```

```
        split(rt, r, a, c);
        split(a, l, a, b);
        setAdd(b, v);
        rt = merge(merge(a, b), c);
    }

    void updRev(int l, int r) {
        assert(0 <= l && l <= r && r <= askSz());
        int a, b, c;
        split(rt, r, a, c);
        split(a, l, a, b);
        rev(b);
        rt = merge(merge(a, b), c);
    }

    T askSum(int l, int r) {
        assert(0 <= l && l <= r && r <= askSz());
        int a, b, c;
        split(rt, r, a, c);
        split(a, l, a, b);
        T ans = sum(b);
        rt = merge(merge(a, b), c);
        return ans;
    }

    optional<T> ask(int pos) {
        if (pos < 0 || pos >= askSz()) return nullopt;
        int a, b, c;
        split(rt, pos + 1, a, c);
        split(a, pos, a, b);
        T ans = tr[b].v;
        rt = merge(merge(a, b), c);
        return ans;
    }

    vector<T> askAll() {
        vector<T> res;
        res.reserve(askSz());
        auto dfs = [&](auto &&go, int p) -> void {
            if (!p) return;
            push(p);
            go(go, tr[p].l);
            res.push_back(tr[p].v);
            go(go, tr[p].r);
        };
        dfs(dfs, rt);
        return res;
    }
};
```

10C - PBDS有序多重集 (GNU环境) .cpp

70 lines · C++ · 03 - 数据结构

```

/*
用途：GNU PBDS 有序多重集；FHQ Treap 的 GNU 环境高性能替代实现。
限制：仅 GCC/libstdc++ 可用，Clang/libc++、MSVC 不可用，因此与可移植的 10A 同时保留。
排他和 kth 均为 1-indexed；重复键通过唯一编号区分。
性能：默认使用宏展开后的 int；若大规模数据因此超时或超空间，可将纯下标、状态或计数局部改为 i32。
*/
#ifdef int
#pragma push_macro("int")
#undef int
#define LYE_RESTORE_INT
#endif
#include <ext/pb_ds/assoc_container.hpp>
#include <ext/pb_ds/tree_policy.hpp>
#ifdef LYE_RESTORE_INT
#pragma pop_macro("int")
#undef LYE_RESTORE_INT
#endif

template <class Key> struct OSet {
    using P = pair<Key, int>;
    using Tree = __gnu_pbds::tree<P, __gnu_pbds::null_type, less<P>,
                                   __gnu_pbds::rb_tree_tag,
                                   __gnu_pbds::tree_order_statistics_node_update>;

    Tree t;
    int id = 0;

    int insert(const Key& x) {
        int k = ++id;
        t.insert({x, k});
        return k;
    }

    bool erase(const Key& x, int k) { return t.erase({x, k}); }

    bool erase(const Key& x) {
        auto it = t.lower_bound({x, 0});
        if (it == t.end() || it->first != x) return false;
        t.erase(it);
        return true;
    }

    int askSz() const { return t.size(); }
    bool askEmp() const { return t.empty(); }
    void clear() { t.clear(), id = 0; }
    int askCnt(const Key& x) const {
        return t.order_of_key({x, numeric_limits<int>::max()}) - t.order_of_key({x, 0});
    }
    int askRank(const Key& x) const { return t.order_of_key({x, 0}) + 1; }

    optional<Key> askKth(int k) const {
        if (k < 1 || k > askSz()) return nullopt;
        return t.find_by_order(k - 1)->first;
    }

    optional<Key> askPre(const Key& x) const {
        auto it = t.lower_bound({x, 0});
        if (it == t.begin()) return nullopt;
        return (--it)->first;
    }
}

```

```
optional<Key> askNxt(const Key& x) const {
    auto it = t.upper_bound({x, numeric_limits<int>::max()});
    return it == t.end() ? nullopt : optional<Key>(it->first);
}

optional<Key> askGE(const Key& x) const {
    auto it = t.lower_bound({x, 0});
    return it == t.end() ? nullopt : optional<Key>(it->first);
}
};
```

10D - vector有序多重集（小数据环境）.cpp

38 lines · C++ · 03 - 数据结构

```

/*
用途：用排序 vector 模拟有序多重集；接口与常见平衡树题一致。
环境：查询  $O(\log n)$ ，插入/删除  $O(n)$ ； $n$  较小、修改少或只想极短实现时使用。
*/
template <class T> struct OSet {
    vector<T> a;

    int askSz() const { return a.size(); }
    bool askEmp() const { return a.empty(); }
    void clear() { a.clear(); }

    void insert(const T &x) { a.insert(lower_bound(a.begin(), a.end(), x), x); }

    bool erase(const T &x) {
        auto p = lower_bound(a.begin(), a.end(), x);
        if (p == a.end() || *p != x) return false;
        a.erase(p);
        return true;
    }

    int askRank(const T &x) const { return lower_bound(a.begin(), a.end(), x) - a.begin() + 1; }
    int askCnt(const T &x) const { return upper_bound(a.begin(), a.end(), x) - lower_bound(a.begin(), a.end(), x); }

    optional<T> askKth(int k) const {
        if (k < 1 || k > askSz()) return nullopt;
        return a[k - 1];
    }

    optional<T> askPre(const T &x) const {
        auto p = lower_bound(a.begin(), a.end(), x);
        return p == a.begin() ? nullopt : optional<T>(*--p);
    }

    optional<T> askNxt(const T &x) const {
        auto p = upper_bound(a.begin(), a.end(), x);
        return p == a.end() ? nullopt : optional<T>(*p);
    }
};

```


10E - PBDS有序集合与映射 (GNU环境) .cpp

32 lines · C++ · 03 - 数据结构

```

/*
用途：GNU PBDS 唯一键有序集合/映射，支持排名与第 k 小；10C 另行处理重复键。
限制：仅 GCC/libstdc++ 可用。默认红黑树；切换 Tag 可用 splay_tree_tag / ov_tree_tag。

OST<T> 常用操作 (k 为 0-indexed)：
insert/erase/find/lower_bound/upper_bound
order_of_key(x)          严格小于 x 的元素数
find_by_order(k)         第 k 小迭代器，越界等于 end()
a.split(x, b)            a 保留 <= x, b 得到 > x
a.join(b)                两棵树键域必须严格分离，完成后 b 为空
OMT<K,V> 还支持 t[k]、find_by_order(k)->second。
禁止用 less_equal 模拟 multiset：它不满足严格弱序，会破坏 erase/join 等接口。
*/
#ifdef int
#pragma push_macro("int")
#undef int
#define LYE_RESTORE_INT
#endif
#include <ext/pb_ds/assoc_container.hpp>
#include <ext/pb_ds/tree_policy.hpp>
#ifdef LYE_RESTORE_INT
#pragma pop_macro("int")
#undef LYE_RESTORE_INT
#endif

template <class K, class Cmp = less<K>, class Tag = __gnu_pbds::rb_tree_tag>
using OST = __gnu_pbds::tree<K, __gnu_pbds::null_type, Cmp, Tag,
                             __gnu_pbds::tree_order_statistics_node_update>;

template <class K, class V, class Cmp = less<K>, class Tag = __gnu_pbds::rb_tree_tag>
using OMT = __gnu_pbds::tree<K, V, Cmp, Tag,
                             __gnu_pbds::tree_order_statistics_node_update>;

```

11A - KDTree (二维静态查询) .cpp

108 lines · C++ · 03 - 数据结构

```

/*
用途：二维静态 KD Tree，支持最近点（可排除一个编号）与闭矩形计数。
点下标为 build 时的 0-indexed 顺序。D 默认 i128，防止坐标差平方溢出 int。
平均查询接近  $O(\sqrt{n})$ ；最坏  $O(n)$ ，构造  $O(n \log n)$ 。
性能：默认使用宏展开后的 int；若大规模数据因此超时或超空间，可将纯下标、状态或计数局部改为 i32。
*/

using i128 = __int128_t;

template <class T = int, class D = i128> struct KDTree {
    struct Point {
        T x{}, y{};
    };
    struct Node {
        Point p;
        T mnx{}, mxx{}, mny{}, mxy{};
        int l = -1, r = -1, id = -1, sz = 1;
    };

    vector<pair<Point, int>> a;
    vector<Node> tr;
    int rt = -1;

    KDTree() = default;
    explicit KDTree(const vector<Point>& p) { build(p); }

    static D sq(T x, T y) {
        D d = D(x) - D(y);
        return d * d;
    }
    static D dis(const Point& a, const Point& b) { return sq(a.x, b.x) + sq(a.y, b.y); }

    void pull(int u) {
        Node& x = tr[u];
        x.mnx = x.mxx = x.p.x, x.mny = x.mxy = x.p.y, x.sz = 1;
        for (int v : {x.l, x.r}) if (v != -1) {
            x.mnx = min(x.mnx, tr[v].mnx), x.mxx = max(x.mxx, tr[v].mxx);
            x.mny = min(x.mny, tr[v].mny), x.mxy = max(x.mxy, tr[v].mxy);
            x.sz += tr[v].sz;
        }
    }

    int make(int l, int r, int d) {
        if (l >= r) return -1;
        int m = (l + r) >> 1;
        auto cmp = [d](const auto& x, const auto& y) {
            T a = d ? x.first.y : x.first.x;
            T b = d ? y.first.y : y.first.x;
            return a != b ? a < b : x.second < y.second;
        };
        nth_element(a.begin() + l, a.begin() + m, a.begin() + r, cmp);
        int u = tr.size();
        tr.push_back({a[m].first, a[m].first.x, a[m].first.x, a[m].first.y, a[m].first.y,
            -1, -1, a[m].second, 1});
        tr[u].l = make(l, m, d ^ 1);
        tr[u].r = make(m + 1, r, d ^ 1);
        pull(u);
        return u;
    }
}

```

```

void build(const vector<Point>& p) {
    a.clear(), tr.clear();
    a.reserve(p.size()), tr.reserve(p.size());
    for (int i = 0; i < p.size(); ++i) a.push_back({p[i], i});
    rt = make(0, a.size(), 0);
}

D box(int u, const Point& p) const {
    D r{};
    if (p.x < tr[u].mnx) r += sq(p.x, tr[u].mnx);
    if (p.x > tr[u].mxx) r += sq(p.x, tr[u].mxx);
    if (p.y < tr[u].mny) r += sq(p.y, tr[u].mny);
    if (p.y > tr[u].mxy) r += sq(p.y, tr[u].mxy);
    return r;
}

void near(int u, const Point& p, int ban, optional<D>& z) const {
    if (u == -1) return;
    if (tr[u].id != ban) {
        D d = dis(p, tr[u].p);
        if (!z || d < *z) z = d;
    }
    int x = tr[u].l, y = tr[u].r;
    D dx = x == -1 ? D(-1) : box(x, p);
    D dy = y == -1 ? D(-1) : box(y, p);
    if (y != -1 && (x == -1 || dy < dx)) swap(x, y), swap(dx, dy);
    if (x != -1 && (!z || dx <= *z)) near(x, p, ban, z);
    if (y != -1 && (!z || dy <= *z)) near(y, p, ban, z);
}

optional<D> askNear(const Point& p, int ban = -1) const {
    optional<D> r;
    near(rt, p, ban, r);
    return r;
}

int rect(int u, T x1, T y1, T x2, T y2) const {
    if (u == -1 || tr[u].mxx < x1 || x2 < tr[u].mnx || tr[u].mxy < y1 || y2 < tr[u].mny) return 0;
    if (x1 <= tr[u].mnx && tr[u].mxx <= x2 && y1 <= tr[u].mny && tr[u].mxy <= y2) return tr[u].sz;
    int r = x1 <= tr[u].p.x && tr[u].p.x <= x2 && y1 <= tr[u].p.y && tr[u].p.y <= y2;
    return r + rect(tr[u].l, x1, y1, x2, y2) + rect(tr[u].r, x1, y1, x2, y2);
}

int ask(T x1, T y1, T x2, T y2) const {
    if (x1 > x2 || y1 > y2) return 0;
    return rect(rt, x1, y1, x2, y2);
}
};

```

11B - KDTree (二维动态插入与偏序计数) .cpp

95 lines · C++ · 03 - 数据结构

```

/*
用途：二维动态 KD Tree，插入点并查询  $x \leq qx$  且  $y \leq qy$  的点数；重复点分别计数。
复杂度：替罪羊式失衡重构，均摊表现良好；单次查询最坏仍可能  $O(n)$ 。
注意：节点下标显式 int。
性能：默认使用宏展开后的 int；若大规模数据因此超时或超空间，可将纯下标、状态或计数局部改为 i32。
*/

template <class T = int> struct KDTree {
    struct Node {
        T x = 0, y = 0, ax = 0, bx = 0, ay = 0, by = 0;
        int l = 0, r = 0, sz = 1, d = 0;
    };

    vector<Node> t{{}};
    int rt = 0;

    int sz(int p) const { return p ? t[p].sz : 0; }

    void pull(int p) {
        auto &u = t[p];
        u.sz = 1;
        u.ax = u.bx = u.x;
        u.ay = u.by = u.y;
        for (int v : {u.l, u.r}) if (v) {
            u.sz += t[v].sz;
            u.ax = min(u.ax, t[v].ax);
            u.bx = max(u.bx, t[v].bx);
            u.ay = min(u.ay, t[v].ay);
            u.by = max(u.by, t[v].by);
        }
    }

    int build(vector<int> &a, int l, int r, int d = 0) {
        if (l >= r) return 0;
        int m = (l + r) >> 1;
        nth_element(a.begin() + l, a.begin() + m, a.begin() + r, [&](int p, int q) {
            if (!d) return pair(t[p].x, t[p].y) < pair(t[q].x, t[q].y);
            return pair(t[p].y, t[p].x) < pair(t[q].y, t[q].x);
        });
        int p = a[m];
        t[p].d = d;
        t[p].l = build(a, l, m, d ^ 1);
        t[p].r = build(a, m + 1, r, d ^ 1);
        pull(p);
        return p;
    }

    void flat(int p, vector<int> &a) {
        if (!p) return;
        flat(t[p].l, a);
        a.push_back(p);
        flat(t[p].r, a);
    }

    void rebuild(int &p) {
        int d = t[p].d;
        vector<int> a;
        a.reserve(t[p].sz);
        flat(p, a);
        p = build(a, 0, a.size(), d);
    }
};

```

```
    }

    bool bad(int p) const { return max(sz(t[p].l), sz(t[p].r)) * 4 > sz(p) * 3; }

    void ins(int &p, int q) {
        if (!p) {
            p = q;
            pull(p);
            return;
        }
        if ((!t[p].d && pair(t[q].x, t[q].y) < pair(t[p].x, t[p].y)) ||
            (t[p].d && pair(t[q].y, t[q].x) < pair(t[p].y, t[p].x)))
            ins(t[p].l, q);
        else
            ins(t[p].r, q);
        pull(p);
        if (bad(p)) rebuild(p);
    }

    void insert(T x, T y) {
        Node u;
        u.x = u.ax = u.bx = x;
        u.y = u.ay = u.by = y;
        t.push_back(u);
        ins(rt, t.size() - 1);
    }

    int ask0(int p, T x, T y) const {
        if (!p || x < t[p].ax || y < t[p].ay) return 0;
        if (t[p].bx <= x && t[p].by <= y) return t[p].sz;
        return (t[p].x <= x && t[p].y <= y) + ask0(t[p].l, x, y) + ask0(t[p].r, x, y);
    }

    int ask(T x, T y) const { return ask0(rt, x, y); }
};
```

12 - 珂朵莉树 (ODT, 随机区间赋值环境) .cpp

88 lines · C++ · 03 - 数据结构

```

/*
用途：维护分段常值序列，支持半开区间 [l,r) 赋值、加法、第 k 小与幂和。
前提：只有当区间赋值足够随机、区间数长期很少时才使用；不存在可靠通用最坏复杂度。
自动取模：askSum<M> 令 M 为 MInt 即可让中间过程自动取模。
*/

template <class T, class I = int> struct ODT {
    struct Node {
        I l, r;
        mutable T v;
        bool operator<(const Node &o) const { return l < o.l; }
    };

    using It = typename set<Node>::iterator;
    I n = 0;
    set<Node> s;

    ODT() = default;
    explicit ODT(const vector<T> &a) { build(a); }

    void build(const vector<T> &a) {
        n = a.size();
        s.clear();
        for (I l = 0, r; l < n; l = r) {
            for (r = l + 1; r < n && a[r] == a[l]; ++r) {}
            s.insert({l, r, a[l]});
        }
    }

    It split(I p) {
        if (p >= n) return s.end();
        auto q = prev(s.upper_bound({p, p, T{}}));
        if (q->l == p) return q;
        auto [l, r, v] = *q;
        s.erase(q);
        s.insert({l, p, v});
        return s.insert({p, r, v}).first;
    }

    void update(I l, I r, const T &x) {
        auto e = split(r), b = split(l);
        s.erase(b, e);
        auto p = s.insert({l, r, x}).first;
        if (p != s.begin()) {
            auto q = prev(p);
            if (q->r == p->l && q->v == p->v) {
                I nl = q->l;
                s.erase(q);
                s.erase(p);
                p = s.insert({nl, r, x}).first;
            }
        }
        auto q = next(p);
        if (q != s.end() && p->r == q->l && p->v == q->v) {
            I nl = p->l;
            I nr = q->r;
            s.erase(q);
            s.erase(p);
            s.insert({nl, nr, x});
        }
    }
}

```

```
    }

    void updAdd(I l, I r, const T &x) {
        auto e = split(r), b = split(l);
        for (auto p = b; p != e; ++p) p->v += x;
    }

    T askKth(I l, I r, I k) {
        auto e = split(r), b = split(l);
        vector<pair<T, I>> a;
        for (auto p = b; p != e; ++p) a.push_back({p->v, p->r - p->l});
        sort(a.begin(), a.end());
        for (auto [x, c] : a) if ((k -= c) <= 0) return x;
        throw out_of_range("ODT::askKth");
    }

    template <class M> M askSum(I l, I r, int e) {
        auto qp = [](M a, int b) {
            M z = 1;
            for (; b; b >>= 1, a *= a) if (b & 1) z *= a;
            return z;
        };
        auto q = split(r), b = split(l);
        M z = 0;
        for (auto p = b; p != q; ++p) z += qp(M(p->v), e) * M(p->r - p->l);
        return z;
    }
};
```

13A - PBDS开放寻址哈希表 (gp_hash_table, GNU环境) .cpp

49 lines · C++ · 03 - 数据结构

```

/*
用途：GNU PBDS gp_hash_table；开放寻址，常数小，适合大量唯一键的映射/集合。
限制：仅 GCC/libstdc++ 可用；增删可能使全部迭代器失效，不要长期保存迭代器。

HMap<K,V>：h[k]、insert、find、erase、size、empty、clear。
HSet<K>：insert、find、erase、size、empty、clear；不允许重复键。
默认 resize policy 不开放手动 resize/reserve，不能照搬 unordered_map::reserve。
Hash 对整数、字符串及 pair 均可用，并加入运行时盐以避免构造碰撞。
*/

using u64 = uint64_t;

#ifdef int
#pragma push_macro("int")
#undef int
#define LYE_RESTORE_INT
#endif
#include <ext/pb_ds/assoc_container.hpp>
#ifdef LYE_RESTORE_INT
#pragma pop_macro("int")
#undef LYE_RESTORE_INT
#endif

struct Hash {
    inline static const u64 r =
        chrono::steady_clock::now().time_since_epoch().count();

    static u64 mix(u64 x) {
        x += 0x9e3779b97f4a7c15ULL;
        x = (x ^ (x >> 30)) * 0xbf58476d1ce4e5b9ULL;
        x = (x ^ (x >> 27)) * 0x94d049bb133111ebULL;
        return x ^ (x >> 31);
    }

    template <class T> size_t operator()(const T &x) const {
        return mix(u64(hash<T>{}(x)) + r);
    }

    template <class A, class B> size_t operator()(const pair<A, B> &x) const {
        u64 a = (*this)(x.first), b = (*this)(x.second);
        return mix(a ^ (b + 0x9e3779b97f4a7c15ULL + (a << 6) + (a >> 2)));
    }
};

template <class K, class V, class H = Hash, class Eq = equal_to<K>>
using HMap = __gnu_pbds::gp_hash_table<K, V, H, Eq>;

template <class K, class H = Hash, class Eq = equal_to<K>>
using HSet = __gnu_pbds::gp_hash_table<K, __gnu_pbds::null_type, H, Eq>;

```


13B - PBDS拉链哈希表 (cc_hash_table, GNU环境) .cpp

48 lines · C++ · 03 - 数据结构

```

/*
用途：GNU PBDS cc_hash_table；拉链法哈希，适合希望避免开放寻址聚簇或存较大值的环境。
限制：仅 GCC/libstdc++ 可用；通常比 13A 多指针和节点分配开销，优先实测后选型。

HMap<K,V>: h[k]、insert、find、erase、size、empty、clear。
HSet<K>: insert、find、erase、size、empty、clear；不允许重复键。
13A/13B 是同一哈希字典问题的不同实现，通常只粘贴其中一个。
*/

using u64 = uint64_t;

#ifdef int
#pragma push_macro("int")
#undef int
#define LYE_RESTORE_INT
#endif
#include <ext/pb_ds/assoc_container.hpp>
#ifdef LYE_RESTORE_INT
#pragma pop_macro("int")
#undef LYE_RESTORE_INT
#endif

struct Hash {
    inline static const u64 r =
        chrono::steady_clock::now().time_since_epoch().count();

    static u64 mix(u64 x) {
        x += 0x9e3779b97f4a7c15ULL;
        x = (x ^ (x >> 30)) * 0xbf58476d1ce4e5b9ULL;
        x = (x ^ (x >> 27)) * 0x94d049bb133111ebULL;
        return x ^ (x >> 31);
    }

    template <class T> size_t operator()(const T &x) const {
        return mix(u64(hash<T>{}(x)) + r);
    }

    template <class A, class B> size_t operator()(const pair<A, B> &x) const {
        u64 a = (*this)(x.first), b = (*this)(x.second);
        return mix(a ^ (b + 0x9e3779b97f4a7c15ULL + (a << 6) + (a >> 2)));
    }
};

template <class K, class V, class H = Hash, class Eq = equal_to<K>>
using HMap = __gnu_pbds::cc_hash_table<K, V, H, Eq>;

template <class K, class H = Hash, class Eq = equal_to<K>>
using HSet = __gnu_pbds::cc_hash_table<K, __gnu_pbds::null_type, H, Eq>;

```

00A - 图遍历 (BFS、DFS、二分图判定) .cpp

104 lines · C++ · 04 - 图论

```

/*
用途：无权图的 BFS/迭代 DFS；另附无向图二分图染色。
约定：点编号 1..n。Graph 可按需加有向边或无向边；
Bipartite 只应加入无向边。BFS 的 dis 为 -1 表示不可达。
复杂度：每次遍历或染色  $O(n + m)$ 。
*/
struct Graph {
    struct BFSResult {
        vector<int> dis;
        vector<int> fa;
        vector<int> ord;
    };

    int n;
    vector<vector<int>> g;

    explicit Graph(int n) : n(n), g(n + 1) {}

    void add(int u, int v) {
        g[u].push_back(v);
    }

    void add2(int u, int v) {
        add(u, v);
        add(v, u);
    }

    BFSResult bfs(int s) const {
        BFSResult res{vector<int>(n + 1, -1), vector<int>(n + 1, -1), {}};
        queue<int> q;
        res.dis[s] = 0;
        q.push(s);
        while (!q.empty()) {
            int u = q.front();
            q.pop();
            res.ord.push_back(u);
            for (int v : g[u]) {
                if (res.dis[v] != -1) continue;
                res.dis[v] = res.dis[u] + 1;
                res.fa[v] = u;
                q.push(v);
            }
        }
        return res;
    }

    // 显式栈版先序 DFS，避免链状图递归爆栈。
    vector<int> dfs(int s) const {
        vector<char> vis(n + 1, false);
        vector<int> it(n + 1), ord;
        vector<int> stk = {s};
        vis[s] = true;
        ord.push_back(s);
        while (!stk.empty()) {
            int u = stk.back();
            if (it[u] == g[u].size()) {
                stk.pop_back();
                continue;
            }
            int v = g[u][it[u]++];

```

```
        if (vis[v]) continue;
        vis[v] = true;
        ord.push_back(v);
        stk.push_back(v);
    }
    return ord;
}
};

struct Bipartite {
    int n;
    vector<vector<int>> g;

    explicit Bipartite(int n) : n(n), g(n + 1) {}

    void add(int u, int v) {
        g[u].push_back(v);
        g[v].push_back(u);
    }

    // color[u] 为 0/1; 返回 nullopt 表示含奇环。
    optional<vector<int>> solve() const {
        vector<int> col(n + 1, -1);
        for (int s = 1; s <= n; ++s) {
            if (col[s] != -1) continue;
            queue<int> q;
            q.push(s);
            col[s] = 0;
            while (!q.empty()) {
                int u = q.front();
                q.pop();
                for (int v : g[u]) {
                    if (col[v] == -1) {
                        col[v] = col[u] ^ 1;
                        q.push(v);
                    } else if (col[v] == col[u]) {
                        return nullopt;
                    }
                }
            }
        }
        return col;
    }
};
```

00B - 链式前向星（大图环境）.cpp

32 lines · C++ · 04 - 图论

```

/*
用途：大图的连续内存邻接表；与 vector<vector<Edge>> 是同一存图问题的内存/缓存特化分支。
点编号 0..n-1，head[u] 起沿 edge[i].next 遍历。预先 reserve(m) 可避免扩容。
性能：默认使用宏展开后的 int；若大规模数据因此超时或超空间，可将纯下标、状态或计数局部改为 i32。
*/

template <class W = int> struct Graph {
    struct Edge {
        int v, nxt;
        W w;
    };
    int n;
    vector<int> hd;
    vector<Edge> e;

    explicit Graph(int n, int m = 0) : n(n), hd(n, -1) {
        assert(n >= 0 && m >= 0);
        e.reserve(m);
    }

    int add(int u, int v, W w = W(1)) {
        assert(0 <= u && u < n && 0 <= v && v < n);
        e.push_back({v, hd[u], w});
        return hd[u] = e.size() - 1;
    }

    array<int, 2> add2(int u, int v, W w = W(1)) {
        int a = add(u, v, w);
        int b = add(v, u, w);
        return {a, b};
    }
};

```

01A - 单源最短路 (Dijkstra) (无负权) .cpp

53 lines · C++ · 04 - 图论

```

/*
用途：非负边权单源最短路。
用法：Dijkstra<T> g(n)；若权值类型不同写 Dijkstra<T>。
约定：点号和父亲数组使用 int，距离类型由模板 T 决定。
性能：默认使用宏展开后的 int；若大规模数据因此超时或超空间，可将纯下标、状态或计数局部改为 i32。
*/

template <class T = int> struct Dijkstra {
    static constexpr T INF = numeric_limits<T>::max() / 4;

    struct Edge {
        int v;
        T w;
    };

    int n;
    vector<vector<Edge>> g;

    explicit Dijkstra(int n) : n(n), g(n + 1) {
        assert(n >= 0);
    }

    void add(int u, int v, T w) {
        assert(w >= 0);
        g[u].push_back({v, w});
    }

    void add2(int u, int v, T w) {
        add(u, v, w);
        add(v, u, w);
    }

    vector<T> solve(int s, vector<int>* fa = nullptr) const {
        vector<T> dis(n + 1, INF);
        if (fa) fa->assign(n + 1, -1);
        using S = pair<T, int>;
        priority_queue<S, vector<S>, greater<S>> q;
        dis[s] = 0;
        q.push({0, s});
        while (!q.empty()) {
            auto [d, u] = q.top();
            q.pop();
            if (d != dis[u]) continue;
            for (auto [v, w] : g[u]) {
                if (dis[v] <= d + w) continue;
                dis[v] = d + w;
                if (fa) (*fa)[v] = u;
                q.push({dis[v], v});
            }
        }
        return dis;
    }
};

```

01B - 单源最短路 (Bellman-ford, 负权图) .cpp

72 lines · C++ · 04 - 图论

```

/*
用途：允许负边权的单源最短路，并标记从源点可达的负环所能影响的点。
约定：点编号 1..n，边均为有向边；所有有限距离应能放进 int。
复杂度：O(nm)。负环不影响的点，其 distance 才有最短路意义。
*/

struct Bellman {
    static constexpr int INF = numeric_limits<int>::max() / 4;

    struct Edge {
        int u, v;
        int w;
    };

    struct Result {
        vector<int> dis;
        vector<char> neg;

        bool askNeg() const {
            return any_of(neg.begin() + 1,
                          neg.end(), [](char v) { return v; });
        }
    };

    int n;
    vector<Edge> e;
    vector<vector<int>> g;

    explicit Bellman(int n) : n(n), g(n + 1) {}

    void add(int u, int v, int w) {
        e.push_back({u, v, w});
        g[u].push_back(v);
    }

    Result solve(int s) const {
        vector<int> dis(n + 1, INF);
        dis[s] = 0;

        for (int k = 1; k < n; ++k) {
            bool chg = false;
            for (auto [u, v, w] : e) {
                if (dis[u] == INF) continue;
                if (dis[v] > dis[u] + w) {
                    dis[v] = dis[u] + w;
                    chg = true;
                }
            }
            if (!chg) break;
        }

        vector<char> neg(n + 1, false);
        queue<int> q;
        for (auto [u, v, w] : e) {
            if (dis[u] != INF && dis[v] > dis[u] + w && !neg[v]) {
                neg[v] = true;
                q.push(v);
            }
        }
        while (!q.empty()) {

```

```
        int u = q.front();
        q.pop();
        for (int v : g[u]) {
            if (!neg[v]) {
                neg[v] = true;
                q.push(v);
            }
        }
    }
    return {move(dis), move(neg)};
};
```

01C - 单源最短路 (SPFA, 负权图) .cpp

53 lines · C++ · 04 - 图论

```

/*
用途：允许负边权的单源最短路；若存在源点可达负环则返回 nullopt。
约定：点编号 1..n，边均为有向边；最坏复杂度  $O(nm)$ ，通常仅在数据温和时使用。
所有有限距离应能放进 int。
*/

struct SPFA {
    static constexpr int INF = numeric_limits<int>::max() / 4;

    struct Edge {
        int v;
        int w;
    };

    int n;
    vector<vector<Edge>> g;

    explicit SPFA(int n) : n(n), g(n + 1) {}

    void add(int u, int v, int w) {
        g[u].push_back({v, w});
    }

    // 返回 nullopt 表示存在从 source 可达的负环。
    optional<vector<int>> solve(int s) const {
        vector<int> dis(n + 1, INF);
        vector<int> m(n + 1);
        vector<char> inq(n + 1, false);
        queue<int> q;

        dis[s] = 0;
        q.push(s);
        inq[s] = true;

        while (!q.empty()) {
            int u = q.front();
            q.pop();
            inq[u] = false;

            for (auto [v, w] : g[u]) {
                if (dis[v] <= dis[u] + w) continue;
                dis[v] = dis[u] + w;
                m[v] = m[u] + 1;
                if (m[v] >= n) return nullopt;
                if (!inq[v]) {
                    inq[v] = true;
                    q.push(v);
                }
            }
        }
        return dis;
    }
};

```


01D - 多源最短路 (APSP) .cpp

44 lines · C++ · 04 - 图论

```

/*
用途：Floyd-Warshall 全源最短路。
约定：点编号 1..n；允许负边权，但不允许负环。调用 solve 后 dis[i][j] 即答案。
复杂度：O(n^3) 时间、O(n^2) 空间，适合 n 约 500 以内的稠密图。
*/

struct Floyd {
    static constexpr int INF = numeric_limits<int>::max() / 4;

    int n;
    vector<vector<int>> dis;

    explicit Floyd(int n) : n(n), dis(n + 1, vector<int>(n + 1, INF)) {
        for (int i = 1; i <= n; ++i) dis[i][i] = 0;
    }

    void add(int u, int v, int w) {
        dis[u][v] = min(dis[u][v], w);
    }

    void add2(int u, int v, int w) {
        add(u, v, w);
        add(v, u, w);
    }

    void solve() {
        for (int k = 1; k <= n; ++k) {
            for (int i = 1; i <= n; ++i) {
                if (dis[i][k] == INF) continue;
                for (int j = 1; j <= n; ++j) {
                    if (dis[k][j] == INF) continue;
                    dis[i][j] = min(dis[i][j], dis[i][k] + dis[k][j]);
                }
            }
        }

        bool askNeg() const {
            for (int i = 1; i <= n; ++i) {
                if (dis[i][i] < 0) return true;
            }
            return false;
        }
    };
};

```

01E - 0-1 BFS.cpp

51 lines · C++ · 04 - 图论

```

/*
用途：边权只为 0 或 1 时的单源最短路，比 Dijkstra 更轻。
约定：点编号 1..n；add 加有向边，add2 加无向边。
复杂度：O(n + m)。
*/

struct ZeroOneBFS {
    static constexpr int INF = numeric_limits<int>::max() / 4;

    struct Edge {
        int v;
        int w;
    };

    int n;
    vector<vector<Edge>> g;

    explicit ZeroOneBFS(int n) : n(n), g(n + 1) {}

    void add(int u, int v, int w) {
        assert(w == 0 || w == 1);
        g[u].push_back({v, w});
    }

    void add2(int u, int v, int w) {
        add(u, v, w);
        add(v, u, w);
    }

    vector<int> solve(int s) const {
        vector<int> dis(n + 1, INF);
        deque<int> q;
        dis[s] = 0;
        q.push_back(s);

        while (!q.empty()) {
            int u = q.front();
            q.pop_front();
            for (auto [v, w] : g[u]) {
                if (dis[v] <= dis[u] + w) continue;
                dis[v] = dis[u] + w;
                if (w == 0) {
                    q.push_front(v);
                } else {
                    q.push_back(v);
                }
            }
        }
        return dis;
    }
};

```

01F - 差分约束.cpp

51 lines · C++ · 04 - 图论

```

/*
用途：差分约束系统。addLE(u,v,w) 表示  $x[v]-x[u]\leq w$ ，对应边  $u\rightarrow v$  权  $w$ 。
solve 返回一组可行解；存在矛盾（负环）时返回 nullopt。变量编号  $0..n-1$ 。
复杂度：SPFA 平均较快，最坏  $O(nm)$ ；对抗数据可改用 Bellman-Ford。
性能：默认使用宏展开后的 int；若大规模数据因此超时或超空间，可将纯下标、状态或计数局部改为 i32。
*/

template <class T = int> struct DiffCons {
    struct Edge {
        int v;
        T w;
    };

    int n;
    vector<vector<Edge>> g;
    explicit DiffCons(int n) : n(n), g(n) { assert(n >= 0); }

    void addLE(int u, int v, T w) {
        assert(0 <= u && u < n && 0 <= v && v < n);
        g[u].push_back({v, w});
    }
    void addGE(int u, int v, T w) { addLE(v, u, -w); }
    void addEQ(int u, int v, T w) {
        addLE(u, v, w);
        addGE(u, v, w);
    }

    optional<vector<T>> solve() const {
        vector<T> d(n, T{});
        vector<int> len(n), q;
        vector<char> in(n, true);
        q.reserve(n * 2);
        for (int i = 0; i < n; ++i) q.push_back(i); // 等价于权 0 超级源。
        for (int h = 0; h < q.size(); ++h) {
            int u = q[h];
            in[u] = false;
            for (auto [v, w] : g[u]) {
                if (d[v] <= d[u] + w) continue;
                d[v] = d[u] + w;
                len[v] = len[u] + 1;
                if (len[v] >= n) return nullopt;
                if (!in[v]) in[v] = true, q.push_back(v);
            }
            if (h > 2 * n && h * 2 > q.size()) {
                q.erase(q.begin(), q.begin() + h + 1);
                h = -1;
            }
        }
        return d;
    }
};

```

01G - Johnson全源最短路（稀疏负权图）.cpp

55 lines · C++ · 04 - 图论

```

/*
用途：稀疏图全源最短路，允许负边；有负环时返回 nullopt。
复杂度： $O(nm+n(m+n)\log n)$ ，空间  $O(n^2+m)$ 。点号  $1..n$ 。
约定： $T$  为有符号距离类型；边权和任意简单路长度应落在  $T$  的安全范围内。
性能：默认使用宏展开后的 int；若大规模数据因此超时或超空间，可将纯下标、状态或计数局部改为 i32。
*/

template <class T = int> struct Johnson {
    static constexpr T INF = numeric_limits<T>::max() / 4;
    struct E { int u, v; T w; };
    int n;
    vector<E> e;
    vector<vector<pair<int, T>>> g;

    explicit Johnson(int _n) : n(_n), g(_n + 1) {
        static_assert(is_signed_v<T>);
        assert(n >= 0);
    }

    void add(int u, int v, T w) {
        assert(1 <= u && u <= n && 1 <= v && v <= n);
        e.push_back({u, v, w});
    }

    optional<vector<vector<T>>> solve() {
        vector<T> h(n + 1);
        for (int k = 1; k <= n; ++k) {
            bool f = false;
            for (auto [u, v, w] : e) if (h[v] > h[u] + w)
                h[v] = h[u] + w, f = true;
            if (!f) break;
            if (k == n) return nullopt;
        }
        g.assign(n + 1, {});
        for (auto [u, v, w] : e) {
            T x = w + h[u] - h[v];
            assert(x >= 0);
            g[u].push_back({v, x});
        }

        vector<vector<T>> a(n + 1, vector<T>(n + 1, INF));
        for (int s = 1; s <= n; ++s) {
            vector<T> d(n + 1, INF);
            priority_queue<pair<T, int>, vector<pair<T, int>>, greater<pair<T, int>>> q;
            d[s] = 0, q.push({0, s});
            while (!q.empty()) {
                auto [du, u] = q.top(); q.pop();
                if (du != d[u]) continue;
                for (auto [v, w] : g[u]) if (du <= INF - w && d[v] > du + w)
                    d[v] = du + w, q.push({d[v], v});
            }
            for (int v = 1; v <= n; ++v) if (d[v] != INF) a[s][v] = d[v] - h[s] + h[v];
        }
        return a;
    }
};

```

02A - 最小生成树 (Kruskal) .cpp

70 lines · C++ · 04 - 图论

```

/*
用途：无向带权图的最小生成森林；ok 为真时即最小生成树。
约定：点编号 1..n，可含重边和负边权。
复杂度：O(m log m)。
*/

struct MST {

    struct Edge {
        int u, v;
        int w;
    };

    struct Result {
        bool ok;
        int w; // 非连通时为最小生成森林的权值。
        vector<Edge> vis; // 非连通时为最小生成森林的边。
    };

    struct DSU {
        vector<int> fa, sz;

        explicit DSU(int n) : fa(n + 1), sz(n + 1, 1) {
            iota(fa.begin(), fa.end(), 0);
        }

        int find(int x) {
            while (x != fa[x]) {
                x = fa[x] = fa[fa[x]];
            }
            return x;
        }

        bool merge(int x, int y) {
            x = find(x);
            y = find(y);
            if (x == y) return false;
            if (sz[x] < sz[y]) swap(x, y);
            fa[y] = x;
            sz[x] += sz[y];
            return true;
        }
    };

    int n;
    vector<Edge> e;

    explicit MST(int n) : n(n) {}

    void add(int u, int v, int w) {
        e.push_back({u, v, w});
    }

    Result solve() const {
        vector<Edge> ord = e;
        sort(ord.begin(), ord.end(), [](const Edge& a, const Edge& b) {
            return a.w < b.w;
        });

        DSU d(n);

```

```
    int sum = 0;
    vector<Edge> vis;
    for (const Edge& x : ord) {
        if (!d.merge(x.u, x.v)) continue;
        sum += x.w;
        vis.push_back(x);
    }
    return {vis.size() == n - 1, sum, move(vis)};
};
```

02B - 最小生成树 (Prim) .cpp

62 lines · C++ · 04 - 图论

```

/*
用途：无向带权图的最小生成森林；ok 为真时即最小生成树。
约定：点编号 1..n，可含重边和负边权。
复杂度：O(m log n)。稠密图通常可改为 O(n^2) 朴素 Prim。
*/

struct MST {

    struct Edge {
        int v;
        int w;
    };

    struct CEdge {
        int u, v;
        int w;
    };

    struct Result {
        bool ok;
        int w; // 非连通时为最小生成森林的权值。
        vector<CEdge> vis; // 非连通时为最小生成森林的边。
    };

    int n;
    vector<vector<Edge>> g;

    explicit MST(int n) : n(n), g(n + 1) {}

    void add(int u, int v, int w) {
        g[u].push_back({v, w});
        g[v].push_back({u, w});
    }

    Result solve() const {
        using State = tuple<int, int, int>; // weight, vertex, parent
        priority_queue<State, vector<State>, greater<State>> q;
        vector<char> use(n + 1, false);
        vector<CEdge> vis;
        int sum = 0;
        int cc = 0;

        for (int s = 1; s <= n; ++s) {
            if (use[s]) continue;
            ++cc;
            q.push({0, s, 0});
            while (!q.empty()) {
                auto [w, u, fa] = q.top();
                q.pop();
                if (use[u]) continue;
                use[u] = true;
                sum += w;
                if (fa != 0) vis.push_back({fa, u, w});

                for (auto [v, w] : g[u]) {
                    if (!use[v]) q.push({w, v, u});
                }
            }
        }

        return {cc <= 1, sum, move(vis)};
    }
};

```

```
    }  
};
```


02C - 有向最小生成树 (Chu-Liu-Edmonds) .cpp

63 lines · C++ · 04 - 图论

```

/*
用途：有向图以 root 为根的最小树形图（每个非根点恰有一条入边，且均从根可达）。
约定：点编号 0..n-1，允许负权和重边；返回 nullopt 表示不存在。仅返回最小权值。
复杂度：O(nm)。
性能：默认使用宏展开后的 int；若大规模数据因此超时或超空间，可将纯下标、状态或计数局部改为 i32。
*/

template <class T = int> struct DMST {
    struct Edge {
        int u, v;
        T w;
    };

    int n;
    vector<Edge> e;
    explicit DMST(int n) : n(n) { assert(n >= 0); }

    void add(int u, int v, T w) {
        assert(0 <= u && u < n && 0 <= v && v < n);
        e.push_back({u, v, w});
    }

    optional<T> solve(int rt) const {
        assert(0 <= rt && rt < n);
        const T inf = numeric_limits<T>::max() / 4;
        int z = n, r = rt;
        vector<Edge> a = e;
        T ans{};
        while (true) {
            vector<T> in(z, inf);
            vector<int> pre(z, -1);
            for (auto [u, v, w] : a) {
                if (u != v && w < in[v]) in[v] = w, pre[v] = u;
            }
            in[r] = T{};
            for (int i = 0; i < z; ++i) {
                if (in[i] == inf) return nullopt;
                ans += in[i];
            }

            int cnt = 0;
            vector<int> id(z, -1), vis(z, -1);
            for (int i = 0; i < z; ++i) {
                int v = i;
                while (vis[v] != i && id[v] == -1 && v != r) vis[v] = i, v = pre[v];
                if (v == r || id[v] != -1) continue;
                for (int u = pre[v]; u != v; u = pre[u]) id[u] = cnt;
                id[v] = cnt++;
            }
            if (!cnt) break;
            for (int i = 0; i < z; ++i) if (id[i] == -1) id[i] = cnt++;

            vector<Edge> b;
            b.reserve(a.size());
            for (auto [u, v, w] : a) {
                int x = id[u], y = id[v];
                if (x != y) b.push_back({x, y, w - in[v]});
            }
            r = id[r], z = cnt, a.swap(b);
        }
    }
};

```

```
        return ans;  
    }  
};
```

03A - 拓扑排序 (Kahn) .cpp

37 lines · C++ · 04 - 图论

```
/*
用途：有向无环图 (DAG) 的拓扑排序与判环。
约定：点编号 1..n，add 添加有向边。
复杂度：O(n + m)。solve() 返回 nullopt 表示图中存在有向环。
*/
struct Topo {
    int n;
    vector<vector<int>> g;
    vector<int> in;

    explicit Topo(int n) : n(n), g(n + 1), in(n + 1) {}

    void add(int u, int v) {
        g[u].push_back(v);
        ++in[v];
    }

    optional<vector<int>> solve() const {
        vector<int> deg = in;
        queue<int> q;
        for (int u = 1; u <= n; ++u) {
            if (deg[u] == 0) q.push(u);
        }

        vector<int> ord;
        while (!q.empty()) {
            int u = q.front();
            q.pop();
            ord.push_back(u);
            for (int v : g[u]) {
                if (--deg[v] == 0) q.push(v);
            }
        }
        if (ord.size() != n) return nullopt;
        return ord;
    }
};
```

03B - 强连通分量 (Tarjan) .cpp

74 lines · C++ · 04 - 图论

```

/*
用途：有向图强连通分量、缩点 DAG。
约定：点编号 1..n，add 添加有向边。bel 编号为 0..cc-1，
编号顺序不承诺拓扑性质；dag 是去重后的缩点图。
复杂度：O(n + m)。
*/
struct SCC {
    struct Result {
        int cc;
        vector<int> bel;
        vector<vector<int>> scc;
        vector<vector<int>> dag;
    };

    int n;
    vector<vector<int>> g;

    explicit SCC(int n) : n(n), g(n + 1) {}

    void add(int u, int v) {
        g[u].push_back(v);
    }

    Result solve() const {
        vector<int> dfn(n + 1), low(n + 1), stk, bel(n + 1, -1);
        vector<char> ins(n + 1, false);
        vector<vector<int>> scc;
        int tim = 0;

        auto dfs = [&](auto&& go, int u) -> void {
            dfn[u] = low[u] = ++tim;
            stk.push_back(u);
            ins[u] = true;

            for (int v : g[u]) {
                if (!dfn[v]) {
                    go(go, v);
                    low[u] = min(low[u], low[v]);
                } else if (ins[v]) {
                    low[u] = min(low[u], dfn[v]);
                }
            }

            if (dfn[u] != low[u]) return;
            int id = scc.size();
            scc.push_back({});
            while (true) {
                int v = stk.back();
                stk.pop_back();
                ins[v] = false;
                bel[v] = id;
                scc.back().push_back(v);
                if (v == u) break;
            }
        };

        for (int u = 1; u <= n; ++u) {
            if (!dfn[u]) dfs(dfs, u);
        }
    }
};

```

```
vector<vector<int>> dag(scc.size());
for (int u = 1; u <= n; ++u) {
    for (int v : g[u]) {
        int fr = bel[u], to = bel[v];
        if (fr != to) dag[fr].push_back(to);
    }
}
for (auto& e : dag) {
    sort(e.begin(), e.end());
    e.erase(unique(e.begin(), e.end()), e.end());
}
return {scc.size(), move(bel), move(scc), move(dag)};
}
};
```

03C - 2-SAT.cpp

83 lines · C++ · 04 - 图论

```

/*
用途：布尔可满足性（每个子句至多两个文字）。
约定：变量编号 0..m-1；lit(x,true) 表示 x，lit(x,false) 表示 !x。
接口：add2(x,vx,y,vy) 添加 (x == vx) OR (y == vy)，force(x,vx) 强制取值。
复杂度：O(V + E)。solve() 返回 nullopt 表示无解，否则 ans[x] 是变量 x 的取值。
*/
struct TwoSAT {
    int m;
    vector<vector<int>> g;

    explicit TwoSAT(int m) : m(m), g(2 * m) {}

    int lit(int x, bool val) const {
        assert(0 <= x && x < m);
        return 2 * x + (val ? 0 : 1);
    }

    static int neg(int id) {
        return id ^ 1;
    }

    void add(int fr, int to) {
        g[fr].push_back(to);
    }

    void addOr(int a, int b) {
        add(neg(a), b);
        add(neg(b), a);
    }

    void add2(int x, bool vx, int y, bool vy) {
        addOr(lit(x, vx), lit(y, vy));
    }

    void force(int x, bool val) {
        int a = lit(x, val);
        addOr(a, a);
    }

    optional<vector<char>> solve() const {
        const int n = 2 * m;
        vector<int> dfn(n), low(n), stk, bel(n, -1);
        vector<char> ins(n, false);
        int tim = 0, cc = 0;

        auto dfs = [&](auto&& go, int u) -> void {
            dfn[u] = low[u] = ++tim;
            stk.push_back(u);
            ins[u] = true;
            for (int v : g[u]) {
                if (!dfn[v]) {
                    go(go, v);
                    low[u] = min(low[u], low[v]);
                } else if (ins[v]) {
                    low[u] = min(low[u], dfn[v]);
                }
            }
            if (dfn[u] != low[u]) return;
            while (true) {
                int v = stk.back();

```

```
        stk.pop_back();
        ins[v] = false;
        bel[v] = cc;
        if (v == u) break;
    }
    ++cc;
};

for (int u = 0; u < n; ++u) {
    if (!dfn[u]) dfs(dfs, u);
}

vector<char> ans(m);
for (int x = 0; x < m; ++x) {
    int vx = lit(x, true);
    int vy = lit(x, false);
    if (bel[vx] == bel[vy]) return nullopt;
    // Tarjan 的分量按出边方向从后向前编号，编号更小者应优先取真。
    ans[x] = bel[vx] < bel[vy];
}
return ans;
}
};
```

04A - 边双连通分量（桥）.cpp

81 lines · C++ · 04 - 图论

```

/*
用途：无向图找桥、删桥后的边双连通分量，以及桥树（非连通图时为桥森林）。
约定：点编号 1..n；支持重边，边编号为 add 的返回值（0..m-1）。
复杂度：O(n + m)。
*/
struct EBCC {
    struct Result {
        int cc;
        vector<int> bel;           // 点所属边双连通分量，编号 0..componentCount-1。
        vector<char> br;          // 对应 add 的边编号。
        vector<vector<int>> tr;    // 分量编号上的桥森林。
    };

    int n;
    vector<pair<int, int>> e;
    vector<vector<pair<int, int>>> g;

    explicit EBCC(int n) : n(n), g(n + 1) {}

    int add(int u, int v) {
        int id = e.size();
        e.push_back({u, v});
        g[u].push_back({v, id});
        g[v].push_back({u, id});
        return id;
    }

    Result solve() const {
        const int m = e.size();
        vector<int> dfn(n + 1), low(n + 1);
        vector<char> br(m, false);
        int tim = 0;

        auto dfs = [&](auto&& go, int u, int pe) -> void {
            dfn[u] = low[u] = ++tim;
            for (auto [v, id] : g[u]) {
                if (id == pe) continue;
                if (!dfn[v]) {
                    go(go, v, id);
                    low[u] = min(low[u], low[v]);
                    if (low[v] > dfn[u]) br[id] = true;
                } else {
                    low[u] = min(low[u], dfn[v]);
                }
            }
        };

        for (int u = 1; u <= n; ++u) {
            if (!dfn[u]) dfs(dfs, u, -1);
        }

        vector<int> bel(n + 1, -1);
        int cc = 0;
        for (int s = 1; s <= n; ++s) {
            if (bel[s] != -1) continue;
            stack<int> st;
            st.push(s);
            bel[s] = cc;
            while (!st.empty()) {
                int u = st.top();

```



```
        st.pop();
        for (auto [v, id] : g[u]) {
            if (br[id] || bel[v] != -1) continue;
            bel[v] = cc;
            st.push(v);
        }
    }
    ++cc;
}

vector<vector<int>> tr(cc);
for (int id = 0; id < m; ++id) {
    if (!br[id]) continue;
    auto [u, v] = e[id];
    int x = bel[u], y = bel[v];
    tr[x].push_back(y);
    tr[y].push_back(x);
}
return {cc, move(bel), move(br), move(tr)};
}
};
```

04B - 点双连通分量（割点、圆方树）.cpp

90 lines · C++ · 04 - 图论

```

/*
用途：无向图找割点、点双连通分量，并建立圆方树（block-cut tree）。
约定：点编号 1..n；支持重边；默认不放自环。原图非连通时 blockCutTree 为森林。
结果中原点仍为 1..n，第 i 个点双连通分量对应圆方树节点 n + i + 1。
复杂度：O(n + m)。
*/
struct VBCC {
    struct Edge {
        int u, v;
    };

    struct Result {
        vector<char> cut;
        vector<vector<int>> cc;
        vector<vector<int>> tr;
    };

    int n;
    vector<Edge> e;
    vector<vector<pair<int, int>>> g;

    explicit VBCC(int n) : n(n), g(n + 1) {}

    int add(int u, int v) {
        int id = e.size();
        e.push_back({u, v});
        g[u].push_back({v, id});
        g[v].push_back({u, id});
        return id;
    }

    Result solve() const {
        vector<int> dfn(n + 1), low(n + 1), stk;
        vector<char> cut(n + 1, false);
        vector<vector<int>> cc;
        int tim = 0;

        auto dfs = [&](auto&& go, int u, int pe) -> void {
            dfn[u] = low[u] = ++tim;
            int son = 0;
            for (auto [v, id] : g[u]) {
                if (id == pe) continue;
                if (!dfn[v]) {
                    ++son;
                    stk.push_back(id);
                    go(go, v, id);
                    low[u] = min(low[u], low[v]);

                    if (low[v] >= dfn[u]) {
                        if (pe != -1 || son > 1) cut[u] = true;
                        vector<int> cur;
                        while (true) {
                            int eid = stk.back();
                            stk.pop_back();
                            cur.push_back(e[eid].u);
                            cur.push_back(e[eid].v);
                            if (eid == id) break;
                        }
                        sort(cur.begin(), cur.end());
                        cur.erase(unique(cur.begin(), cur.end()), cur.end());
                    }
                }
            }
        };
    }
};

```

```
        cc.push_back(move(cur));
    }
    } else if (dfn[v] < dfn[u]) {
        stk.push_back(id);
        low[u] = min(low[u], dfn[v]);
    }
}
};

for (int u = 1; u <= n; ++u) {
    if (dfn[u]) continue;
    if (g[u].empty()) {
        dfn[u] = low[u] = ++tim;
        cc.push_back({u});
    } else {
        dfs(dfs, u, -1);
    }
}

vector<vector<int>> tr(n + cc.size() + 1);
for (int id = 0; id < cc.size(); ++id) {
    int b = n + id + 1;
    for (int u : cc[id]) {
        tr[u].push_back(b);
        tr[b].push_back(u);
    }
}
return {move(cut), move(cc), move(tr)};
}
};
```

05A - 欧拉路（有向、无向）.cpp

97 lines · C++ · 04 - 图论

```

/*
用途：Hierholzer 求欧拉通路/回路，自动检查度数条件与非孤立部分连通性。
约定：点编号 1..n；构造时指定 dir。返回的点列长度为 m + 1；无解返回 nullopt。
没有边时返回 {1}。支持重边和自环。
复杂度：O(n + m)。
*/
struct EulerTrail {
    int n;
    bool dir;
    int m = 0;
    vector<vector<pair<int, int>>> g;
    vector<int> in, out, deg;

    EulerTrail(int n, bool dir)
        : n(n), dir(dir), g(n + 1), in(n + 1), out(n + 1), deg(n + 1) {}

    int add(int u, int v) {
        int id = m++;
        g[u].push_back({v, id});
        if (dir) {
            ++out[u];
            ++in[v];
        } else {
            g[v].push_back({u, id});
            ++deg[u];
            ++deg[v];
        }
        return id;
    }

    optional<vector<int>> solve() const {
        if (m == 0) return vector<int>{1};
        int s = -1;

        if (dir) {
            int cs = 0, ct = 0;
            for (int u = 1; u <= n; ++u) {
                int d = out[u] - in[u];
                if (d == 1) {
                    ++cs;
                    s = u;
                } else if (d == -1) {
                    ++ct;
                } else if (d != 0) {
                    return nullopt;
                }
            }
            if (cs != ct || cs > 1) return nullopt;
            if (s == -1) {
                for (int u = 1; u <= n; ++u) {
                    if (out[u] > 0) {
                        s = u;
                        break;
                    }
                }
            }
        } else {
            int odd = 0;
            for (int u = 1; u <= n; ++u) {
                if (deg[u] & 1) {

```

```
        ++odd;
        s = u;
    }
}
if (odd != 0 && odd != 2) return nullopt;
if (s == -1) {
    for (int u = 1; u <= n; ++u) {
        if (deg[u] > 0) {
            s = u;
            break;
        }
    }
}
}

vector<int> it(n + 1);
vector<char> vis(m, false);
vector<int> stk = {s}, pth;
while (!stk.empty()) {
    int u = stk.back();
    while (it[u] < g[u].size() && vis[g[u][it[u]].second]) {
        ++it[u];
    }
    if (it[u] == g[u].size()) {
        pth.push_back(u);
        stk.pop_back();
    } else {
        auto [v, id] = g[u][it[u]++];
        vis[id] = true;
        stk.push_back(v);
    }
}
if (pth.size() != m + 1) return nullopt;
reverse(pth.begin(), pth.end());
return pth;
}

};
```

06A - 二分图最大匹配 (Hopcroft-Karp) .cpp

113 lines · C++ · 04 - 图论

```

/*
用途：二分图最大匹配，并可由最大匹配求最小点覆盖 (Kőnig 定理)。
约定：左部编号 1..nl，右部编号 1..nr；add(l,r)。
复杂度：最大匹配  $O(m \sqrt{n})$ ；askCov 为  $O(n + m)$ 。
*/
struct HK {
    struct Result {
        int cnt;
        vector<int> ml;    // matchLeft[u] = 匹配到的右部点，0 表示未匹配。
        vector<int> mr;    // matchRight[v] = 匹配到的左部点，0 表示未匹配。
    };

    struct Cover {
        vector<char> l;
        vector<char> r;
    };

    int nl, nr;
    vector<vector<int>>> g;

    HK(int nl, int nr) : nl(nl), nr(nr), g(nl + 1) {}

    void add(int l, int r) {
        assert(1 <= l && l <= nl && 1 <= r && r <= nr);
        g[l].push_back(r);
    }

    Result solve() const {
        const int INF = numeric_limits<int>::max();
        vector<int> ml(nl + 1), mr(nr + 1), dis(nl + 1);

        auto bfs = [&]() -> bool {
            queue<int> q;
            for (int u = 1; u <= nl; ++u) {
                if (ml[u] == 0) {
                    dis[u] = 0;
                    q.push(u);
                } else {
                    dis[u] = INF;
                }
            }

            int lim = INF;
            while (!q.empty()) {
                int u = q.front();
                q.pop();
                if (dis[u] >= lim) continue;
                for (int v : g[u]) {
                    int x = mr[v];
                    if (x == 0) {
                        lim = dis[u] + 1;
                    } else if (dis[x] == INF) {
                        dis[x] = dis[u] + 1;
                        q.push(x);
                    }
                }
            }

            return lim != INF;
        };

        while (bfs()) {
            for (int u = 1; u <= nl; ++u) {
                if (ml[u] == 0) continue;
                int v = g[u][0];
                int x = mr[v];
                if (x == 0) {
                    ml[u] = v;
                    mr[v] = u;
                } else {
                    int y = ml[x];
                    ml[x] = v;
                    mr[v] = x;
                    ml[y] = 0;
                    mr[mr[y]] = 0;
                }
            }
        }

        return Result{ml.size() - 1, ml, mr};
    }
};

```

```

auto dfs = [&](auto&& go, int u) -> bool {
    for (int v : g[u]) {
        int x = mr[v];
        if (x == 0 || (dis[x] == dis[u] + 1 && go(go, x))) {
            ml[u] = v;
            mr[v] = u;
            return true;
        }
    }
    dis[u] = INF;
    return false;
};

int cnt = 0;
while (bfs()) {
    for (int u = 1; u <= nl; ++u) {
        if (ml[u] == 0 && dfs(dfs, u)) ++cnt;
    }
}
return {cnt, move(ml), move(mr)};
}

// 传入 solve 的结果；返回（左部未访问点）+（右部访问点）。
Cover askCov(const Result& mat) const {
    vector<char> vl(nl + 1, false), vr(nr + 1, false);
    queue<int> q;
    for (int u = 1; u <= nl; ++u) {
        if (mat.ml[u] == 0) {
            vl[u] = true;
            q.push(u);
        }
    }

    while (!q.empty()) {
        int u = q.front();
        q.pop();
        for (int v : g[u]) {
            if (mat.ml[u] == v || vr[v]) continue;
            vr[v] = true;
            int x = mat.mr[v];
            if (x != 0 && !vl[x]) {
                vl[x] = true;
                q.push(x);
            }
        }
    }

    Cover res{vector<char>(nl + 1), vector<char>(nr + 1)};
    for (int u = 1; u <= nl; ++u) res.l[u] = !vl[u];
    for (int v = 1; v <= nr; ++v) res.r[v] = vr[v];
    return res;
}
};

```

06B - 一般图最大匹配（带花树）.cpp

110 lines · C++ · 04 - 图论

```

/*
用途：Edmonds 带花树求一般无向图最大基数匹配。
约定：点编号 1..n；支持重边，默认不放自环。mat[u] 为匹配点，0 表示未匹配。
复杂度：O(n^3)，适合中等规模图。
*/
struct Blossom {
    struct Result {
        int cnt;
        vector<int> mat;
    };

    int n;
    vector<vector<int>> g;

    explicit Blossom(int n) : n(n), g(n + 1) {}

    void add(int u, int v) {
        assert(u != v);
        g[u].push_back(v);
        g[v].push_back(u);
    }

    Result solve() const {
        vector<int> mat(n + 1);
        vector<int> fa(n + 1), bas(n + 1);
        vector<char> use(n + 1), in(n + 1);

        auto lca = [&](int a, int b) -> int {
            vector<char> vis(n + 1, false);
            while (true) {
                a = bas[a];
                vis[a] = true;
                if (mat[a] == 0) break;
                a = fa[mat[a]];
            }
            while (true) {
                b = bas[b];
                if (vis[b]) return b;
                b = fa[mat[b]];
            }
        };

        auto tag = [&](int u, int rt, int son) -> void {
            while (bas[u] != rt) {
                in[bas[u]] = true;
                in[bas[mat[u]]] = true;
                fa[u] = son;
                son = mat[u];
                u = fa[mat[u]];
            }
        };

        auto bfs = [&](int s) -> int {
            fill(use.begin(), use.end(), false);
            fill(fa.begin(), fa.end(), 0);
            iota(bas.begin(), bas.end(), 0);
            queue<int> q;
            q.push(s);
            use[s] = true;

```



```

while (!q.empty()) {
    int u = q.front();
    q.pop();
    for (int v : g[u]) {
        if (bas[u] == bas[v] || mat[u] == v) continue;

        if (v == s || (mat[v] != 0 && fa[mat[v]] != 0)) {
            int rt = lca(u, v);
            fill(in.begin(), in.end(), false);
            tag(u, rt, v);
            tag(v, rt, u);
            for (int x = 1; x <= n; ++x) {
                if (!in[bas[x]]) continue;
                bas[x] = rt;
                if (!use[x]) {
                    use[x] = true;
                    q.push(x);
                }
            }
        } else if (fa[v] == 0) {
            fa[v] = u;
            if (mat[v] == 0) return v;
            v = mat[v];
            use[v] = true;
            q.push(v);
        }
    }
}
return 0;
};

for (int s = 1; s <= n; ++s) {
    if (mat[s] != 0) continue;
    int t = bfs(s);
    while (t != 0) {
        int pre = fa[t];
        int nxt = mat[pre];
        mat[t] = pre;
        mat[pre] = t;
        t = nxt;
    }
}

int cnt = 0;
for (int u = 1; u <= n; ++u) {
    if (mat[u] > u) ++cnt;
}
return {cnt, move(mat)};
}
};

```

06C - 二分图最优匹配 (Hungarian、KM) .cpp

93 lines · C++ · 04 - 图论

```

/*
用途：完全二分图的最小/最大权完备匹配 (Hungarian / KM) 。
约定：左部大小 <= 右部大小；每个左部点都必须匹配一个不同右部点，需为每一对点设置权值。
askMin/askMax 分别返回最小/最大权值；权值及答案应远离 int 边界。
复杂度：O(leftSize^2 * rightSize) 。
*/

struct KM {
    static constexpr int INF = numeric_limits<int>::max() / 4;

    struct Result {
        int val;
        vector<int> ml;    // matchLeft[u] = 匹配到的右部点。
        vector<int> mr;    // matchRight[v] = 匹配到的左部点，0 表示未使用。
    };

    int nl, nr;
    vector<vector<int>> cst;

    KM(int nl, int nr)
        : nl(nl), nr(nr), cst(nl + 1, vector<int>(nr + 1)) {
        assert(nl <= nr);
    }

    void modify(int l, int r, int v) {
        assert(1 <= l && l <= nl && 1 <= r && r <= nr);
        cst[l][r] = v;
    }

    Result askMin() const {
        vector<int> hl(nl + 1), hr(nr + 1);
        vector<int> mr(nr + 1), pre(nr + 1);

        for (int x = 1; x <= nl; ++x) {
            mr[0] = x;
            int r0 = 0;
            vector<int> mn(nr + 1, INF);
            vector<char> use(nr + 1, false);

            do {
                use[r0] = true;
                int l = mr[r0];
                int d = INF;
                int r = 0;
                for (int y = 1; y <= nr; ++y) {
                    if (use[y]) continue;
                    int rc = cst[l][y] - hl[l] - hr[y];
                    if (rc < mn[y]) {
                        mn[y] = rc;
                        pre[y] = r0;
                    }
                    if (mn[y] < d) {
                        d = mn[y];
                        r = y;
                    }
                }
            } while (r != 0);
            for (int y = 0; y <= nr; ++y) {
                if (use[y]) {
                    hl[mr[y]] += d;
                    hr[y] -= d;
                }
            }
        }
    }
};

```

```
        } else {
            mn[y] -= d;
        }
    }
    r0 = r;
} while (mr[r0] != 0);

do {
    int prv = pre[r0];
    mr[r0] = mr[prv];
    r0 = prv;
} while (r0 != 0);
}

vector<int> ml(nl + 1);
for (int y = 1; y <= nr; ++y) {
    if (mr[y] != 0) ml[mr[y]] = y;
}
return {-hr[0], move(ml), move(mr)};
}

Result askMax() const {
    KM neg(nl, nr);
    for (int x = 1; x <= nl; ++x) {
        for (int y = 1; y <= nr; ++y) {
            neg.cst[x][y] = -cst[x][y];
        }
    }
    Result res = neg.askMin();
    res.val = -res.val;
    return res;
}
};
```

06D - 一般图最大权匹配（带权带花树）.cpp

224 lines · C++ · 04 - 图论

```

/*
用途：一般无向图最大权匹配（允许点不匹配），Edmonds 带权带花树。
约定：原点编号 1..n， $n < N$ ，只考虑正权边；重边取最大权。复杂度  $O(n^3)$ 。
若要求完美匹配且有负权，可先给每条边统一加足够大的常数，再检查匹配边数为  $n/2$ 。
点、花与矩阵下标使用 int。
性能：默认使用宏展开后的 int；若大规模数据因此超时或超空间，可将纯下标、状态或计数局部改为 i32。
*/

namespace Blossom {
using W = int;
constexpr int N = 405;
constexpr int V = N * 2 + 5;
constexpr W INF = numeric_limits<W>::max() / 4;

struct Edge {
    int u = 0, v = 0;
    W w = 0;
};
struct Result {
    W w = 0;
    vector<array<int, 2>> es;
};

inline Edge e[V][V];
inline W lab[V];
inline int n, m, tim, ql, qr;
inline int lk[V], sl[V], st[V], fa[V], bel[V][V], typ[V], vis[V], que[V];
inline vector<int> cyc[V];

inline W slack(const Edge& x) { return lab[x.u] + lab[x.v] - 2 * x.w; }

void upd(int u, int v) {
    if (!sl[v] || slack(e[u][v]) < slack(e[sl[v]][v])) sl[v] = u;
}

void slack(int v) {
    sl[v] = 0;
    for (int u = 1; u <= n; ++u)
        if (e[u][v].w > 0 && st[u] != v && typ[st[u]] == 0) upd(u, v);
}

void push(int u) {
    if (u <= n) que[++qr] = u;
    else for (int v : cyc[u]) push(v);
}

void setSt(int u, int v) {
    st[u] = v;
    if (u > n) for (int x : cyc[u]) setSt(x, v);
}

int rot(int u, int v) {
    int k = find(cyc[u].begin(), cyc[u].end(), v) - cyc[u].begin();
    if (k & 1) {
        reverse(cyc[u].begin() + 1, cyc[u].end());
        return cyc[u].size() - k;
    }
    return k;
}
}

```

```

void match(int u, int v) {
    lk[u] = e[u][v].v;
    if (u <= n) return;
    Edge w = e[u][v];
    int x = bel[u][w.u], y = rot(u, x);
    for (int i = 0; i < y; ++i) match(cyc[u][i], cyc[u][i ^ 1]);
    match(x, v);
    rotate(cyc[u].begin(), cyc[u].begin() + y, cyc[u].end());
}

void augment(int u, int v) {
    int w = st[lk[u]];
    match(u, v);
    if (!w) return;
    match(w, st[fa[w]]);
    augment(st[fa[w]], w);
}

int lca(int u, int v) {
    for (++tim; u || v; swap(u, v)) {
        if (!u) continue;
        if (vis[u] == tim) return u;
        vis[u] = tim;
        if ((u = st[lk[u]])) u = st[fa[u]];
    }
    return 0;
}

void add(int u, int a, int v) {
    int x = n + 1;
    while (x <= m && st[x]) ++x;
    if (x > m) ++m;
    assert(m < V);
    lab[x] = 0, typ[x] = 0, st[x] = x, lk[x] = lk[a];
    cyc[x].clear();
    cyc[x].push_back(a);
    int i, j = 0;
    for (i = u; i != a; i = st[fa[j]]) {
        cyc[x].push_back(i);
        cyc[x].push_back(j = st[lk[i]]);
        push(j);
    }
    reverse(cyc[x].begin() + 1, cyc[x].end());
    for (i = v; i != a; i = st[fa[j]]) {
        cyc[x].push_back(i);
        cyc[x].push_back(j = st[lk[i]]);
        push(j);
    }
    setSt(x, x);
    for (i = 1; i <= m; ++i) e[x][i] = e[i][x] = {};
    fill(bel[x], bel[x] + n + 1, 0);
    for (int z : cyc[x]) {
        for (int y = 1; y <= m; ++y) {
            if (!e[x][y].w || slack(e[z][y]) < slack(e[x][y])) e[x][y] = e[z][y], e[y][x] = e[y][z];
        }
        for (int y = 1; y <= n; ++y) if (bel[z][y]) bel[x][y] = z;
    }
    slack(x);
}

void expand(int u) {
    for (int x : cyc[u]) setSt(x, x);
    int a = bel[u][e[u][fa[u]].u], r = rot(u, a);
    for (int i = 0; i < r; i += 2) {

```

```

        int x = cyc[u][i], y = cyc[u][i + 1];
        fa[x] = e[y][x].u;
        typ[x] = 1;
        typ[y] = sl[x] = 0;
        slack(y), push(y);
    }
    typ[a] = 1, fa[a] = fa[u];
    for (int i = r + 1; i < cyc[u].size(); ++i) typ[cyc[u][i]] = -1, slack(cyc[u][i]);
    st[u] = 0;
}

bool onEdge(const Edge& x) {
    int u = st[x.u], v = st[x.v];
    if (typ[v] == -1) {
        fa[v] = x.u, typ[v] = 1;
        int a = st[lk[v]];
        sl[v] = sl[a] = typ[a] = 0;
        push(a);
    } else if (typ[v] == 0) {
        int a = lca(u, v);
        if (!a) {
            augment(u, v), augment(v, u);
            return true;
        }
        add(u, a, v);
    }
    return false;
}

bool bfs() {
    fill(typ + 1, typ + m + 1, -1);
    fill(sl + 1, sl + m + 1, 0);
    ql = 1, qr = 0;
    for (int i = 1; i <= m; ++i) if (st[i] == i && !lk[i]) {
        fa[i] = typ[i] = 0;
        push(i);
    }
    if (ql > qr) return false;
    while (true) {
        while (ql <= qr) {
            int u = que[ql++];
            if (typ[st[u]] == 1) continue;
            for (int v = 1; v <= n; ++v) if (e[u][v].w > 0 && st[u] != st[v]) {
                if (slack(e[u][v])) upd(u, st[v]);
                else if (onEdge(e[u][v])) return true;
            }
        }

        W d = INF;
        for (int i = n + 1; i <= m; ++i) if (st[i] == i && typ[i] == 1) d = min(d, lab[i] / 2);
        for (int i = 1; i <= m; ++i) if (st[i] == i && sl[i] && typ[i] != 1)
            d = min(d, slack(e[sl[i]][i]) / (typ[i] == 0 ? 2 : 1));
        if (d == INF) return false;

        for (int i = 1; i <= n; ++i) if (typ[st[i]] != -1) {
            lab[i] += (2 * typ[st[i]] - 1) * d;
            if (lab[i] <= 0) return false;
        }
        for (int i = n + 1; i <= m; ++i) if (st[i] == i && typ[i] != -1)
            lab[i] += (2 - 4 * typ[i]) * d;

        ql = 1, qr = 0;
        for (int i = 1; i <= m; ++i)
            if (st[i] == i && sl[i] && st[sl[i]] != i && !slack(e[sl[i]][i]) && onEdge(e[sl[i]][i]))

```

```

        return true;
    for (int i = n + 1; i <= m; ++i) if (st[i] == i && typ[i] == 1 && !lab[i]) expand(i);
}
}

Result solve(int _n, const vector<tuple<int, int, W>>& es) {
    assert(0 <= _n && _n < N);
    n = m = _n, tim = 0;
    int lim = min<int>(V, 2 * n + 5);
    fill(lk, lk + lim, 0), fill(st, st + lim, 0), fill(vis, vis + lim, 0);
    for (int i = 0; i < lim; ++i) {
        fill(e[i], e[i] + lim, Edge{});
        fill(bel[i], bel[i] + lim, 0);
        cyc[i].clear();
    }
    for (int i = 1; i <= n; ++i) st[i] = i;
    W mx = 0;
    for (int i = 1; i <= n; ++i) for (int j = 1; j <= n; ++j) e[i][j] = {i, j, 0};
    for (auto [u, v, w] : es) {
        assert(1 <= u && u <= n && 1 <= v && v <= n);
        if (u == v || w <= 0) continue;
        if (w > e[u][v].w) e[u][v] = {u, v, w}, e[v][u] = {v, u, w};
        mx = max(mx, w);
    }
    for (int i = 1; i <= n; ++i) {
        lab[i] = mx;
        for (int j = 1; j <= n; ++j) if (i == j) bel[i][j] = i;
    }
    while (bfs()) {}
    Result r;
    for (int i = 1; i <= n; ++i) if (lk[i] && i < lk[i]) {
        r.w += e[i][lk[i]].w;
        r.es.push_back({i, lk[i]});
    }
    return r;
}
} // namespace Blossom

```

07 - 全局最小割 (Stoer-Wagner) .cpp

55 lines · C++ · 04 - 图论

```

/*
用途：无向非负权图的全局最小割，不指定源汇；同时返回割的一侧原点集合。
约定：点编号 0..n-1；重边容量累加。n<2 时割值为 0。
复杂度：O(n^3) 时间、O(n^2) 空间，适合稠密中小图。
性能：默认使用宏展开后的 int；若大规模数据因此超时或超空间，可将纯下标、状态或计数局部改为 i32。
*/

template <class Cap = int> struct MinCut {
    struct Result {
        Cap val;
        vector<int> sd;
    };

    int n;
    vector<vector<Cap>> w;
    explicit MinCut(int n) : n(n), w(n, vector<Cap>(n)) { assert(n >= 0); }

    void add(int u, int v, Cap c) {
        assert(0 <= u && u < n && 0 <= v && v < n && c >= 0);
        if (u == v) return;
        w[u][v] += c, w[v][u] += c;
    }

    Result solve() const {
        if (n < 2) return {Cap{}, n ? vector<int>{0} : vector<int>{}};
        vector<vector<Cap>> a = w;
        vector<int> v(n);
        iota(v.begin(), v.end(), 0);
        vector<vector<int>> g(n);
        for (int i = 0; i < n; ++i) g[i] = {i};
        Cap ans = numeric_limits<Cap>::max();
        vector<int> sd;

        while (v.size() > 1) {
            vector<Cap> d(n);
            vector<char> use(n);
            int pre = -1, sel = -1;
            for (int k = 0; k < v.size(); ++k) {
                sel = -1;
                for (int x : v) if (!use[x] && (sel == -1 || d[x] > d[sel])) sel = x;
                if (k + 1 == v.size()) {
                    if (d[sel] < ans) ans = d[sel], sd = g[sel];
                    for (int x : v) if (x != pre) a[pre][x] += a[sel][x], a[x][pre] = a[pre][x];
                    g[pre].insert(g[pre].end(), g[sel].begin(), g[sel].end());
                    v.erase(find(v.begin(), v.end(), sel));
                    break;
                }
                use[sel] = true;
                for (int x : v) if (!use[x]) d[x] += a[sel][x];
                pre = sel;
            }
        }
        return {ans, move(sd)};
    }
};

```


08 - 最大团（位集分支限界）.cpp

55 lines · C++ · 04 - 图论

```

/*
用途：求 n<=64 无向简单图的最大团及一组方案。
复杂度：最坏指数级；贪心染色给上界，适合中小规模稠密图。
性能：邻接与候选集使用 u64，避免 #define int long long 扩大状态。
性能：默认使用宏展开后的 int；若大规模数据因此超时或超空间，可将纯下标、状态或计数局部改为 i32。
*/

using u64 = uint64_t;

struct Clique {
    int n;
    vector<u64> g;
    vector<int> now, ans;

    explicit Clique(int _n) : n(_n), g(_n) { assert(0 <= n && n <= 64); }
    void add(int u, int v) {
        assert(0 <= u && u < n && 0 <= v && v < n && u != v);
        g[u] |= u64{1} << v;
        g[v] |= u64{1} << u;
    }

    vector<int> solve() {
        now.clear(), ans.clear();
        dfs(n == 64 ? ~u64{1} : (u64{1} << n) - 1);
        return ans;
    }

private:
    void color(u64 p, vector<int>& v, vector<int>& c) {
        for (int k = 1; p; ++k) {
            u64 q = p;
            while (q) {
                int x = __builtin_ctzll(q);
                u64 b = u64{1} << x;
                v.push_back(x), c.push_back(k);
                p ^= b, q ^= b, q &= ~g[x];
            }
        }
    }

    void dfs(u64 p) {
        if (!p) {
            if (now.size() > ans.size()) ans = now;
            return;
        }
        vector<int> v, c;
        color(p, v, c);
        for (int i = v.size(); i--;) {
            if (now.size() + c[i] <= ans.size()) return;
            int x = v[i];
            now.push_back(x), dfs(p & g[x]), now.pop_back();
            p &= ~(u64{1} << x);
        }
    }
};

```

09 - 平面嵌入面与对偶图.cpp

90 lines · C++ · 04 - 图论

```

/*
用途：已知连通直线平面嵌入（无交叉）时，枚举面并建立带权对偶图。
约定：每条无向边给一个权；对偶边连接其左右两面。桥的左右面相同，不加入对偶自环。
复杂度： $O(m \log m)$ ，主要为每点按极角排序。
性能：默认使用宏展开后的 int；若大规模数据因此超时或超空间，可将纯下标、状态或计数局部改为 i32。
*/

using i128 = __int128_t;

template <class T = int, class W = int, class D = i128> struct PlanarDual {
    struct P { T x, y; };
    struct E { int u, v; W w; };

    vector<P> p;
    vector<E> e;
    vector<array<int, 2>> he;
    vector<int> rev, nxt, bel;
    vector<vector<int>> g;
    vector<D> ar;
    int out = -1;

    PlanarDual() = default;
    explicit PlanarDual(vector<P> p) : p(move(p)), g(this->p.size()) {}

    void add(int u, int v, W w = 1) {
        int id = e.size(), a = he.size();
        e.push_back({u, v, w});
        he.push_back({u, v});
        he.push_back({v, u});
        rev.push_back(a + 1);
        rev.push_back(a);
        g[u].push_back(a);
        g[v].push_back(a + 1);
    }

    D cr(int a, int b, int c) const {
        return D(p[b].x - p[a].x) * (p[c].y - p[a].y) - D(p[b].y - p[a].y) * (p[c].x - p[a].x);
    }

    void build() {
        int m = he.size();
        vector<int> pos(m);
        auto hf = [&](T x, T y) { return y < 0 || (y == 0 && x < 0); };
        for (int u = 0; u < g.size(); ++u) {
            sort(g[u].begin(), g[u].end(), [&](int a, int b) {
                auto [ua, x] = he[a];
                auto [ub, y] = he[b];
                T ax = p[x].x - p[u].x, ay = p[x].y - p[u].y;
                T bx = p[y].x - p[u].x, by = p[y].y - p[u].y;
                if (hf(ax, ay) != hf(bx, by)) return hf(ax, ay) < hf(bx, by);
                D z = D(ax) * by - D(ay) * bx;
                if (z) return z > 0;
                return D(ax) * ax + D(ay) * ay < D(bx) * bx + D(by) * by;
            });
            for (int i = 0; i < g[u].size(); ++i) pos[g[u][i]] = i;
        }

        nxt.assign(m, -1);
        for (int h = 0; h < m; ++h) {
            int v = he[h][1], k = pos[rev[h]], d = g[v].size();

```

```
        nxt[h] = g[v][(k + d - 1) % d];
    }

    bel.assign(m, -1);
    ar.clear();
    for (int s = 0; s < m; ++s) if (bel[s] < 0) {
        int f = ar.size(), h = s;
        D a = 0;
        do {
            bel[h] = f;
            auto [u, v] = he[h];
            a += D(p[u].x) * p[v].y - D(p[u].y) * p[v].x;
            h = nxt[h];
        } while (h != s);
        ar.push_back(a);
    }
    out = min_element(ar.begin(), ar.end()) - ar.begin();
}

vector<vector<pair<int, W>>> ask() const {
    vector<vector<pair<int, W>>> d(ar.size());
    for (int i = 0; i < e.size(); ++i) {
        int a = bel[i << 1], b = bel[i << 1 | 1];
        if (a == b) continue;
        d[a].push_back({b, e[i].w});
        d[b].push_back({a, e[i].w});
    }
    return d;
}

};
```

01A - 最大流 (Dinic) .cpp

138 lines · C++ · 05 - 网络流

```

/*
用途：Dinic 最大流、残量图最小割恢复；作为短代码与通用默认分支。
约定：容量类型用 Cap 模板适配；点号和反向边下标使用 int。
性能：默认使用宏展开后的 int；若大规模数据因此超时或超空间，可将纯下标、状态或计数局部改为 i32。
*/

template <class Cap = int> struct MaxFlow {
    static constexpr Cap INF = numeric_limits<Cap>::max() / 4;

    struct Edge {
        int v, r;
        Cap c, oc;
    };

    struct EdgeInfo {
        int u, v;
        Cap c, f;
    };

    int n;
    vector<vector<Edge>> g;
    vector<array<int, 2>> pos;
    vector<int> dep, cur;

    explicit MaxFlow(int n) : n(n), g(n) {
        assert(n >= 0);
    }

    int add(int u, int v, Cap c) {
        assert(0 <= u && u < n && 0 <= v && v < n && c >= 0);
        int x = g[u].size();
        int y = g[v].size() + (u == v);
        g[u].push_back({v, y, c, c});
        g[v].push_back({u, x, 0, 0});
        pos.push_back({u, x});
        return pos.size() - 1;
    }

    void add2(int u, int v, Cap c) {
        add(u, v, c);
        add(v, u, c);
    }

    void reset() {
        for (auto& a : g)
            for (Edge& e : a) e.c = e.oc;
    }

    EdgeInfo askEdge(int id) const {
        assert(0 <= id && id < pos.size());
        auto [u, i] = pos[id];
        const Edge& e = g[u][i];
        return {u, e.v, e.oc, e.oc - e.c};
    }

    Cap askFlow(int id) const {
        return askEdge(id).f;
    }

    vector<EdgeInfo> askAll() const {

```

```

    vector<EdgeInfo> a;
    a.reserve(pos.size());
    for (int i = 0; i < pos.size(); ++i) a.push_back(askEdge(i));
    return a;
}

void erase(int id) {
    assert(0 <= id && id < pos.size());
    auto [u, i] = pos[id];
    Edge& e = g[u][i];
    Edge& f = g[e.v][e.r];
    e.c = f.c = 0;
}

bool bfs(int s, int t) {
    dep.assign(n, -1);
    queue<int> q;
    dep[s] = 0;
    q.push(s);
    while (!q.empty()) {
        int u = q.front();
        q.pop();
        for (const Edge& e : g[u]) {
            if (e.c <= 0 || dep[e.v] != -1) continue;
            dep[e.v] = dep[u] + 1;
            q.push(e.v);
        }
    }
    return dep[t] != -1;
}

Cap dfs(int u, int t, Cap f) {
    if (u == t || !f) return f;
    for (int& i = cur[u]; i < g[u].size(); ++i) {
        Edge& e = g[u][i];
        if (e.c <= 0 || dep[e.v] != dep[u] + 1) continue;
        Cap x = dfs(e.v, t, min(f, e.c));
        if (!x) continue;
        e.c -= x;
        g[e.v][e.r].c += x;
        return x;
    }
    return 0;
}

Cap flow(int s, int t, Cap lim = INF) {
    assert(0 <= s && s < n && 0 <= t && t < n && s != t && lim >= 0);
    Cap r = 0;
    while (r < lim && bfs(s, t)) {
        cur.assign(n, 0);
        while (r < lim) {
            Cap x = dfs(s, t, min(lim - r, INF));
            if (!x) break;
            r += x;
        }
    }
    return r;
}

vector<char> askCut(int s) const {
    assert(0 <= s && s < n);
    vector<char> a(n);
    queue<int> q;
    q.push(s);

```

```
        a[s] = 1;
        while (!q.empty()) {
            int u = q.front();
            q.pop();
            for (const Edge& e : g[u]) {
                if (e.c > 0 && !a[e.v]) {
                    a[e.v] = 1;
                    q.push(e.v);
                }
            }
        }
        return a;
    }
};
```

01B - 最大流 (HLPP) .cpp

183 lines · C++ · 05 - 网络流

/*

用途：最高标号预流推进 (HLPP) 最大流，含反向 BFS 初始标号、当前弧与 gap 优化。

适合点边较多、Dinic 分层增广偏慢的图；小图和建模题通常优先更短的 01A Dinic。

约定：点号/边下标使用 int，容量由 Cap 适配；点编号 0..n-1。

性能：默认使用宏展开后的 int；若大规模数据因此超时或超空间，可将纯下标、状态或计数局部改为 i32。

*/

```
template <class Cap = int> struct MaxFlow {
    static constexpr Cap INF = numeric_limits<Cap>::max() / 4;

    struct Edge {
        int v, r;
        Cap c, oc;
    };
    struct EdgeInfo {
        int u, v;
        Cap c, f;
    };

    int n;
    vector<vector<Edge>> g;
    vector<array<int, 2>> pos;
    vector<Cap> ex;
    vector<int> h, cnt, cur;
    vector<vector<int>> b;
    int s = -1, t = -1, hi = 0, lmh = 0;

    explicit MaxFlow(int n) : n(n), g(n) { assert(n >= 0); }

    int add(int u, int v, Cap c) {
        assert(0 <= u && u < n && 0 <= v && v < n && c >= 0);
        int x = g[u].size();
        int y = g[v].size() + (u == v);
        g[u].push_back({v, y, c, c});
        g[v].push_back({u, x, 0, 0});
        pos.push_back({u, x});
        return pos.size() - 1;
    }

    void add2(int u, int v, Cap c) {
        add(u, v, c), add(v, u, c);
    }

    void reset() {
        for (auto& a : g) for (auto& e : a) e.c = e.oc;
    }

    EdgeInfo askEdge(int id) const {
        assert(0 <= id && id < pos.size());
        auto [u, i] = pos[id];
        const auto& e = g[u][i];
        return {u, e.v, e.oc, e.oc - e.c};
    }

    Cap askFlow(int id) const {
        return askEdge(id).f;
    }

    vector<EdgeInfo> askAll() const {
        vector<EdgeInfo> a;

```

```

    a.reserve(pos.size());
    for (int i = 0; i < pos.size(); ++i) a.push_back(askEdge(i));
    return a;
}

void erase(int id) {
    assert(0 <= id && id < pos.size());
    auto [u, i] = pos[id];
    Edge& e = g[u][i];
    Edge& f = g[e.v][e.r];
    e.c = f.c = 0;
}

void active(int u) {
    if (u == s || u == t || ex[u] <= 0 || h[u] >= lmh) return;
    b[h[u]].push_back(u);
    hi = max(hi, h[u]);
}

void label() {
    lmh = 2 * n + 1;
    h.assign(n, n + 1); // 不可达汇点的点仍可沿反向边把预流退回源点。
    queue<int> q;
    h[t] = 0, q.push(t);
    while (!q.empty()) {
        int u = q.front();
        q.pop();
        for (const auto& e : g[u]) {
            const auto& r = g[e.v][e.r];
            if (r.c <= 0 || h[e.v] != n + 1) continue;
            h[e.v] = h[u] + 1;
            q.push(e.v);
        }
    }
    h[s] = n;
    cnt.assign(lmh + 1, 0);
    for (int x : h) ++cnt[x];
    cur.assign(n, 0);
    b.assign(lmh + 1, {});
    hi = 0;
    for (int u = 0; u < n; ++u) active(u);
}

void push(int u, Edge& e) {
    Cap x = min(ex[u], e.c);
    if (x <= 0 || h[u] != h[e.v] + 1) return;
    bool z = ex[e.v] == 0;
    e.c -= x;
    g[e.v][e.r].c += x;
    ex[u] -= x, ex[e.v] += x;
    if (z) active(e.v);
}

void relabel(int u) {
    int old = h[u], nh = lmh;
    for (const auto& e : g[u]) if (e.c > 0) nh = min(nh, min<int>(lmh, h[e.v] + 1));
    --cnt[old];
    h[u] = nh;
    ++cnt[nh];
    cur[u] = 0;

    if (old < n && cnt[old] == 0) {
        for (int v = 0; v < n; ++v) {
            if (v == s || v == t || h[v] <= old || h[v] >= n) continue;

```



```

        --cnt[h[v]];
        h[v] = n + 1;
        ++cnt[h[v]];
        active(v);
    }
}

void push(int u) {
    while (ex[u] > 0 && h[u] < lmh) {
        if (cur[u] == g[u].size()) {
            relabel(u);
            continue;
        }
        Edge& e = g[u][cur[u]];
        if (e.c > 0 && h[u] == h[e.v] + 1) push(u, e);
        else ++cur[u];
    }
    active(u);
}

Cap flow(int _s, int _t) {
    assert(0 <= _s && _s < n && 0 <= _t && _t < n && _s != _t);
    s = _s, t = _t;
    ex.assign(n, Cap{});
    label();
    for (auto& e : g[s]) {
        if (e.v == s || e.c <= 0) continue;
        Cap x = e.c;
        e.c = 0;
        g[e.v][e.r].c += x;
        ex[s] -= x, ex[e.v] += x;
        active(e.v);
    }
    while (hi >= 0) {
        if (b[hi].empty()) {
            --hi;
            continue;
        }
        int u = b[hi].back();
        b[hi].pop_back();
        if (h[u] != hi || ex[u] <= 0) continue;
        push(u);
    }
    return ex[t];
}

vector<char> askCut(int s) const {
    vector<char> a(n);
    queue<int> q;
    q.push(s), a[s] = true;
    while (!q.empty()) {
        int u = q.front();
        q.pop();
        for (const auto& e : g[u]) if (e.c > 0 && !a[e.v]) a[e.v] = true, q.push(e.v);
    }
    return a;
}
};

```

01C - 最大流（HLPP，极致优化）.cpp

205 lines · C++ · 05 - 网络流

```

/*
用途：highest-label 预流推进的极低常数版，使用反向 BFS、gap、指针当前弧与周期重标号。
用法：接口与 01A / 01B 的 MaxFlow 一致；大图卡常时替换使用。
约定：点编号 0..n-1，Cap 必须是有符号整型；容量默认使用宏展开后的 int。
性能：点边下标必须保留 i32；改成宏展开后的 int 会增大边表并降低缓存命中率。
*/

using i32 = int32_t;

template <class Cap = int> struct MaxFlow {
    static constexpr Cap INF = numeric_limits<Cap>::max() / 4;

    struct Edge {
        i32 v, r;
        Cap f, c;
    };
    struct EdgeInfo {
        int u, v;
        Cap c, f;
    };

    i32 n, s = -1, t = -1, hi = 0, lm = 0;
    int wk = 0;
    vector<vector<Edge>> g;
    vector<array<i32, 2>> pos;
    vector<Cap> ex;
    vector<i32> h, cnt;
    vector<Edge*> cur;
    vector<vector<i32>> b;

    explicit MaxFlow(int n) : n(n), g(n) {
        assert(0 <= n && n <= numeric_limits<i32>::max());
    }

    int add(int u, int v, Cap c) {
        assert(0 <= u && u < n && 0 <= v && v < n && c >= 0);
        i32 x = g[u].size(), y = g[v].size() + (u == v);
        g[u].push_back({i32(v), y, 0, c});
        g[v].push_back({i32(u), x, 0, 0});
        pos.push_back({i32(u), x});
        return pos.size() - 1;
    }

    void add2(int u, int v, Cap c) {
        add(u, v, c), add(v, u, c);
    }

    void reset() {
        for (auto& a : g) {
            for (auto& e : a) e.c += e.f, e.f = 0;
        }
    }

    EdgeInfo askEdge(int id) const {
        int m = pos.size();
        assert(0 <= id && id < m);
        auto [u, i] = pos[id];
        const auto& e = g[u][i];
        return {u, e.v, e.c + e.f, e.f};
    }
}

```

```

Cap askFlow(int id) const {
    return askEdge(id).f;
}

vector<EdgeInfo> askAll() const {
    vector<EdgeInfo> a;
    int m = pos.size();
    a.reserve(m);
    for (int i = 0; i < m; ++i) a.push_back(askEdge(i));
    return a;
}

void erase(int id) {
    int m = pos.size();
    assert(0 <= id && id < m);
    auto [u, i] = pos[id];
    auto& e = g[u][i];
    auto& r = g[e.v][e.r];
    e.f = e.c = r.f = r.c = 0;
}

void active(i32 u) {
    if (u == s || u == t || ex[u] <= 0 || h[u] >= lm) return;
    b[h[u]].push_back(u);
    hi = max(hi, h[u]);
}

void bfs() {
    lm = 2 * n + 1;
    h.assign(n, n + 1);
    cnt.assign(lm + 1, 0);
    cur.resize(n);
    b.assign(lm + 1, {});
    queue<i32> q;
    h[t] = 0, q.push(t);
    while (!q.empty()) {
        i32 u = q.front();
        q.pop();
        for (const auto& e : g[u]) {
            if (e.v == s || h[e.v] != n + 1 || g[e.v][e.r].c <= 0) continue;
            h[e.v] = h[u] + 1;
            q.push(e.v);
        }
    }
    h[s] = n;
    for (i32 u = 0; u < n; ++u) ++cnt[h[u]], cur[u] = g[u].data();
    hi = 0, wk = 0;
    for (i32 u = 0; u < n; ++u) active(u);
}

void gap(i32 k, i32 z) {
    for (i32 u = 0; u < n; ++u) {
        if (u == s || u == t || h[u] <= k || h[u] >= n + 1) continue;
        --cnt[h[u]];
        h[u] = n + 1;
        ++cnt[h[u]];
        cur[u] = g[u].data();
        if (u != z) active(u);
    }
}

void label(i32 u) {
    i32 oh = h[u], nh = lm;

```

```

    for (const auto& e : g[u]) {
        ++wk;
        if (e.c > 0) nh = min(nh, i32(h[e.v] + 1));
    }
    --cnt[oh];
    h[u] = nh;
    ++cnt[nh];
    cur[u] = g[u].data();
    if (oh < n && !cnt[oh]) gap(oh, u);
}

void push(i32 u, Edge& e) {
    Cap f = min(ex[u], e.c);
    if (f <= 0 || h[u] != h[e.v] + 1) return;
    bool z = ex[e.v] == 0;
    auto& r = g[e.v][e.r];
    e.f += f, e.c -= f;
    r.f -= f, r.c += f;
    ex[u] -= f, ex[e.v] += f;
    if (z) active(e.v);
}

void work(i32 u) {
    Edge* ed = g[u].data() + g[u].size();
    while (ex[u] > 0 && h[u] < lm) {
        if (cur[u] == ed) {
            label(u);
            ed = g[u].data() + g[u].size();
        } else if (cur[u]->c > 0 && h[u] == h[cur[u]->v] + 1) {
            push(u, *cur[u]);
        } else {
            ++cur[u], ++wk;
        }
    }
    active(u);
}

Cap flow(int _s, int _t) {
    assert(0 <= _s && _s < n && 0 <= _t && _t < n && _s != _t);
    s = _s, t = _t;
    ex.assign(n, Cap{});
    bfs();
    for (auto& e : g[s]) {
        if (e.v == s || e.c <= 0) continue;
        Cap f = e.c;
        auto& r = g[e.v][e.r];
        e.f += f, e.c = 0;
        r.f -= f, r.c += f;
        ex[s] -= f, ex[e.v] += f;
        active(e.v);
    }
    int m = pos.size();
    while (hi >= 0) {
        if (b[hi].empty()) {
            --hi;
            continue;
        }
        i32 u = b[hi].back();
        b[hi].pop_back();
        if (h[u] != hi || ex[u] <= 0) continue;
        work(u);
        if (wk > 4 * m + n) bfs();
    }
    return ex[t];
}

```

```
    }

    vector<char> askCut(int s) const {
        assert(0 <= s && s < n);
        vector<char> a(n);
        queue<i32> q;
        q.push(s), a[s] = true;
        while (!q.empty()) {
            i32 u = q.front();
            q.pop();
            for (const auto& e : g[u]) {
                if (e.c > 0 && !a[e.v]) a[e.v] = true, q.push(e.v);
            }
        }
        return a;
    }
};
```

02 - 最小割.cpp

28 lines · C++ · 05 - 网络流

```
/*
用途：S-T 最小割值及源点侧点集恢复。
依赖：01A - 最大流 (Dinic) 的 MaxFlow。
约定：点编号 0..n-1；solve 会从初始容量开始计算，可重复调用。
复杂度：与 Dinic 一致。
*/

template <class Cap = int> struct MinCut {
    struct Result {
        Cap val;
        vector<char> sd;
    };

    int n;
    MaxFlow<Cap> mf;

    explicit MinCut(int n) : n(n), mf(n) {}

    int add(int u, int v, Cap c) {
        return mf.add(u, v, c);
    }

    Result solve(int s, int t) {
        mf.reset();
        Cap val = mf.flow(s, t);
        return {val, mf.askCut(s)};
    }
};
```

02A - 最大权闭合图（最小割建模）.cpp

46 lines · C++ · 05 - 网络流

```

/*
用途：最大权闭合图。若选择 u 必须选择 v，则 add(u,v)。
依赖：01A - 最大流 (Dinic) 的 MaxFlow。
约定：点编号 0..n-1；modify 可对同一点多次调用。依赖边容量应大于所有可能答案。
复杂度：一次 Dinic 最大流。
*/

template <class Cap = int> struct MaxClosure {
    static constexpr Cap INF = numeric_limits<Cap>::max() / 4;

    struct Result {
        Cap val;
        vector<char> vis;
    };

    int n;
    int s, t;
    MaxFlow<Cap> mf;
    Cap sum = 0;

    explicit MaxClosure(int n) : n(n), s(n), t(n + 1), mf(n + 2) {}

    void modify(int u, Cap w) {
        assert(0 <= u && u < n);
        if (w > 0) {
            mf.add(s, u, w);
            sum += w;
        } else if (w < 0) {
            mf.add(u, t, -w);
        }
    }

    void add(int u, int v, Cap c = INF) {
        assert(0 <= u && u < n && 0 <= v && v < n && c >= 0);
        mf.add(u, v, c);
    }

    Result solve() {
        mf.reset();
        Cap cut = mf.flow(s, t);
        vector<char> sd = mf.askCut(s);
        vector<char> vis(n, false);
        for (int u = 0; u < n; ++u) vis[u] = sd[u];
        return {sum - cut, move(vis)};
    }
};

```

03 - 最小割树.cpp

122 lines · C++ · 05 - 网络流

/*

用途：Gomory-Hu Tree（无向图任意两点最小割）。

依赖：01A - 最大流（Dinic）的 MaxFlow。

约定：点编号 $0..n-1$ ，边容量非负；build 后 g 是一棵带权树，

任意两点最小割等于树路径上的最小边权。 $ask(u,v)$ 单次 $O(\log n)$ 。

复杂度：建树调用 $n-1$ 次最大流，额外 $O(n \log n)$ 预处理。

*/

```
template <class Cap = int> struct GHTree {
    static constexpr Cap INF = numeric_limits<Cap>::max() / 4;

    int n;
    MaxFlow<Cap> mf;
    vector<vector<pair<int, Cap>>> g;
    vector<int> dep;
    vector<vector<int>> up;
    vector<vector<Cap>> mn;
    bool ok = false;

    explicit GHTree(int n) : n(n), mf(n), g(n) {
        assert(n > 0);
    }

    void add(int u, int v, Cap c) {
        assert(0 <= u && u < n && 0 <= v && v < n && c >= 0);
        mf.add(u, v, c);
        mf.add(v, u, c);
        ok = false;
    }

    void build() {
        vector<int> fa(n, 0);
        vector<Cap> cut(n);
        fa[0] = -1;

        for (int s = 1; s < n; ++s) {
            int t = fa[s];
            mf.reset();
            Cap val = mf.flow(s, t);
            vector<char> sd = mf.askCut(s);
            cut[s] = val;

            for (int u = s + 1; u < n; ++u) {
                if (fa[u] == t && sd[u]) fa[u] = s;
            }
            if (fa[t] != -1 && sd[fa[t]]) {
                fa[s] = fa[t];
                fa[t] = s;
                cut[s] = cut[t];
                cut[t] = val;
            }
        }

        g.assign(n, {});
        for (int u = 1; u < n; ++u) {
            g[u].push_back({fa[u], cut[u]});
            g[fa[u]].push_back({u, cut[u]});
        }
        init();
        ok = true;
    }
};
```



```

    }

    Cap ask(int u, int v) const {
        assert(ok && 0 <= u && u < n && 0 <= v && v < n);
        if (u == v) return 0;
        Cap ans = INF;
        if (dep[u] < dep[v]) swap(u, v);
        int d = dep[u] - dep[v];
        for (int bit = 0; d > 0; ++bit, d >>= 1) {
            if (d & 1) {
                ans = min(ans, mn[bit][u]);
                u = up[bit][u];
            }
        }
        if (u == v) return ans;

        for (int bit = up.size(); bit--;) {
            if (up[bit][u] == up[bit][v]) continue;
            ans = min(ans, mn[bit][u]);
            ans = min(ans, mn[bit][v]);
            u = up[bit][u];
            v = up[bit][v];
        }
        ans = min(ans, mn[0][u]);
        ans = min(ans, mn[0][v]);
        return ans;
    }

private:
    void init() {
        int log = 1;
        while ((1 << log) <= n) ++log;
        dep.assign(n, 0);
        up.assign(log, vector<int>(n));
        mn.assign(log, vector<Cap>(n, INF));

        queue<int> q;
        vector<char> vis(n, false);
        q.push(0);
        vis[0] = true;
        up[0][0] = 0;
        while (!q.empty()) {
            int u = q.front();
            q.pop();
            for (auto [v, w] : g[u]) {
                if (vis[v]) continue;
                vis[v] = true;
                dep[v] = dep[u] + 1;
                up[0][v] = u;
                mn[0][v] = w;
                q.push(v);
            }
        }

        for (int bit = 1; bit < log; ++bit) {
            for (int u = 0; u < n; ++u) {
                up[bit][u] = up[bit - 1][up[bit - 1][u]];
                mn[bit][u] = min(mn[bit - 1][u], mn[bit - 1][up[bit - 1][u]]);
            }
        }
    }
};

```

04 - 最小费用最大流 (MinCostFlow) .cpp

155 lines · C++ · 05 - 网络流

```
/*
用途：连续最短增广路最小费用最大流，支持初始负费用边。
约定：点编号 0..n-1，容量非负；不存在从源点可达的负费用环。
flow 不自动 reset，若需重复求解请先 reset。返回 nullopt 表示检测到可达负环。
复杂度：每次增广 Dijkstra  $O(m \log n)$ ，总次数至多为增广次数。
*/
```

```
template <class Cap = int, class C = int> struct MCMF {
    static constexpr Cap FI = numeric_limits<Cap>::max() / 4;
    static constexpr C CI = numeric_limits<C>::max() / 4;

    struct Edge {
        int v, r;
        Cap c, oc;
        C w;
    };

    struct EdgeInfo {
        int u, v;
        Cap c, f;
        C w;
    };

    struct Result {
        Cap f;
        C c;
    };

    int n;
    vector<vector<Edge>> g;
    vector<pair<int, int>> pos;

    explicit MCMF(int n) : n(n), g(n) {}

    int add(int u, int v, Cap c, C w) {
        assert(0 <= u && u < n && 0 <= v && v < n && c >= 0);
        int id = g[u].size();
        int r = g[v].size() + (u == v);
        g[u].push_back({v, r, c, c, w});
        g[v].push_back({u, id, 0, 0, -w});
        pos.push_back({u, id});
        return pos.size() - 1;
    }

    void reset() {
        for (auto& a : g) {
            for (Edge& e : a) e.c = e.oc;
        }
    }

    EdgeInfo askEdge(int id) const {
        assert(0 <= id && id < pos.size());
        auto [u, i] = pos[id];
        const Edge& e = g[u][i];
        return {u, e.v, e.oc, e.oc - e.c, e.w};
    }

    vector<EdgeInfo> askAll() const {
        vector<EdgeInfo> res;
        res.reserve(pos.size());
```

```

    for (int id = 0; id < pos.size(); ++id) res.push_back(askEdge(id));
    return res;
}

optional<Result> flow(int s, int t, Cap lim = FI) {
    static_assert(std::is_signed_v<C>, "Cost must be signed because reverse edges negate it.");
    assert(0 <= s && s < n && 0 <= t && t < n && s != t && lim >= 0);

    vector<C> h;
    if (!initPot(s, h)) return nullopt;

    Cap f = 0;
    C c = 0;
    while (f < lim) {
        vector<C> dis(n, CI);
        vector<pair<int, int>> fa(n, {-1, -1});
        priority_queue<pair<C, int>, vector<pair<C, int>>, greater<pair<C, int>>> pq;
        dis[s] = 0;
        pq.push({0, s});

        while (!pq.empty()) {
            auto [du, u] = pq.top();
            pq.pop();
            if (du != dis[u]) continue;
            for (int i = 0; i < g[u].size(); ++i) {
                const Edge& e = g[u][i];
                if (e.c <= 0) continue;
                C rc = e.w + h[u] - h[e.v];
                if (dis[e.v] > du + rc) {
                    dis[e.v] = du + rc;
                    fa[e.v] = {u, i};
                    pq.push({dis[e.v], e.v});
                }
            }
        }
        if (fa[t].first == -1) break;

        for (int u = 0; u < n; ++u) {
            if (dis[u] != CI) h[u] += dis[u];
        }

        Cap aug = lim - f;
        C pc = 0;
        for (int u = t; u != s; ) {
            auto [pre, i] = fa[u];
            aug = min(aug, g[pre][i].c);
            pc += g[pre][i].w;
            u = pre;
        }
        for (int u = t; u != s; ) {
            auto [pre, i] = fa[u];
            Edge& e = g[pre][i];
            e.c -= aug;
            g[e.v][e.r].c += aug;
            u = pre;
        }
        f += aug;
        c += static_cast<C>(aug) * pc;
    }
    return Result{f, c};
}

private:
    bool initPot(int s, vector<C>& h) const {

```

```
vector<C> dis(n, CI);
vector<int> pec(n);
vector<char> inq(n, false);
queue<int> q;
dis[s] = 0;
q.push(s);
inq[s] = true;

while (!q.empty()) {
    int u = q.front();
    q.pop();
    inq[u] = false;
    for (const Edge& e : g[u]) {
        if (e.c <= 0 || dis[e.v] <= dis[u] + e.w) continue;
        dis[e.v] = dis[u] + e.w;
        pec[e.v] = pec[u] + 1;
        if (pec[e.v] >= n) return false;
        if (!inq[e.v]) {
            inq[e.v] = true;
            q.push(e.v);
        }
    }
}

h.assign(n, 0);
for (int u = 0; u < n; ++u) {
    if (dis[u] != CI) h[u] = dis[u];
}
return true;
}
};
```

05 - 上下界网络流.cpp

98 lines · C++ · 05 - 网络流

```

/*
用途：带下界/上界的可行循环流，以及带源汇的最大流。
依赖：01A - 最大流 (Dinic)。
约定：点编号  $0..n-1$ ， $\text{add}(u,v,lo,hi)$  满足  $0 \leq lo \leq hi$ 。
ask 求可行循环流； $\text{flow}(s,t)$  求满足上下界后的最大 S-T 流。
复杂度：常数次 Dinic 最大流。
*/

template <class Cap = int> struct BoundFlow {
    static constexpr Cap INF = numeric_limits<Cap>::max() / 4;

    struct Edge {
        int u, v;
        Cap lo, hi;
    };

    struct Result {
        bool ok;
        Cap val;           // 循环流时为 0；S-T 流时为最大流值。
        vector<Cap> a;      // 与 add 顺序对应，已包含下界。
    };

    int n;
    vector<Edge> e;

    explicit BoundFlow(int n) : n(n) {}

    int add(int u, int v, Cap lo, Cap hi) {
        assert(0 <= u && u < n && 0 <= v < n && 0 <= lo && lo <= hi);
        e.push_back({u, v, lo, hi});
        return e.size() - 1;
    }

    Result ask() const {
        static_assert(std::is_signed_v<Cap>, "Cap must be signed because balances may be negative.");
        Network z = build(false, -1, -1);
        Cap f = z.mf.flow(z.ss, z.tt);
        if (f != z.nd) return {false, 0, {}};
        return {true, 0, get(z)};
    }

    Result flow(int s, int t) const {
        static_assert(std::is_signed_v<Cap>, "Cap must be signed because balances may be negative.");
        assert(0 <= s && s < n && 0 <= t && t < n && s != t);
        Network z = build(true, s, t);
        Cap f = z.mf.flow(z.ss, z.tt);
        if (f != z.nd) return {false, 0, {}};

        Cap bas = z.mf.askFlow(z.rid);
        for (int id : z.aid) z.mf.erase(id);
        z.mf.erase(z.rid);
        Cap add = z.mf.flow(s, t);
        return {true, bas + add, get(z)};
    }

private:
    struct Network {
        MaxFlow<Cap> mf;
        int ss, tt;
        Cap nd = 0;
    };

```

```
vector<int> eid;
vector<int> aid;
int rid = -1;

explicit Network(int n) : mf(n + 2), ss(n), tt(n + 1) {}
};

Network build(bool bk, int s, int t) const {
    Network z(n);
    vector<Cap> bal(n, 0);
    z.eid.reserve(e.size());
    for (const Edge& x : e) {
        z.eid.push_back(z.mf.add(x.u, x.v, x.hi - x.lo));
        bal[x.u] -= x.lo;
        bal[x.v] += x.lo;
    }
    if (bk) {
        z.rid = z.mf.add(t, s, INF);
    }
    for (int u = 0; u < n; ++u) {
        if (bal[u] > 0) {
            z.aid.push_back(z.mf.add(z.ss, u, bal[u]));
            z.nd += bal[u];
        } else if (bal[u] < 0) {
            z.aid.push_back(z.mf.add(u, z.tt, -bal[u]));
        }
    }
    return z;
}

vector<Cap> get(const Network& z) const {
    vector<Cap> res(e.size());
    for (int id = 0; id < e.size(); ++id) {
        res[id] = e[id].lo + z.mf.askFlow(z.eid[id]);
    }
    return res;
}
};
```

01 - 素数（线性筛）.cpp

63 lines · C++ · 06 - 数学

```

/*
用途：一次预处理 1..n 的最小质因子、质数表、欧拉函数和莫比乌斯函数。
复杂度：预处理  $O(n)$ ，空间  $O(n)$ 。适合 n 能整体预处理（通常  $\leq 1e7$ ）的场景。
接口：lp[x]、pri、phi[x]、mu[x]、ask(x)、factor(x)。
若 n 很大而查询很少：改用 01F - Miller-Rabin + 01FA - Pollard-Rho，不要开大筛。
*/
struct Sieve {
    int n = 0;
    vector<int> pri, lp, phi, mu;

    Sieve() = default;
    explicit Sieve(int n) { init(n); }

    void init(int _n) {
        assert(_n >= 0);
        n = _n;
        pri.clear();
        lp.assign(n + 1, 0);
        phi.assign(n + 1, 0);
        mu.assign(n + 1, 0);
        if (n >= 1) {
            lp[1] = phi[1] = mu[1] = 1;
        }
        for (int x = 2; x <= n; ++x) {
            if (lp[x] == 0) {
                lp[x] = x;
                pri.push_back(x);
                phi[x] = x - 1;
                mu[x] = -1;
            }
            for (int p : pri) {
                if (p > n / x) break;
                int y = p * x;
                lp[y] = p;
                if (x % p == 0) {
                    phi[y] = phi[x] * p;
                    mu[y] = 0;
                    break;
                }
                phi[y] = phi[x] * (p - 1);
                mu[y] = -mu[x];
            }
        }
    }

    bool ask(int x) const {
        return 2 <= x && x <= n && lp[x] == x;
    }

    vector<pair<int, int>> factor(int x) const {
        assert(1 <= x && x <= n);
        vector<pair<int, int>> res;
        while (x > 1) {
            int p = lp[x], e = 0;
            do {
                x /= p;
                ++e;
            } while (x % p == 0);
            res.push_back({p, e});
        }
    }
}

```

```
        return res;
    }
};
```


01A - 质因数分解（最小质因子）.cpp

18 lines · C++ · 06 - 数学

```
/*
用途：使用最小质因子表分解小范围整数。
前提：先构造 LinearSieve sieve(maxValue)，且  $1 \leq n \leq \text{sieve.limit}$ 。
复杂度： $O(\log n)$ ，适合大量、范围已知的查询；大整数请使用 01FA - Pollard-Rho。
*/
inline vector<pair<int, int>> factor(int n, const vector<int>& lp) {
    assert(1 <= n && n < lp.size());
    vector<pair<int, int>> res;
    while (n > 1) {
        int p = lp[n], e = 0;
        do {
            n /= p;
            ++e;
        } while (n % p == 0);
        res.push_back({p, e});
    }
    return res;
}
```

01B - 欧拉函数（单点试除）.cpp

16 lines · C++ · 06 - 数学

```
/*
用途：计算单个  $\phi(n)$ 。
复杂度： $O(\sqrt{n})$ ，无须预处理；需要整张  $\phi$  表时使用 01C，需要  $\text{minPrime} / \mu$  等更多信息时使用 01 - 素数（线性筛）。
*/

inline int phi(int n) {
    assert(n >= 1);
    int res = n;
    for (int p = 2; p <= n / p; ++p) {
        if (n % p != 0) continue;
        while (n % p == 0) n /= p;
        res = res / p * (p - 1);
    }
    if (n > 1) res = res / n * (n - 1);
    return res;
}
```

01C - 欧拉函数（整表筛）.cpp

15 lines · C++ · 06 - 数学

```
/*
用途：预处理  $\phi(1..n)$ 。
复杂度： $O(n \log \log n)$ ；若同时还需  $\minPrime / \mu$ ，使用 01 - 素数（线性筛）的 LinearSieve::phi (0(n))。
*/
inline vector<int> phiAll(int n) {
    assert(n >= 0);
    vector<int> phi(n + 1);
    iota(phi.begin(), phi.end(), 0);
    if (n >= 1) phi[1] = 1;
    for (int p = 2; p <= n; ++p) {
        if (phi[p] != p) continue;
        for (int x = p; x <= n; x += p) phi[x] = phi[x] / p * (p - 1);
    }
    return phi;
}
```

01D - 莫比乌斯函数 (Mertens、杜教筛) .cpp

35 lines · C++ · 06 - 数学

```

/*
用途：杜教筛求  $M(n) = \sum_{i \leq n} \mu(i)$ 。
依赖：01 - 素数（线性筛 / Sieve）。预处理范围  $\text{lim}$  内  $O(1)$ ，超过后按整除分块递归并记忆化。
通常取  $\text{lim}$  约  $n^{2/3}$  或按内存设置；若只需  $\mu[1..N]$ ，直接使用 Sieve::mu。
*/

struct Mertens {
    Sieve sv;
    vector<int> pre;
    unordered_map<int, int> mp;

    Mertens() = default;
    explicit Mertens(int lim) { init(lim); }

    void init(int lim) {
        assert(lim >= 1);
        sv.init(lim);
        pre.assign(lim + 1, 0);
        for (int i = 1; i <= lim; ++i) pre[i] = pre[i - 1] + sv.mu[i];
        mp.clear();
    }

    int ask(int n) {
        assert(n >= 0 && sv.n > 0);
        if (n <= sv.n) return pre[n];
        if (auto it = mp.find(n); it != mp.end()) return it->second;
        int ans = 1;
        for (int l = 2, r; l <= n; l = r + 1) {
            int q = n / l;
            r = n / q;
            ans -= (r - l + 1) * ask(q);
        }
        return mp[n] = ans;
    }
};

```

01DA - 莫比乌斯反演（约数和变换）.cpp

45 lines · C++ · 06 - 数学

```

/*
莫比乌斯反演的两种常见方向（下标从 1 开始，mu 取 LinearSieve::mu）：

$$F(n) = \sum_{d|n} f(d) \iff f(n) = \sum_{d|n} \mu(d) F(n/d);$$


$$G(n) = \sum_{n|d} f(d) \iff f(n) = \sum_{k \geq 1} \mu(k) G(nk)。$$

以下函数都是  $O(N \log N)$ ，用于把公式直接落实为数组变换。
*/
template <class T> vector<T> zetaDiv(const vector<T>& f) {
    int n = f.size() - 1;
    assert(n >= 0);
    vector<T> res(n + 1, T(0));
    for (int d = 1; d <= n; ++d)
        for (int x = d; x <= n; x += d) res[x] += f[d];
    return res;
}

template <class T> vector<T> mobDiv(const vector<T>& F, const vector<int>& mu) {
    int n = F.size() - 1;
    assert(mu.size() > n);
    vector<T> res(n + 1, T(0));
    for (int d = 1; d <= n; ++d) {
        if (mu[d] == 0) continue;
        for (int x = d; x <= n; x += d) res[x] += T(mu[d]) * F[x / d];
    }
    return res;
}

template <class T> vector<T> zetaMul(const vector<T>& f) {
    int n = f.size() - 1;
    assert(n >= 0);
    vector<T> res(n + 1, T(0));
    for (int d = 1; d <= n; ++d)
        for (int x = d; x <= n; x += d) res[d] += f[x];
    return res;
}

template <class T> vector<T> mobMul(const vector<T>& G, const vector<int>& mu) {
    int n = G.size() - 1;
    assert(mu.size() > n);
    vector<T> res(n + 1, T(0));
    for (int k = 1; k <= n; ++k) {
        if (mu[k] == 0) continue;
        for (int d = 1; d * k <= n; ++d) res[d] += T(mu[k]) * G[d * k];
    }
    return res;
}

```

01DB - 加权莫比乌斯函数 (d^2) .cpp

59 lines · C++ · 06 - 数学

```

/*
用途：杜教筛求  $S(n)=\sum_{d\leq n} \mu(d)*d^2$  (模质数)。
这是 01D 的加权版本，不能用普通  $M(n)$  替代；常用于带  $d^2$  权值的整除分块式。
依赖：02A - 自动取模 (Mint)。Mint 的模数应为大于 3 的质数。
*/

template <class Z> struct Mobius2 {
    int n = 0;
    vector<int> lp, pri, mu;
    vector<Z> pre;
    unordered_map<int, Z> mp;

    Mobius2() = default;
    explicit Mobius2(int lim) { init(lim); }

    void init(int lim) {
        assert(lim >= 1);
        n = lim;
        lp.assign(n + 1, 0);
        mu.assign(n + 1, 0);
        pri.clear();
        pre.assign(n + 1, Z(0));
        lp[1] = 1;
        mu[1] = 1;
        for (int x = 2; x <= n; ++x) {
            if (lp[x] == 0) lp[x] = x, mu[x] = -1, pri.push_back(x);
            for (int p : pri) {
                if (p > n / x) break;
                int y = p * x;
                lp[y] = p;
                if (x % p == 0) {
                    mu[y] = 0;
                    break;
                }
                mu[y] = -mu[x];
            }
        }
        for (int i = 1; i <= n; ++i) pre[i] = pre[i - 1] + Z(mu[i]) * Z(i) * i;
        mp.clear();
    }

    static Z sum2(int n) {
        Z x = Z(n);
        return x * (x + 1) * (x * 2 + 1) / Z(6);
    }

    Z ask(int x) {
        assert(x >= 0 && n > 0);
        if (x <= n) return pre[x];
        if (auto it = mp.find(x); it != mp.end()) return it->second;
        Z ans = 1;
        for (int l = 2, r; l <= x; l = r + 1) {
            int q = x / l;
            r = x / q;
            ans -= (sum2(r) - sum2(l - 1)) * ask(q);
        }
        return mp[x] = ans;
    }
};

```

01DC - 狄利克雷卷积与逆.cpp

35 lines · C++ · 06 - 数学

```

/*
用途：在下标 1..N 上计算 Dirichlet convolution：
    (f*g)(n)=sum_{d|n} f(d)g(n/d)，并求 f 的 Dirichlet 逆。
前提：数组下标 0 被忽略；求逆要求 f(1) 可逆。复杂度 O(N log N)。
关系：与 01DA 的约数 zeta / Möbius 变换配套使用。
*/
namespace Dirichlet {
template <class T> vector<T> mul(const vector<T>& l, const vector<T>& r) {
    int n = min(l.size(), r.size()) - 1;
    assert(n >= 0);
    vector<T> ans(n + 1, T(0));
    for (int d = 1; d <= n; ++d) {
        for (int k = d; k <= n; k += d) {
            ans[k] += l[d] * r[k / d];
        }
    }
    return ans;
}

template <class Z> vector<Z> inverse(const vector<Z>& f) {
    int n = f.size() - 1;
    assert(n >= 1 && f[1] != Z(0));
    vector<Z> ans(n + 1, Z(0));
    ans[1] = f[1].inv();
    vector<Z> sum(n + 1, Z(0));
    for (int d = 2; d <= n; ++d) sum[d] += f[d] * ans[1];
    for (int val = 2; val <= n; ++val) {
        ans[val] = -sum[val] * ans[1];
        for (int d = 2; val <= n / d; ++d) {
            sum[val * d] += f[d] * ans[val];
        }
    }
    return ans;
}
} // namespace Dirichlet

```

01EA - 检查是否是素数的幂（试除）.cpp

14 lines · C++ · 06 - 数学

/*

用途：判断 n 是否可写成 p^k (p 为质数, $k \geq 1$)。

复杂度： $O(\sqrt{n})$ ，无预处理；适合单次或少量查询。已筛范围内的大量查询使用 01EB。

*/

```
inline bool ask(int n) {
    if (n <= 1) return false;
    for (int p = 2; p <= n / p; ++p) {
        if (n % p != 0) continue;
        do n /= p; while (n % p == 0);
        return n == 1;
    }
    return true; // 没有真因子，n 本身为质数。
}
```


01EB - 检查是否是素数的幂（最小质因子）.cpp

12 lines · C++ · 06 - 数学

```
/*
用途：判断  $n$  是否可写成  $p^k$  ( $p$  为质数,  $k \geq 1$ )。
前提：使用 01 - 素数（线性筛）得到 minPrime, 且  $n$  在筛范围内。
复杂度： $O(\log n)$ ；大量、范围已知的查询优于 01EA。
*/
inline bool ask(int n, const vector<int>& lp) {
    if (n <= 1 || n >= lp.size()) return false;
    int p = lp[n];
    if (p <= 1) return false;
    do n /= p; while (n % p == 0);
    return n == 1;
}
```

01F - Miller-Rabin 素性检验.cpp

63 lines · C++ · 06 - 数学

```

/*
用途：64 位整数确定性素性检验。
复杂度： $O(\log n)$  次模乘；下列底数对 u64 范围确定正确。
环境：GCC/Clang 用 int 为  $O(1)$  模乘；其他编译器自动退化到  $O(\log \text{mod})$  加法模乘。
*/

using u64 = uint64_t;
using u128 = __uint128_t;

namespace Miller {

inline u64 mul(u64 a, u64 b, u64 mod) {
    assert(mod > 0);
    a %= mod;
    b %= mod;
#ifdef __SIZEOF_INT128__
    return u128(a) * b % mod;
#else
    u64 res = 0;
    while (b > 0) {
        if (b & 1) res = res >= mod - a ? res - (mod - a) : res + a;
        a = a >= mod - a ? a - (mod - a) : a + a;
        b >>= 1;
    }
    return res;
#endif
}

inline u64 power(u64 a, u64 e, u64 mod) {
    u64 res = 1 % mod;
    while (e > 0) {
        if (e & 1) res = mul(res, a, mod);
        a = mul(a, a, mod);
        e >>= 1;
    }
    return res;
}

inline bool ask(u64 n) {
    if (n < 2) return false;
    for (u64 p : {2ULL, 3ULL, 5ULL, 7ULL, 11ULL, 13ULL, 17ULL, 19ULL, 23ULL, 29ULL, 31ULL, 37ULL}) {
        if (n % p == 0) return n == p;
    }
    u64 d = n - 1;
    int s = 0;
    while ((d & 1) == 0) d >>= 1, ++s;
    for (u64 a : {2ULL, 325ULL, 9375ULL, 28178ULL, 450775ULL, 9780504ULL, 1795265022ULL}) {
        if (a % n == 0) continue;
        u64 x = power(a % n, d, n);
        if (x == 1 || x == n - 1) continue;
        bool ok = true;
        for (int r = 1; r < s; ++r) {
            x = mul(x, x, n);
            if (x == n - 1) {
                ok = false;
                break;
            }
        }
        if (!ok) return false;
    }
}

```

```
        return true;
    }
} // namespace Miller
```

01FA - Pollard-Rho 质因数分解.cpp

58 lines · C++ · 06 - 数学

```

/*
用途：分解 u64 范围的大整数。
依赖：01F - Miller-Rabin 素性检验。
复杂度：期望约  $O(n^{1/4})$ ；适合 n 很大、查询不多，不能替代小范围线性筛。
*/

using u64 = uint64_t;

namespace Pollard {

inline u64 add(u64 a, u64 b, u64 mod) {
    return a >= mod - b ? a - (mod - b) : a + b;
}

inline u64 next(u64 x, u64 c, u64 mod) {
    return add(Miller::mul(x, x, mod), c, mod);
}

inline u64 rand(u64 l, u64 r) {
    static mt19937_64 rng(chrono::steady_clock::now().time_since_epoch().count());
    return uniform_int_distribution<u64>(l, r)(rng);
}

inline u64 find(u64 n) {
    if (n % 2 == 0) return 2;
    if (n % 3 == 0) return 3;
    while (true) {
        u64 c = rand(1, n - 1);
        u64 x = rand(0, n - 1), y = x, d = 1;
        while (d == 1) {
            x = next(x, c, n);
            y = next(next(y, c, n), c, n);
            u64 dif = x >= y ? x - y : y - x;
            d = gcd(dif, n);
        }
        if (d != n) return d;
    }
}

inline void dfs(u64 n, vector<u64>& fac) {
    if (n == 1) return;
    if (Miller::ask(n)) {
        fac.push_back(n);
        return;
    }
    u64 d = find(n);
    dfs(d, fac);
    dfs(n / d, fac);
}

inline vector<u64> factor(u64 n) {
    assert(n >= 1);
    vector<u64> fac;
    dfs(n, fac);
    sort(fac.begin(), fac.end());
    return fac;
}

} // namespace Pollard

```

01G - 素性检验（试除）.cpp

12 lines · C++ · 06 - 数学

/*

用途：无预处理地判断单个小整数是否为质数。

复杂度： $O(\sqrt{n})$ 。与 01F 是同一问题的不同环境：小数、少量查询用本模板；int 大数用 Miller-Rabin。

*/

```
inline bool ask(int x) {
    if (x < 2) return false;
    for (int d = 2; d <= x / d; ++d) {
        if (x % d == 0) return false;
    }
    return true;
}
```

01H - 约数函数（枚举、个数、和）.cpp

70 lines · C++ · 06 - 数学

```

/*
用途：已知  $n = \prod p_i^{e_i}$  的质因数分解后，枚举全部约数并计算约数个数、约数和、欧拉函数。
依赖：分解可由 01A（小范围）或 01FA（64 位）得到。约数和以 i128 返回，避免中间溢出。
复杂度：枚举  $O(\tau(n))$ ，其余  $O(\text{质因子种数})$ 。
*/

using i128 = __int128_t;

namespace Divisor {
using Factor = vector<pair<int, int>>;

inline void check(const Factor& fac) {
    for (auto [pri, e] : fac) assert(pri > 1 && e >= 1);
}

inline vector<int> getDiv(const Factor& fac) {
    check(fac);
    vector<int> d{1};
    for (auto [pri, e] : fac) {
        int n = d.size();
        int pw = 1;
        for (int i = 1; i <= e; ++i) {
            assert(pw <= numeric_limits<int>::max() / pri);
            pw *= pri;
            for (int i = 0; i < n; ++i) {
                assert(d[i] <= numeric_limits<int>::max() / pw);
                d.push_back(d[i] * pw);
            }
        }
    }
    sort(d.begin(), d.end());
    return d;
}

inline int askCnt(const Factor& fac) {
    check(fac);
    int ans = 1;
    for (auto [pri, e] : fac) {
        assert(ans <= numeric_limits<int>::max() / (e + 1));
        ans *= e + 1;
    }
    return ans;
}

inline i128 askSum(const Factor& fac) {
    check(fac);
    i128 ans = 1;
    for (auto [pri, e] : fac) {
        i128 pw = 1, sum = 1;
        for (int i = 1; i <= e; ++i) {
            pw *= pri;
            sum += pw;
        }
        ans *= sum;
    }
    return ans;
}

inline int phi(const Factor& fac) {
    check(fac);

```

```
    i128 ans = 1;
    for (auto [pri, e] : fac) {
        i128 pk = 1;
        for (int i = 1; i < e; ++i) pk *= pri;
        ans *= pk * (pri - 1);
    }
    assert(ans <= numeric_limits<int>::max());
    return ans;
}
} // namespace Divisor
```

011- 质数计数 (Lehmer) .cpp

85 lines · C++ · 06 - 数学

```

/*
用途：计算  $\pi(n)$ ，即不超过  $n$  的质数个数。
复杂度：Lehmer prime counting，单次通常远快于开到  $n$  的筛；默认预处理到  $5e6$ ，适合  $n$  约  $1e13$  及以内。
前提：若  $n$  更大， $\text{init}(\text{limit})$  的  $\text{limit}$  至少应覆盖  $\sqrt{n}$ ；大量小范围查询直接使用 01 - 线性筛。
*/

using i128 = __int128_t;

using ld = long double;

struct Lehmer {
    int lim = 0;
    vector<int> pri, pc;
    unordered_map<int, int> mp;

    explicit Lehmer(int mx = 5'000'000) {
        init(mx);
    }

    void init(int mx) {
        assert(mx >= 2);
        lim = mx;
        vector<bool> vis(mx + 1, false);
        pc.assign(mx + 1, 0);
        pri.clear();
        for (int x = 2; x <= mx; ++x) {
            if (!vis[x]) pri.push_back(x);
            for (int p : pri) {
                if (p > mx / x) break;
                vis[p * x] = true;
                if (x % p == 0) break;
            }
        }
        for (int x = 1; x <= mx; ++x) pc[x] = pc[x - 1] + (!vis[x] && x >= 2);
        mp.clear();
    }

    static int isqrt(int val) {
        assert(val >= 0);
        int rt = sqrtl(ld(val));
        while (i128(rt + 1) * (rt + 1) <= val) ++rt;
        while (i128(rt) * rt > val) --rt;
        return rt;
    }

    static int icbrt(int val) {
        assert(val >= 0);
        int rt = cbrtl(ld(val));
        while (i128(rt + 1) * (rt + 1) * (rt + 1) <= val) ++rt;
        while (i128(rt) * rt * rt > val) --rt;
        return rt;
    }

    int phi(int val, int pi) const {
        if (pi == 0) return val;
        if (pi == 1) return val - val / 2;
        if (pi < pri.size() &&
            pri[pi - 1] * pri[pi - 1] > val) {
            assert(val <= lim);
            return pc[val] - pi + 1;
        }
    }
}

```



```
    }
    return phi(val, pi - 1) - phi(val / pri[pi - 1], pi - 1);
}

int ask(int val) {
    assert(val >= 0);
    if (val <= lim) return pc[val];
    if (auto it = mp.find(val); it != mp.end()) return it->second;

    int a = ask(isqrt(isqrt(val)));
    int b = ask(isqrt(val));
    int c = ask(icbrt(val));
    assert(b <= pri.size());
    int ans = phi(val, a) + (b + a - 2) * (b - a + 1) / 2;
    for (int i = a; i < b; ++i) {
        int q = val / pri[i];
        ans -= ask(q);
        if (i < c) {
            int hi = ask(isqrt(q));
            for (int j = i; j < hi; ++j) ans -= ask(q / pri[j]) - j;
        }
    }
    return mp[val] = ans;
}

};
```

01J - 质数和 (Min_25筛) .cpp

71 lines · C++ · 06 - 数学

```

/*
用途：求不超过 n 的所有质数之和；这是 Min_25 筛的基础质数幂和筛法。
复杂度：约  $O(n^{3/4}/\log n)$  时间、 $O(\sqrt{n})$  空间，适合 n 远大于可直接筛的范围。
T 默认 i128 做精确计算，也可传 MInt 自动取模；三角和先整除 2，兼容偶数模数。
性能：默认使用宏展开后的 int；若大规模数据因此超时或超空间，可将纯下标、状态或计数局部改为 i32。
*/

using i128 = __int128_t;

using ld = long double;

template <class T = i128> struct Min25 {
    int n = 0, s = 0;
    vector<int> w;
    vector<int> id1, id2, pr;
    vector<T> g, pre;

    int id(int x) const { return x <= s ? id1[x] : id2[n / x]; }

    static T tri(int x) {
        int a = x, b = x + 1;
        if (a & 1) b >>= 1;
        else a >>= 1;
        return T(a) * T(b);
    }

    T askSum(int _n) {
        assert(_n >= 0);
        n = _n;
        if (n < 2) return T{};
        s = sqrtl(static_cast<ld>(n));
        while ((s + 1) <= n / (s + 1)) ++s;
        while (s > n / s) --s;

        vector<char> vis(s + 1);
        pr.clear();
        pre.assign(1, T{});
        for (int i = 2; i <= s; ++i) {
            if (!vis[i]) {
                pr.push_back(i);
                pre.push_back(pre.back() + T(i));
            }
            for (int p : pr) {
                if (i * p > s) break;
                vis[i * p] = true;
                if (i % p == 0) break;
            }
        }

        w.clear(), g.clear();
        id1.assign(s + 1, 0), id2.assign(s + 1, 0);
        for (int l = 1, r; l <= n; l = r + 1) {
            int v = n / l;
            r = n / v;
            int k = w.size();
            w.push_back(v);
            g.push_back(tri(v) - T(1));
            if (v <= s) id1[v] = k;
            else id2[n / v] = k;
        }
    }
};

```

```
    for (int i = 0; i < pr.size(); ++i) {
        int p = pr[i];
        if (p > n / p) break;
        int pp = p * p;
        for (int j = 0; j < w.size() && w[j] >= pp; ++j)
            g[j] -= T(p) * (g[id(w[j] / p)] - pre[i]);
    }
    return g[id(n)];
};
```

02 - 快速幂 (ksm) .cpp

44 lines · C++ · 06 - 数学

```

/*
用途：64 位安全模乘、模幂与费马逆元。
前提：inv 仅在 mod 为质数且 a 非 0 (mod mod) 时使用。
环境：GCC/Clang 的 int 为  $O(\log e)$ ；无 int 时模乘退化为  $O(\log \log \text{mod})$ 。
*/

using u64 = uint64_t;
using u128 = __uint128_t;

namespace ModArith64 {

inline u64 mul(u64 a, u64 b, u64 mod) {
    assert(mod > 0);
    a %= mod;
    b %= mod;
#ifdef __SIZEOF_INT128__
    return u128(a) * b % mod;
#else
    u64 res = 0;
    while (b > 0) {
        if (b & 1) res = res >= mod - a ? res - (mod - a) : res + a;
        a = a >= mod - a ? a - (mod - a) : a + a;
        b >>= 1;
    }
    return res;
#endif
}

inline u64 power(u64 a, u64 e, u64 mod) {
    assert(mod > 0);
    u64 res = 1 % mod;
    while (e > 0) {
        if (e & 1) res = mul(res, a, mod);
        a = mul(a, a, mod);
        e >>= 1;
    }
    return res;
}

inline u64 inv(u64 a, u64 mod) {
    assert(mod > 1 && a % mod != 0);
    return power(a % mod, mod - 2, mod);
}
} // namespace ModArith64

```

02A - 自动取模 (MInt) .cpp

153 lines · C++ · 06 - 数学

```

/*
用途：自动取模整数。直接写 MInt<P>，P=0 时先 MInt<0>::setMod(p)。
接口：pow(n)、inv()、四则运算、输入输出；默认 Z 为模 998244353。
约定：inv()/除法要求模数为质数；动态模数需满足  $1 < p < 2^{31}$ 。
性能：内部值固定为 i32，故在 #define int long long 下也不会把大数组翻倍。
*/

using i32 = int32_t;

constexpr i32 mod = 998244353;

template <class T> T mypow(T a, int n) {
    assert(n >= 0);
    T r = T(1);
    while (n) {
        if (n & 1) r *= a;
        a *= a;
        n >>= 1;
    }
    return r;
}

template <i32 P> struct MInt {
    static_assert(P >= 0);

    i32 x = 0;
    inline static i32 md = P ? P : mod;

    MInt() = default;
    MInt(int v) : x(norm(v)) {}

    static i32 askMod() {
        return P ? P : md;
    }

    static void setMod(i32 v) {
        static_assert(P == 0, "Only MInt<0> supports a dynamic modulus.");
        assert(v > 1);
        md = v;
    }

    static i32 norm(int v) {
        v %= askMod();
        if (v < 0) v += askMod();
        return static_cast<i32>(v);
    }

    i32 val() const {
        return x;
    }

    explicit operator i32() const {
        return x;
    }

    bool ask0() const {
        return x == 0;
    }

    MInt pow(int n) const {

```

```
        return mypow(*this, n);
    }

    MInt inv() const {
        assert(x);
        return pow(askMod() - 2);
    }

    MInt operator-() const {
        return MInt(x ? askMod() - x : 0);
    }

    MInt& operator+=(MInt a) {
        i32 p = askMod();
        if (x >= p - a.x) x -= p - a.x;
        else x += a.x;
        return *this;
    }

    MInt& operator--(MInt a) {
        x -= a.x;
        if (x < 0) x += askMod();
        return *this;
    }

    MInt& operator*=(MInt a) {
        x = static_cast<i32>(static_cast<int>(x) * a.x % askMod());
        return *this;
    }

    MInt& operator/=(MInt a) {
        return *this *= a.inv();
    }

    MInt& operator++() {
        return *this += 1;
    }

    MInt operator++(i32) {
        MInt a = *this;
        ++*this;
        return a;
    }

    MInt& operator--() {
        return *this -= 1;
    }

    MInt operator--(i32) {
        MInt a = *this;
        --*this;
        return a;
    }

    friend MInt operator+(MInt a, MInt b) {
        return a += b;
    }

    friend MInt operator-(MInt a, MInt b) {
        return a -= b;
    }

    friend MInt operator*(MInt a, MInt b) {
        return a *= b;
    }
```

```
    }

    friend MInt operator/(MInt a, MInt b) {
        return a /= b;
    }

    friend bool operator==(MInt a, MInt b) {
        return a.x == b.x;
    }

    friend bool operator!=(MInt a, MInt b) {
        return a.x != b.x;
    }

    friend istream& operator>>(istream& is, MInt& a) {
        int x;
        is >> x;
        a = MInt(x);
        return is;
    }

    friend ostream& operator<<(ostream& os, const MInt& a) {
        return os << a.x;
    }
};

template <i32 V, i32 P> const MInt<P> IV = MInt<P>(V).inv();

using Z = MInt<mod>;
```

02B - 裴蜀定理.cpp

15 lines · C++ · 06 - 数学

```
/*
裴蜀定理：整数  $a_1 \dots a_k$  的所有线性组合恰为  $\gcd(a_1, \dots, a_k)$  的倍数。
用途：判断方程  $\sum a_i * x_i = \text{target}$  是否有整数解。
*/

inline int gcdAll(const vector<int>& a) {
    int res = 0;
    for (int val : a) res = gcd(res, val);
    return res >= 0 ? res : -res;
}

inline bool ask(const vector<int>& coe, int t) {
    int d = gcdAll(coe);
    return d == 0 ? t == 0 : t % d == 0;
}
```


02BA - 扩展欧几里得 (exgcd) .cpp

23 lines · C++ · 06 - 数学

```
/*
用途：求  $ax + by = \gcd(a, b)$  的一组系数，以及任意模数下的逆元。
 $\text{inv}(a, m)$  有值当且仅当  $\gcd(a, m) = 1$ ； $m$  不要求为质数。
*/

struct ExgcdRes {
    int gcd, x, y;
};

inline ExgcdRes exgcd(int a, int b) {
    if (b == 0) return {a >= 0 ? a : -a, a >= 0 ? 1 : -1, 0};
    auto sub = exgcd(b, a % b);
    return {sub.gcd, sub.y, sub.x - (a / b) * sub.y};
}

inline optional<int> inv(int a, int mod) {
    assert(mod > 0);
    auto res = exgcd(a, mod);
    if (res.gcd != 1) return nullopt;
    int inv = res.x % mod;
    if (inv < 0) inv += mod;
    return inv;
}
```

02BAA - 中国剩余定理.cpp

57 lines · C++ · 06 - 数学

```

/*
用途：广义中国剩余定理，合并  $x \equiv \text{residue} \pmod{\text{modulus}}$ ；模数不要求互质。
依赖：02 - 快速幂 (ModArith64) 与 02BA - 扩展欧几里得 (inv)。
返回 nullopt：方程无解，或合并后的 lcm 超出 int。
*/

struct CRTResult {
    int r; // 始终规范到  $[0, \text{modulus})$ 。
    int mod;
};

inline int norm(int val, int mod) {
    val %= mod;
    return val < 0 ? val + mod : val;
}

inline optional<CRTResult> merge(CRTResult x, CRTResult y) {
    assert(x.mod > 0 && y.mod > 0);
    x.r = norm(x.r, x.mod);
    y.r = norm(y.r, y.mod);

    int g = gcd(x.mod, y.mod);
    int dif = y.r - x.r;
    if (dif % g != 0) return nullopt;

    int b = y.mod / g;
    if (x.mod > numeric_limits<int>::max() / b) return nullopt;
    int lcm = x.mod * b;

    int k = 0;
    if (b != 1) {
        auto iv = inv(x.mod / g, b);
        assert(iv.has_value());
        int rhs = norm(dif / g, b);
        k = ModArith64::mul(rhs, *iv, b);
    }

    int add = ModArith64::mul(x.mod, k, lcm);
    int r = x.r >= lcm - add
        ? x.r - (lcm - add)
        : x.r + add;

    return CRTResult{r, lcm};
}

inline optional<CRTResult> crt(const vector<CRTResult>& eq) {
    if (eq.empty()) return CRTResult{0, 1};
    CRTResult res = eq[0];
    if (res.mod <= 0) return nullopt;
    res.r = norm(res.r, res.mod);
    for (int i = 1; i < eq.size(); ++i) {
        if (eq[i].mod <= 0) return nullopt;
        auto z = merge(res, eq[i]);
        if (!z) return nullopt;
        res = *z;
    }
    return res;
}

```

02BAAB - 扩展 Lucas（组合数模合数）.cpp

86 lines · C++ · 06 - 数学

```

/*
用途：计算  $C(n, k) \bmod m$ ， $m$  可以是合数。
依赖：02 - ModArith64、02BA - inv、02BAA - 广义 CRT。
前提：每个质数幂  $p^q$  会预处理  $O(p^q)$  数组，适合  $m$  的各质因子幂可开（典型  $m \leq 2e6$ ）；质数模优先用 03 / 03A。
*/

namespace ExLucas {
struct Comb {
    int pri, pk;
    int pe = 0;
    vector<int> fac;

    Comb(int p, int pk, int pe)
        : pri(p), pk(pk), pe(pe) {
        assert(pri >= 2 && pk >= pri && pk <= numeric_limits<int>::max());
        fac.assign(pk + 1, 1);
        for (int val = 1; val <= pk; ++val) {
            fac[val] = fac[val - 1];
            if (val % pri != 0) {
                fac[val] = ModArith64::mul(fac[val], val, pk);
            }
        }
    }

    int cntP(int val) const {
        int ans = 0;
        while (val > 0) val /= pri, ans += val;
        return ans;
    }

    int facP(int val) const {
        if (val == 0) return 1;
        int ans = ModArith64::power(fac[pk], val / pk, pk);
        ans = ModArith64::mul(ans, fac[val % pk], pk);
        return ModArith64::mul(ans, facP(val / pri), pk);
    }

    int C(int n, int k) const {
        if (k < 0 || k > n) return 0;
        int e = cntP(n) - cntP(k) - cntP(n - k);
        if (e >= pe) return 0;

        int a = facP(n);
        int b1 = facP(k);
        int br = facP(n - k);
        auto il = inv(b1, pk);
        auto ir = inv(br, pk);
        assert(il && ir);
        int ans = ModArith64::mul(a, *il, pk);
        ans = ModArith64::mul(ans, *ir, pk);
        ans = ModArith64::mul(ans, ModArith64::power(pri, e, pk), pk);
        return ans;
    }
};

inline vector<tuple<int, int, int>> factor(int mod) {
    assert(mod >= 1);
    vector<tuple<int, int, int>> fac;
    for (int d = 2; d <= mod / d; ++d) {
        if (mod % d != 0) continue;

```

```
        int pk = 1;
        int e = 0;
        do {
            mod /= d;
            pk *= d;
            ++e;
        } while (mod % d == 0);
        fac.emplace_back(d, pk, e);
    }
    if (mod > 1) fac.emplace_back(mod, mod, 1);
    return fac;
}

inline int C(int n, int k, int mod) {
    assert(n >= 0 && mod > 0);
    if (k < 0 || k > n || mod == 1) return 0;
    vector<CRTResult> eq;
    for (auto [pri, pk, e] : factor(mod)) {
        Comb cal(pri, pk, e);
        eq.push_back({cal.C(n, k), pk});
    }
    auto res = crt(eq);
    assert(res && res->mod == mod);
    return res->r;
}
} // namespace ExLucas
```

02C - BSGS.cpp

71 lines · C++ · 06 - 数学

```

/*
用途：求最小非负  $x$ ，使  $a^x \equiv b \pmod{m}$ 。
依赖：02 - 快速幂 (ModArith64) 和 02BA - 扩展欧几里得 (inv)。
接口：bsgs 要求  $\gcd(a, m) = 1$ ；exBSGS 允许不互素。均返回 optional<int>。
复杂度： $O(\sqrt{m})$  时间和空间；模数需在 signed int 范围内。
*/
using ld = long double;

namespace BSGS {
inline int norm(int val, int mod) {
    val %= mod;
    return val < 0 ? val + mod : val;
}

inline optional<int> bsgs(int a, int b, int mod) {
    assert(mod > 0);
    if (mod == 1) return 0;
    a = norm(a, mod);
    b = norm(b, mod);
    if (b == 1) return 0;
    if (gcd(a, mod) != 1) return nullopt;

    int blk = sqrtl(ld(mod));
    while (blk < mod / blk || (blk == mod / blk && blk * blk < mod)) ++blk;

    unordered_map<int, int> bb;
    bb.reserve(blk * 2 + 1);
    int cur = 1;
    for (int j = 0; j < blk; ++j) {
        if (!bb.count(cur)) bb.emplace(cur, j);
        cur = ModArith64::mul(cur, a, mod);
    }

    auto ia = inv(a, mod);
    assert(ia.has_value());
    int gs = ModArith64::power(*ia, blk, mod);
    cur = b;
    for (int i = 0; i <= blk; ++i) {
        auto it = bb.find(cur);
        if (it != bb.end()) return i * blk + it->second;
        cur = ModArith64::mul(cur, gs, mod);
    }
    return nullopt;
}

inline optional<int> exBSGS(int a, int b, int mod) {
    assert(mod > 0);
    if (mod == 1) return 0;
    a = norm(a, mod);
    b = norm(b, mod);
    if (b == 1) return 0;

    int ban = 0, scl = 1;
    while (true) {
        int g = gcd(a, mod);
        if (g == 1) break;
        if (b == scl) return ban;
        if (b % g != 0) return nullopt;
        b /= g;
        mod /= g;
    }
}

```

```
        ++ban;
        if (mod == 1) return ban;
        scl = ModArith64::mul(scl, a / g, mod);
    }

    auto is = inv(scl, mod);
    assert(is.has_value());
    auto rem = bsgs(a, ModArith64::mul(b, *is, mod), mod);
    return rem ? optional<int>(*rem + ban) : nullopt;
}
} // namespace BSGS
```

02D - 逆元线性预处理.cpp

13 lines · C++ · 06 - 数学

```
/*
用途：预处理  $1..n$  在质数模下的逆元。
用法：auto iv = invAll<Z>(n)，其中 Z 可为任意 MInt<P>。
前提：Z::askMod() 为质数且  $n < \text{mod}$ ；复杂度  $O(n)$ 。
*/
template <class Z> vector<Z> invAll(int n) {
    assert(n >= 0 && n < Z::askMod());
    vector<Z> iv(n + 1);
    if (n) iv[1] = Z(1);
    int p = Z::askMod();
    for (int i = 2; i <= n; ++i) iv[i] = Z(p - p / i) * iv[p % i];
    return iv;
}
```

02E - 模平方根 (Tonelli-Shanks) .cpp

32 lines · C++ · 06 - 数学

```

/*
用途：求  $x^2 = a \pmod{p}$  的一组解；p 必须为质数。
依赖：02A - 自动取模 (MInt)。无解返回 nullopt。
*/

template <class Z> optional<Z> sqrt(Z a) {
    int p = Z::askMod();
    assert(p >= 2);
    if (a == Z(0) || p == 2) return a;
    if (a.pow((p - 1) / 2) != Z(1)) return nullopt;
    if (p % 4 == 3) return a.pow((p + 1) / 4);

    int s = 0;
    int q = p - 1;
    while (!(q & 1)) q >>= 1, ++s;
    Z z = 2;
    while (z.pow((p - 1) / 2) == Z(1)) ++z;
    Z x = a.pow((q + 1) / 2), b = a.pow(q), c = z.pow(q);
    int m = s;
    while (b != Z(1)) {
        int i = 1;
        Z d = b * b;
        while (i < m && d != Z(1)) d *= d, ++i;
        assert(i < m);
        Z t = c.pow(1LL << (m - i - 1));
        x *= t;
        c = t * t;
        b *= c;
        m = i;
    }
    return x;
}

```


02F - 原根.cpp

91 lines · C++ · 06 - 数学

```

/*
用途：判断模数是否有原根，寻找最小/全部原根，并判断给定数是否为原根。
前提：依赖 02 - 快速幂 (ModArith64)。原根仅存在于  $2$ 、 $4$ 、 $p^k$ 、 $2p^k$  ( $p$  为奇质数)。
接口：exist(n) 判断是否存在；solve(n) 返回最小原根，不存在返回  $-1$ ；roots(n) 返回递增的全部原根；ask(g, n) 判断  $g$ 。
复杂度：试除分解  $O(\sqrt{n})$ ，单次验证  $O(\omega(\phi(n)) \log n)$ ；枚举全部原根  $O(\phi(n) \log n)$ 。
*/

namespace PrimRoot {
inline vector<int> factor(int val) {
    assert(val >= 1);
    vector<int> fac;
    for (int d = 2; d <= val / d; ++d) {
        if (val % d != 0) continue;
        fac.push_back(d);
        do val /= d; while (val % d == 0);
    }
    if (val > 1) fac.push_back(val);
    return fac;
}

inline bool exist(int mod) {
    assert(mod >= 2);
    if (mod == 2 || mod == 4) return true;
    int x = mod, c = 0;
    while (x % 2 == 0) x /= 2, ++c;
    if (c > 1 || x == 1) return false;
    int p = 0;
    for (int d = 3; d <= x / d; d += 2) {
        if (x % d == 0) {
            p = d;
            break;
        }
    }
    if (!p) return true;
    while (x % p == 0) x /= p;
    return x == 1;
}

inline int phi(int mod) {
    int res = mod;
    for (int p : factor(mod)) res = res / p * (p - 1);
    return res;
}

inline bool ask(int g, int mod, int ph, const vector<int>& fac) {
    g %= mod;
    if (g <= 0) g += mod;
    if (gcd(g, mod) != 1) return false;
    for (int p : fac) {
        if (ModArith64::power(g, ph / p, mod) == 1) return false;
    }
    return true;
}

inline bool ask(int g, int mod) {
    assert(mod >= 2);
    if (!exist(mod)) return false;
    int ph = phi(mod);
    return ask(g, mod, ph, factor(ph));
}

```

```
inline int find(int mod, int ph, const vector<int>& fac) {
    for (int g = 1; g < mod; ++g) {
        if (ask(g, mod, ph, fac)) return g;
    }
    return -1;
}

inline int solve(int mod) {
    assert(mod >= 2);
    if (!exist(mod)) return -1;
    int ph = phi(mod);
    return find(mod, ph, factor(ph));
}

inline vector<int> roots(int mod) {
    assert(mod >= 2);
    if (!exist(mod)) return {};
    int ph = phi(mod);
    vector<int> fac = factor(ph);
    int g = find(mod, ph, fac), cnt = phi(ph);
    vector<int> res;
    res.reserve(cnt);
    for (int k = 1, x = g; k <= ph; ++k) {
        if (gcd(k, ph) == 1) res.push_back(x);
        x = ModArith64::mul(x, g, mod);
    }
    sort(res.begin(), res.end());
    return res;
}
} // namespace PrimRoot
```

02G - 整除分块求和 (floor_sum) .cpp

30 lines · C++ · 06 - 数学

```
/*
用途：求  $\sum_{0 \leq i < n} \text{floor}((a*i+b)/m)$ ，常用于整除分块、格点计数与欧几里得递归。
前提：n,m,a,b 均非负且  $m > 0$ 。返回 i128，调用方按答案范围输出。
复杂度： $O(\log(\max(a,m)))$ 。
*/

using i128 = __int128_t;

namespace FloorSum {
inline i128 flrSum(int n, int m, int a, int b) {
    assert(n >= 0 && m > 0 && a >= 0 && b >= 0);
    i128 cnt = n, mod = m, coe = a, off = b, ans = 0;
    while (true) {
        if (coe >= mod) {
            ans += (cnt - 1) * cnt * (coe / mod) / 2;
            coe %= mod;
        }
        if (off >= mod) {
            ans += cnt * (off / mod);
            off %= mod;
        }
        i128 mx = coe * cnt + off;
        if (mx < mod) break;
        cnt = mx / mod;
        off = mx % mod;
        swap(mod, coe);
    }
    return ans;
}
} // namespace FloorSum
```

03 - 组合数学 (Comb, with Z) .cpp

49 lines · C++ · 06 - 数学

```

/*
用途：阶乘、逆阶乘与  $C(n,k)$ 、 $A(n,k)$ 。
用法：Comb<Z> c(n)；Z 可为任意 MInt<P>。
前提：模数为质数，且查询  $n < \text{mod}$ ；首次扩展  $O(n)$ ，之后  $O(1)$ 。
*/
template <class Z> struct Comb {
    int n = 0;
    vector<Z> f{Z(1)}, g{Z(1)};

    Comb() = default;
    explicit Comb(int m) {
        init(m);
    }

    void init(int m) {
        assert(m >= 0 && m < Z::askMod());
        if (m <= n) return;
        f.resize(m + 1);
        g.resize(m + 1);
        for (int i = n + 1; i <= m; ++i) f[i] = f[i - 1] * i;
        g[m] = f[m].inv();
        for (int i = m; i > n; --i) g[i - 1] = g[i] * i;
        n = m;
    }

    Z fac(int x) {
        assert(x >= 0);
        if (x > n) init(x);
        return f[x];
    }

    Z ifac(int x) {
        assert(x >= 0);
        if (x > n) init(x);
        return g[x];
    }

    Z C(int x, int k) {
        if (x < 0 || k < 0 || k > x) return Z(0);
        return fac(x) * ifac(k) * ifac(x - k);
    }

    Z A(int x, int k) {
        if (x < 0 || k < 0 || k > x) return Z(0);
        return fac(x) * ifac(x - k);
    }
};

template <class Z> using Comb = Comb<Z>;

```

03A - Lucas 定理.cpp

44 lines · C++ · 06 - 数学

```

/*
用途：小质数 p 下求  $C(n, k)$ ， $n, k$  可很大。
用法：LucasPrime<Z> c(p)；要求  $Z::askMod() == p$ 。
*/

template <class Z> struct LucasPrime {
    int p = 0;
    vector<Z> f, g;

    LucasPrime() = default;

    explicit LucasPrime(int p) {
        init(p);
    }

    void init(int x) {
        assert(x >= 2 && Z::askMod() == x);
        p = x;
        f.assign(p, Z(1));
        g.assign(p, Z(1));
        for (int i = 1; i < p; ++i) f[i] = f[i - 1] * i;
        g[p - 1] = f[p - 1].inv();
        for (int i = p - 1; i; --i) g[i - 1] = g[i] * i;
    }

    Z smallC(int n, int k) const {
        if (k < 0 || k > n) return Z(0);
        return f[n] * g[k] * g[n - k];
    }

    Z C(int n, int k) const {
        assert(p);
        if (k < 0 || k > n) return Z(0);
        Z r = 1;
        while (n || k) {
            int x = n % p, y = k % p;
            if (y > x) return Z(0);
            r *= smallC(x, y);
            n /= p;
            k /= p;
        }
        return r;
    }
};

```

03B - 康托展开（暴力，小 n）.cpp

46 lines · C++ · 06 - 数学

```

/*
用途：求 1..n 排列的精确康托展开排名（从 1 开始），并支持反排名。
复杂度： $O(n^2)$ ，int 精确范围  $n \leq 20$ 。大 n 且只需模意义排名时使用 03C。
*/
struct Cantor {
    static void check(const vector<int>& p) {
        int n = p.size();
        vector<char> vis(n + 1, false);
        for (int val : p) {
            assert(1 <= val && val <= n && !vis[val]);
            vis[val] = true;
        }
    }

    static int askRank(const vector<int>& p) {
        int n = p.size();
        assert(n <= 20);
        check(p);
        vector<int> fac(n + 1, 1);
        for (int i = 1; i <= n; ++i) fac[i] = fac[i - 1] * i;
        int rk = 1;
        for (int i = 0; i < n; ++i) {
            int cnt = 0;
            for (int j = i + 1; j < n; ++j) cnt += p[j] < p[i];
            rk += cnt * fac[n - 1 - i];
        }
        return rk;
    }

    static vector<int> askPerm(int n, int rk) {
        assert(0 <= n && n <= 20);
        vector<int> fac(n + 1, 1);
        for (int i = 1; i <= n; ++i) fac[i] = fac[i - 1] * i;
        assert(1 <= rk && rk <= fac[n]);
        --rk;
        vector<int> vis(n), p;
        iota(vis.begin(), vis.end(), 1);
        for (int rem = n; rem >= 1; --rem) {
            int id = rk / fac[rem - 1];
            rk %= fac[rem - 1];
            p.push_back(vis[id]);
            vis.erase(vis.begin() + id);
        }
        return p;
    }
};

```

03C - 康托展开（树状数组，大 n）.cpp

88 lines · C++ · 06 - 数学

```

/*
用途：求 1..n 排列的康托展开排名模 mod。
复杂度：O(n log n)。结果只保留模 mod；n <= 20 且需要精确排名或反排名时使用 03B。
*/

using i128 = __int128_t;

struct Cantor {
    static int mul(int a, int b, int mod) {
        assert(mod > 0);
        a %= mod;
        b %= mod;
#ifdef __SIZEOF_INT128__
        return i128(a) * b % mod;
#else
        int res = 0;
        while (b > 0) {
            if (b & 1) res = res >= mod - a ? res - (mod - a) : res + a;
            a = a >= mod - a ? a - (mod - a) : a + a;
            b >>= 1;
        }
        return res;
#endif
    }

    static void check(const vector<int>& p) {
        int n = p.size();
        vector<char> vis(n + 1, false);
        for (int val : p) {
            assert(1 <= val && val <= n && !vis[val]);
            vis[val] = true;
        }
    }

    static int askMod(const vector<int>& p, int mod) {
        assert(mod > 0);
        int n = p.size();
        check(p);
        struct BIT {
            int n;
            vector<int> t;
            explicit BIT(int n) : n(n), t(n + 1, 0) {}
            void modify(int x, int v) { for (; x <= n; x += x & -x) t[x] += v; }
            int ask(int x) const { int r = 0; for (; x > 0; x -= x & -x) r += t[x]; return r; }
        } bit(n);

        vector<int> fac(n + 1, 1 % mod);
        for (int i = 1; i <= n; ++i) fac[i] = mul(fac[i - 1], i, mod);
        for (int x = 1; x <= n; ++x) bit.modify(x, 1);

        int rk = 1 % mod;
        for (int i = 0; i < n; ++i) {
            int cnt = bit.ask(p[i] - 1);
            int get = mul(cnt, fac[n - 1 - i], mod);
            rk = rk >= mod - get ? rk - (mod - get) : rk + get;
            bit.modify(p[i], -1);
        }
        return rk;
    }
}

```

```
// 质数模时优先使用本重载，Z 自动完成加乘取模。
template <class Z> static Z askRank(const vector<int>& p) {
    int n = p.size();
    check(p);
    struct BIT {
        int n;
        vector<int> t;
        explicit BIT(int n) : n(n), t(n + 1) {}
        void modify(int x, int v) {
            for (; x <= n; x += x & -x) t[x] += v;
        }
        int ask(int x) const {
            int r = 0;
            for (; x; x -= x & -x) r += t[x];
            return r;
        }
    } b(n);
    vector<Z> f(n + 1, Z(1));
    for (int i = 1; i <= n; ++i) f[i] = f[i - 1] * i;
    for (int x = 1; x <= n; ++x) b.modify(x, 1);
    Z r = 1;
    for (int i = 0; i < n; ++i) {
        r += Z(b.ask(p[i] - 1)) * f[n - 1 - i];
        b.modify(p[i], -1);
    }
    return r;
}
};
```


03D - 卡特兰数.cpp

11 lines · C++ · 06 - 数学

```
/*
用途：预处理 Catalan(0..n)。
前提：Z 的模数为质数，n < mod。
*/
template <class Z> vector<Z> catalan(int n) {
    assert(n >= 0 && n < Z::askMod());
    vector<Z> a(n + 1);
    a[0] = 1;
    for (int i = 1; i <= n; ++i) a[i] = a[i - 1] * Z(4 * i - 2) / Z(i + 1);
    return a;
}
```

03E - 斯特林数（第一、二类）.cpp

33 lines · C++ · 06 - 数学

```

/*
用途：预处理无符号第一类、第二类 Stirling 数。
复杂度： $O(n^2)$ 。
*/
template <class Z> struct Stirling {
    int n = 0;
    vector<vector<Z>> a, b;

    Stirling() = default;

    explicit Stirling(int m) {
        init(m);
    }

    void init(int m) {
        assert(m >= 0);
        n = m;
        a.assign(n + 1, vector<Z>(n + 1));
        b.assign(n + 1, vector<Z>(n + 1));
        a[0][0] = b[0][0] = 1;
        for (int i = 1; i <= n; ++i) {
            for (int j = 1; j <= i; ++j) {
                a[i][j] = a[i - 1][j - 1] + Z(i - 1) * a[i - 1][j];
                b[i][j] = b[i - 1][j - 1] + Z(j) * b[i - 1][j];
            }
        }
    }

    Z stir1(int x, int k) const {
        assert(0 <= x && x <= n && 0 <= k && k <= x);
        return (x - k) & 1 ? -a[x][k] : a[x][k];
    }
};

```

03F - 错排数与贝尔数.cpp

26 lines · C++ · 06 - 数学

```
/*
用途：预处理错排数  $D(0..n)$  与 Bell 数  $B(0..n)$ 。
仅要求  $Z$  支持加减乘。
*/
template <class Z> vector<Z> derange(int n) {
    assert(n >= 0);
    vector<Z> a(n + 1);
    a[0] = 1;
    for (int i = 2; i <= n; ++i) a[i] = Z(i - 1) * (a[i - 1] + a[i - 2]);
    return a;
}

template <class Z> vector<Z> bell(int n) {
    assert(n >= 0);
    vector<vector<Z>> a(n + 1);
    vector<Z> b(n + 1);
    a[0] = {Z(1)};
    b[0] = 1;
    for (int i = 1; i <= n; ++i) {
        a[i].assign(i + 1, Z(0));
        a[i][0] = a[i - 1][i - 1];
        for (int j = 1; j <= i; ++j) a[i][j] = a[i][j - 1] + a[i - 1][j - 1];
        b[i] = a[i][0];
    }
    return b;
}
```

03G - 拉格朗日插值（任意横坐标）.cpp

23 lines · C++ · 06 - 数学

```
/*
用途：给定互异横坐标点  $(x[i], y[i])$ ，在  $p$  处求值。
前提： $Z$  为质数模，横坐标两两不同。
*/
template <class Z> Z lagr(const vector<Z>& x, const vector<Z>& y, Z p) {
    int n = x.size();
    assert(n && y.size() == n);
    for (int i = 0; i < n; ++i) {
        if (p == x[i]) return y[i];
        for (int j = 0; j < i; ++j) assert(x[i] != x[j]);
    }
    Z r = 0;
    for (int i = 0; i < n; ++i) {
        Z a = 1, b = 1;
        for (int j = 0; j < n; ++j) {
            if (i == j) continue;
            a *= p - x[j];
            b *= x[i] - x[j];
        }
        r += y[i] * a / b;
    }
    return r;
}
```

03H - 拉格朗日插值（连续横坐标）.cpp

24 lines · C++ · 06 - 数学

```
/*
用途：已知  $f(0..n-1)$ ，在任意整数  $x$  处求次数小于  $n$  的多项式值。
前提： $Z$  为质数模， $n < \text{mod}$ ；复杂度  $O(n)$ 。
*/

template <class Z> Z lagr(const vector<Z>& a, int x) {
    int n = a.size();
    assert(n && n < Z::askMod());
    if (0 <= x && x < n) return a[x];
    Z p = x;
    vector<Z> l(n + 1, Z(1)), r(n + 1, Z(1)), f(n, Z(1)), g(n, Z(1));
    for (int i = 0; i < n; ++i) l[i + 1] = l[i] * (p - Z(i));
    for (int i = n - 1; i >= 0; --i) r[i] = r[i + 1] * (p - Z(i));
    for (int i = 1; i < n; ++i) f[i] = f[i - 1] * i;
    g[n - 1] = f[n - 1].inv();
    for (int i = n - 1; i; --i) g[i - 1] = g[i] * i;
    Z ans = 0;
    for (int i = 0; i < n; ++i) {
        Z t = a[i] * l[i] * r[i + 1] * g[i] * g[n - 1 - i];
        if ((n - 1 - i) & 1) t = -t;
        ans += t;
    }
    return ans;
}
```

03l - 牛顿插值（差分级数）.cpp

32 lines · C++ · 06 - 数学

```

/*
用途：整数横坐标  $0..n-1$  上的 Newton 前向差分级数。
构造  $O(n^2)$ ，单次 ask  $O(n)$ ；Z 为质数模且  $n < \text{mod}$ 。
*/
template <class Z> struct Newton {
    vector<Z> d;

    Newton() = default;

    explicit Newton(vector<Z> a) {
        init(move(a));
    }

    void init(vector<Z> a) {
        assert(a.size() < Z::askMod());
        d.clear();
        while (!a.empty()) {
            d.push_back(a[0]);
            for (int i = 0; i + 1 < a.size(); ++i) a[i] = a[i + 1] - a[i];
            a.pop_back();
        }
    }

    Z ask(Z x) const {
        Z r = 0, c = 1;
        for (int i = 0; i < d.size(); ++i) {
            r += d[i] * c;
            c *= (x - Z(i)) / Z(i + 1);
        }
        return r;
    }
};

```

03J - 整数分拆（五边形数）.cpp

23 lines · C++ · 06 - 数学

```
/*
用途：Euler 五边形数定理预处理整数分拆  $p(0..n)$ 。
Z 仅需支持加减；复杂度  $O(n \sqrt{n})$ 。
*/

template <class Z> vector<Z> part(int n) {
    assert(n >= 0);
    vector<Z> a(n + 1);
    a[0] = 1;
    for (int i = 1; i <= n; ++i) {
        Z s = 0;
        for (int k = 1;; ++k) {
            int x = k * (3 * k - 1) / 2;
            if (x > i) break;
            Z t = k & 1 ? Z(1) : Z(-1);
            s += t * a[i - x];
            int y = k * (3 * k + 1) / 2;
            if (y <= i) s += t * a[i - y];
        }
        a[i] = s;
    }
    return a;
}
```

03K - Berlekamp-Massey (最短线性递推) .cpp

32 lines · C++ · 06 - 数学

```

/*
用途：求最短递推  $a[t] = \sum c[i] a[t-1-i]$ 。
前提：Z 为域；返回的系数可直接交给 03KA。
*/
template <class Z> vector<Z> BM(const vector<Z>& a) {
    vector<Z> c{Z(1)}, b{Z(1)};
    int l = 0, m = 1;
    Z d = 1;
    for (int i = 0; i < a.size(); ++i) {
        Z x = a[i];
        for (int j = 1; j <= l; ++j) x += c[j] * a[i - j];
        if (x == Z(0)) {
            ++m;
            continue;
        }
        vector<Z> t = c;
        Z q = x / d;
        if (c.size() < b.size() + m) c.resize(b.size() + m);
        for (int j = 0; j < b.size(); ++j) c[j + m] -= q * b[j];
        if (l * 2 <= i) {
            l = i + 1 - l;
            b = move(t);
            d = x;
            m = 1;
        } else {
            ++m;
        }
    }
    vector<Z> r(l);
    for (int i = 0; i < l; ++i) r[i] = -c[i + 1];
    return r;
}

```


03KA - 线性递推第 n 项 (Kitamasa) .cpp

34 lines · C++ · 06 - 数学

```
/*
用途：已知递推系数  $c$  和前  $k$  项  $a$ ，在  $O(k^2 \log n)$  求第  $n$  项。
*/

namespace LinearRec {
template <class Z> vector<Z> combine(const vector<Z>& a, const vector<Z>& b, const vector<Z>& c) {
    int k = c.size();
    vector<Z> t(k * 2 - 1);
    for (int i = 0; i < k; ++i)
        for (int j = 0; j < k; ++j) t[i + j] += a[i] * b[j];
    for (int i = k * 2 - 2; i >= k; --i)
        for (int j = 0; j < k; ++j) t[i - 1 - j] += t[i] * c[j];
    t.resize(k);
    return t;
}

template <class Z> Z ask(const vector<Z>& a, const vector<Z>& c, int n) {
    int k = c.size();
    assert(k && a.size() == k && n >= 0);
    if (n < k) return a[n];
    vector<Z> r(k), x(k);
    r[0] = 1;
    if (k == 1) x[0] = c[0];
    else x[1] = 1;
    while (n) {
        if (n & 1) r = combine(r, x, c);
        x = combine(x, x, c);
        n >>= 1;
    }
    Z ans = 0;
    for (int i = 0; i < k; ++i) ans += r[i] * a[i];
    return ans;
}
} // namespace LinearRec
```

03L - Burnside 引理（群作用计数）.cpp

12 lines · C++ · 06 - 数学

```
/*
用途：群作用下的轨道数；f(i) 返回第 i 个变换的不动点数。
前提：群大小在 Z 中可逆。
*/
namespace Burnside {
template <class Z, class F> Z burn(int n, F f) {
    assert(n > 0);
    Z s = 0;
    for (int i = 0; i < n; ++i) s += f(i);
    return s / Z(n);
}
} // namespace Burnside
```

04A - 容斥 (bitmask, 小 m) .cpp

57 lines · C++ · 06 - 数学

```

/*
用途：计算  $[1, n]$  中至少被一个 divisor 整除的数的个数。
复杂度： $O(2^m \cdot m)$ ， $m$  为去重、去冗余后的 divisor 数，要求  $m \leq 22$ 。
与 04B 是同一问题的两种实现； $m$  较大且 lcm 常很快超过  $n$  时可改用 04B。
*/

using i128 = __int128_t;

namespace IE {
inline vector<int> norm(vector<int> d) {
    for (int val : d) assert(val > 0);
    sort(d.begin(), d.end());
    d.erase(unique(d.begin(), d.end()), d.end());
    vector<int> res;
    for (int val : d) {
        bool bad = false;
        for (int kp : res) {
            if (val % kp == 0) {
                bad = true;
                break;
            }
        }
        if (!bad) res.push_back(val);
    }
    return res;
}

inline bool lcm(int a, int b, int lim, int& res) {
    int x = a / gcd(a, b);
    if (x > lim / b) return false;
    res = x * b;
    return true;
}

inline int solve(int n, vector<int> d) {
    assert(n >= 0);
    d = norm(move(d));
    int m = d.size();
    assert(m <= 22);
    i128 ans = 0;
    for (int msk = 1; msk < (1ULL << m); ++msk) {
        int cur = 1;
        bool ok = true;
        for (int i = 0; i < m; ++i) {
            if ((msk >> i & 1) && !lcm(cur, d[i], n, cur)) {
                ok = false;
                break;
            }
        }
        if (!ok) continue;
        if (__builtin_popcountll(msk) & 1) ans += n / cur;
        else ans -= n / cur;
    }
    assert(ans >= numeric_limits<int>::min() && ans <= numeric_limits<int>::max());
    return ans;
}
} // namespace IE

```

04B - 容斥 (DFS 剪枝) .cpp

51 lines · C++ · 06 - 数学

```

/*
用途：计算  $[1, n]$  中至少被一个 divisor 整除的数的个数。
复杂度：最坏  $O(2^m)$ ，但会在  $\text{lcm} > n$  时剪枝；适合 divisor 较多且 lcm 很快超界的情形。
与 04A 是同一问题的两种实现； $m \leq 22$  且希望实现路径固定时使用 04A。
*/

using i128 = __int128_t;

namespace IE {
inline vector<int> norm(vector<int> d) {
    for (int val : d) assert(val > 0);
    sort(d.begin(), d.end());
    d.erase(unique(d.begin(), d.end()), d.end());
    vector<int> res;
    for (int val : d) {
        bool bad = false;
        for (int kp : res) {
            if (val % kp == 0) {
                bad = true;
                break;
            }
        }
        if (!bad) res.push_back(val);
    }
    return res;
}

inline bool lcm(int a, int b, int lim, int& res) {
    int x = a / gcd(a, b);
    if (x > lim / b) return false;
    res = x * b;
    return true;
}

inline int solve(int n, vector<int> d) {
    assert(n >= 0);
    d = norm(move(d));
    i128 ans = 0;
    function<void(int, int, int)> dfs = [&](int st, int cur, int sgn) {
        for (int i = st; i < d.size(); ++i) {
            int nxt;
            if (!lcm(cur, d[i], n, nxt)) continue;
            ans += sgn * i128(n / nxt);
            dfs(i + 1, nxt, -sgn);
        }
    };
    dfs(0, 1, 1);
    assert(ans >= numeric_limits<int>::min() && ans <= numeric_limits<int>::max());
    return ans;
}
} // namespace IE

```

05 - 矩阵 (Matrix, with Int) .cpp

103 lines · C++ · 06 - 数学

```
template <class T> struct Matrix {
    int n, m;
    vector<vector<T>> a;

    Matrix(int n = 0, int m = 0, T val = T()) : n(n), m(m), a(n, vector<T>(m, val)) {}

    static Matrix unit(int n) {
        Matrix I(n, n);
        for (int i = 0; i < n; i++) I.a[i][i] = T(1);
        return I;
    }

    vector<T> &operator[](int i) {
        return a[i];
    }
    const vector<T> &operator[](int i) const {
        return a[i];
    }

    Matrix operator+(const Matrix &o) const {
        assert(n == o.n && m == o.m);
        Matrix res(n, m);
        for (int i = 0; i < n; i++)
            for (int j = 0; j < m; j++) res[i][j] = a[i][j] + o[i][j];
        return res;
    }

    Matrix operator-(const Matrix &o) const {
        assert(n == o.n && m == o.m);
        Matrix res(n, m);
        for (int i = 0; i < n; i++)
            for (int j = 0; j < m; j++) res[i][j] = a[i][j] - o[i][j];
        return res;
    }

    Matrix operator*(const Matrix &o) const {
        assert(m == o.n);
        Matrix res(n, o.m, T(0));
        for (int i = 0; i < n; i++)
            for (int k = 0; k < m; k++)
                for (int j = 0; j < o.m; j++) res[i][j] = res[i][j] + a[i][k] * o[k][j];
        return res;
    }

    Matrix operator*(const T &k) const {
        Matrix res(n, m);
        for (int i = 0; i < n; i++)
            for (int j = 0; j < m; j++) res[i][j] = a[i][j] * k;
        return res;
    }

    Matrix trans() const {
        Matrix res(m, n);
        for (int i = 0; i < n; i++)
            for (int j = 0; j < m; j++) res[j][i] = a[i][j];
        return res;
    }

    Matrix ksm(int exp) const {
```

```
        assert(n == m);
        assert(exp >= 0);
        Matrix bas = *this;
        Matrix res = unit(n);
        while (exp > 0) {
            if (exp & 1) res = res * bas;
            bas = bas * bas;
            exp >>= 1;
        }
        return res;
    }
};

template <class T> T det(int n, Matrix<T> &mat) {
    T det = T(1);
    int sgn = 1;

    for (int i = 0; i < n; i++) {
        int piv = i;
        while (piv < n && mat[piv][i] == T()) piv++;
        if (piv == n) return T();

        if (piv != i) {
            swap(mat[i], mat[piv]);
            sgn = -sgn;
        }

        det *= mat[i][i];
        for (int j = i + 1; j < n; j++) {
            if (!(mat[j][i] == T())) {
                T k = mat[j][i] / mat[i][i];
                for (int k = i; k < n; k++) {
                    mat[j][k] -= k * mat[i][k];
                }
            }
        }
    }

    if (sgn == -1) {
        det = T() - det;
    }
    return det;
}
```

05A - Min-Plus矩阵 (MinPlusMatrix, with Int) .cpp

62 lines · C++ · 06 - 数学

// 性能：默认使用宏展开后的 `int`；若大规模数据因此超时或超空间，可将纯下标、状态或计数局部改为 `i32`。

```
template <int N, class T = int> struct Matrix {
    static constexpr T INF = numeric_limits<T>::max() / 4;
    array<array<T, N>, N> a;

    Matrix(T v = INF) {
        for (int i = 0; i < N; ++i) a[i].fill(v);
    }

    static Matrix unit() {
        Matrix res;
        for (int i = 0; i < N; ++i) res[i][i] = 0;
        return res;
    }

    array<T, N> &operator[](int i) {
        return a[i];
    }
    const array<T, N> &operator[](int i) const {
        return a[i];
    }

    Matrix operator+(const Matrix &o) const {
        Matrix res;
        for (int i = 0; i < N; ++i)
            for (int j = 0; j < N; ++j) res[i][j] = min(a[i][j], o[i][j]);
        return res;
    }

    Matrix operator*(const Matrix &o) const {
        Matrix res;
        for (int i = 0; i < N; ++i) {
            for (int k = 0; k < N; ++k) {
                if (a[i][k] >= INF) continue;
                for (int j = 0; j < N; ++j) {
                    if (o[k][j] >= INF) continue;
                    res[i][j] = min(res[i][j], a[i][k] + o[k][j]);
                }
            }
        }
        return res;
    }

    Matrix &operator+=(const Matrix &o) {
        return *this = *this + o;
    }
    Matrix &operator*=(const Matrix &o) {
        return *this = *this * o;
    }

    Matrix ksm(int b) const {
        assert(b >= 0);
        Matrix a = *this, res = unit();
        while (b) {
            if (b & 1) res *= a;
            a *= a;
            b >>= 1;
        }
        return res;
    }
};
```

```
    }  
};
```


05B - 高斯消元（一般线性方程组）.cpp

49 lines · C++ · 06 - 数学

```

/*
用途：求一般实系数 m 元线性方程组，区分唯一解、无穷多解、无解。
复杂度： $O(m^2 * rows)$ 。浮点题请按数据范围调整 eps；精确模意义方程不能直接套用。
与 05C 都用于线性方程组，但 05C 只适合对称、无零主元的特殊矩阵。
*/
using ld = long double;

enum class GaussStat { Unique, Infinite, Inconsistent };

struct GaussRes {
    GaussStat st;
    vector<ld> sol; // Infinite 时给出一个自由元均为 0 的特解。
    vector<int> pc;
};

inline GaussRes gauss(vector<vector<ld>> a, ld eps = 1e-12L) {
    int n = a.size();
    if (n == 0) return {GaussStat::Unique, {}, {}};
    int m = a[0].size() - 1;
    assert(m >= 0);
    for (const auto& row : a) assert(row.size() == m + 1);

    vector<int> pc;
    int pr = 0;
    for (int c = 0; c < m && pr < n; ++c) {
        int vis = pr;
        for (int row = pr + 1; row < n; ++row) {
            if (fabsl(a[row][c]) > fabsl(a[vis][c])) vis = row;
        }
        if (fabsl(a[vis][c]) <= eps) continue;
        swap(a[vis], a[pr]);
        ld iv = 1 / a[pr][c];
        for (int j = c; j <= m; ++j) a[pr][j] *= iv;
        for (int row = 0; row < n; ++row) {
            if (row == pr || fabsl(a[row][c]) <= eps) continue;
            ld coe = a[row][c];
            for (int j = c; j <= m; ++j) a[row][j] -= coe * a[pr][j];
        }
        pc.push_back(c);
        ++pr;
    }

    for (int row = pr; row < n; ++row) {
        if (fabsl(a[row][m]) > eps) return {GaussStat::Inconsistent, {}, pc};
    }
    vector<ld> sol(m, 0);
    for (int row = 0; row < pr; ++row) sol[pc[row]] = a[row][m];
    return {pr == m ? GaussStat::Unique : GaussStat::Infinite, sol, pc};
}

```

05C - 对称 LDLT 分解（线性方程组）.cpp

41 lines · C++ · 06 - 数学

```

/*
用途：以 LDLT 分解求对称线性方程组。
复杂度：O(n3)。不做主元交换，要求各顺序主子式不出现零枢轴；否则使用 05B。
与 05B 是同一问题的不同实现：本模板常数和内存更小，但适用条件更强。
*/
using ld = long double;

inline optional<vector<ld>> solve(
    const vector<vector<ld>>& mat, const vector<ld>& rhs, ld eps = 1e-12L) {
    int n = mat.size();
    if (rhs.size() != n) return nullopt;
    for (const auto& row : mat) if (row.size() != n) return nullopt;

    vector<vector<ld>> lo(n, vector<ld>(n, 0));
    vector<ld> dia(n);
    for (int i = 0; i < n; ++i) {
        for (int j = 0; j < i; ++j) {
            ld val = mat[i][j];
            for (int k = 0; k < j; ++k) val -= lo[i][k] * dia[k] * lo[j][k];
            if (fabsl(dia[j]) <= eps) return nullopt;
            lo[i][j] = val / dia[j];
        }
        ld val = mat[i][i];
        for (int k = 0; k < i; ++k) val -= lo[i][k] * lo[i][k] * dia[k];
        if (fabsl(val) <= eps) return nullopt;
        dia[i] = val;
        lo[i][i] = 1;
    }

    vector<ld> y(n), z(n), ans(n);
    for (int i = 0; i < n; ++i) {
        y[i] = rhs[i];
        for (int j = 0; j < i; ++j) y[i] -= lo[i][j] * y[j];
        z[i] = y[i] / dia[i];
    }
    for (int i = n - 1; i >= 0; --i) {
        ans[i] = z[i];
        for (int j = i + 1; j < n; ++j) ans[i] -= lo[j][i] * ans[j];
    }
    return ans;
}

```

05D - 高斯消元（模质数）.cpp

42 lines · C++ · 06 - 数学

```

/*
用途：质数模域上的线性方程组，区分唯一解、无穷多解、无解。
依赖：02A - 自动取模 (MInt) 。
*/
enum class GaussStat { Unique, Infinite, Inconsistent };

template <class Z> struct GaussRes {
    GaussStat st;
    vector<Z> x;
    vector<int> p;
};

template <class Z> GaussRes<Z> gauss(vector<vector<Z>> a) {
    int n = a.size();
    if (!n) return {GaussStat::Unique, {}, {}};
    int m = a[0].size() - 1;
    assert(m >= 0);
    for (const auto& v : a) assert(v.size() == m + 1);

    vector<int> p;
    int r = 0;
    for (int c = 0; c < m && r < n; ++c) {
        int x = r;
        while (x < n && a[x][c] == Z(0)) ++x;
        if (x == n) continue;
        swap(a[x], a[r]);
        Z iv = a[r][c].inv();
        for (int j = c; j <= m; ++j) a[r][j] *= iv;
        for (int i = 0; i < n; ++i) {
            if (i == r || a[i][c] == Z(0)) continue;
            Z z = a[i][c];
            for (int j = c; j <= m; ++j) a[i][j] -= z * a[r][j];
        }
        p.push_back(c);
        ++r;
    }
    for (int i = r; i < n; ++i)
        if (a[i][m] != Z(0)) return {GaussStat::Inconsistent, {}, p};
    vector<Z> x(m);
    for (int i = 0; i < r; ++i) x[p[i]] = a[i][m];
    return {r == m ? GaussStat::Unique : GaussStat::Infinite, x, p};
}

```

05E - 行列式（模质数）.cpp

26 lines · C++ · 06 - 数学

```
/*
用途：域上的方阵行列式；MInt 时模数须为质数。
*/
template <class Z> Z det(vector<vector<Z>> a) {
    int n = a.size();
    for (const auto& v : a) assert(v.size() == n);
    Z r = 1;
    for (int c = 0; c < n; ++c) {
        int p = c;
        while (p < n && a[p][c] == Z(0)) ++p;
        if (p == n) return Z(0);
        if (p != c) {
            swap(a[p], a[c]);
            r = -r;
        }
        Z x = a[c][c];
        r *= x;
        Z iv = x.inv();
        for (int i = c + 1; i < n; ++i) {
            if (a[i][c] == Z(0)) continue;
            Z q = a[i][c] * iv;
            for (int j = c; j < n; ++j) a[i][j] -= q * a[c][j];
        }
    }
    return r;
}
```

05F - 线性规划（单纯形）.cpp

127 lines · C++ · 06 - 数学

```

/*
用途：求  $\max c \cdot x$ ，满足  $A \cdot x \leq b, x \geq 0$  的浮点线性规划。
返回最优、无界或无解；使用两阶段单纯形法。浮点数据请按量级调整 EPS。
复杂度：单纯形最坏指数，竞赛中的中小规模通常可用；不能处理整数规划。
*/
using ld = long double;

enum class LPStatus { Optimal, Infeasible, Unbounded };

struct LPRes {
    LPStatus st;
    ld ans = 0;
    vector<ld> sol;
};

struct Simplex {
    static constexpr ld EPS = 1e-12L;
    int n, m;
    vector<int> bas, nb;
    vector<vector<ld>> a;

    Simplex(const vector<vector<ld>>& A, const vector<ld>& b,
            const vector<ld>& c)
        : n(b.size()), m(c.size()), bas(n),
          nb(m + 1), a(n + 2, vector<ld>(m + 2, 0)) {
        assert(A.size() == n);
        for (int i = 0; i < n; ++i) {
            assert(A[i].size() == m);
            for (int j = 0; j < m; ++j) a[i][j] = A[i][j];
        }
        for (int i = 0; i < n; ++i) {
            bas[i] = m + i;
            a[i][m] = -1;
            a[i][m + 1] = b[i];
        }
        for (int j = 0; j < m; ++j) {
            nb[j] = j;
            a[n][j] = -c[j];
        }
        nb[m] = -1;
        a[n + 1][m] = 1;
    }

    void pivot(int row, int c) {
        ld inv = 1 / a[row][c];
        for (int i = 0; i < n + 2; ++i) {
            if (i == row) continue;
            for (int j = 0; j < m + 2; ++j) {
                if (j == c) continue;
                a[i][j] -= a[row][j] * a[i][c] * inv;
            }
        }
        for (int j = 0; j < m + 2; ++j) if (j != c) a[row][j] *= inv;
        for (int i = 0; i < n + 2; ++i) if (i != row) a[i][c] *= -inv;
        a[row][c] = inv;
        swap(bas[row], nb[c]);
    }

    bool simplex(int ph) {
        int obj = ph == 1 ? n + 1 : n;

```

```

while (true) {
    int in = -1;
    for (int c = 0; c <= m; ++c) {
        if (ph == 2 && nb[c] == -1) continue;
        if (in == -1 || a[obj][c] < a[obj][in] - EPS ||
            (fabsl(a[obj][c] - a[obj][in]) <= EPS &&
             nb[c] < nb[in])) {
            in = c;
        }
    }
    if (a[obj][in] >= -EPS) return true;

    int out = -1;
    for (int row = 0; row < n; ++row) {
        if (a[row][in] <= EPS) continue;
        if (out == -1 ||
            a[row][m + 1] / a[row][in] <
            a[out][m + 1] / a[out][in] - EPS ||
            (fabsl(a[row][m + 1] / a[row][in] -
                a[out][m + 1] / a[out][in]) <= EPS &&
             bas[row] < bas[out])) {
            out = row;
        }
    }
    if (out == -1) return false;
    pivot(out, in);
}

LPRes solve() {
    if (n == 0) {
        for (int j = 0; j < m; ++j)
            if (a[n][j] < -EPS) return {LPStatus::Unbounded, 0, {}};
        return {LPStatus::Optimal, 0, vector<ld>(m, 0)};
    }

    int row = 0;
    for (int i = 1; i < n; ++i)
        if (a[i][m + 1] < a[row][m + 1]) row = i;
    if (a[row][m + 1] < -EPS) {
        pivot(row, m);
        if (!simplex(1) || a[n + 1][m + 1] < -EPS) {
            return {LPStatus::Infeasible, 0, {}};
        }
        if (fabsl(a[n + 1][m + 1]) > EPS) {
            return {LPStatus::Infeasible, 0, {}};
        }
        for (int i = 0; i < n; ++i) {
            if (bas[i] != -1) continue;
            int c = 0;
            for (int j = 1; j <= m; ++j) {
                if (a[i][j] < a[i][c] - EPS ||
                    (fabsl(a[i][j] - a[i][c]) <= EPS && nb[j] < nb[c])) {
                    c = j;
                }
            }
            pivot(i, c);
        }
    }
    if (!simplex(2)) return {LPStatus::Unbounded, 0, {}};
    vector<ld> sol(m, 0);
    for (int i = 0; i < n; ++i) {
        if (bas[i] < m) sol[bas[i]] = a[i][m + 1];
    }
}

```

```
        return {LPStatus::Optimal, a[n][m + 1], sol};  
    }  
};
```

05G - 高斯消元 (GF(2) 位集) .cpp

41 lines · C++ · 06 - 数学

```

/*
用途：在 GF(2) 上求异或线性方程组。每行 bitset 的 [0,variables) 为系数，variables 位为右端常数。
前提：variables <= MAX_VARIABLES；加减与乘法均为 xor/and。
复杂度：O(rows * variables * MAX_VARIABLES / word_bits)，适合布尔方程、异或约束。
性能：默认使用宏展开后的 int；若大规模数据因此超时或超空间，可将纯下标、状态或计数局部改为 i32。
*/
enum class GaussStat { Unique, Infinite, Inconsistent };

template <int M> struct GaussRes {
    GaussStat st;
    vector<int> sol; // Infinite 时自由元取 0 的一组特解。
    vector<int> pc;
};

template <int M>
GaussRes<M> gauss(vector<bitset<M + 1>> a, int m) {
    assert(0 <= m && m <= M);
    int n = a.size();
    vector<int> pc;
    int pr = 0;
    for (int c = 0; c < m && pr < n; ++c) {
        int vis = pr;
        while (vis < n && !a[vis][c]) ++vis;
        if (vis == n) continue;
        swap(a[vis], a[pr]);
        for (int row = 0; row < n; ++row) {
            if (row != pr && a[row][c]) a[row] ^= a[pr];
        }
        pc.push_back(c);
        ++pr;
    }
    for (int row = pr; row < n; ++row) {
        bool has = false;
        for (int c = 0; c < m; ++c) has = has || a[row][c];
        if (!has && a[row][m]) return {GaussStat::Inconsistent, {}, pc};
    }
    vector<int> sol(m, 0);
    for (int row = 0; row < pr; ++row) sol[pc[row]] = a[row][m];
    return {pr == m ? GaussStat::Unique : GaussStat::Infinite, sol, pc};
}

```


05H - 矩阵树定理 (Kirchhoff) .cpp

21 lines · C++ · 06 - 数学

```
/*
用途：无向带权图生成树数。
依赖：05E - 行列式（模质数）；Z 通常为 MInt<P>。
*/
template <class Z> Z kirch(int n, const vector<tuple<int, int, Z>>& e) {
    assert(n >= 0);
    if (n <= 1) return Z(1);
    vector<vector<Z>> a(n, vector<Z>(n));
    for (auto [u, v, w] : e) {
        assert(0 <= u && u < n && 0 <= v && v < n);
        if (u == v) continue;
        a[u][u] += w;
        a[v][v] += w;
        a[u][v] -= w;
        a[v][u] -= w;
    }
    vector<vector<Z>> b(n - 1, vector<Z>(n - 1));
    for (int i = 0; i + 1 < n; ++i)
        for (int j = 0; j + 1 < n; ++j) b[i][j] = a[i][j];
    return det(move(b));
}
```

06 - 分数 (Frac, with Int) .cpp

69 lines · C++ · 06 - 数学

```
template <class T> struct Frac {
    T x, y;
    Frac() : x(0), y(1) {}
    Frac(T a, T b) : x(a), y(b) {
        assert(y != 0);
        T g = gcd(x, y);
        assert(g != 0);
        x /= g, y /= g;
        if (y < 0) x = -x, y = -y;
    }

    constexpr double val() const {
        return 1.0 * x / y;
    }

    friend ostream &operator<<(ostream &o, const Frac &b) {
        T g = gcd(b.x, b.y);
        if (b.y == g) {
            return o << b.x / g;
        } else {
            return o << b.x / g << "/" << b.y / g;
        }
    }

    Frac &operator+=(const Frac o) {
        *this = Frac(x * o.y + y * o.x, y * o.y);
        return *this;
    }

    Frac &operator-=(const Frac o) {
        *this = Frac(x * o.y - y * o.x, y * o.y);
        return *this;
    }

    Frac &operator*=(const Frac o) {
        *this = Frac(x * o.x, y * o.y);
        return *this;
    }

    Frac &operator/=(const Frac o) {
        assert(o.x != 0);
        *this = Frac(x * o.y, y * o.x);
        return *this;
    }

    constexpr Frac operator-() const {
        return Frac(-x, y);
    }

    friend Frac operator+(Frac a, const Frac &b) {
        return a += b;
    }

    friend Frac operator-(Frac a, const Frac &b) {
        return a -= b;
    }

    friend Frac operator*(Frac a, const Frac &b) {
        return a *= b;
    }

    friend Frac operator/(Frac a, const Frac &b) {
        return a /= b;
    }

    friend bool operator<(const Frac &a, const Frac &b) {
        return a.x * b.y < a.y * b.x;
    }

    friend bool operator>(const Frac &a, const Frac &b) {
```

```
        return a.x * b.y > a.y * b.x;
    }
    friend bool operator==(const Frac &a, const Frac &b) {
        return a.x * b.y == a.y * b.x;
    }
    friend bool operator!=(const Frac &a, const Frac &b) {
        return a.x * b.y != a.y * b.x;
    }
};
```

06A - 复数 (Complex, with Int) .cpp

81 lines · C++ · 06 - 数学

```
template <typename T> class Complex {
public:
    T a, b;

    constexpr Complex(T r = 0, T i = 0) : a(r), b(i) {}

    constexpr Complex operator+(const Complex &rhs) const {
        return {a + rhs.a, b + rhs.b};
    }

    constexpr Complex operator-(const Complex &rhs) const {
        return {a - rhs.a, b - rhs.b};
    }

    constexpr Complex operator*(const Complex &rhs) const {
        return {a * rhs.a - b * rhs.b, a * rhs.b + b * rhs.a};
    }

    Complex operator/(const Complex &rhs) const {
        T den = rhs.a * rhs.a + rhs.b * rhs.b;
        assert(den != 0);
        return {(a * rhs.a + b * rhs.b) / den, (b * rhs.a - a * rhs.b) / den};
    }

    Complex inv() const {
        T den = a * a + b * b;
        assert(den != 0);
        return Complex(a / den, -b / den);
    }

    Complex &operator+=(const Complex &rhs) {
        *this = (*this) + rhs;
        return *this;
    }

    Complex &operator-=(const Complex &rhs) {
        *this = (*this) - rhs;
        return *this;
    }

    Complex &operator*=(const Complex &rhs) {
        *this = (*this) * rhs;
        return *this;
    }

    Complex &operator/=(const Complex &rhs) {
        *this = (*this) / rhs;
        return *this;
    }

    T norm() const {
        return a * a + b * b;
    }

    bool operator==(const Complex &rhs) const {
        return a == rhs.a && b == rhs.b;
    }

    bool operator!=(const Complex &rhs) const {
        return !(*this == rhs);
    }
};
```

```
    }

    bool operator<(const Complex &rhs) const {
        return norm() < rhs.norm();
    }

    bool operator>(const Complex &rhs) const {
        return norm() > rhs.norm();
    }

    friend ostream &operator<<(ostream &os, const Complex &c) {
        os << "(" << c.a;
        if (c.b >= 0) os << "+";
        os << c.b << "i";
        return os;
    }
};

template <class T> Complex<T> conj(const Complex<T> &z) {
    return Complex<T>(z.a, -z.b);
}
```

06B - 三次单位根复数 (Complex3) .cpp

55 lines · C++ · 06 - 数学

```
// x^3 = 1, x != 1
template <class T> struct Complex3 {
    T a, b;
    Complex3(T a = T(), T b = T()) : a(a), b(b) {}

    Complex3 operator+(const Complex3 &rhs) const {
        return {a + rhs.a, b + rhs.b};
    }

    Complex3 operator-(const Complex3 &rhs) const {
        return {a - rhs.a, b - rhs.b};
    }

    Complex3 operator*(const Complex3 &rhs) const {
        T ac = a * rhs.a;
        T bd = b * rhs.b;
        T ad = a * rhs.b;
        T bc = b * rhs.a;

        return {ac - bd, ad + bc - bd};
    }

    Complex3 operator/(const Complex3 &rhs) const {
        return (*this) * rhs.inv();
    }

    Complex3 inv() const {
        T z = a * a - a * b + b * b;
        assert(z != 0);
        return Complex3<T>((a - b) / z, -b / z);
    }

    Complex3 &operator+=(const Complex3 &rhs) {
        *this = (*this) + rhs;
        return *this;
    }

    Complex3 &operator-=(const Complex3 &rhs) {
        *this = (*this) - rhs;
        return *this;
    }

    Complex3 &operator*=(const Complex3 &rhs) {
        *this = (*this) * rhs;
        return *this;
    }

    Complex3 &operator/=(const Complex3 &rhs) {
        *this = (*this) / rhs;
        return *this;
    }

    bool ask0() const {
        return a == 0 && b == 0;
    }
};
```

07 - 平面几何 (Point&Line, with Int) .cpp

348 lines · C++ · 06 - 数学

```
using ld = long double;

template <class T> struct Point {
    T x;
    T y;
    Point(const T &_x = 0, const T &_y = 0) : x(_x), y(_y) {}

    template <class U> operator Point<U>() {
        return Point<U>(U(x), U(y));
    }
    Point &operator+=(const Point &p) & {
        x += p.x;
        y += p.y;
        return *this;
    }
    Point &operator-=(const Point &p) & {
        x -= p.x;
        y -= p.y;
        return *this;
    }
    Point &operator*=(const T &v) & {
        x *= v;
        y *= v;
        return *this;
    }
    Point &operator/=(const T &v) & {
        x /= v;
        y /= v;
        return *this;
    }
    Point operator-() const {
        return Point(-x, -y);
    }
    friend Point operator+(Point a, const Point &b) {
        return a += b;
    }
    friend Point operator-(Point a, const Point &b) {
        return a -= b;
    }
    friend Point operator*(Point a, const T &b) {
        return a *= b;
    }
    friend Point operator/(Point a, const T &b) {
        return a /= b;
    }
    friend Point operator*(const T &a, Point b) {
        return b *= a;
    }
    friend bool operator==(const Point &a, const Point &b) {
        return a.x == b.x && a.y == b.y;
    }
    friend bool operator!=(const Point &a, const Point &b) {
        return !(a == b);
    }
    friend istream &operator>>(istream &is, Point &p) {
        return is >> p.x >> p.y;
    }
    friend ostream &operator<<(ostream &os, const Point &p) {
        return os << "(" << p.x << ", " << p.y << ")";
    }
}
```

```

};

template <class T> struct Line {
    Point<T> a;
    Point<T> b;
    Line(const Point<T> &a = Point<T>(), const Point<T> &b = Point<T>()) : a(_a), b(_b) {}
};

template <class T> T dot(const Point<T> &a, const Point<T> &b) {
    return a.x * b.x + a.y * b.y;
}

template <class T> T cross(const Point<T> &a, const Point<T> &b) {
    return a.x * b.y - a.y * b.x;
}

template <class T> T square(const Point<T> &p) {
    return dot(p, p);
}

template <class T> double length(const Point<T> &p) {
    return sqrt(square(p));
}

template <class T> double length(const Line<T> &l) {
    return length(l.a - l.b);
}

template <class T> Point<T> norm(const Point<T> &p) {
    return p / length(p);
}

template <class T> bool isPara(const Line<T> &l1, const Line<T> &l2) {
    return cross(l1.b - l1.a, l2.b - l2.a) == 0;
}

template <class T> double dis(const Point<T> &a, const Point<T> &b) {
    return length(a - b);
}

template <class T> double disPL(const Point<T> &p, const Line<T> &l) {
    return abs(cross(l.a - l.b, l.a - p)) / length(l);
}

template <class T> double disPS(const Point<T> &p, const Line<T> &l) {
    if (dot(p - l.a, l.b - l.a) < 0) {
        return dis(p, l.a);
    }
    if (dot(p - l.b, l.a - l.b) < 0) {
        return dis(p, l.b);
    }
    return disPL(p, l);
}

template <class T> Point<T> rotate(const Point<T> &a) {
    return Point(-a.y, a.x);
}

template <class T> int sgn(const Point<T> &a) {
    return a.y > 0 || (a.y == 0 && a.x > 0) ? 1 : -1;
}

template <class T> bool side(const Point<T> &p, const Line<T> &l) {
    return cross(l.b - l.a, p - l.a) > 0;
}

```



```

}

template <class T> Point<T> inter(const Line<T> &l1, const Line<T> &l2) {
    return l1.a + (l1.b - l1.a) * (cross(l2.b - l2.a, l1.a - l2.a) / cross(l2.b - l2.a, l1.a - l1.b));
}

template <class T> bool onSeg(const Point<T> &p, const Line<T> &l) {
    return cross(p - l.a, l.b - l.a) == 0 && min(l.a.x, l.b.x) <= p.x && p.x <= max(l.a.x, l.b.x) &&
        min(l.a.y, l.b.y) <= p.y && p.y <= max(l.a.y, l.b.y);
}

template <class T> bool inPoly(const Point<T> &a, const vector<Point<T>> &p) {
    int n = p.size();
    for (int i = 0; i < n; i++) {
        if (onSeg(a, Line(p[i], p[(i + 1) % n]))) {
            return true;
        }
    }

    int t = 0;
    for (int i = 0; i < n; i++) {
        auto u = p[i];
        auto v = p[(i + 1) % n];
        if (u.x < a.x && v.x >= a.x && side(a, Line(v, u))) {
            t ^= 1;
        }
        if (u.x >= a.x && v.x < a.x && side(a, Line(u, v))) {
            t ^= 1;
        }
    }

    return t == 1;
}

template <class T> tuple<int, Point<T>, Point<T>> segInt(const Line<T> &l1, const Line<T> &l2) {
    if (max(l1.a.x, l1.b.x) < min(l2.a.x, l2.b.x)) {
        return {0, Point<T>(), Point<T>()};
    }
    if (min(l1.a.x, l1.b.x) > max(l2.a.x, l2.b.x)) {
        return {0, Point<T>(), Point<T>()};
    }
    if (max(l1.a.y, l1.b.y) < min(l2.a.y, l2.b.y)) {
        return {0, Point<T>(), Point<T>()};
    }
    if (min(l1.a.y, l1.b.y) > max(l2.a.y, l2.b.y)) {
        return {0, Point<T>(), Point<T>()};
    }
    if (cross(l1.b - l1.a, l2.b - l2.a) == 0) {
        if (cross(l1.b - l1.a, l2.a - l1.a) != 0) {
            return {0, Point<T>(), Point<T>()};
        } else {
            auto xx1 = max(l1.a.x, l1.b.x);
            auto nx1 = min(l1.a.x, l1.b.x);
            auto yy1 = max(l1.a.y, l1.b.y);
            auto ny1 = min(l1.a.y, l1.b.y);
            auto xx2 = max(l2.a.x, l2.b.x);
            auto nx2 = min(l2.a.x, l2.b.x);
            auto yy2 = max(l2.a.y, l2.b.y);
            auto ny2 = min(l2.a.y, l2.b.y);
            Point<T> p1(max(nx1, nx2), max(ny1, ny2));
            Point<T> p2(min(xx1, xx2), min(yy1, yy2));
            if (!onSeg(p1, l1)) {
                swap(p1.y, p2.y);
            }
        }
    }
}

```

```

        if (p1 == p2) {
            return {3, p1, p2};
        } else {
            return {2, p1, p2};
        }
    }
}

auto cp1 = cross(l2.a - l1.a, l2.b - l1.a);
auto cp2 = cross(l2.a - l1.b, l2.b - l1.b);
auto cp3 = cross(l1.a - l2.a, l1.b - l2.a);
auto cp4 = cross(l1.a - l2.b, l1.b - l2.b);

if ((cp1 > 0 && cp2 > 0) || (cp1 < 0 && cp2 < 0) || (cp3 > 0 && cp4 > 0) || (cp3 < 0 && cp4 < 0))
{
    return {0, Point<T>(), Point<T>()};
}

Point p = inter(l1, l2);
if (cp1 != 0 && cp2 != 0 && cp3 != 0 && cp4 != 0) {
    return {1, p, p};
} else {
    return {3, p, p};
}
}

template <class T> double disSS(const Line<T> &l1, const Line<T> &l2) {
    if (get<0>(segInt(l1, l2)) != 0) {
        return 0.0;
    }
    return min({disPS(l1.a, l2), disPS(l1.b, l2), disPS(l2.a, l1), disPS(l2.b, l1)});
}

template <class T> bool segIn(const Line<T> &l, const vector<Point<T>> &p) {
    int n = p.size();
    if (!inPoly(l.a, p)) {
        return false;
    }
    if (!inPoly(l.b, p)) {
        return false;
    }
    for (int i = 0; i < n; i++) {
        auto u = p[i];
        auto v = p[(i + 1) % n];
        auto w = p[(i + 2) % n];
        auto [t, p1, p2] = segInt(l, Line(u, v));

        if (t == 1) {
            return false;
        }
        if (t == 0) {
            continue;
        }
        if (t == 2) {
            if (onSeg(v, l) && v != l.a && v != l.b) {
                if (cross(v - u, w - v) > 0) {
                    return false;
                }
            }
        }
    }
    } else {
        if (p1 != u && p1 != v) {
            if (side(l.a, Line(v, u)) || side(l.b, Line(v, u))) {
                return false;
            }
        } else if (p1 == v) {

```

```

        if (l.a == v) {
            if (side(u, l)) {
                if (side(w, l) && side(w, Line(u, v))) {
                    return false;
                }
            } else {
                if (side(w, l) || side(w, Line(u, v))) {
                    return false;
                }
            }
        } else if (l.b == v) {
            if (side(u, Line(l.b, l.a))) {
                if (side(w, Line(l.b, l.a)) && side(w, Line(u, v))) {
                    return false;
                }
            } else {
                if (side(w, Line(l.b, l.a)) || side(w, Line(u, v))) {
                    return false;
                }
            }
        } else {
            if (side(u, l)) {
                if (side(w, Line(l.b, l.a)) || side(w, Line(u, v))) {
                    return false;
                }
            } else {
                if (side(w, l) || side(w, Line(u, v))) {
                    return false;
                }
            }
        }
    }
}

return true;
}

template <class T> vector<Point<T>> hp(vector<Line<T>> a) {
    sort(a.begin(), a.end(), [&](auto l1, auto l2) {
        auto d1 = l1.b - l1.a;
        auto d2 = l2.b - l2.a;

        if (sgn(d1) != sgn(d2)) {
            return sgn(d1) == 1;
        }

        return cross(d1, d2) > 0;
    });

    deque<Line<T>> ls;
    deque<Point<T>> ps;
    for (auto l : a) {
        if (ls.empty()) {
            ls.push_back(l);
            continue;
        }

        while (!ps.empty() && !side(ps.back(), l)) {
            ps.pop_back();
            ls.pop_back();
        }

        while (!ps.empty() && !side(ps[0], l)) {
            ps.pop_front();
        }
    }
}

```

```
        ls.pop_front();
    }

    if (cross(l.b - l.a, ls.back().b - ls.back().a) == 0) {
        if (dot(l.b - l.a, ls.back().b - ls.back().a) > 0) {
            if (!side(ls.back().a, l)) {
                assert(ls.size() == 1);
                ls[0] = l;
            }
            continue;
        }
        return {};
    }

    ps.push_back(inter(ls.back(), l));
    ls.push_back(l);
}

while (!ps.empty() && !side(ps.back(), ls[0])) {
    ps.pop_back();
    ls.pop_back();
}
if (ls.size() <= 2) {
    return {};
}
ps.push_back(inter(ls[0], ls.back()));

return vector(ps.begin(), ps.end());
}

using Real = ld;
using P = Point<Real>;

constexpr Real eps = 0;
```

07A - 凸包与旋转卡壳（依赖07）.cpp

89 lines · C++ · 06 - 数学

```

/*
依赖：07 - 平面几何 (Point&Line, with Int)。
用途：二维凸包、凸多边形直径、 $O(\log n)$  点包含判定。凸包不重复首点并按逆时针排列。
整数坐标时 T 应能容纳叉积和距离平方；坐标接近  $1e9$  时通常用 int。
性能：默认使用宏展开后的 int；若大规模数据因此超时或超空间，可将纯下标、状态或计数局部改为 i32。
*/
template <class T>
vector<Point<T>> convex(vector<Point<T>> a, bool kp = false) {
    sort(a.begin(), a.end(), [](const auto& p, const auto& q) {
        return p.x != q.x ? p.x < q.x : p.y < q.y;
    });
    a.erase(unique(a.begin(), a.end()), a.end());
    int n = a.size();
    if (n <= 2) return a;
    bool col = true;
    for (int i = 2; i < n; ++i) col &= cross(a[1] - a[0], a[i] - a[0]) == 0;
    if (kp && col) return a;

    auto ch = [&](auto fst, auto lst) {
        vector<Point<T>> h;
        for (; fst != lst; ++fst) {
            const auto& p = *fst;
            while (h.size() >= 2) {
                T c = cross(h.back() - h[h.size() - 2], p - h.back());
                if (kp ? c >= 0 : c > 0) break;
                h.pop_back();
            }
            h.push_back(p);
        }
        return h;
    };
    auto lo = ch(a.begin(), a.end());
    auto up = ch(a.rbegin(), a.rend());
    lo.pop_back(), up.pop_back();
    lo.insert(lo.end(), up.begin(), up.end());
    return lo;
}

template <class T> T area2(const vector<Point<T>>& p) {
    T r{};
    for (int i = 0, n = p.size(); i < n; ++i) r += cross(p[i], p[(i + 1) % n]);
    return r;
}

// 输入为逆时针凸多边形；返回 {最大距离平方, 端点下标}。
template <class T>
tuple<T, int, int> diam2(const vector<Point<T>>& p) {
    int n = p.size();
    if (!n) return {T{}, -1, -1};
    if (n == 1) return {T{}, 0, 0};
    auto ab = [](T x) { return x < 0 ? -x : x; };
    T ans{};
    int x = 0, y = 1, j = 1;
    auto upd = [&](int u, int v) {
        T d = square(p[u] - p[v]);
        if (d > ans) ans = d, x = u, y = v;
    };
    for (int i = 0; i < n; ++i) {
        int k = (i + 1) % n;
        while (ab(cross(p[k] - p[i], p[(j + 1) % n] - p[i])) >

```

```

        ab(cross(p[k] - p[i], p[j] - p[i])) j = (j + 1) % n;
        upd(i, j), upd(k, j);
    }
    return {ans, x, y};
}

```

// -1: 外部, 0: 边界, 1: 内部; 要求 p 为逆时针凸多边形且无重复首点。

```

template <class T> int inConv(const vector<Point<T>>& p, const Point<T>& q) {
    int n = p.size();
    auto on = [&](const Point<T>& a, const Point<T>& b) {
        return cross(q - a, b - a) == 0 && dot(q - a, q - b) <= 0;
    };
    if (!n) return -1;
    if (n == 1) return q == p[0] ? 0 : -1;
    if (n == 2) return on(p[0], p[1]) ? 0 : -1;
    T a = cross(p[1] - p[0], q - p[0]);
    T b = cross(p[n - 1] - p[0], q - p[0]);
    if (a < 0 || b > 0) return -1;
    if (!a) return on(p[0], p[1]) ? 0 : -1;
    if (!b) return on(p[0], p[n - 1]) ? 0 : -1;
    int l = 1, r = n - 1;
    while (r - l > 1) {
        int m = (l + r) >> 1;
        if (cross(p[m] - p[0], q - p[0]) >= 0) l = m;
        else r = m;
    }
    T c = cross(p[r] - p[l], q - p[l]);
    return c < 0 ? -1 : c == 0 ? 0 : 1;
}

```

07AA - 闵可夫斯基和（依赖07A）.cpp

42 lines · C++ · 06 - 数学

```

/*
依赖：07 - 平面几何、07A - 凸包。
用途：求两个点集凸包的闵可夫斯基和；结果为逆时针凸包，不重复首点、去共线中间点。
复杂度：含两次凸包为  $O(n \log n + m \log m)$ ，已给凸包时主体为  $O(n+m)$ 。
性能：默认使用宏展开后的 int；若大规模数据因此超时或超空间，可将纯下标、状态或计数局部改为 i32。
*/
template <class T>
vector<Point<T>> minkSum(vector<Point<T>> a, vector<Point<T>> b) {
    a = convex(move(a)), b = convex(move(b));
    if (a.empty() || b.empty()) return {};
    if (a.size() == 1) {
        for (auto& p : b) p += a[0];
        return b;
    }
    if (b.size() == 1) {
        for (auto& p : a) p += b[0];
        return a;
    }
    auto z = [](vector<Point<T>>& p) {
        auto it = min_element(p.begin(), p.end(), [](const auto& x, const auto& y) {
            return x.y != y.y ? x.y < y.y : x.x < y.x;
        });
        rotate(p.begin(), it, p.end());
    };
    z(a), z(b);
    vector<Point<T>> x(a.size()), y(b.size());
    for (int i = 0; i < a.size(); ++i) x[i] = a[(i + 1) % a.size()] - a[i];
    for (int i = 0; i < b.size(); ++i) y[i] = b[(i + 1) % b.size()] - b[i];

    int i = 0, j = 0;
    Point<T> p = a[0] + b[0];
    vector<Point<T>> c{p};
    while (i < x.size() || j < y.size()) {
        Point<T> d;
        if (j == y.size() || (i < x.size() && cross(x[i], y[j]) > 0)) d = x[i++];
        else if (i == x.size() || cross(x[i], y[j]) < 0) d = y[j++];
        else d = x[i++] + y[j++];
        p += d, c.push_back(p);
    }
    c.pop_back();
    return convex(move(c));
}

```

07B - 圆与最小覆盖圆（依赖07）.cpp

77 lines · C++ · 06 - 数学

```

/*
依赖：07 中的 Real、P、Point/Line、dot/cross/square/rotate。
用途：圆与直线/圆的交点、三点外接圆、随机增量最小覆盖圆。
浮点比较统一使用 EP；重合圆有无穷交点时 interCC 返回空数组，需按题意另判。
性能：默认使用宏展开后的 int；若大规模数据因此超时或超空间，可将纯下标、状态或计数局部改为 i32。
*/
constexpr Real EP = 1e-12L;
struct Circle {
    P o;
    Real r = -1;
};

inline bool inCir(const Circle& c, const P& p) {
    return c.r >= 0 && square(p - c.o) <= c.r * c.r + EP;
}

inline Circle diaCir(const P& a, const P& b) {
    P o = (a + b) / Real(2);
    return {o, sqrtl(square(a - b)) / 2};
}

inline Circle cir3(const P& a, const P& b, const P& c) {
    P x = b - a, y = c - a;
    Real d = 2 * cross(x, y);
    if (fabsl(d) <= EP) {
        Circle r = diaCir(a, b);
        for (auto [u, v] : {pair<P, P>{a, c}, pair<P, P>{b, c}}) {
            Circle q = diaCir(u, v);
            if (q.r > r.r) r = q;
        }
        return r;
    }
    P o = a + P((y.y * square(x) - x.y * square(y)) / d,
                (x.x * square(y) - y.x * square(x)) / d);
    return {o, sqrtl(square(o - a))};
}

inline vector<P> interCL(const Circle& c, const Line<Real>& l) {
    P d = l.b - l.a;
    Real z = square(d);
    assert(z > EP);
    P h = l.a + d * (dot(c.o - l.a, d) / z);
    Real q = c.r * c.r - square(h - c.o);
    if (q < -EP) return {};
    if (q <= EP) return {h};
    P v = d * sqrtl(q / z);
    return {h - v, h + v};
}

inline vector<P> interCC(const Circle& a, const Circle& b) {
    P d = b.o - a.o;
    Real z = sqrtl(square(d));
    if (z <= EP) return {};
    if (z > a.r + b.r + EP || z < fabsl(a.r - b.r) - EP) return {};
    Real x = (z * z + a.r * a.r - b.r * b.r) / (2 * z);
    Real h2 = a.r * a.r - x * x;
    P v = d / z;
    P p = a.o + v * x;
    if (h2 <= EP) return {p};
    P q = rotate(v) * sqrtl(max<Real>(0, h2));
}

```



```
    return {p - q, p + q};
}

inline Circle minCir(vector<P> p, int sd = 712367821) {
    if (p.empty()) return {{0, 0}, 0};
    shuffle(p.begin(), p.end(), mt19937_64(sd));
    Circle c{p[0], 0};
    for (int i = 1; i < p.size(); ++i) if (!inCir(c, p[i])) {
        c = {p[i], 0};
        for (int j = 0; j < i; ++j) if (!inCir(c, p[j])) {
            c = diaCir(p[i], p[j]);
            for (int k = 0; k < j; ++k)
                if (!inCir(c, p[k])) c = cir3(p[i], p[j], p[k]);
        }
    }
    return c;
}
```

07C - 三维几何.cpp

46 lines · C++ · 06 - 数学

```

/*
用途：三维点、向量、直线与平面的基础运算。整数 T 可精确做点积/叉积/体积判断；距离返回 long double。
*/
using ld = long double;

template <class T> struct Point3 {
    T x{}, y{}, z{};
    Point3() = default;
    Point3(T x, T y, T z) : x(x), y(y), z(z) {}
    Point3& operator+=(const Point3& p) { x += p.x, y += p.y, z += p.z; return *this; }
    Point3& operator-=(const Point3& p) { x -= p.x, y -= p.y, z -= p.z; return *this; }
    Point3& operator*=(const T& k) { x *= k, y *= k, z *= k; return *this; }
    friend Point3 operator+(Point3 a, const Point3& b) { return a + b; }
    friend Point3 operator-(Point3 a, const Point3& b) { return a - b; }
    friend Point3 operator*(Point3 a, const T& k) { return a * k; }
    friend bool operator==(const Point3& a, const Point3& b) { return a.x == b.x && a.y == b.y && a.z == b.z; }
};

template <class T> T dot(const Point3<T>& a, const Point3<T>& b) {
    return a.x * b.x + a.y * b.y + a.z * b.z;
}

template <class T> Point3<T> cross(const Point3<T>& a, const Point3<T>& b) {
    return {a.y * b.z - a.z * b.y, a.z * b.x - a.x * b.z, a.x * b.y - a.y * b.x};
}

template <class T> T square(const Point3<T>& a) { return dot(a, a); }
template <class T> T volume6(const Point3<T>& a, const Point3<T>& b, const Point3<T>& c, const Point3<T>& d) {
    return dot(b - a, cross(c - a, d - a));
}

template <class T> ld disPL(const Point3<T>& p, const Point3<T>& a, const Point3<T>& b) {
    return sqrtl(static_cast<ld>(square(cross(p - a, b - a))) /
        static_cast<ld>(square(b - a)));
}

template <class T> ld disPP(const Point3<T>& p, const Point3<T>& a, const Point3<T>& n) {
    return fabsl(static_cast<ld>(dot(p - a, n))) /
        sqrtl(static_cast<ld>(square(n)));
}

// 直线 p+t*d 与平面 (x-a)·n=0 的交点；平时时返回 nullopt。
template <class T>
optional<Point3<ld>> interLP(const Point3<T>& p, const Point3<T>& d,
    const Point3<T>& a, const Point3<T>& n) {
    T z = dot(d, n);
    if (z == 0) return nullopt;
    ld t = static_cast<ld>(dot(a - p, n)) / static_cast<ld>(z);
    return Point3<ld>(p.x + t * d.x, p.y + t * d.y, p.z + t * d.z);
}

```

07D - 平面最近点对 (依赖07) .cpp

52 lines · C++ · 06 - 数学

```

/*
依赖：07 - 平面几何中的 Point。
用途：求平面点集最近点对；返回 {距离平方, 原下标1, 原下标2}，不足两点返回 {INF, -1, -1}。
复杂度：分治  $O(n \log n)$ 。距离统一转 long double，避免整数坐标作差/平方溢出。
性能：默认使用宏展开后的 int；若大规模数据因此超时或超空间，可将纯下标、状态或计数局部改为 i32。
*/
using ld = long double;

template <class T>
tuple<ld, int, int> close(vector<Point<T>> p) {
    struct N { Point<T> p; int id; };
    vector<N> a;
    for (int i = 0; i < p.size(); ++i) a.push_back({p[i], i});
    sort(a.begin(), a.end(), [](const N& x, const N& y) {
        return x.p.x != y.p.x ? x.p.x < y.p.x : x.p.y < y.p.y;
    });
    ld ans = numeric_limits<ld>::infinity();
    int ax = -1, ay = -1;
    auto upd = [&](const N& x, const N& y) {
        ld dx = ld(x.p.x) - ld(y.p.x);
        ld dy = ld(x.p.y) - ld(y.p.y);
        ld d = dx * dx + dy * dy;
        if (d < ans) ans = d, ax = x.id, ay = y.id;
    };
    auto cy = [](const N& x, const N& y) {
        return x.p.y != y.p.y ? x.p.y < y.p.y : x.p.x < y.p.x;
    };
    auto dfs = [&](auto&& go, int l, int r) -> void {
        if (r - l <= 3) {
            for (int i = l; i < r; ++i) for (int j = i + 1; j < r; ++j) upd(a[i], a[j]);
            sort(a.begin() + l, a.begin() + r, cy);
            return;
        }
        int m = (l + r) >> 1;
        ld mx = ld(a[m].p.x);
        go(go, l, m), go(go, m, r);
        inplace_merge(a.begin() + l, a.begin() + m, a.begin() + r, cy);
        vector<N> q;
        for (int i = l; i < r; ++i) {
            ld dx = ld(a[i].p.x) - mx;
            if (dx * dx >= ans) continue;
            for (int j = q.size(); j--;) {
                ld dy = ld(a[i].p.y) - ld(q[j].p.y);
                if (dy * dy >= ans) break;
                upd(a[i], q[j]);
            }
            q.push_back(a[i]);
        }
    };
    dfs(dfs, 0, a.size());
    return {ans, ax, ay};
}

```

08 - 快速数论变换 (NTT) .cpp

77 lines · C++ · 06 - 数学

```

/*
用途：高性能整数 NTT 卷积。
用法：auto c = NTT<998244353, 3>::mul(a, b)。
约定：系数和下标缓存使用 int。
自动取模版本使用 08B / 08C 的 Poly<Z, 3>。
性能：默认使用宏展开后的 int；若大规模数据因此超时或超空间，可将纯下标、状态或计数局部改为 i32。
*/

template <int P, int G> struct NTT {

    static int pow(int a, int n) {
        int r = 1;
        while (n) {
            if (n & 1) r = r * a % P;
            a = a * a % P;
            n >>= 1;
        }
        return r;
    }

    static void ntt(vector<int>& a, bool iv) {
        int n = a.size();
        assert(n && !(n & (n - 1)));
        static vector<int> rv;
        static vector<int> rt{0, 1};
        if (rv.size() != n) {
            int k = __builtin_ctz(static_cast<unsigned>(n));
            rv.assign(n, 0);
            if (n > 1)
                for (int i = 0; i < n; ++i) rv[i] = (rv[i >> 1] >> 1) | ((i & 1) << (k - 1));
        }
        if (rt.size() < n) {
            int k = __builtin_ctz(static_cast<unsigned>(rt.size()));
            rt.resize(n);
            while ((1LL << k) < n) {
                int z = pow(G, (P - 1) >> (k + 1));
                for (int i = 1LL << (k - 1); i < (1LL << k); ++i) {
                    rt[i << 1] = rt[i];
                    rt[i << 1 | 1] = rt[i] * z % P;
                }
                ++k;
            }
        }
        for (int i = 0; i < n; ++i)
            if (i < rv[i]) swap(a[i], a[rv[i]]);
        for (int len = 1; len < n; len <= 1) {
            for (int i = 0; i < n; i += len << 1) {
                for (int j = 0; j < len; ++j) {
                    int u = a[i + j];
                    int v = a[i + j + len] * rt[len + j] % P;
                    a[i + j] = (u + v) % P;
                    a[i + j + len] = (u - v + P) % P;
                }
            }
        }
        if (iv) {
            reverse(a.begin() + 1, a.end());
            int x = pow(n, P - 2);
            for (int& v : a) v = v * x % P;
        }
    }
};

```

```
    }

    static vector<int> mul(vector<int> a, vector<int> b) {
        if (a.empty() || b.empty()) return {};
        int m = a.size() + b.size() - 1;
        int n = 1;
        while (n < m) n <=< 1;
        a.resize(n);
        b.resize(n);
        ntt(a, false);
        ntt(b, false);
        for (int i = 0; i < n; ++i) a[i] = a[i] * b[i] % P;
        ntt(a, true);
        a.resize(m);
        return a;
    }
};
```

08A - 多项式 (Poly, int) (NTT) .cpp

183 lines · C++ · 06 - 数学

```

/*
用途：高性能整数系数多项式（乘法、求逆、ln/exp、求值、插值）。
依赖：08 - 快速数论变换 (NTT)。
用法：using P = Poly<998244353, 3>; 系数容器为 vector<int>。
约定：这里刻意不用 MInt，系数/下标缓存使用 int。
自动取模版本使用 08B / 08C 的 Poly<Z, 3>。
性能：默认使用宏展开后的 int；若大规模数据因此超时或超空间，可将纯下标、状态或计数局部改为 i32。
*/

template <int P, int G = 3> struct Poly {

    static int pw(int a, int n) {
        return NTT<P, G>::pow(a, n);
    }

    static void ntt(vector<int>& a, bool iv) {
        NTT<P, G>::ntt(a, iv);
    }

    static vector<int> add(vector<int> a, vector<int> b) {
        int n = max(a.size(), b.size());
        a.resize(n);
        b.resize(n);
        for (int i = 0; i < n; ++i) {
            a[i] += b[i];
            if (a[i] >= P) a[i] -= P;
        }
        return a;
    }

    static vector<int> sub(vector<int> a, vector<int> b) {
        int n = max(a.size(), b.size());
        a.resize(n);
        b.resize(n);
        for (int i = 0; i < n; ++i) {
            a[i] -= b[i];
            if (a[i] < 0) a[i] += P;
        }
        return a;
    }

    static vector<int> mul(vector<int> a, vector<int> b) {
        return NTT<P, G>::mul(move(a), move(b));
    }

    static vector<int> inv(const vector<int>& a, int n) {
        assert(n >= 0);
        if (!n) return {};
        assert(!a.empty() && a[0]);
        if (n == 1) return {pw(a[0], P - 2)};
        vector<int> b = inv(a, (n + 1) >> 1);
        int m = 1;
        while (m < n << 1) m <=< 1;
        vector<int> c(a.begin(), a.begin() + min<int>(a.size(), n));
        c.resize(m);
        b.resize(m);
        ntt(c, false);
        ntt(b, false);
        for (int i = 0; i < m; ++i) {
            b[i] = (2 * b[i] - c[i] * b[i] % P * b[i] % P + P) % P;
        }
    }
};

```

```

    }
    ntt(b, true);
    b.resize(n);
    return b;
}

static vector<int> div(const vector<int>& a, const vector<int>& b) {
    assert(!b.empty() && b.back());
    int n = a.size(), m = b.size();
    if (n < m) return {0};
    vector<int> x(a.rbegin(), a.rend()), y(b.rbegin(), b.rend());
    y = inv(y, n - m + 1);
    vector<int> q = mul(x, y);
    q.resize(n - m + 1);
    reverse(q.begin(), q.end());
    return q;
}

static vector<int> mod(const vector<int>& a, const vector<int>& b) {
    vector<int> r = sub(a, mul(div(a, b), b));
    r.resize(min<int>(r.size(), b.size() - 1));
    return r;
}

static vector<int> der(const vector<int>& a) {
    int n = a.size();
    if (n <= 1) return {};
    vector<int> b(n - 1);
    for (int i = 1; i < n; ++i) b[i - 1] = a[i] * i % P;
    return b;
}

static vector<int> integ(const vector<int>& a) {
    int n = a.size();
    vector<int> b(n + 1), iv(n + 1);
    if (n) iv[1] = 1;
    for (int i = 2; i <= n; ++i) iv[i] = P - (P / i) * iv[P % i] % P;
    for (int i = 0; i < n; ++i) b[i + 1] = a[i] * iv[i + 1] % P;
    return b;
}

static vector<int> ln(const vector<int>& a, int n) {
    assert(n >= 0);
    if (!n) return {};
    assert(!a.empty() && a[0]);
    vector<int> b = integ(mul(der(a), inv(a, n)));
    b.resize(n);
    return b;
}

static vector<int> exp(const vector<int>& a, int n) {
    assert(n >= 0);
    if (!n) return {};
    assert(a.empty() || !a[0]);
    vector<int> b{1};
    for (int k = 2; k < n << 1; k <= 1) {
        int m = min(k, n);
        vector<int> c(a.begin(), a.begin() + min<int>(a.size(), m));
        c.resize(m);
        c = sub(c, ln(b, m));
        ++c[0];
        if (c[0] == P) c[0] = 0;
        b = mul(b, c);
        b.resize(m);
    }
}

```

```

    }
    b.resize(n);
    return b;
}

static void build(vector<vector<int>>& t, const vector<int>& x, int u, int l, int r) {
    if (r - l == 1) {
        t[u] = {x[l] ? P - x[l] : 0, 1};
        return;
    }
    int m = (l + r) >> 1;
    build(t, x, u << 1, l, m);
    build(t, x, u << 1 | 1, m, r);
    t[u] = mul(t[u << 1], t[u << 1 | 1]);
}

static void eval0(const vector<int>& f, const vector<vector<int>>& t, vector<int>& y, int u, int l,
int r) {
    if (r - l == 1) {
        if (f.empty()) y[l] = 0;
        else if (f.size() == 1) y[l] = f[0];
        else y[l] = mod(f, t[u])[0];
        return;
    }
    vector<int> g = mod(f, t[u]);
    int m = (l + r) >> 1;
    eval0(g, t, y, u << 1, l, m);
    eval0(g, t, y, u << 1 | 1, m, r);
}

static vector<int> eval(const vector<int>& f, const vector<int>& x) {
    int n = x.size();
    if (!n) return {};
    vector<vector<int>> t(n << 2 | 1);
    build(t, x, 1, 0, n);
    vector<int> y(n);
    eval0(f, t, y, 1, 0, n);
    return y;
}

static vector<int> interp(const vector<int>& x, const vector<int>& y) {
    int n = x.size();
    assert(y.size() == n);
    if (!n) return {};
    vector<vector<int>> t(n << 2 | 1);
    build(t, x, 1, 0, n);
    vector<int> d = eval(der(t[1]), x);
    vector<int> w(n);
    for (int i = 0; i < n; ++i) w[i] = y[i] * pw(d[i], P - 2) % P;
    function<vector<int>(int, int, int)> dfs = [&](int u, int l, int r) -> vector<int> {
        if (r - l == 1) return {w[l]};
        int m = (l + r) >> 1;
        return add(mul(dfs(u << 1, l, m), t[u << 1 | 1]), mul(dfs(u << 1 | 1, m, r), t[u << 1]));
    };
    return dfs(1, 0, n);
}

};

using Poly998 = Poly<998244353, 3>;

```


08B - 多项式 (Poly, Z) (NTT) .cpp

197 lines · C++ · 06 - 数学

```

/*
用途：完整自动取模 NTT 多项式：乘法、逆、ln/exp、多点求值、插值。
用法：using P = Poly<Z, 3>; Z 为 MInt<P>，模数须是 NTT 友好质数。
性能：默认使用宏展开后的 int；若大规模数据因此超时或超空间，可将纯下标、状态或计数局部改为 i32。
*/
template <class Z, int G = 3> struct Poly {

    static void ntt(vector<Z>& a, bool iv) {
        int n = a.size();
        assert(n && !(n & (n - 1)));
        vector<int> rv(n);
        int k = __builtin_ctz(static_cast<unsigned>(n));
        if (n > 1)
            for (int i = 0; i < n; ++i) rv[i] = (rv[i >> 1] >> 1) | ((i & 1) << (k - 1));
        for (int i = 0; i < n; ++i)
            if (i < rv[i]) swap(a[i], a[rv[i]]);
        for (int len = 2; len <= n; len <= 1) {
            Z wn = mypow(Z(G), (Z::askMod() - 1) / len);
            if (iv) wn = wn.inv();
            for (int i = 0; i < n; i += len) {
                Z w = 1;
                for (int j = 0; j < len / 2; ++j) {
                    Z x = a[i + j], y = a[i + j + len / 2] * w;
                    a[i + j] = x + y;
                    a[i + j + len / 2] = x - y;
                    w *= wn;
                }
            }
        }
        if (iv) {
            Z x = Z(n).inv();
            for (Z& v : a) v *= x;
        }
    }

    static vector<Z> add(vector<Z> a, vector<Z> b) {
        int n = max(a.size(), b.size());
        a.resize(n);
        b.resize(n);
        for (int i = 0; i < n; ++i) a[i] += b[i];
        return a;
    }

    static vector<Z> sub(vector<Z> a, vector<Z> b) {
        int n = max(a.size(), b.size());
        a.resize(n);
        b.resize(n);
        for (int i = 0; i < n; ++i) a[i] -= b[i];
        return a;
    }

    static vector<Z> mul(vector<Z> a, vector<Z> b) {
        if (a.empty() || b.empty()) return {};
        int m = a.size() + b.size() - 1, n = 1;
        while (n < m) n <= 1;
        a.resize(n);
        b.resize(n);
        ntt(a, false);
        ntt(b, false);
        for (int i = 0; i < n; ++i) a[i] *= b[i];
    }
};

```

```

    ntt(a, true);
    a.resize(m);
    return a;
}

static vector<Z> inv(const vector<Z>& a, int n) {
    assert(n >= 0);
    if (!n) return {};
    assert(!a.empty() && a[0] != Z(0));
    if (n == 1) return {a[0].inv()};
    vector<Z> b = inv(a, (n + 1) >> 1);
    int m = 1;
    while (m < n << 1) m <=< 1;
    vector<Z> c(a.begin(), a.begin() + min<int>(a.size(), n));
    c.resize(m);
    b.resize(m);
    ntt(c, false);
    ntt(b, false);
    for (int i = 0; i < m; ++i) b[i] *= Z(2) - c[i] * b[i];
    ntt(b, true);
    b.resize(n);
    return b;
}

static vector<Z> div(const vector<Z>& a, const vector<Z>& b) {
    assert(!b.empty() && b.back() != Z(0));
    int n = a.size(), m = b.size();
    if (n < m) return {Z(0)};
    vector<Z> x(a.rbegin(), a.rend()), y(b.rbegin(), b.rend());
    y = inv(y, n - m + 1);
    vector<Z> q = mul(x, y);
    q.resize(n - m + 1);
    reverse(q.begin(), q.end());
    return q;
}

static vector<Z> mod(const vector<Z>& a, const vector<Z>& b) {
    vector<Z> r = sub(a, mul(div(a, b), b));
    r.resize(min<int>(r.size(), b.size() - 1));
    return r;
}

static vector<Z> der(const vector<Z>& a) {
    int n = a.size();
    if (n <= 1) return {};
    vector<Z> b(n - 1);
    for (int i = 1; i < n; ++i) b[i - 1] = a[i] * Z(i);
    return b;
}

static vector<Z> integ(const vector<Z>& a) {
    int n = a.size();
    vector<Z> b(n + 1);
    for (int i = 0; i < n; ++i) b[i + 1] = a[i] / Z(i + 1);
    return b;
}

static vector<Z> ln(const vector<Z>& a, int n) {
    assert(n >= 0);
    if (!n) return {};
    assert(!a.empty() && a[0] != Z(0));
    vector<Z> b = integ(mul(der(a), inv(a, n)));
    b.resize(n);
    return b;
}

```

```

}

static vector<Z> exp(const vector<Z>& a, int n) {
    assert(n >= 0);
    if (!n) return {};
    assert(a.empty() || a[0] == Z(0));
    vector<Z> b{Z(1)};
    for (int k = 2; k < n << 1; k <= 1) {
        int m = min(k, n);
        vector<Z> c(a.begin(), a.begin() + min<int>(a.size(), m));
        c.resize(m);
        c = sub(c, ln(b, m));
        c[0] += Z(1);
        b = mul(b, c);
        b.resize(m);
    }
    b.resize(n);
    return b;
}

static void build(vector<vector<Z>>& t, const vector<Z>& x, int u, int l, int r) {
    if (r - l == 1) {
        t[u] = {-x[l], Z(1)};
        return;
    }
    int m = (l + r) >> 1;
    build(t, x, u << 1, l, m);
    build(t, x, u << 1 | 1, m, r);
    t[u] = mul(t[u << 1], t[u << 1 | 1]);
}

static void eval0(const vector<Z>& f, const vector<vector<Z>>& t, vector<Z>& y, int u, int l, int r) {
    if (r - l == 1) {
        if (f.empty()) y[l] = Z(0);
        else if (f.size() == 1) y[l] = f[0];
        else y[l] = mod(f, t[u])[0];
        return;
    }
    vector<Z> g = mod(f, t[u]);
    int m = (l + r) >> 1;
    eval0(g, t, y, u << 1, l, m);
    eval0(g, t, y, u << 1 | 1, m, r);
}

static vector<Z> eval(const vector<Z>& f, const vector<Z>& x) {
    int n = x.size();
    if (!n) return {};
    vector<vector<Z>> t(n << 2 | 1);
    build(t, x, 1, 0, n);
    vector<Z> y(n);
    eval0(f, t, y, 1, 0, n);
    return y;
}

static vector<Z> interp(const vector<Z>& x, const vector<Z>& y) {
    int n = x.size();
    assert(y.size() == n);
    if (!n) return {};
    vector<vector<Z>> t(n << 2 | 1);
    build(t, x, 1, 0, n);
    vector<Z> d = eval(der(t[1]), x), w(n);
    for (int i = 0; i < n; ++i) w[i] = y[i] / d[i];
    function<vector<Z>(int, int, int)> dfs = [&](int u, int l, int r) -> vector<Z> {

```

```
        if (r - l == 1) return {w[l]};
        int m = (l + r) >> 1;
        return add(mul(dfs(u << 1, l, m), t[u << 1 | 1]), mul(dfs(u << 1 | 1, m, r), t[u << 1]));
    };
    return dfs(1, 0, n);
}

};

template <class Z> using NTT = Poly<Z, 3>;
```

08C - 轻量多项式 (Poly, Z) (NTT, ln, exp) .cpp

116 lines · C++ · 06 - 数学

```

/*
用途：轻量 NTT 多项式（乘法、求逆、ln、exp）。
用法：using P = Poly<Z, 3>; Z 为任意 MInt<P>，当前模数须为 NTT 友好质数。
性能：默认使用宏展开后的 int；若大规模数据因此超时或超空间，可将纯下标、状态或计数局部改为 i32。
*/
template <class Z, int G = 3> struct Poly {
    static vector<Z> cut(vector<Z> a, int n) {
        a.resize(n);
        return a;
    }

    static vector<Z> add(vector<Z> a, const vector<Z> &b) {
        if (a.size() < b.size()) a.resize(b.size());
        for (int i = 0; i < b.size(); ++i) a[i] += b[i];
        return a;
    }

    static vector<Z> sub(vector<Z> a, const vector<Z> &b) {
        if (a.size() < b.size()) a.resize(b.size());
        for (int i = 0; i < b.size(); ++i) a[i] -= b[i];
        return a;
    }

    static void ntt(vector<Z> &a, bool inv) {
        int n = a.size();
        if (n <= 1) return;
        vector<int> rev(n);
        int lg = __builtin_ctzll(n);
        for (int i = 0; i < n; ++i) {
            rev[i] = (rev[i >> 1] >> 1) | ((i & 1) << (lg - 1));
            if (i < rev[i]) swap(a[i], a[rev[i]]);
        }
        for (int len = 2; len <= n; len <= 1) {
            Z wn = mypow(Z(G), (Z::askMod() - 1) / len);
            if (inv) wn = wn.inv();
            for (int i = 0; i < n; i += len) {
                Z w = 1;
                for (int j = 0; j < len / 2; ++j) {
                    Z x = a[i + j], y = a[i + j + len / 2] * w;
                    a[i + j] = x + y;
                    a[i + j + len / 2] = x - y;
                    w *= wn;
                }
            }
        }
        if (inv) {
            Z iv = Z(n).inv();
            for (auto &x : a) x *= iv;
        }
    }

    static vector<Z> mul(vector<Z> a, vector<Z> b) {
        if (a.empty() || b.empty()) return {};
        int nd = a.size() + b.size() - 1;
        int n = 1;
        while (n < nd) n <= 1;
        a.resize(n), b.resize(n);
        ntt(a, false), ntt(b, false);
        for (int i = 0; i < n; ++i) a[i] *= b[i];
        ntt(a, true);
    }

```

```

        a.resize(nd);
        return a;
    }

    static vector<Z> der(const vector<Z> &a) {
        if (a.size() <= 1) return {};
        vector<Z> b(a.size() - 1);
        for (int i = 1; i < a.size(); ++i) b[i - 1] = a[i] * Z(i);
        return b;
    }

    static vector<Z> integ(const vector<Z> &a) {
        vector<Z> b(a.size() + 1);
        for (int i = 0; i < a.size(); ++i) b[i + 1] = a[i] / Z(i + 1);
        return b;
    }

    static vector<Z> inv(const vector<Z> &a, int m) {
        assert(m >= 0);
        if (m == 0) return {};
        assert(!a.empty() && a[0] != Z(0));
        vector<Z> b{a[0].inv()};
        for (int k = 2; k < 2 * m; k <= 1) {
            vector<Z> f = cut(a, min(k, m));
            vector<Z> t = mul(mul(b, b), f);
            b.resize(min(k, m));
            for (int i = 0; i < b.size(); ++i) b[i] += b[i] - t[i];
        }
        b.resize(m);
        return b;
    }

    static vector<Z> ln(const vector<Z> &a, int m) {
        assert(m >= 0);
        if (m == 0) return {};
        assert(!a.empty() && a[0] == Z(1));
        vector<Z> b = integ(mul(der(a), inv(a, m)));
        b.resize(m);
        return b;
    }

    static vector<Z> exp(const vector<Z> &a, int m) {
        assert(m >= 0);
        if (m == 0) return {};
        assert(a.empty() || a[0] == Z(0));
        vector<Z> b{1};
        for (int k = 2; k < 2 * m; k <= 1) {
            int n = min(k, m);
            vector<Z> t = sub(cut(a, n), ln(b, n));
            t[0] += Z(1);
            b = cut(mul(b, t), n);
        }
        b.resize(m);
        return b;
    }
};

```

08D - 快速傅里叶变换 (FFT, with Complex) .cpp

68 lines · C++ · 06 - 数学

```

struct FFT {
    using cd = Complex<double>;
    static constexpr double PI = 3.141592653589793238462643383279502884;

    void fft(vector<cd> &a, bool inv) {
        int n = a.size();
        static vector<int> rev;
        static vector<cd> rts{{0, 0}, {1, 0}};

        if (rev.size() != n) {
            int k = __builtin_ctz(n);
            rev.assign(n, 0);
            for (int i = 0; i < n; i++) rev[i] = (rev[i >> 1] >> 1) | ((i & 1) << (k - 1));
        }

        if (rts.size() < n) {
            int k = __builtin_ctz(rts.size());
            rts.resize(n);
            while ((1 << k) < n) {
                double da = 2 * PI / (1 << (k + 1));
                for (int i = 1 << (k - 1); i < (1 << k); i++) {
                    rts[i << 1] = rts[i];
                    double ag = da * (2 * i + 1 - (1 << k));
                    rts[i << 1 | 1] = cd(cos(ag), sin(ag));
                }
                k++;
            }
        }

        for (int i = 0; i < n; i++)
            if (i < rev[i]) swap(a[i], a[rev[i]]);

        for (int len = 1; len < n; len <= 1) {
            for (int i = 0; i < n; i += len << 1) {
                for (int j = 0; j < len; j++) {
                    cd u = a[i + j];
                    cd v = a[i + j + len] * rts[len + j];
                    a[i + j] = u + v;
                    a[i + j + len] = u - v;
                }
            }
        }

        if (inv) {
            reverse(a.begin() + 1, a.end());
            for (cd &x : a) x /= n;
        }
    }

    vector<int> mul(const vector<int> &a, const vector<int> &b) {
        if (a.empty() || b.empty()) return {};
        int nd = a.size() + b.size() - 1;
        vector<cd> fa(a.begin(), a.end()), fb(b.begin(), b.end());
        int n = 1;
        while (n < nd) n <= 1;
        fa.resize(n);
        fb.resize(n);
        fft(fa, false);
        fft(fb, false);
    }
}

```

```
    for (int i = 0; i < n; i++) fa[i] *= fb[i];
    fft(fa, true);

    vector<int> res(nd);
    for (int i = 0; i < nd; i++) res[i] = llround(fa[i].a);
    return res;
}
};
```


08E - 快速傅里叶变换 (ModFFT, with Complex) .cpp

96 lines · C++ · 06 - 数学

```

/*
用途：任意 32 位模数卷积（系数按 15 位拆分）。
依赖：06A - 复数 (Complex)。
用法：原始整数写 mul(vector<int>, vector<int>, p);
      自动取模写 mul(vector<Z>, vector<Z>), Z 为 MInt<P>。
性能：默认使用宏展开后的 int；若大规模数据因此超时或超空间，可将纯下标、状态或计数局部改为 i32。
*/

struct FFT {
    using C = Complex<double>;

    static constexpr double pi = 3.141592653589793238462643383279502884;

    static void fft(vector<C>& a, bool iv) {
        int n = a.size();
        assert(n && !(n & (n - 1)));
        static vector<int> rv;
        static vector<C> rt{{0, 0}, {1, 0}};
        if (rv.size() != n) {
            int k = __builtin_ctz(static_cast<unsigned>(n));
            rv.assign(n, 0);
            if (n > 1)
                for (int i = 0; i < n; ++i) rv[i] = (rv[i >> 1] >> 1) | ((i & 1) << (k - 1));
        }
        if (rt.size() < n) {
            int k = __builtin_ctz(static_cast<unsigned>(rt.size()));
            rt.resize(n);
            while ((1LL << k) < n) {
                double ag = 2 * pi / (1LL << (k + 1));
                for (int i = 1LL << (k - 1); i < (1LL << k); ++i) {
                    rt[i << 1] = rt[i];
                    double x = ag * (2 * i + 1 - (1LL << k));
                    rt[i << 1 | 1] = C(cos(x), sin(x));
                }
                ++k;
            }
        }
        for (int i = 0; i < n; ++i)
            if (i < rv[i]) swap(a[i], a[rv[i]]);
        for (int len = 1; len < n; len <= 1) {
            for (int i = 0; i < n; i += len << 1) {
                for (int j = 0; j < len; ++j) {
                    C x = a[i + j], y = a[i + j + len] * rt[len + j];
                    a[i + j] = x + y;
                    a[i + j + len] = x - y;
                }
            }
        }
        if (iv) {
            reverse(a.begin() + 1, a.end());
            for (C& x : a) x /= n;
        }
    }

    static vector<int> mul(const vector<int>& a, const vector<int>& b, int p) {
        if (a.empty() || b.empty()) return {};
        assert(p > 0);
        int m = a.size() + b.size() - 1, n = 1;
        while (n < m) n <= 1;
        vector<C> x(n), y(n);
    }
}

```

```

    for (int i = 0; i < a.size(); ++i) x[i] = C(a[i] & 32767, a[i] >> 15);
    for (int i = 0; i < b.size(); ++i) y[i] = C(b[i] & 32767, b[i] >> 15);
    fft(x, false);
    fft(y, false);
    vector<C> u(n), v(n);
    for (int i = 0; i < n; ++i) {
        int j = (n - i) & (n - 1);
        C a1 = (x[i] + conj(x[j])) * C(0.5, 0);
        C a2 = (x[i] - conj(x[j])) * C(0, -0.5);
        C b1 = (y[i] + conj(y[j])) * C(0.5, 0);
        C b2 = (y[i] - conj(y[j])) * C(0, -0.5);
        u[i] = a1 * b1 + a2 * b2 * C(0, 1);
        v[i] = a1 * b2 + a2 * b1;
    }
    fft(u, true);
    fft(v, true);
    vector<int> r(m);
    for (int i = 0; i < m; ++i) {
        int x0 = llround(u[i].a) % p;
        int y0 = llround(v[i].a) % p;
        int z0 = llround(u[i].b) % p;
        r[i] = (x0 + (y0 << 15) + (z0 << 30)) % p;
    }
    return r;
}

template <class Z> static vector<Z> mul(const vector<Z>& x, const vector<Z>& y) {
    vector<int> a(x.size()), b(y.size());
    for (int i = 0; i < x.size(); ++i) a[i] = static_cast<int>(x[i]);
    for (int i = 0; i < y.size(); ++i) b[i] = static_cast<int>(y[i]);
    vector<int> v = mul(a, b, Z::askMod());
    vector<Z> r(v.size());
    for (int i = 0; i < v.size(); ++i) r[i] = Z(v[i]);
    return r;
}
};

```

08F - 多模快速数论变换 (CRTNTT, with NTT) .cpp

72 lines · C++ · 06 - 数学

```

/*
用途：用三模 NTT + CRT 做任意 32 位模数卷积。
依赖：08 - 快速数论变换 (NTT)。
用法：原始整数写 mul(vector<int>, vector<int>, p);
      自动取模写 mul(vector<Z>, vector<Z>), Z 为 MInt<P>。
性能：默认使用宏展开后的 int；若大规模数据因此超时或超空间，可将纯下标、状态或计数局部改为 i32。
*/

using i128 = __int128_t;

struct CRTNTT {
    static constexpr int p1 = 998244353;
    static constexpr int p2 = 1004535809;
    static constexpr int p3 = 104857601;

    NTT<p1, 3> a;
    NTT<p2, 3> b;
    NTT<p3, 3> c;

    static int mul(int a, int b, int p) {
        assert(p > 0);
        a %= p;
        b %= p;
#ifdef __SIZEOF_INT128__
        return i128(a) * b % p;
#else
        int r = 0;
        while (b) {
            if (b & 1) r = r >= p - a ? r - (p - a) : r + a;
            a = a >= p - a ? a - (p - a) : a + a;
            b >>= 1;
        }
        return r;
#endif
    }

    static int crt(int a, int b, int c, int p) {
        assert(p > 0);
        static constexpr int m1 = p1, m2 = p2, m3 = p3;
        static constexpr int i1 = 669690699;
        static constexpr int i2 = 99747662;
        int x = a;
        int t = (b - x) * i1 % m2;
        if (t < 0) t += m2;
        x += t * m1;
        t = (c - x % m3) * i2 % m3;
        if (t < 0) t += m3;
        int y = mul(t, mul(m1 % p, m2 % p, p), p);
        int z = x % p;
        return z >= p - y ? z - (p - y) : z + y;
    }

    vector<int> mul(const vector<int>& x, const vector<int>& y, int p) {
        if (x.empty() || y.empty()) return {};
        auto u = a.mul(x, y);
        auto v = b.mul(x, y);
        auto w = c.mul(x, y);
        vector<int> r(u.size());
        for (int i = 0; i < u.size(); ++i) r[i] = crt(u[i], v[i], w[i], p);
        return r;
    }
}

```

```
    }

    template <class Z> vector<Z> mul(const vector<Z>& x, const vector<Z>& y) {
        vector<int> a(x.size()), b(y.size());
        for (int i = 0; i < x.size(); ++i) a[i] = static_cast<int>(x[i]);
        for (int i = 0; i < y.size(); ++i) b[i] = static_cast<int>(y[i]);
        vector<int> v = mul(a, b, Z::askMod());
        vector<Z> r(v.size());
        for (int i = 0; i < v.size(); ++i) r[i] = Z(v[i]);
        return r;
    }
};
```

08G - FWT (OR、AND、XOR 卷积) .cpp

42 lines · C++ · 06 - 数学

```

/*
用途：长度  $2^k$  的 OR / AND / XOR 卷积。
前提：XOR 逆变换要求  $2$  在  $Z$  中可逆。
*/
enum class FWType { Or, And, Xor };

template <class Z> void fwt(vector<Z>& a, FWType tp, bool iv) {
    int n = a.size();
    assert(n && !(n & (n - 1)));
    for (int l = 1; l < n; l <= 1) {
        for (int i = 0; i < n; i += l < 1) {
            for (int j = 0; j < l; ++j) {
                Z& x = a[i + j];
                Z& y = a[i + j + l];
                if (tp == FWType::Or) {
                    if (iv) y -= x;
                    else y += x;
                } else if (tp == FWType::And) {
                    if (iv) x -= y;
                    else x += y;
                } else {
                    Z u = x, v = y;
                    x = u + v;
                    y = u - v;
                }
            }
        }
    }
    if (tp == FWType::Xor && iv) {
        Z x = Z(n).inv();
        for (Z& v : a) v *= x;
    }
}

template <class Z> vector<Z> conv(vector<Z> a, vector<Z> b, FWType tp) {
    assert(a.size() == b.size() && !a.empty());
    fwt(a, tp, false);
    fwt(b, tp, false);
    for (int i = 0; i < a.size(); ++i) a[i] *= b[i];
    fwt(a, tp, true);
    return a;
}

```

08H - SOS DP (子集、超集 Zeta 变换) .cpp

39 lines · C++ · 06 - 数学

```

/*
用途：对所有 mask 计算子集和或超集和，以及对应的 Möbius 逆变换。
前提：数组长度为  $2^k$ ，元素类型 T 支持 += 和 -=。
复杂度： $O(k \cdot 2^k)$ 。常用于按位包含关系计数、容斥和子集 DP。
*/
namespace SOSDP {
inline int bits(int siz) {
    assert(siz > 0 && (siz & (siz - 1)) == 0);
    return __builtin_ctz((unsigned)siz);
}

template <class T> void zetaSub(vector<T>& a) {
    int k = bits(a.size());
    for (int i = 0; i < k; ++i)
        for (int msk = 0; msk < a.size(); ++msk)
            if (msk >> i & 1) a[msk] += a[msk ^ (1 << i)];
}

template <class T> void mobSub(vector<T>& a) {
    int k = bits(a.size());
    for (int i = 0; i < k; ++i)
        for (int msk = 0; msk < a.size(); ++msk)
            if (msk >> i & 1) a[msk] -= a[msk ^ (1 << i)];
}

template <class T> void zetaSup(vector<T>& a) {
    int k = bits(a.size());
    for (int i = 0; i < k; ++i)
        for (int msk = 0; msk < a.size(); ++msk)
            if ((msk >> i & 1) == 0) a[msk] += a[msk | (1 << i)];
}

template <class T> void mobSup(vector<T>& a) {
    int k = bits(a.size());
    for (int i = 0; i < k; ++i)
        for (int msk = 0; msk < a.size(); ++msk)
            if ((msk >> i & 1) == 0) a[msk] -= a[msk | (1 << i)];
}
} // namespace SOSDP

```

08HA - 子集卷积 (Subset Convolution) .cpp

44 lines · C++ · 06 - 数学

```

/*
用途：c[S] = sum(A subset S) a[A] b[S-A]。
前提：长度为  $2^k$ ，Z 支持加减乘；复杂度  $O(k^2 2^k)$ 。
*/
namespace SubConv {
template <class Z> void zetaSub(vector<Z>& a) {
    int n = a.size(), k = __builtin_ctz((unsigned)n);
    assert(n && !(n & (n - 1)));
    for (int i = 0; i < k; ++i)
        for (int s = 0; s < n; ++s)
            if (s >> i & 1) a[s] += a[s ^ (1 << i)];
}

template <class Z> void mobSub(vector<Z>& a) {
    int n = a.size(), k = __builtin_ctz((unsigned)n);
    assert(n && !(n & (n - 1)));
    for (int i = 0; i < k; ++i)
        for (int s = 0; s < n; ++s)
            if (s >> i & 1) a[s] -= a[s ^ (1 << i)];
}

template <class Z> vector<Z> mul(const vector<Z>& a, const vector<Z>& b) {
    int n = a.size(), k = __builtin_ctz((unsigned)n);
    assert(n && !(n & (n - 1)) && b.size() == n);
    vector<vector<Z>> x(k + 1, vector<Z>(n)), y(k + 1, vector<Z>(n)), z(k + 1, vector<Z>(n));
    for (int s = 0; s < n; ++s) {
        int d = __builtin_popcount((unsigned)s);
        x[d][s] = a[s];
        y[d][s] = b[s];
    }
    for (int i = 0; i <= k; ++i) {
        zetaSub(x[i]);
        zetaSub(y[i]);
    }
    for (int i = 0; i <= k; ++i) {
        for (int j = 0; j <= i; ++j)
            for (int s = 0; s < n; ++s) z[i][s] += x[j][s] * y[i - j][s];
        mobSub(z[i]);
    }
    vector<Z> r(n);
    for (int s = 0; s < n; ++s) r[s] = z[__builtin_popcount((unsigned)s)][s];
    return r;
}
} // namespace SubConv

```

08l - 快速数论变换 (NTT, 极致优化) .cpp

254 lines · C++ · 06 - 数学

```

/*
用途: 32 位 Montgomery + radix-4 NTT, 接口与 08 的 NTT<P,G> 一致。
用法: auto c = NTT<998244353, 3>::mul(a, b); 卡常时替换 08。
限制: P 为小于  $2^{30}$  的奇质数, G 为原根, 变换长度整除 P-1。
性能: 模数值与变换缓存必须保留 u32, 循环下标保留 i32; 改用宏展开后的 int 会显著变慢并扩大内存。
*/

using i32 = int32_t;
using u32 = uint32_t;
using u64 = uint64_t;

template <u32 P, u32 G> struct NTT {
    struct Z {
        u32 x = 0;

        static constexpr u32 askR() {
            u32 r = P;
            for (i32 i = 0; i < 5; ++i) r *= 2 - P * r;
            return -r;
        }

        static constexpr u32 mr = askR();
        static constexpr u32 m2 = -u64(P) % P;

        static constexpr u32 red(u64 x) {
            return (x + u64(u32(x) * mr) * P) >> 32;
        }

        Z() = default;
        Z(int v) {
            v %= P;
            if (v < 0) v += P;
            x = red(u64(v) * m2);
        }

        static Z raw(u32 x) {
            Z a;
            a.x = x;
            return a;
        }

        u32 val() const {
            u32 v = red(x);
            return v >= P ? v - P : v;
        }

        Z pow(int n) const {
            Z a = *this, r = 1;
            while (n) {
                if (n & 1) r *= a;
                a *= a;
                n >>= 1;
            }
            return r;
        }

        Z inv() const {
            return pow(P - 2);
        }
    };
};

```



```

Z& operator+=(Z a) {
    x += a.x;
    if (x >= 2 * P) x -= 2 * P;
    return *this;
}

Z& operator-=(Z a) {
    x += 2 * P - a.x;
    if (x >= 2 * P) x -= 2 * P;
    return *this;
}

Z& operator*=(Z a) {
    x = red(u64(x) * a.x);
    return *this;
}

friend Z operator+(Z a, Z b) { return a += b; }
friend Z operator-(Z a, Z b) { return a -= b; }
friend Z operator*(Z a, Z b) { return a *= b; }
};

static_assert(P & 1);
static_assert(P < (1u << 30));
static_assert(u32(-NTT::Z::mr * P) == 1);

static inline bool ok = false;
static inline array<Z, 30> rt, ir, q2, iq2, q3, iq3;

static void init() {
    if (ok) return;
    ok = true;
    i32 k = __builtin_ctz(P - 1);
    rt[k] = Z(G).pow((P - 1) >> k);
    ir[k] = rt[k].inv();
    for (i32 i = k; i--;) {
        rt[i] = rt[i + 1] * rt[i + 1];
        ir[i] = ir[i + 1] * ir[i + 1];
    }
    Z a = 1, b = 1;
    for (i32 i = 0; i <= k - 2; ++i) {
        q2[i] = rt[i + 2] * a;
        iq2[i] = ir[i + 2] * b;
        a *= ir[i + 2], b *= rt[i + 2];
    }
    a = b = 1;
    for (i32 i = 0; i <= k - 3; ++i) {
        q3[i] = rt[i + 3] * a;
        iq3[i] = ir[i + 3] * b;
        a *= ir[i + 3], b *= rt[i + 3];
    }
}

static void ntt(vector<Z>& a, bool iv) {
    i32 n = a.size();
    assert(n && !(n & (n - 1)));
    i32 h = __builtin_ctz(u32(n));
    assert(h <= __builtin_ctz(P - 1));
    init();

    if (!iv) {
        i32 len = 0;
        while (len < h) {
            if (h - len == 1) {

```

```

        i32 p = 1 << (h - len - 1);
        Z w = 1;
        for (i32 s = 0; s < (1 << len); ++s) {
            i32 o = s << (h - len);
            for (i32 i = 0; i < p; ++i) {
                Z x = a[o + i], y = a[o + i + p] * w;
                a[o + i] = x + y;
                a[o + i + p] = x - y;
            }
            w *= q2[__builtin_ctz(~u32(s))];
        }
        ++len;
    } else {
        i32 p = 1 << (h - len - 2);
        Z w = 1, im = rt[2];
        for (i32 s = 0; s < (1 << len); ++s) {
            Z w2 = w * w, w3 = w2 * w;
            i32 o = s << (h - len);
            for (i32 i = 0; i < p; ++i) {
                Z a0 = a[o + i];
                Z a1 = a[o + i + p] * w;
                Z a2 = a[o + i + 2 * p] * w2;
                Z a3 = a[o + i + 3 * p] * w3;
                Z x = (a1 - a3) * im;
                a[o + i] = a0 + a1 + a2 + a3;
                a[o + i + p] = a0 - a1 + a2 - a3;
                a[o + i + 2 * p] = a0 - a2 + x;
                a[o + i + 3 * p] = a0 - a2 - x;
            }
            w *= q3[__builtin_ctz(~u32(s))];
        }
        len += 2;
    }
}
} else {
    i32 len = h;
    while (len) {
        if (len == 1) {
            i32 p = 1 << (h - len);
            Z w = 1;
            for (i32 s = 0; s < (1 << (len - 1)); ++s) {
                i32 o = s << (h - len + 1);
                for (i32 i = 0; i < p; ++i) {
                    Z x = a[o + i], y = a[o + i + p];
                    a[o + i] = x + y;
                    a[o + i + p] = (x - y) * w;
                }
                w *= iq2[__builtin_ctz(~u32(s))];
            }
            --len;
        } else {
            i32 p = 1 << (h - len);
            Z w = 1, im = ir[2];
            for (i32 s = 0; s < (1 << (len - 2)); ++s) {
                Z w2 = w * w, w3 = w2 * w;
                i32 o = s << (h - len + 2);
                for (i32 i = 0; i < p; ++i) {
                    Z a0 = a[o + i];
                    Z a1 = a[o + i + p];
                    Z a2 = a[o + i + 2 * p];
                    Z a3 = a[o + i + 3 * p];
                    Z x = (a2 - a3) * im;
                    a[o + i] = a0 + a1 + a2 + a3;
                    a[o + i + p] = (a0 - a1 + x) * w;
                }
            }
        }
    }
}

```

```

        a[0 + i + 2 * p] = (a0 + a1 - a2 - a3) * w2;
        a[0 + i + 3 * p] = (a0 - a1 - x) * w3;
    }
    w *= iq3[__builtin_ctz(~u32(s))];
}
len -= 2;
}
}
Z z = Z(n).inv();
for (auto& x : a) x *= z;
}

static u32 norm(int x) {
    x %= P;
    if (x < 0) x += P;
    return x;
}

static vector<u32> mul32(const vector<int>& a, const vector<int>& b) {
    if (a.empty() || b.empty()) return {};
    i32 an = a.size(), bn = b.size();
    i32 m = an + bn - 1;
    if (min(an, bn) <= 32) {
        vector<Z> z(m);
        for (i32 i = 0; i < an; ++i)
            for (i32 j = 0; j < bn; ++j) z[i + j] += Z(a[i]) * Z(b[j]);
        vector<u32> r(m);
        for (i32 i = 0; i < m; ++i) r[i] = z[i].val();
        return r;
    }

    i32 n = 1;
    while (n < m) n <= 1;
    if (m - 2 <= n / 2) {
        vector<int> x(a.begin(), a.end() - 1), y(b.begin(), b.end() - 1);
        u32 al = norm(a.back()), bl = norm(b.back());
        auto r = mul32(x, y);
        r.resize(m);
        r[m - 1] = u64(al) * bl % P;
        for (i32 i = 0; i < an - 1; ++i)
            r[i + bn - 1] = (r[i + bn - 1] + u64(norm(x[i])) * bl) % P;
        for (i32 i = 0; i < bn - 1; ++i)
            r[i + an - 1] = (r[i + an - 1] + u64(norm(y[i])) * al) % P;
        return r;
    }

    vector<Z> x(n), y(n);
    for (i32 i = 0; i < an; ++i) x[i] = Z(a[i]);
    for (i32 i = 0; i < bn; ++i) y[i] = Z(b[i]);
    bool eq = a == b;
    ntt(x, false);
    if (eq) y = x;
    else ntt(y, false);
    for (i32 i = 0; i < n; ++i) x[i] *= y[i];
    ntt(x, true);
    vector<u32> r(m);
    for (i32 i = 0; i < m; ++i) r[i] = x[i].val();
    return r;
}

static vector<int> mul(const vector<int>& a, const vector<int>& b) {
    auto x = mul32(a, b);
    return vector<int>(x.begin(), x.end());
}

```

```
    }  
};
```

081A - 任意模卷积 (CRTNTT, 极致优化) .cpp

49 lines · C++ · 06 - 数学

```

/*
用途：三次极低常数 NTT + CRT 的精确任意模卷积，可过针对浮点拆分的 FFT killer。
依赖：081 - 快速数论变换 (NTT, 极致优化)。
用法：CRTNTT f; auto c = f.mul(a, b, p)。输入会先规范到  $[0, p)$ 。
限制： $p \leq 2^{32}-1$ ，变换长度不超过  $2^{21}$ ；三模乘积必须大于单项卷积的绝对上界。
性能：三份余数缓存必须保留 u32；改用宏展开后的 int 会额外占用大量内存并降低缓存命中率。
*/

using u32 = uint32_t;
using u64 = uint64_t;

struct CRTNTT {
    static constexpr u32 p1 = 998244353;
    static constexpr u32 p2 = 1004535809;
    static constexpr u32 p3 = 104857601;
    static constexpr u32 i1 = 669690699;
    static constexpr u32 i2 = 99747662;

    static int norm(int x, int p) {
        x %= p;
        if (x < 0) x += p;
        return x;
    }

    vector<int> mul(const vector<int>& x, const vector<int>& y, int p) const {
        if (x.empty() || y.empty()) return {};
        assert(0 < p && p <= 4294967295LL);
        int nx = x.size(), ny = y.size();
        assert(nx + ny - 1 <= (1LL << 21));
        vector<int> a(nx), b(ny);
        for (int i = 0; i < nx; ++i) a[i] = norm(x[i], p);
        for (int i = 0; i < ny; ++i) b[i] = norm(y[i], p);

        auto a1 = NTT<p1, 3>::mul32(a, b);
        auto a2 = NTT<p2, 3>::mul32(a, b);
        auto a3 = NTT<p3, 3>::mul32(a, b);
        vector<int> r(a1.size());
        int n = r.size();
        u64 m1 = p1 % p;
        u64 m2 = m1 * (p2 % p) % p;
        for (int i = 0; i < n; ++i) {
            u64 x1 = a1[i];
            u64 x2 = (u64(a2[i]) + p2 - x1 % p2) * i1 % p2;
            u64 x3 = (u64(a3[i]) + p3 - (x1 + x2 * p1) % p3) * i2 % p3;
            r[i] = (x1 % p + x2 % p * m1 + x3 % p * m2) % p;
        }
        return r;
    }
};

```

09 - 博弈论 (Nim、SG) .cpp

142 lines · C++ · 06 - 数学

```

/*
适用范围：有限、无环、无偏（双方可走的边相同）、正常游戏规则（不能走者负）。
Nim：异或非 0 为先手必胜；misere 单独处理“最后一步输”的反常 Nim。
SG：多个独立子游戏的 Grundy 值异或；buildSG 要求所有转移均指向更小编号状态。
性能：默认使用宏展开后的 int；若大规模数据因此超时或超空间，可将纯下标、状态或计数局部改为 i32。
*/

namespace Game {

inline bool nim(const vector<int>& pil) {
    int xs = 0;
    for (int pil : pil) {
        assert(pil >= 0);
        xs ^= pil;
    }
    return xs != 0;
}

// 返回 {堆下标, 操作后剩余石子数}；必败态返回 nullopt。
inline optional<pair<int, int>> nimMove(const vector<int>& a) {
    int x = 0;
    for (int v : a) assert(v >= 0), x ^= v;
    if (!x) return nullopt;
    for (int i = 0; i < a.size(); ++i) {
        int v = a[i] ^ x;
        if (v < a[i]) return pair<int, int>{i, v};
    }
    return nullopt;
}

inline bool bash(int n, int m) {
    assert(n >= 0 && m > 0);
    return n % (m + 1) != 0;
}

// 每次可同时修改至多 k 堆；P 态当且仅当每一二进制位的 1 数量均为 k+1 的倍数。
inline bool moore(const vector<int>& a, int k) {
    assert(k >= 1);
    for (int b = 0; b < 64; ++b) {
        int c = 0;
        for (auto x : a) c += x >> b & 1;
        if (c % (k + 1)) return true;
    }
    return false;
}

// a[0] 表示第 1 级台阶；只异或奇数级台阶。
inline bool stair(const vector<int>& a) {
    int x = 0;
    for (int i = 0; i < a.size(); i += 2) assert(a[i] >= 0), x ^= a[i];
    return x != 0;
}

inline bool misere(const vector<int>& pil) {
    bool one = true;
    int cnt = 0;
    int xs = 0;
    for (int pil : pil) {
        assert(pil >= 0);
        one &= pil <= 1;
    }
}

```

```

        cnt += pil > 0;
        xs ^= pil;
    }
    return one ? (cnt % 2 == 0) : (xs != 0);
}

inline int mex(const vector<int>& a) {
    vector<char> vis(a.size() + 1);
    for (int x : a) if (0 <= x && x <= a.size()) vis[x] = true;
    for (int x = 0;; ++x) if (!vis[x]) return x;
}

template <class F> vector<int> buildSG(int n, F to) {
    assert(n >= 0);
    vector<int> sg(n);
    for (int s = 0; s < n; ++s) {
        vector<int> a;
        for (int v : to(s)) {
            assert(0 <= v && v < s);
            a.push_back(sg[v]);
        }
        sg[s] = mex(a);
    }
    return sg;
}

inline vector<int> subSG(int n, vector<int> mv) {
    assert(n >= 0);
    for (int x : mv) assert(x > 0);
    sort(mv.begin(), mv.end());
    mv.erase(unique(mv.begin(), mv.end()), mv.end());
    return buildSG(n + 1, [&](int s) {
        vector<int> a;
        for (int x : mv) {
            if (x > s) break;
            a.push_back(s - x);
        }
        return a;
    });
}

inline bool sumSG(const vector<int>& a) {
    int x = 0;
    for (int v : a) {
        assert(v >= 0);
        x ^= v;
    }
    return x != 0;
}

inline bool antiSG(const vector<int>& a) {
    int x = 0, big = 0;
    for (int v : a) assert(v >= 0), x ^= v, big += v > 1;
    return big ? x != 0 : x == 0;
}

inline int lasker(int x) {
    assert(x >= 0);
    if (!x) return 0;
    return x % 4 == 0 ? x - 1 : x % 4 == 3 ? x + 1 : x;
}

inline int treeSG(const vector<vector<int>>& g, int rt = 1) {
    auto dfs = [&](auto&& go, int u, int fa) -> int {

```

```
        int x = 0;
        for (int v : g[u]) if (v != fa) x ^= go(go, v, u) + 1;
        return x;
    };
    return dfs(dfs, rt, -1);
}

inline bool fib(int n) {
    if (n <= 1) return false;
    int a = 1, b = 2;
    while (b < n) {
        if (numeric_limits<int>::max() - b < a) return true;
        auto c = a + b;
        a = b, b = c;
    }
    return b != n;
}
} // namespace Game
```


09A - Wythoff Nim.cpp

23 lines · C++ · 06 - 数学

```
/*
用途：判断 Wythoff Nim（两堆，每步可取一堆任意个或两堆各取相同正数）的必败态。
结论：设  $a \leq b$ 、 $k = b - a$ ，则  $(a, b)$  为必败态当且仅当  $a = \text{floor}(k * \phi)$ 。
浮点实现适合  $a, b$  约  $1e15$  及以内；更大范围需额外做精确无理数比较。
*/
using ld = long double;

namespace WythoffNim {
inline int flrPhi(int k) {
    static const ld phi = (1.0L + sqrtl(5.0L)) / 2.0L;
    return floorl(ld(k) * phi);
}

inline pair<int, int> lose(int k) {
    int fst = flrPhi(k);
    return {fst, fst + k};
}

inline bool ask(int fst, int b) {
    if (fst > b) swap(fst, b);
    return fst == flrPhi(b - fst);
}
} // namespace WythoffNim
```

10A - 圆周率计算 (Machin, with FixedPoint) .cpp

38 lines · C++ · 06 - 数学

```
/*
用途：用 Machin 公式计算圆周率。
依赖：高精度 01A (或 01B) BigInt + 02A FixedPoint。适合约  $2 * 10^3$  位及以下；更高位数使用 10B。
*/

FixedPoint machin(int n) {
    const int ex = 40;
    const int pre = n + ex;
    FixedPoint::setPre(pre);

    auto arc = [&](int iv) -> FixedPoint {
        FixedPoint trm = FixedPoint(1) / FixedPoint(iv);
        FixedPoint sum = trm;
        int sgn = -1;
        FixedPoint iv2 = FixedPoint(1) / FixedPoint(iv * iv);
        FixedPoint eps(BigInt(1), pre + 15, 1);
        for (int k = 3;; k += 2) {
            trm = trm * iv2;
            FixedPoint cur = trm / FixedPoint(k);
            if (cur.abs() < eps) break;
            if (sgn > 0)
                sum = sum + cur;
            else
                sum = sum - cur;
            sgn = -sgn;
        }
        sum.round(FixedPoint::pre);
        sum.norm();
        return sum;
    };

    FixedPoint a1 = arc(5);
    FixedPoint a2 = arc(239);
    FixedPoint pi = a1 * FixedPoint(16) - a2 * FixedPoint(4);
    pi.round(n);
    pi.norm();
    return pi;
}
```

10B - 圆周率计算 (Chudnovsky, with FixedPoint) .cpp

45 lines · C++ · 06 - 数学

```

/*
用途：用 Chudnovsky 公式计算圆周率。
依赖：高精度 01B BigInt + 02A FixedPoint。适合约  $2 * 10^5$  位及以下；低精度时 10A 常数更小。
*/
FixedPoint chud(int n) {
    const int ex = 20;
    const int pre = n + ex;
    FixedPoint::setPre(pre);

    auto fac = [](int x) -> BigInt {
        BigInt res(1);
        for (int i = 2; i <= x; ++i) res *= BigInt(i);
        return res;
    };
    auto tf = [](const BigInt& x) -> FixedPoint {
        return FixedPoint(x, 0, 1);
    };
    auto sf = [&](const FixedPoint& x) -> FixedPoint {
        FixedPoint y(to_string(sqrt(stod(x.str()))));
        for (int i = 0; i < 80; ++i) {
            y = (y + x / y) / FixedPoint(2);
        }
        y.round(pre);
        y.norm();
        return y;
    };

    FixedPoint sum(0);
    string es = "0." + string(pre + 5, '0') + "1";
    FixedPoint eps(es);

    for (int k = 0;; ++k) {
        BigInt a = fac(6 * k) * (BigInt(545140134) * k + BigInt(13591409));
        BigInt d = fac(3 * k) * BigInt::pow(fac(k), 3) * BigInt::pow(BigInt(640320), 3 * k);
        FixedPoint trm = tf(a) / tf(d);
        if (k % 2 == 1) trm = FixedPoint(0) - trm;
        sum += trm;
        if (trm.abs() < eps) break;
    }

    FixedPoint pi = FixedPoint(426880) * sf(FixedPoint(10005)) / sum;
    pi.round(n);
    pi.norm();
    return pi;
}

```

11A - 牛顿迭代（方程、平方根、开 k 次方）.cpp

49 lines · C++ · 06 - 数学

```

/*
用途：Newton-Raphson 数值求根，以及平方根、非负 k 次根。
前提：函数在根附近可导且导数不接近 0；失败返回 nullopt。浮点题应按量级设置 eps。
复杂度：收敛区间内通常二次收敛；远离根或多重根时不保证收敛。
*/
using ld = long double;

namespace Newton {
template <class F, class DF>
optional<ld> solve(F f, DF df, ld x,
                  ld eps = 1e-12L, int lim = 100) {
    assert(eps > 0 && lim > 0);
    ld cur = x;
    for (int it = 0; it < lim; ++it) {
        ld val = f(cur);
        ld slp = df(cur);
        if (!isfinite(val) || !isfinite(slp) || fabs(slp) <= eps) return nullopt;
        ld nxt = cur - val / slp;
        if (!isfinite(nxt)) return nullopt;
        if (fabs(nxt - cur) <= eps * max(ld(1), fabs(nxt))) return nxt;
        cur = nxt;
    }
    return nullopt;
}

inline ld sqrt(ld val, ld eps = 1e-15L) {
    assert(val >= 0);
    if (val == 0) return 0;
    ld cur = max(ld(1), val);
    for (int it = 0; it < 200; ++it) {
        ld nxt = (cur + val / cur) / 2;
        if (fabs(nxt - cur) <= eps * max(ld(1), fabs(nxt))) return nxt;
        cur = nxt;
    }
    return cur;
}

inline ld root(ld val, int k, ld eps = 1e-15L) {
    assert(val >= 0 && k >= 1);
    if (val == 0 || k == 1) return val;
    ld cur = max(ld(1), pow(val, 1.0L / k));
    for (int it = 0; it < 200; ++it) {
        ld nxt = (ld(k - 1) * cur + val / pow(cur, k - 1)) / k;
        if (fabs(nxt - cur) <= eps * max(ld(1), fabs(nxt))) return nxt;
        cur = nxt;
    }
    return cur;
}
} // namespace Newton

```

11B - 自适应辛普森积分.cpp

37 lines · C++ · 06 - 数学

```

/*
用途：数值计算定积分  $\int [l, r] f(x) dx$ 。
前提：函数在区间上足够光滑；奇点、极高频振荡、间断点应先分段处理。
复杂度：自适应递归，eps 为绝对误差目标，maxDepth 防止病态数据无限递归。
*/
using ld = long double;

namespace Simpson {
template <class F>
ld solve(F f, ld l, ld r, ld eps = 1e-10L, int dep = 30) {
    assert(eps > 0 && dep >= 0);
    if (l > r) return -solve(f, r, l, eps, dep);
    auto sp = [](ld a, ld b, ld x, ld y, ld z) {
        return (b - a) * (x + 4 * y + z) / 6;
    };

    ld mid = (l + r) / 2;
    ld fl = f(l), fm = f(mid), fr = f(r);
    ld s = sp(l, r, fl, fm, fr);
    std::function<ld(ld, ld, ld, ld, ld, ld, ld, int)> dfs;
    dfs = [&](ld a, ld b, ld x, ld y, ld z, ld v,
              ld e, int d) -> ld {
        ld m = (a + b) / 2;
        ld lm = (a + m) / 2, rm = (m + b) / 2;
        ld xl = f(lm), xr = f(rm);
        ld sl = sp(a, m, x, xl, y);
        ld sr = sp(m, b, y, xr, z);
        ld err = sl + sr - v;
        if (d == 0 || fabsl(err) <= 15 * e) {
            return sl + sr + err / 15;
        }
        return dfs(a, m, x, xl, y, sl, e / 2, d - 1) +
            dfs(m, b, y, xr, z, sr, e / 2, d - 1);
    };
    return dfs(l, r, fl, fm, fr, s, eps, dep);
}
} // namespace Simpson

```

11C - 三分搜索（实数、整数）.cpp

36 lines · C++ · 06 - 数学

```
/*
用途：在单峰/单谷函数上找最小值点；实数版返回近似 argmin，整数版返回精确 argmin。
前提：目标函数在给定闭区间上单峰（或单谷）；不满足时不能套用。
复杂度：实数版  $O(\text{iterations})$ ，整数版  $O(\log(r-l))$  次后常数枚举。
*/
using ld = long double;

namespace Ternary {
template <class F>
ld argmin(ld l, ld r, F f, int lim = 200) {
    assert(l <= r && lim > 0);
    for (int it = 0; it < lim; ++it) {
        ld fst = (2 * l + r) / 3;
        ld b = (l + 2 * r) / 3;
        if (f(fst) <= f(b)) r = b;
        else l = fst;
    }
    return (l + r) / 2;
}

template <class F>
int argmin(int l, int r, F f) {
    assert(l <= r);
    while (r - l > 3) {
        int fst = l + (r - l) / 3;
        int b = r - (r - l) / 3;
        if (f(fst) <= f(b)) r = b - 1;
        else l = fst + 1;
    }
    int ans = l;
    for (int val = l + 1; val <= r; ++val) {
        if (f(val) < f(ans)) ans = val;
    }
    return ans;
}
} // namespace Ternary
```

11D - 数值求根（二分、割线）.cpp

45 lines · C++ · 06 - 数学

```

/*
用途：连续函数的二分求根和割线法求根。
前提：二分要求端点异号或已有根；割线法需要足够好的两个初值。失败返回 nullopt。
复杂度：二分  $O(\log((r-l)/\epsilon))$ ；割线法通常超线性收敛但无全局保证。
*/
using ld = long double;

namespace Root {
template <class F>
optional<ld> bisect(F f, ld l, ld r, ld eps = 1e-12L,
                  int lim = 200) {
    assert(l <= r && eps > 0 && lim > 0);
    ld fl = f(l), fr = f(r);
    if (fabsl(fl) <= eps) return l;
    if (fabsl(fr) <= eps) return r;
    if ((fl < 0) == (fr < 0)) return nullopt;
    for (int it = 0; it < lim; ++it) {
        ld mid = (l + r) / 2;
        ld fm = f(mid);
        if (fabsl(fm) <= eps || r - l <= eps) return mid;
        if ((fl < 0) == (fm < 0)) l = mid, fl = fm;
        else r = mid, fr = fm;
    }
    return (l + r) / 2;
}

template <class F>
optional<ld> secant(F f, ld fst, ld b, ld eps = 1e-12L,
                  int lim = 100) {
    assert(eps > 0 && lim > 0);
    ld fa = f(fst), fb = f(b);
    for (int it = 0; it < lim; ++it) {
        ld d = fb - fa;
        if (fabsl(d) <= eps) return nullopt;
        ld nxt = b - fb * (b - fst) / d;
        if (!isfinite(nxt)) return nullopt;
        if (fabsl(nxt - b) <= eps * max(ld(1), fabsl(nxt))) return nxt;
        fst = b;
        fa = fb;
        b = nxt;
        fb = f(b);
    }
    return nullopt;
}
} // namespace Root

```

12A - 位运算与子集枚举.cpp

44 lines · C++ · 06 - 数学

```

/*
用途：常用位操作、子掩码与超掩码枚举。
前提：mask 使用 u64；超集枚举要求 bitCount < 64，实际通常 <= 20。
复杂度：子掩码枚举  $O(2^{\text{popcount}(\text{mask})})$ ，所有 mask 的子掩码总复杂度  $O(3^n)$ 。
*/

using u64 = uint64_t;

namespace BitToolkit {
inline u64 lowbit(u64 val) {
    return val & (0ULL - val);
}

inline int bitLen(u64 val) {
    return val == 0 ? 0 : 64 - __builtin_clzll(val);
}

inline int ctz(u64 val) {
    return val == 0 ? 64 : __builtin_ctzll(val);
}

template <class F> void submask(u64 msk, F vis) {
    u64 s = msk;
    while (true) {
        vis(s);
        if (s == 0) break;
        s = (s - 1) & msk;
    }
}

template <class F> void supset(u64 msk, int k, F vis) {
    assert(0 <= k && k < 64 && (msk >> k) == 0);
    u64 all = (1ULL << k) - 1;
    submask(all ^ msk, [&](u64 c) { vis(msk | c); });
}

inline u64 nxtComb(u64 val) {
    assert(val != 0);
    u64 lb = lowbit(val);
    u64 nxt = val + lb;
    assert(nxt > val);
    return nxt | (((val ^ nxt) / lb) >> 2);
}
} // namespace BitToolkit

```


12B - 异或线性基 (XorBasis64) .cpp

59 lines · C++ · 06 - 数学

```

/*
用途：64 位异或线性基：插入、可表示性、最大异或、化简后的基向量。
复杂度：每次插入/查询  $O(64)$ ，化简  $O(64^2)$ 。可合并、可持久化等扩展见数据结构目录的线性基。
*/

using u64 = uint64_t;

struct LB {
    array<u64, 64> a{};
    int rk = 0;

    bool insert(u64 val) {
        for (int bit = 63; bit >= 0; --bit) {
            if (((val >> bit) & 1ULL) == 0) continue;
            if (a[bit] == 0) {
                a[bit] = val;
                ++rk;
                return true;
            }
            val ^= a[bit];
        }
        return false;
    }

    bool ask(u64 val) const {
        for (int bit = 63; bit >= 0; --bit) {
            if (((val >> bit) & 1ULL) == 0) continue;
            if (a[bit] == 0) return false;
            val ^= a[bit];
        }
        return true;
    }

    u64 askMax(u64 sd = 0) const {
        u64 ans = sd;
        for (int bit = 63; bit >= 0; --bit) {
            if ((ans ^ a[bit]) > ans) ans ^= a[bit];
        }
        return ans;
    }

    void reduce() {
        for (int hi = 63; hi >= 0; --hi) {
            if (a[hi] == 0) continue;
            for (int low = hi - 1; low >= 0; --low) {
                if (a[low] != 0 && ((a[hi] >> low) & 1ULL)) a[hi] ^= a[low];
            }
        }
    }

    vector<u64> askVec() const {
        LB z = *this;
        z.reduce();
        vector<u64> res;
        for (int bit = 0; bit < 64; ++bit)
            if (z.a[bit] != 0) res.push_back(z.a[bit]);
        return res;
    }
};

```

12C - 二进制 GCD 与 Gray 编码.cpp

30 lines · C++ · 06 - 数学

```
/*
用途：Stein（二进制）gcd，以及 Gray 编码互转。
复杂度：binaryGcd 为  $O(\log \max(a,b))$ ；Gray 编码/解码  $O(1)$  /  $O(\log \text{word bits})$ 。
*/

using u64 = uint64_t;

namespace BinArith {
inline u64 gcd(u64 l, u64 r) {
    if (l == 0) return r;
    if (r == 0) return l;
    int s = __builtin_ctzll(l | r);
    l >>= __builtin_ctzll(l);
    r >>= __builtin_ctzll(r);
    if (l > r) swap(l, r);
    r -= l;
} while (r != 0);
return l << s;
}

inline u64 gray(u64 val) {
    return val ^ (val >> 1);
}

inline u64 igray(u64 g) {
    for (int sh = 1; sh < 64; sh <= 1) g ^= g >> sh;
    return g;
}
} // namespace BinArith
```

13A - Fibonacci (快速倍增) .cpp

16 lines · C++ · 06 - 数学

```
/*
用途:  $O(\log n)$  求 Fibonacci; T 可为整数、MInt 或高精整数。
*/

template <class T> pair<T, T> fib2(int n) {
    assert(n >= 0);
    if (!n) return {T(0), T(1)};
    auto [x, y] = fib2<T>(n >> 1);
    T a = x * (T(2) * y - x);
    T b = x * x + y * y;
    return n & 1 ? pair<T, T>{b, a + b} : pair<T, T>{a, b};
}

template <class T> T fib(int n) {
    return fib2<T>(n).first;
}
```

13B - 约瑟夫环.cpp

15 lines · C++ · 06 - 数学

```
/*
用途：n 人 (0..n-1) 从 0 开始，每次数 k 个并删除，求最后幸存者。
复杂度：O(n)，适合 n 可线性枚举；返回 0-indexed 或 1-indexed 答案。
*/

inline int joseph(int n, int stp) {
    assert(n >= 1 && stp >= 1);
    int ans = 0;
    for (int siz = 2; siz <= n; ++siz) ans = (ans + stp) % siz;
    return ans;
}

inline int joseph1(int n, int stp) {
    return joseph(n, stp) + 1;
}
```

01A - KMP.cpp

43 lines · C++ · 07 - 字符串

```

/*
用途：单模式串匹配、border 与前缀函数。
约定：前缀函数和匹配位置使用 int；空模式串匹配所有 n+1 个位置。
性能：默认使用宏展开后的 int；若大规模数据因此超时或超空间，可将纯下标、状态或计数局部改为 i32。
*/
struct KMP {

    string p;
    vector<int> pi;

    explicit KMP(const string& s) : p(s) {
        build();
    }

    void build() {
        int n = p.size();
        pi.assign(n, 0);
        for (int i = 1, j = 0; i < n; ++i) {
            while (j && p[i] != p[j]) j = pi[j - 1];
            if (p[i] == p[j]) ++j;
            pi[i] = j;
        }
    }

    vector<int> ask(const string& s) const {
        vector<int> a;
        int n = s.size(), m = p.size();
        if (!m) {
            a.resize(n + 1);
            iota(a.begin(), a.end(), 0);
            return a;
        }
        for (int i = 0, j = 0; i < n; ++i) {
            while (j && s[i] != p[j]) j = pi[j - 1];
            if (s[i] == p[j]) ++j;
            if (j == m) {
                a.push_back(i - m + 1);
                j = pi[j - 1];
            }
        }
        return a;
    }
};

```

01AA - KMP自动机.cpp

63 lines · C++ · 07 - 字符串

```

/*
用途：KMP 的 DFA 转移，适合数字 DP、避免/统计模式串出现次数。
用法：状态为已匹配前缀长度 0..|p|；askNxt 后用 ask 判断是否匹配完整模式串。
约定：字符集为连续 [F,F+A)，允许空模式串。
性能：默认使用宏展开后的 int；若大规模数据因此超时或超空间，可将纯下标、状态或计数局部改为 i32。
*/
template <int A = 26, char F = 'a'> struct KMP {

    string p;
    vector<int> pi;
    vector<array<int, A>> to;

    KMP() = default;

    explicit KMP(const string& s) {
        build(s);
    }

    int id(char c) const {
        int x = c - F;
        assert(0 <= x && x < A);
        return x;
    }

    void build(const string& s) {
        p = s;
        int n = p.size();
        for (char c : p) id(c);
        pi.assign(n, 0);
        for (int i = 1, j = 0; i < n; ++i) {
            while (j && p[i] != p[j]) j = pi[j - 1];
            if (p[i] == p[j]) ++j;
            pi[i] = j;
        }
        to.assign(n + 1, array<int, A>());
        for (int i = 0; i <= n; ++i) {
            for (int x = 0; x < A; ++x) {
                int j = i;
                char c = static_cast<char>(F + x);
                while (j && (j == n || p[j] != c)) j = pi[j - 1];
                if (j < n && p[j] == c) ++j;
                to[i][x] = j;
            }
        }
    }

    int askSz() const {
        return p.size();
    }

    int askNxt(int p, int x) const {
        assert(0 <= p && p <= askSz() && 0 <= x && x < A);
        return to[p][x];
    }

    int askNxt(int p, char c) const {
        return askNxt(p, id(c));
    }

    bool ask(int p) const {

```

```
        return p == askSz();  
    }  
};
```

01B - Z函数（扩展KMP）.cpp

47 lines · C++ · 07 - 字符串

```

/*
用途：Z 函数、两串每个后缀与模式串的最长公共前缀。
约定：z[0]=|s|；askLCP(p, s)[i] 是 p 与 s[i..] 的 LCP 长度。
性能：默认使用宏展开后的 int；若大规模数据因此超时或超空间，可将纯下标、状态或计数局部改为 i32。
*/
struct ZFunction {

    template <class S> static vector<int> build(const S& s) {
        int n = s.size();
        vector<int> z(n);
        if (!n) return z;
        z[0] = n;
        for (int l = 0, r = 0, i = 1; i < n; ++i) {
            if (i < r) z[i] = min<int>(r - i, z[i - l]);
            while (i + z[i] < n && s[z[i]] == s[i + z[i]]) ++z[i];
            if (i + z[i] > r) l = i, r = i + z[i];
        }
        return z;
    }

    template <class P, class S> static vector<int> askLCP(const P& p, const S& s) {
        int n = s.size(), m = p.size();
        vector<int> a(n);
        if (!m) return a;
        auto z = build(p);
        for (int l = 0, r = 0, i = 0; i < n; ++i) {
            if (i < r) a[i] = min<int>(r - i, z[i - l]);
            while (a[i] < m && i + a[i] < n && p[a[i]] == s[i + a[i]]) ++a[i];
            if (i + a[i] > r) l = i, r = i + a[i];
        }
        return a;
    }

    template <class P, class S> static vector<int> ask(const P& p, const S& s) {
        int n = s.size(), m = p.size();
        if (!m) {
            vector<int> a(n + 1);
            iota(a.begin(), a.end(), 0);
            return a;
        }
        auto z = askLCP(p, s);
        vector<int> a;
        for (int i = 0; i < n; ++i)
            if (z[i] == m) a.push_back(i);
        return a;
    }
};

```


02A - Manacher.cpp

108 lines · C++ · 07 - 字符串

```

/*
用途：回文半径、区间回文判定与最长回文。
模板开关：Manacher<true, false> 只求偶回文；Manacher<false, true> 只求奇回文。
约定：半径数组使用 int。
性能：默认使用宏展开后的 int；若大规模数据因此超时或超空间，可将纯下标、状态或计数局部改为 i32。
*/
template <bool E = true, bool O = true> struct Manacher {

    int n = 0;
    vector<int> o, e;

    Manacher() = default;

    template <class A> Manacher(int n, A a) {
        init(n, a);
    }

    template <class It> Manacher(It l, It r) {
        init(r - l, [&](int i) { return *(l + i); });
    }

    template <class S> explicit Manacher(const S& s) : Manacher(s.begin(), s.end()) {}

    template <class A> void init(int m, A a) {
        assert(m >= 0);
        n = m;
        if constexpr (O) {
            o.assign(n, 0);
            for (int l = 0, r = -1, i = 0; i < n; ++i) {
                int k = i <= r ? min(o[l + r - i], r - i) + 1 : 1;
                while (k <= i && k < n - i && a(i - k) == a(i + k)) ++k;
                o[i] = k - 1;
                if (i + k - 1 > r) l = i - k + 1, r = i + k - 1;
            }
        } else {
            o.clear();
        }
        if constexpr (E) {
            e.assign(n, 0);
            for (int l = 0, r = -1, i = 0; i < n; ++i) {
                int k = i <= r ? min(e[l + r - i + 1], r - i + 1) + 1 : 1;
                while (k <= i && k < n - i + 1 && a(i - k) == a(i + k - 1)) ++k;
                e[i] = k - 1;
                if (i + k - 2 > r) l = i - k + 1, r = i + k - 2;
            }
        } else {
            e.clear();
        }
    }

    template <bool Odd> int askRad(int x) const {
        assert(0 <= x && x < n);
        if constexpr (Odd) {
            static_assert(0, "odd radii are disabled");
            return o[x];
        } else {
            static_assert(E, "even radii are disabled");
            return e[x];
        }
    }
}

```

```

template <bool Odd> bool askPal(int l, int r) const {
    assert(0 <= l && l <= r && r < n);
    if constexpr (Odd) {
        static_assert(0, "odd radii are disabled");
        assert(!((l ^ r) & 1));
        return o[(l + r) >> 1] >= (r - l) / 2;
    } else {
        static_assert(E, "even radii are disabled");
        assert((l ^ r) & 1);
        return e[(l + r + 1) >> 1] > (r - l) / 2;
    }
}

bool ask(int l, int r) const {
    assert(0 <= l && l <= r && r < n);
    if ((l ^ r) & 1) {
        if constexpr (E) return askPal<false>(l, r);
        assert(false);
        return false;
    }
    if constexpr (O) return askPal<true>(l, r);
    assert(false);
    return false;
}

array<int, 2> askMax() const {
    array<int, 2> a{0, 0};
    if constexpr (O) {
        for (int i = 0; i < n; ++i) {
            array<int, 2> b{i - o[i], i + o[i] + 1};
            if (b[1] - b[0] > a[1] - a[0]) a = b;
        }
    }
    if constexpr (E) {
        for (int i = 0; i < n; ++i) {
            array<int, 2> b{i - e[i], i + e[i]};
            if (b[1] - b[0] > a[1] - a[0]) a = b;
        }
    }
    return a;
}

int askLen() const {
    auto a = askMax();
    return a[1] - a[0];
}
};

```

02B - 回文自动机 (PalindromicTree) .cpp

106 lines · C++ · 07 - 字符串

```

/*
用途：在线维护所有本质不同回文串、每个回文串的出现次数。
用法：append 后 askUni() 是本质不同回文数；askCnt()[state] 是该状态回文的总出现次数。
约定：字符集为连续 [F,F+A)，节点和出现次数使用 int。
性能：默认使用宏展开后的 int；若大规模数据因此超时或超空间，可将纯下标、状态或计数局部改为 i32。
*/
template <int A = 26, char F = 'a'> struct PAM {

    struct Node {
        int len = 0, lk = 0, cnt = 0;
        array<int, A> to{};
    };

    vector<Node> t;
    vector<int> s;
    int lst = 1, mx = 0;

    PAM() {
        init();
    }

    void init() {
        t.assign(2, Node());
        t[0].len = -1;
        s.assign(1, -1);
        lst = 1;
        mx = 0;
    }

    int id(char c) const {
        int x = c - F;
        assert(0 <= x && x < A);
        return x;
    }

    int get(int p, int x) const {
        int n = s.size() - 1;
        while (s[n - 1 - t[p].len] != x) p = t[p].lk;
        return p;
    }

    int append(int x) {
        assert(0 <= x && x < A);
        s.push_back(x);
        int p = get(lst, x);
        if (t[p].to[x]) {
            lst = t[p].to[x];
            ++t[lst].cnt;
            mx = max(mx, t[lst].len);
            return lst;
        }
        int q = t.size();
        t.emplace_back();
        t[q].len = t[p].len + 2;
        t[p].to[x] = q;
        if (t[q].len == 1) {
            t[q].lk = 1;
        } else {
            p = get(t[p].lk, x);
            t[q].lk = t[p].to[x];
        }
    }
};

```

```
    }
    lst = q;
    ++t[lst].cnt;
    mx = max(mx, t[lst].len);
    return lst;
}

int append(char c) {
    return append(id(c));
}

void append(const string& a) {
    for (char c : a) append(c);
}

int askSz() const {
    return t.size();
}

int askUni() const {
    return askSz() - 2;
}

int askMax() const {
    return mx;
}

vector<int> ask0rd() const {
    vector<int> c(s.size()), a(askSz() - 1);
    for (int i = 1; i < askSz(); ++i) ++c[t[i].len];
    for (int i = 1; i < c.size(); ++i) c[i] += c[i - 1];
    for (int i = 1; i < askSz(); ++i) a[--c[t[i].len]] = i;
    return a;
}

vector<int> askCnt() const {
    vector<int> a(askSz());
    for (int i = 1; i < askSz(); ++i) a[i] = t[i].cnt;
    auto o = ask0rd();
    for (int i = o.size(); --i > 0;) {
        int p = o[i];
        a[t[p].lk] += a[p];
    }
    return a;
}
};
```

03 - AC自动机.cpp

119 lines · C++ · 07 - 字符串

```

/*
用途：多模式串匹配自动机。
用法：add 返回模式串终点；build 后 askCnt(text)[end] 是该模式串出现次数。
约定：节点编号、转移表和 BFS 队列使用 int；字符集可通过 A、F 适配。
性能：默认使用宏展开后的 int；若大规模数据因此超时或超空间，可将纯下标、状态或计数局部改为 i32。
*/
template <int A = 26, char F = 'a'> struct ACAM {

    struct Node {
        int len = 0, lk = 0;
        array<int, A> to{};
    };

    vector<Node> t;
    vector<int> ord;
    bool ok = false;

    ACAM() {
        init();
    }

    void init() {
        t.assign(2, Node());
        t[0].to.fill(1);
        t[0].len = -1;
        ord.clear();
        ok = false;
    }

    int node() {
        t.emplace_back();
        return t.size() - 1;
    }

    int id(char c) const {
        int x = c - F;
        assert(0 <= x && x < A);
        return x;
    }

    int add(const string& s) {
        assert(!ok);
        assert(!s.empty());
        int p = 1;
        for (char c : s) {
            int x = id(c);
            if (!t[p].to[x]) {
                t[p].to[x] = node();
                t[t[p].to[x]].len = t[p].len + 1;
            }
            p = t[p].to[x];
        }
        return p;
    }

    void build() {
        if (ok) return;
        queue<int> q;
        q.push(1);
        ord.clear();
    }
};

```

```

    ord.reserve(t.size() - 1);
    ord.push_back(1);
    while (!q.empty()) {
        int x = q.front();
        q.pop();
        for (int i = 0; i < A; ++i) {
            if (!t[x].to[i]) {
                t[x].to[i] = t[t[x].lk].to[i];
            } else {
                t[t[x].to[i]].lk = t[t[x].lk].to[i];
                q.push(t[x].to[i]);
                ord.push_back(t[x].to[i]);
            }
        }
    }
    ok = true;
}

int askNxt(int p, int x) const {
    assert(0 <= p && p < askSz() && 0 <= x && x < A);
    return t[p].to[x];
}

int askNxt(int p, char c) const {
    return askNxt(p, id(c));
}

int askLink(int p) const {
    return t[p].lk;
}

int askLen(int p) const {
    return t[p].len;
}

int askSz() const {
    return t.size();
}

const vector<int>& ask0rd() const {
    assert(ok);
    return ord;
}

vector<int> askCnt(const string& s) const {
    assert(ok);
    vector<int> a(askSz());
    int p = 1;
    for (char c : s) {
        p = askNxt(p, c);
        ++a[p];
    }
    for (int i = ord.size(); --i > 0;) {
        int u = ord[i];
        a[t[u].lk] += a[u];
    }
    return a;
}
};

```

04A - 后缀自动机.cpp

151 lines · C++ · 07 - 字符串

```

/*
用途：后缀自动机，append 后可求不同子串数及各状态出现次数。
约定：状态编号、转移和出现次数使用 int。
性能：默认使用宏展开后的 int；若大规模数据因此超时或超空间，可将纯下标、状态或计数局部改为 i32。
*/
template <int A = 26, char F = 'a'> struct SAM {

    struct Node {
        int len = 0, lk = 0, cnt = 0;
        array<int, A> to{};
    };

    vector<Node> t;
    int lst = 1;

    SAM() {
        init();
    }

    void init() {
        t.assign(2, Node());
        t[0].to.fill(1);
        t[0].len = -1;
        lst = 1;
    }

    int node() {
        t.emplace_back();
        return t.size() - 1;
    }

    int extend(int p, int c) {
        assert(0 <= p && p < askSz() && 0 <= c && c < A);
        if (t[p].to[c]) {
            int q = t[p].to[c];
            if (t[q].len == t[p].len + 1) return q;
            int r = node();
            t[r].len = t[p].len + 1;
            t[r].lk = t[q].lk;
            t[r].to = t[q].to;
            t[q].lk = r;
            while (t[p].to[c] == q) {
                t[p].to[c] = r;
                p = t[p].lk;
            }
            return r;
        }
        int q = node();
        t[q].len = t[p].len + 1;
        while (!t[p].to[c]) {
            t[p].to[c] = q;
            p = t[p].lk;
        }
        t[q].lk = extend(p, c);
        return q;
    }

    int id(char c) const {
        int x = c - F;
        assert(0 <= x && x < A);
    }
};

```

```
        return x;
    }

    int extend(int p, char c) {
        return extend(p, id(c));
    }

    int append(char c) {
        lst = extend(lst, c);
        ++t[lst].cnt;
        return lst;
    }

    void append(const string& s) {
        for (char c : s) append(c);
    }

    int askNxt(int p, int x) const {
        return t[p].to[x];
    }

    int askNxt(int p, char c) const {
        return askNxt(p, id(c));
    }

    int askLink(int p) const {
        return t[p].lk;
    }

    int askLen(int p) const {
        return t[p].len;
    }

    int askSz() const {
        return t.size();
    }

    bool ask(const string& s) const {
        int p = 1;
        for (char c : s) {
            p = askNxt(p, c);
            if (!p) return false;
        }
        return true;
    }

    int askLCS(const string& s) const {
        int p = 1, w = 0, r = 0;
        for (char c : s) {
            int x = id(c);
            while (p != 1 && !t[p].to[x]) {
                p = t[p].lk;
                w = t[p].len;
            }
            if (t[p].to[x]) {
                p = t[p].to[x];
                ++w;
            } else {
                p = 1;
                w = 0;
            }
            r = max(r, w);
        }
        return r;
    }
```



```
    }

    vector<int> ask0rd() const {
        vector<int> c(askSz()), a(askSz() - 1);
        for (int i = 1; i < askSz(); ++i) ++c[t[i].len];
        for (int i = 1; i < askSz(); ++i) c[i] += c[i - 1];
        for (int i = 1; i < askSz(); ++i) a[--c[t[i].len]] = i;
        return a;
    }

    vector<int> askCnt() const {
        vector<int> a(askSz());
        for (int i = 1; i < askSz(); ++i) a[i] = t[i].cnt;
        auto o = ask0rd();
        for (int i = o.size(); --i > 0;) {
            int p = o[i];
            a[t[p].lk] += a[p];
        }
        return a;
    }

    int askUni() const {
        int r = 0;
        for (int i = 2; i < askSz(); ++i) r += t[i].len - t[t[i].lk].len;
        return r;
    }
};
```

04B - 后缀数组 (SuffixArray) .cpp

59 lines · C++ · 07 - 字符串

```

/*
用途：倍增后缀数组与 height (lc) 数组，支持 string 或可比较整数序列。
约定：rank[i] 为后缀 i 的名次；lc[r] 为 sa[r]、sa[r+1] 的 LCP。
性能：默认使用宏展开后的 int；若大规模数据因此超时或超空间，可将纯下标、状态或计数局部改为 i32。
*/
struct SA {

    int n = 0;
    vector<int> sa, rk, lc;

    SA() = default;

    template <class S> explicit SA(const S& s) {
        build(s);
    }

    template <class S> void build(const S& s) {
        n = s.size();
        sa.resize(n);
        rk.resize(n);
        lc.assign(n ? n - 1 : 0, 0);
        if (!n) return;
        iota(sa.begin(), sa.end(), 0);
        sort(sa.begin(), sa.end(), [&](int a, int b) { return s[a] < s[b]; });
        rk[sa[0]] = 0;
        for (int i = 1; i < n; ++i) rk[sa[i]] = rk[sa[i - 1]] + (s[sa[i]] != s[sa[i - 1]]);

        vector<int> a, b, c(n);
        a.reserve(n);
        for (int w = 1; rk[sa[n - 1]] < n - 1; w <= 1) {
            a.clear();
            for (int i = max<int>(0, n - w); i < n; ++i) a.push_back(i);
            for (int i : sa)
                if (i >= w) a.push_back(i - w);
            fill(c.begin(), c.end(), 0);
            for (int i = 0; i < n; ++i) ++c[rk[i]];
            for (int i = 1; i < n; ++i) c[i] += c[i - 1];
            for (int i = n - 1; i >= 0; --i) sa[--c[rk[sa[i]]]] = a[i];
            b = rk;
            rk[sa[0]] = 0;
            auto get = [&](int p) { return p < n ? b[p] : -1; };
            for (int i = 1; i < n; ++i) {
                int x = sa[i - 1], y = sa[i];
                rk[y] = rk[x] + (b[x] != b[y] || get(x + w) != get(y + w));
            }
        }

        for (int i = 0, j = 0; i < n; ++i) {
            if (!rk[i]) {
                j = 0;
                continue;
            }
            int p = sa[rk[i] - 1];
            while (i + j < n && p + j < n && s[i + j] == s[p + j]) ++j;
            lc[rk[i] - 1] = j;
            if (j) --j;
        }
    }
};

```

04C - 后缀数组 (SA-IS, 线性) .cpp

122 lines · C++ · 07 - 字符串

```

/*
用途：SA-IS 后缀数组与 Kasai LCP；字符集已压缩到  $[0, up]$  时为  $O(n)$ 。
用法：build(string) 直接处理字节串；整数序列先压缩后调用 build(a, up)。
约定：返回 sa，所有下标为 int；通用 build 会排序压缩，复杂度为  $O(n \log n)$ 。
性能：默认使用宏展开后的 int；若大规模数据因此超时或超空间，可将纯下标、状态或计数局部改为 i32。
*/
struct SA {

    static vector<int> sais(const vector<int>& s, int up) {
        int n = s.size();
        assert(up >= 0);
        for (int x : s) assert(0 <= x && x <= up);
        if (!n) return {};
        if (n == 1) return {0};
        if (n == 2) return s[0] < s[1] ? vector<int>{0, 1} : vector<int>{1, 0};

        vector<uint8_t> tp(n);
        for (int i = n - 2; i >= 0; --i) {
            tp[i] = s[i] == s[i + 1] ? tp[i + 1] : s[i] < s[i + 1];
        }
        vector<int> sl(up + 2), ss(up + 2), sa(n);
        for (int i = 0; i < n; ++i) {
            if (tp[i])
                ++sl[s[i] + 1];
            else
                ++ss[s[i]];
        }
        for (int i = 0; i <= up; ++i) {
            ss[i] += sl[i];
            sl[i + 1] += ss[i];
        }

        auto ind = [&](const vector<int>& lms) {
            fill(sa.begin(), sa.end(), -1);
            vector<int> b = ss;
            for (int x : lms)
                if (x < n) sa[b[s[x]]++] = x;
            b = sl;
            sa[b[s[n - 1]]++] = n - 1;
            for (int i = 0; i < n; ++i) {
                int x = sa[i];
                if (x > 0 && !tp[x - 1]) sa[b[s[x - 1]]++] = x - 1;
            }
            b = sl;
            for (int i = n - 1; i >= 0; --i) {
                int x = sa[i];
                if (x > 0 && tp[x - 1]) sa[--b[s[x - 1] + 1]] = x - 1;
            }
        };

        vector<int> id(n + 1, -1), lms;
        int m = 0;
        for (int i = 1; i < n; ++i)
            if (!tp[i - 1] && tp[i]) id[i] = m++, lms.push_back(i);
        ind(lms);
        if (!m) return sa;

        vector<int> ord;
        ord.reserve(m);
        for (int x : sa)

```

```

        if (id[x] != -1) ord.push_back(x);
    vector<int> rec(m);
    int rup = 0;
    rec[id[ord[0]]] = 0;
    for (int i = 1; i < m; ++i) {
        int x = ord[i - 1], y = ord[i];
        int rx = id[x] + 1 < m ? lms[id[x] + 1] : n;
        int ry = id[y] + 1 < m ? lms[id[y] + 1] : n;
        bool sam = rx - x == ry - y;
        for (; sam && x < rx; ++x, ++y)
            if (s[x] != s[y]) break;
        if (x == n || s[x] != s[y]) sam = false;
        if (!sam) ++rup;
        rec[id[ord[i]]] = rup;
    }
    auto rsa = sais(rec, rup);
    for (int i = 0; i < m; ++i) ord[i] = lms[rsa[i]];
    ind(ord);
    return sa;
}

static vector<int> build(const string& s) {
    vector<int> a(s.size());
    for (int i = 0; i < s.size(); ++i) a[i] = static_cast<unsigned char>(s[i]);
    return sais(a, 255);
}

static vector<int> build(const vector<int>& s, int up) {
    return sais(s, up);
}

template <class S> static vector<int> build(const S& s) {
    int n = s.size();
    vector<int> a(n), p(n);
    iota(p.begin(), p.end(), 0);
    sort(p.begin(), p.end(), [&](int x, int y) { return s[x] < s[y]; });
    int up = 0;
    for (int i = 0; i < n; ++i) {
        if (i && s[p[i - 1]] != s[p[i]]) ++up;
        a[p[i]] = up;
    }
    return sais(a, up);
}

template <class S> static vector<int> askLCP(const S& s, const vector<int>& sa) {
    int n = s.size();
    assert(sa.size() == n);
    vector<int> rk(n), a(n ? n - 1 : 0);
    for (int i = 0; i < n; ++i) {
        assert(0 <= sa[i] && sa[i] < n);
        rk[sa[i]] = i;
    }
    for (int i = 0, h = 0; i < n; ++i) {
        if (!rk[i]) continue;
        int j = sa[rk[i] - 1];
        while (i + h < n && j + h < n && s[i + h] == s[j + h]) ++h;
        a[rk[i] - 1] = h;
        if (h) --h;
    }
    return a;
}
};

```

05 - 字符串哈希 (StringHash, with T) .cpp

66 lines · C++ · 07 - 字符串

```

/*
用途：静态子串哈希、子串比较与 LCP。
用法：StringHash<u64> h(s, base) 使用自然溢出；StringHash<Z> 使用自动取模类型。
约定：区间为 [l, r)，T 必须支持 +、-、* 和从整数构造；哈希存在碰撞概率。
性能：默认使用宏展开后的 int；若大规模数据因此超时或超空间，可将纯下标、状态或计数局部改为 i32。
*/

using u64 = uint64_t;

template <class T> struct StringHash {
    int n = 0;
    T b{};
    vector<T> h, pw;

    StringHash() = default;

    template <class S> StringHash(const S& s, T x) {
        build(s, x);
    }

    template <class S, class F> StringHash(const S& s, T x, F f) {
        build(s, x, f);
    }

    template <class S> void build(const S& s, T x) {
        build(s, x, [](const auto& c) {
            return static_cast<u64>(static_cast<unsigned char>(c)) + 1;
        });
    }

    template <class S, class F> void build(const S& s, T x, F f) {
        n = s.size();
        b = x;
        h.assign(n + 1, T(0));
        pw.assign(n + 1, T(0));
        pw[0] = T(1);
        for (int i = 0; i < n; ++i) {
            pw[i + 1] = pw[i] * b;
            h[i + 1] = h[i] * b + T(f(s[i]));
        }
    }

    T ask(int l, int r) const {
        assert(0 <= l && l <= r && r <= n);
        return h[r] - h[l] * pw[r - l];
    }

    bool askEq(int l, int r, int x, int y) const {
        assert(r - l == y - x);
        return ask(l, r) == ask(x, y);
    }

    int askLCP(int l, int r) const {
        assert(0 <= l && l <= n && 0 <= r && r <= n);
        int x = 0, y = min(n - l, n - r) + 1;
        while (x + 1 < y) {
            int m = x + (y - x) / 2;
            if (askEq(l, l + m, r, r + m))
                x = m;
        }
    }
};

```

```
        else
            y = m;
    }
    return x;
}
};
```

06 - 最小表示法 (Booth) .cpp

25 lines · C++ · 07 - 字符串

```
/*
用途：循环串的最小或最大字典序表示起点。
用法：booth(s) 求最小表示法；booth<false>(s) 求最大表示法。
约定：返回 0-indexed 起点；空串返回 0；元素类型只需可比较。
性能：默认使用宏展开后的 int；若大规模数据因此超时或超空间，可将纯下标、状态或计数局部改为 i32。
*/
template <bool Min = true, class S> int booth(const S& s) {
    int n = s.size();
    if (!n) return 0;
    int i = 0, j = 1, k = 0;
    while (i < n && j < n && k < n) {
        auto x = s[(i + k) % n], y = s[(j + k) % n];
        if (x == y) {
            ++k;
            continue;
        }
        if ((x > y) == Min)
            i += k + 1;
        else
            j += k + 1;
        if (i == j) ++j;
        k = 0;
    }
    return min(i, j);
}
```

07 - Lyndon 分解 (Duval) .cpp

25 lines · C++ · 07 - 字符串

```
/*
用途：Lyndon 分解，返回每个 Lyndon 因子的半开区间。
约定：返回的因子字典序单调不减；元素类型只需可比较。
性能：默认使用宏展开后的 int；若大规模数据因此超时或超空间，可将纯下标、状态或计数局部改为 i32。
*/
template <class S> vector<array<int, 2>> duval(const S& s) {
    int n = s.size();
    vector<array<int, 2>> a;
    for (int i = 0; i < n; i++) {
        int j = i + 1, k = i;
        while (j < n && s[k] <= s[j]) {
            if (s[k] < s[j])
                k = i;
            else
                ++k;
            ++j;
        }
        int w = j - k;
        while (i <= k) {
            a.push_back({i, i + w});
            i += w;
        }
    }
    return a;
}
```


08A - 子序列自动机（位置表）.cpp

42 lines · C++ · 07 - 字符串

```

/*
用途：判定查询序列是否为固定主序列的子序列；支持任意可排序元素类型。
预处理  $O(n \log \sigma)$ ，单次查询  $O(m(\log \sigma + \log n))$ ，空间  $O(n)$ 。
askPos 返回匹配最后一个元素的 0-indexed 位置；空序列返回 -1。
性能：默认使用宏展开后的 int；若大规模数据因此超时或超空间，可将纯下标、状态或计数局部改为 i32。
*/
template <class T, class Cmp = less<T>> struct SubSeq {
    Cmp cmp;
    vector<T> val;
    vector<vector<int>> pos;

    SubSeq() = default;
    template <class Seq> explicit SubSeq(const Seq& a) { build(a); }

    template <class Seq> void build(const Seq& a) {
        val.assign(a.begin(), a.end());
        sort(val.begin(), val.end(), cmp);
        auto eq = [&](const T& x, const T& y) { return !cmp(x, y) && !cmp(y, x); };
        val.erase(unique(val.begin(), val.end(), eq), val.end());
        pos.assign(val.size(), {});
        int i = 0;
        for (const auto& x : a) {
            int k = lower_bound(val.begin(), val.end(), x, cmp) - val.begin();
            pos[k].push_back(i++);
        }
    }

    template <class Seq> optional<int> askPos(const Seq& a) const {
        int at = -1;
        for (const auto& x : a) {
            auto p = lower_bound(val.begin(), val.end(), x, cmp);
            if (p == val.end() || cmp(x, *p) || cmp(*p, x)) return nullopt;
            int k = p - val.begin();
            auto it = upper_bound(pos[k].begin(), pos[k].end(), at);
            if (it == pos[k].end()) return nullopt;
            at = *it;
        }
        return at;
    }

    template <class Seq> bool ask(const Seq& a) const { return askPos(a).has_value(); }
};

```

08B - 子序列自动机（稠密转移）.cpp

38 lines · C++ · 07 - 字符串

```

/*
用途：小且连续字符集上的子序列判定；以  $O(nA)$  空间换  $O(m)$  查询。
默认处理 'a'..'z'。状态和转移显式使用 int。
性能：默认使用宏展开后的 int；若大规模数据因此超时或超空间，可将纯下标、状态或计数局部改为 i32。
*/
template <int A = 26, char F = 'a'> struct SubSeq {
    int n = 0;
    vector<array<int, A>> nx;

    SubSeq() = default;
    explicit SubSeq(const string& s) { build(s); }

    void build(const string& s) {
        n = s.size();
        nx.assign(n + 1, {});
        nx[n].fill(n);
        for (int i = n; i-- > 0;) {
            nx[i] = nx[i + 1];
            int c = static_cast<unsigned char>(s[i]) - static_cast<unsigned char>(F);
            assert(0 <= c && c < A);
            nx[i][c] = i;
        }
    }

    optional<int> askPos(const string& s) const {
        int p = 0, at = -1;
        for (unsigned char x : s) {
            int c = x - static_cast<unsigned char>(F);
            if (c < 0 || c >= A || p > n) return nullopt;
            int q = nx[p][c];
            if (q == n) return nullopt;
            at = q, p = q + 1;
        }
        return at;
    }

    bool ask(const string& s) const { return askPos(s).has_value(); }
};

```

01A - 整数 (BigInt, 常规) .cpp

387 lines · C++ · 08 - 高精度

```

/*
用途：有符号大整数，常规赛时版。
特点：10^9 进制、O(n^2) 乘法、规范化长除；适合千至数千位且代码体积可控的场景。
替代：万位级乘法或更大输入使用 01B 的 FFT 版，二者均定义 BigInt，不能同时粘贴。
性能：默认使用宏展开后的 int；若大规模数据因此超时或超空间，可将纯下标、状态或计数局部改为 i32。
*/

using u64 = uint64_t;
using i128 = __int128_t;

struct BigInt {

    static constexpr int B = 1000000000u;
    static constexpr int W = 9;

    vector<int> d;
    int s = 0;

    BigInt() = default;
    BigInt(int x) {
        *this = x;
    }
    BigInt(const string& a) {
        read(a);
    }

    static u64 mag(int x) {
        return x < 0 ? u64(-(x + 1)) + 1 : u64(x);
    }

    BigInt& operator=(int x) {
        d.clear();
        if (!x) {
            s = 0;
            return *this;
        }
        s = x < 0 ? -1 : 1;
        for (u64 y = mag(x); y; y /= B) d.push_back(y % B);
        return *this;
    }

    BigInt& operator=(const string& a){
        read(a);
        return *this;
    }

    bool ask0() const {
        return !s;
    }

    void norm() {
        while (!d.empty() && !d.back()) d.pop_back();
        if (d.empty()) s = 0;
    }

    void read(const string& a) {
        d.clear();
        int n = a.size(), p = 0;
        while (p < n && isspace(static_cast<unsigned char>(a[p]))) ++p;
        s = 1;

```

```

    if (p < n && (a[p] == '+' || a[p] == '-')) {
        if (a[p] == '-') s = -1;
        ++p;
    }
    while (p < n && a[p] == '0') ++p;
    if (p == n) {
        s = 0;
        return;
    }
    for (int r = n; r > p; r -= W) {
        int l = max<int>(p, r - W);
        int x = 0;
        for (int i = l; i < r; ++i) {
            assert('0' <= a[i] && a[i] <= '9');
            x = x * 10 + a[i] - '0';
        }
        d.push_back(x);
    }
    norm();
}

string str() const {
    if (ask0()) return "0";
    string a = s < 0 ? "-" : "";
    a += std::to_string(d.back());
    string b;
    for (int i = d.size() - 1; i--;) {
        b = to_string(d[i]);
        a += string(W - b.size(), '0') + b;
    }
    return a;
}

static int cmpAbs(const BigInt& a, const BigInt& b) {
    if (a.d.size() != b.d.size()) return a.d.size() < b.d.size() ? -1 : 1;
    for (int i = a.d.size(); i--;)
        if (a.d[i] != b.d[i]) return a.d[i] < b.d[i] ? -1 : 1;
    return 0;
}

static int cmp(const BigInt& a, const BigInt& b) {
    if (a.s != b.s) return a.s < b.s ? -1 : 1;
    if (!a.s) return 0;
    int x = cmpAbs(a, b);
    return a.s > 0 ? x : -x;
}

static BigInt addAbs(const BigInt& a, const BigInt& b) {
    BigInt c;
    int n = max<int>(a.d.size(), b.d.size());
    c.d.assign(n, 0);
    u64 w = 0;
    for (int i = 0; i < n; ++i) {
        w += i < a.d.size() ? a.d[i] : 0;
        w += i < b.d.size() ? b.d[i] : 0;
        c.d[i] = w % B;
        w /= B;
    }
    if (w) c.d.push_back(w);
    c.s = 1;
    return c;
}

static BigInt subAbs(const BigInt& a, const BigInt& b) {

```

```

    BigInt c;
    c.d.assign(a.d.size(), 0);
    int w = 0;
    for (int i = 0; i < a.d.size(); ++i) {
        int x = a.d[i] - (i < b.d.size() ? b.d[i] : 0) - w;
        if (x < 0) x += B, w = 1;
        else w = 0;
        c.d[i] = x;
    }
    c.s = 1;
    c.norm();
    return c;
}

friend BigInt operator+(const BigInt& a, const BigInt& b) {
    if (!a.s) return b;
    if (!b.s) return a;
    if (a.s == b.s) {
        BigInt c = addAbs(a, b);
        c.s = a.s;
        return c;
    }
    int x = cmpAbs(a, b);
    if (!x) return BigInt(0);
    BigInt c = x > 0 ? subAbs(a, b) : subAbs(b, a);
    c.s = x > 0 ? a.s : b.s;
    return c;
}

friend BigInt operator-(const BigInt& a, const BigInt& b) {
    BigInt c = b;
    c.s = -c.s;
    return a + c;
}

static BigInt mul0(const BigInt& a, const BigInt& b) {
    if (!a.s || !b.s) return BigInt(0);
    BigInt c;
    int n = a.d.size(), m = b.d.size();
    c.d.assign(n + m, 0);
    for (int i = 0; i < n; ++i) {
        u64 w = 0;
        for (int j = 0; j < m || w; ++j) {
            u64 x = c.d[i + j] + w;
            if (j < m) x += u64(a.d[i]) * b.d[j];
            c.d[i + j] = x % B;
            w = x / B;
        }
    }
    c.s = a.s * b.s;
    c.norm();
    return c;
}

static BigInt mulInt(const BigInt& a, int b) {
    if (!a.s || !b) return BigInt(0);
    BigInt c;
    u64 y = mag(b);
    c.d.assign(a.d.size(), 0);
    i128 w = 0;
    for (int i = 0; i < a.d.size() || w; ++i) {
        if (i == c.d.size()) c.d.push_back(0);
        w += i < a.d.size() ? static_cast<i128>(a.d[i]) * y : 0;
        c.d[i] = w % B;
    }
}

```

```

        w /= B;
    }
    c.s = a.s * (b < 0 ? -1 : 1);
    c.norm();
    return c;
}

friend BigInt operator*(const BigInt& a, const BigInt& b) {
    return mul0(a, b);
}

static pair<BigInt, int> divmod(const BigInt& a, int b) {
    if (!b) throw runtime_error("division by zero");
    BigInt c;
    c.d.assign(a.d.size(), 0);
    u64 y = mag(b), w = 0;
    for (int i = a.d.size(); i--;) {
        i128 x = static_cast<i128>(w) * B + a.d[i];
        c.d[i] = x / y;
        w = static_cast<u64>(x % y);
    }
    c.s = a.s * (b < 0 ? -1 : 1);
    c.norm();
    int r = w;
    if (a.s < 0) r = -r;
    return {c, r};
}

static pair<BigInt, BigInt> divmod(const BigInt& a, const BigInt& b) {
    if (!b.s) throw runtime_error("division by zero");
    if (!a.s) return {BigInt(0), BigInt(0)};
    BigInt x = a, y = b;
    x.s = y.s = 1;
    if (cmpAbs(x, y) < 0) return {BigInt(0), a};
    if (y.d.size() == 1) {
        auto [q, r] = divmod(a, b.s * y.d[0]);
        return {q, BigInt(r)};
    }
    u64 f = u64(B) / (u64(y.d.back()) + 1);
    x = mulInt(x, f);
    y = mulInt(y, f);
    BigInt q;
    q.d.assign(x.d.size(), 0);
    BigInt r;
    for (int i = x.d.size(); i--;) {
        if (r.d.empty()) r.d.push_back(x.d[i]);
        else r.d.insert(r.d.begin(), x.d[i]);
        r.s = 1;
        r.norm();
        u64 u = r.d.size() <= y.d.size() ? 0 : r.d[y.d.size()];
        u64 v = r.d.size() < y.d.size() ? 0 : r.d[y.d.size() - 1];
        u64 z = min<u64>(B - 1, (u64(B) * u + v) / y.d.back());
        r -= mulInt(y, z);
        while (r.s < 0) {
            r += y;
            --z;
        }
        q.d[i] = z;
    }
    q.s = a.s * b.s;
    q.norm();
    r = divmod(r, f).first;
    r.s = a.s;
    r.norm();
}

```

```

    return {q, r};
}

friend BigInt operator/(const BigInt& a, const BigInt& b) {
    return divmod(a, b).first;
}

friend BigInt operator%(const BigInt& a, const BigInt& b) {
    return divmod(a, b).second;
}

static BigInt divBig(const BigInt& a, const BigInt& b) {
    return divmod(a, b).first;
}

static BigInt mod(const BigInt& a, const BigInt& b) {
    return divmod(a, b).second;
}

BigInt operator/(int x) const {
    return divmod(*this, x).first;
}

BigInt operator%(int x) const {
    return BigInt(divmod(*this, x).second);
}

static BigInt pow(BigInt a, u64 n) {
    BigInt r(1);
    while (n) {
        if (n & 1) r *= a;
        a *= a;
        n >>= 1;
    }
    return r;
}

static BigInt gcd(BigInt a, BigInt b) {
    if (a.s < 0) a.s = 1;
    if (b.s < 0) b.s = 1;
    while (b.s) {
        BigInt r = a % b;
        a = b;
        b = r;
    }
    return a;
}

static BigInt fac(int n) {
    BigInt r(1);
    for (int i = 2; i <= n; ++i) r = mulInt(r, i);
    return r;
}

static BigInt mul1(const BigInt& a, int x) {
    return mulInt(a, x);
}

static BigInt mul10(BigInt a, int k) {
    if (k < 0) throw runtime_error("negative decimal shift");
    if (!a.s || !k) return a;
    int q = k / W, r = k % W;
    int p = 1;
    for (int i = 0; i < r; ++i) p *= 10;

```

```

        if (r) a = mulInt(a, p);
        if (q) a.d.insert(a.d.begin(), q, 0);
        return a;
    }

    static BigInt div10(BigInt a, int k) {
        if (k < 0) throw runtime_error("negative decimal shift");
        if (!a.s || !k) return a;
        int q = k / W, r = k % W;
        int p = 1;
        for (int i = 0; i < r; ++i) p *= 10;
        if (r) a = divmod(a, p).first;
        if (q >= a.d.size()) return BigInt(0);
        if (q) a.d.erase(a.d.begin(), a.d.begin() + q);
        a.norm();
        return a;
    }

    static void useFFT(bool) {}

    BigInt& operator+=(const BigInt& a) {
        return *this = *this + a;
    }

    BigInt& operator-=(const BigInt& a) {
        return *this = *this - a;
    }

    BigInt& operator*=(const BigInt& a) {
        return *this = *this * a;
    }

    BigInt& operator/=(const BigInt& a) {
        return *this = *this / a;
    }

    friend bool operator==(const BigInt& a, const BigInt& b) {
        return cmp(a, b) == 0;
    }

    friend bool operator!=(const BigInt& a, const BigInt& b) {
        return cmp(a, b) != 0;
    }

    friend bool operator<(const BigInt& a, const BigInt& b) {
        return cmp(a, b) < 0;
    }

    friend bool operator<=(const BigInt& a, const BigInt& b) {
        return cmp(a, b) <= 0;
    }

    friend bool operator>(const BigInt& a, const BigInt& b) {
        return cmp(a, b) > 0;
    }

    friend bool operator>=(const BigInt& a, const BigInt& b) {
        return cmp(a, b) >= 0;
    }

    friend ostream& operator<<(ostream& os, const BigInt& a) {
        return os << a.str();
    }
}

```



```
    friend istream& operator>>(istream& is, BigInt& a) {  
        string s;  
        is >> s;  
        a.read(s);  
        return is;  
    }  
};
```

01B - 整数 (BigInt, 极致优化) .cpp

568 lines · C++ · 08 - 高精度

```

/*
用途：有符号大整数，FFT 自动切换的极致优化版。
特点：10^9 进制、长除、超大乘法自动使用 FFT；与 01A 同名 BigInt，二者不能同时粘贴。
性能：十进制块和 FFT 拆分数组保留 i32/u32，并把两组实系数打包进一次正变换；
      改成宏展开后的 int 或拆成两次正变换都会显著增加内存与常数。
*/

using i32 = int32_t;
using u32 = uint32_t;
using u64 = uint64_t;
using i128 = __int128_t;

struct BigInt {
public:
    static constexpr u32 B = 1000000000u;
    static constexpr i32 D = 9;

private:
    static constexpr double PI = 3.141592653589793238462643383279502884;

    template <typename T>
    class Complex {
    public:
        T a, b;

        constexpr Complex(T r = 0, T i = 0) : a(r), b(i) {}

        constexpr Complex operator+(const Complex &rhs) const {
            return {a + rhs.a, b + rhs.b};
        }

        constexpr Complex operator-(const Complex &rhs) const {
            return {a - rhs.a, b - rhs.b};
        }

        constexpr Complex operator*(const Complex &rhs) const {
            return {
                a * rhs.a - b * rhs.b,
                a * rhs.b + b * rhs.a};
        }

        Complex &operator/=(T v) {
            a /= v;
            b /= v;
            return *this;
        }
    };

    using cd = Complex<double>;

    static void fft(vector<cd> &a, bool inv) {
        i32 n = a.size();
        static vector<i32> rev;
        static vector<cd> rts{{0, 0}, {1, 0}};

        i32 rn = rev.size();
        if (rn != n) {
            i32 k = __builtin_ctz(u32(n));
            rev.assign(n, 0);
            for (i32 i = 0; i < n; i++)
                rev[i] = (rev[i >> 1] >> 1) | ((i & 1) << (k - 1));

```

```

    }

    i32 sn = rts.size();
    if (sn < n) {
        i32 k = __builtin_ctz(u32(sn));
        rts.resize(n);
        while ((i32(1) << k) < n) {
            double da = 2 * PI / (i32(1) << (k + 1));
            for (i32 i = i32(1) << (k - 1); i < (i32(1) << k); i++) {
                rts[i << 1] = rts[i];
                double ag = da * (2 * i + 1 - (i32(1) << k));
                rts[i << 1 | 1] = cd(cos(ag), sin(ag));
            }
            k++;
        }
    }

    for (i32 i = 0; i < n; i++)
        if (i < rev[i])
            swap(a[i], a[rev[i]]);

    for (i32 len = 1; len < n; len <= 1) {
        for (i32 i = 0; i < n; i += len << 1) {
            for (i32 j = 0; j < len; j++) {
                cd u = a[i + j];
                cd v = a[i + j + len] * rts[len + j];
                a[i + j] = u + v;
                a[i + j + len] = u - v;
            }
        }
    }

    if (inv) {
        reverse(a.begin() + 1, a.end());
        for (cd &x : a)
            x /= n;
    }
}

static BigInt mulFFT(const BigInt &a, const BigInt &b) {
    if (a.ask0() || b.ask0()) return BigInt(0);
    constexpr i32 FB = 1000;
    constexpr i32 FD = 3;
    vector<i32> fa, fb;
    i32 na = a.d.size(), nb = b.d.size();
    fa.reserve(na * 3);
    fb.reserve(nb * 3);
    for (i32 i = 0; i < na; ++i) {
        u32 x = a.d[i];
        for (i32 k = 0; k < D / FD; ++k) {
            fa.push_back(x % FB);
            x /= FB;
        }
    }
    for (i32 i = 0; i < nb; ++i) {
        u32 x = b.d[i];
        for (i32 k = 0; k < D / FD; ++k) {
            fb.push_back(x % FB);
            x /= FB;
        }
    }
    while (!fa.empty() && !fa.back()) fa.pop_back();
    while (!fb.empty() && !fb.back()) fb.pop_back();
}

```

```

    i32 n = 1;
    i32 fn = fa.size(), gn = fb.size();
    while (n < fn + gn) n <<= 1;
    vector<cd> x(n), y(n);
    for (i32 i = 0; i < fn; ++i) x[i].a = fa[i];
    for (i32 i = 0; i < gn; ++i) x[i].b = fb[i];
    fft(x, false);
    for (i32 i = 0; i < n; ++i) {
        i32 j = (n - i) & (n - 1);
        cd p{(x[i].a + x[j].a) * 0.5, (x[i].b - x[j].b) * 0.5};
        cd q{(x[i].b + x[j].b) * 0.5, (x[j].a - x[i].a) * 0.5};
        y[i] = p * q;
    }
    fft(y, true);

    vector<i32> c;
    c.reserve(n + 4);
    u64 cy = 0;
    for (i32 i = 0; i < n; ++i) {
        int v = y[i].a + 0.5;
        if (v < 0) v = 0;
        u64 w = cy + static_cast<u64>(v);
        c.push_back(static_cast<i32>(w % FB));
        cy = w / FB;
    }
    while (cy) {
        c.push_back(static_cast<i32>(cy % FB));
        cy /= FB;
    }
    while (!c.empty() && !c.back()) c.pop_back();

    BigInt res;
    i32 cn = c.size();
    res.d.reserve((cn + 2) / 3);
    for (i32 i = 0; i < cn; i += 3) {
        u64 w = 0, p = 1;
        for (i32 j = 0; j < 3 && i + j < cn; ++j) {
            w += static_cast<u64>(c[i + j]) * p;
            p *= FB;
        }
        res.d.push_back(static_cast<u32>(w));
    }
    res.sgn = a.sgn * b.sgn;
    res.norm();
    return res;
}

vector<u32> d;
short sgn;

public:
    BigInt() : sgn(0) {}
    BigInt(int v) { *this = v; }
    BigInt(const string &s) { read(s); }
    static u64 mag(int v) {
        return v < 0 ? u64(-(v + 1)) + 1 : u64(v);
    }
    BigInt &operator=(int v) {
        d.clear();
        if (v == 0) {
            sgn = 0;
            return *this;
        }
        sgn = v < 0 ? -1 : 1;

```

```

        u64 av = mag(v);
        while (av) {
            d.push_back(u32(av % B));
            av /= B;
        }
        return *this;
    }

    BigInt &operator=(const string &s) {
        read(s);
        return *this;
    }

    bool ask0() const { return sgn == 0; }

    void norm() {
        while (!d.empty() && d.back() == 0)
            d.pop_back();
        if (d.empty())
            sgn = 0;
    }

    string str() const {
        if (ask0()) return "0";
        string s = sgn < 0 ? "-" : "";
        s += std::to_string(d.back());
        char buf[32];
        for (i32 i = d.size() - 1; i--;) {
            snprintf(buf, sizeof(buf), "%09u", d[i]);
            s += buf;
        }
        return s;
    }

    void read(const string &s) {
        d.clear();
        i32 n = s.size(), p = 0, e = n;
        while (p < n && isspace(static_cast<unsigned char>(s[p]))) ++p;
        sgn = 1;
        if (p < n && (s[p] == '+' || s[p] == '-')) {
            if (s[p] == '-') sgn = -1;
            ++p;
        }
        while (e > p && isspace(static_cast<unsigned char>(s[e - 1]))) --e;
        while (p < e && s[p] == '0') ++p;
        if (p == e) {
            sgn = 0;
            return;
        }
        for (i32 i = e; i > p; i -= D) {
            u32 x = 0;
            i32 l = max<i32>(p, i - D);
            for (i32 j = l; j < i; ++j) {
                assert('0' <= s[j] && s[j] <= '9');
                x = x * 10 + (s[j] - '0');
            }
            d.push_back(x);
        }

        norm();
    }

public:
    static i32 cmpAbs(const BigInt &a, const BigInt &b) {

```

```

        if (a.d.size() != b.d.size())
            return a.d.size() < b.d.size() ? -1 : 1;
        for (i32 i = a.d.size(); i--;)
            if (a.d[i] != b.d[i])
                return a.d[i] < b.d[i] ? -1 : 1;
        return 0;
    }

    static i32 cmp(const BigInt &a, const BigInt &b) {
        if (a.sgn != b.sgn)
            return a.sgn < b.sgn ? -1 : 1;
        if (a.sgn == 0)
            return 0;
        i32 c = cmpAbs(a, b);
        return a.sgn > 0 ? c : -c;
    }

private:
    static BigInt addAbs(const BigInt &a, const BigInt &b) {
        BigInt res;
        u64 cy = 0;
        i32 na = a.d.size(), nb = b.d.size();
        i32 n = max(na, nb);
        res.d.assign(n, 0);
        for (i32 i = 0; i < n; ++i) {
            u64 av = i < na ? a.d[i] : 0;
            u64 bv = i < nb ? b.d[i] : 0;
            u64 sum = av + bv + cy;
            res.d[i] = u32(sum % B);
            cy = sum / B;
        }
        if (cy)
            res.d.push_back(u32(cy));
        res.sgn = res.d.empty() ? 0 : 1;
        return res;
    }

    static BigInt subAbs(const BigInt &a, const BigInt &b) {
        BigInt res;
        res.d.assign(a.d.size(), 0);
        u64 cy = 0;
        i32 na = a.d.size(), nb = b.d.size();
        for (i32 i = 0; i < na; ++i) {
            u64 av = a.d[i];
            u64 bv = i < nb ? b.d[i] : 0;
            int cur = static_cast<int>(av) - static_cast<int>(bv) - static_cast<int>(cy);
            if (cur < 0) {
                cur += B;
                cy = 1;
            } else
                cy = 0;
            res.d[i] = u32(cur);
        }
        res.norm();
        res.sgn = res.d.empty() ? 0 : 1;
        return res;
    }

public:
    BigInt operator+(const BigInt &v) const {
        if (sgn == 0)
            return v;
        if (v.sgn == 0)
            return *this;
        BigInt res;

```

```

        if (sgn == v.sgn) {
            res = addAbs(*this, v);
            res.sgn = sgn;
        } else {
            i32 cmp = cmpAbs(*this, v);
            if (cmp >= 0) {
                res = subAbs(*this, v);
                res.sgn = sgn;
            } else {
                res = subAbs(v, *this);
                res.sgn = v.sgn;
            }
        }
        if (res.ask0())
            res.sgn = 0;
        return res;
    }

    BigInt operator-(const BigInt &v) const {
        if (v.ask0())
            return *this;
        BigInt tmp = v;
        tmp.sgn = -tmp.sgn;
        return *this + tmp;
    }

    static BigInt mul0(const BigInt &a, const BigInt &b) {
        if (a.ask0() || b.ask0())
            return BigInt(0);
        BigInt res;
        i32 n = a.d.size(), m = b.d.size();
        res.d.assign(n + m + 1, 0);
        for (i32 i = 0; i < n; ++i) {
            u64 cy = 0;
            for (i32 j = 0; j < m || cy; ++j) {
                u64 cur = res.d[i + j] + cy + u64(a.d[i]) * (j < m ? b.d[j] : 0);
                res.d[i + j] = u32(cur % B);
                cy = cur / B;
            }
        }
        res.sgn = a.sgn * b.sgn;
        res.norm();
        return res;
    }

    static BigInt mulInt(const BigInt &a, int m) {
        if (a.ask0() || m == 0)
            return BigInt(0);
        BigInt res;
        short sg2 = m < 0 ? -1 : 1;
        u64 mm = mag(m);
        res.d.assign(a.d.size(), 0);
        u64 cy = 0;
        i32 n = a.d.size();
        for (i32 i = 0; i < n || cy; ++i) {
            i128 cur = cy + static_cast<i128>(i < n ? a.d[i] : 0) * mm;
            if (i == n)
                res.d.push_back(0);
            res.d[i] = static_cast<u32>(cur % B);
            cy = static_cast<u64>(cur / B);
        }
        res.sgn = a.sgn * sg2;
        res.norm();
        return res;
    }

```

```

    }

public:
    static constexpr i32 lim = 1500;
    BigInt operator*(const BigInt &v) const {
        if (ask0() || v.ask0())
            return BigInt(0);
        i32 n = d.size(), m = v.d.size();
        if (min(n, m) == 0)
            return BigInt(0);
        if (n + m < lim)
            return mul0(*this, v);
        else
            return mulFFT(*this, v);
    }

    static pair<BigInt, int> divmod(const BigInt &a, int b) {
        if (b == 0)
            throw runtime_error("div by zero");
        BigInt q;
        q.d.assign(a.d.size(), 0);
        u64 cy = 0, ub = mag(b);
        for (i32 i = a.d.size(); i--;) {
            i128 cur = static_cast<i128>(cy) * B + a.d[i];
            q.d[i] = static_cast<u32>(cur / ub);
            cy = static_cast<u64>(cur % ub);
        }
        q.sgn = (q.d.empty() ? 0 : (a.sgn * (b < 0 ? -1 : 1)));
        q.norm();
        int rem = cy;
        if (a.sgn < 0)
            rem = -rem;
        return {q, rem};
    }

    static pair<BigInt, BigInt> divmod(const BigInt &a, const BigInt &b) {
        if (b.ask0())
            throw runtime_error("div by zero");
        if (a.ask0())
            return {BigInt(0), BigInt(0)};
        i32 sgn = a.sgn * b.sgn;
        BigInt aa = a;
        aa.sgn = 1;
        BigInt bb = b;
        bb.sgn = 1;
        if (cmpAbs(aa, bb) < 0)
            return {BigInt(0), a};

        if (bb.d.size() == 1) {
            auto [q, r] = divmod(a, static_cast<int>(b.sgn) * bb.d[0]);
            return {q, BigInt(r)};
        }
        u64 f = u64(B) / (u64(bb.d.back()) + 1);
        aa = mulInt(aa, f);
        bb = mulInt(bb, f);
        BigInt q;
        q.d.assign(aa.d.size(), 0);
        BigInt r;
        for (i32 i = aa.d.size(); i--;) {
            if (r.d.empty()) r.d.push_back(aa.d[i]);
            else r.d.insert(r.d.begin(), aa.d[i]);
            r.sgn = 1;
            r.norm();
            u64 x = r.d.size() <= bb.d.size() ? 0 : r.d[bb.d.size()];

```



```

        u64 y = r.d.size() < bb.d.size() ? 0 : r.d[bb.d.size() - 1];
        u64 z = min<u64>(B - 1, (B * x + y) / bb.d.back());
        r -= mulInt(bb, z);
        while (r.sgn < 0) {
            r += bb;
            --z;
        }
        q.d[i] = static_cast<u32>(z);
    }
    q.sgn = sgn;
    q.norm();
    r = divmod(r, f).first;
    r.sgn = a.sgn;
    r.norm();
    return {q, r};
}

static BigInt mod(const BigInt &a, const BigInt &b) {
    return divmod(a, b).second;
}

static BigInt divBig(const BigInt &a, const BigInt &b) {
    return divmod(a, b).first;
}

BigInt operator/(const BigInt &v) const { return divBig(*this, v); }
BigInt operator%(const BigInt &v) const { return mod(*this, v); }
BigInt operator/(int v) const { return divmod(*this, v).first; }
BigInt operator%(int v) const { return BigInt(divmod(*this, v).second); }

static BigInt pow(BigInt a, u64 e) {
    BigInt res(1);
    while (e) {
        if (e & 1)
            res = res * a;
        a = a * a;
        e >>= 1;
    }
    return res;
}

static void useFFT(bool) {}

static BigInt gcd(BigInt a, BigInt b) {
    if (a.sgn < 0) a.sgn = 1;
    if (b.sgn < 0) b.sgn = 1;
    while (!b.ask0()) {
        BigInt r = a % b;
        a = b;
        b = r;
    }
    return a;
}

static BigInt fac(int n) {
    BigInt res(1);
    for (int i = 2; i <= n; i++) {
        res = mulInt(res, i);
    }
    return res;
}

static BigInt mul1(const BigInt &a, u32 sm) { return mulInt(a, sm); }

```

```

static BigInt mul10(BigInt a, i32 k) {
    if (k < 0) throw runtime_error("negative decimal shift");
    if (a.ask0() || !k) return a;
    i32 q = k / D, r = k % D;
    u32 p = 1;
    for (i32 i = 0; i < r; ++i) p *= 10;
    if (r) a = mulInt(a, p);
    if (q) a.d.insert(a.d.begin(), q, 0);
    return a;
}

static BigInt div10(BigInt a, i32 k) {
    if (k < 0) throw runtime_error("negative decimal shift");
    if (a.ask0() || !k) return a;
    i32 q = k / D, r = k % D;
    u32 p = 1;
    for (i32 i = 0; i < r; ++i) p *= 10;
    if (r) a = divmod(a, p).first;
    i32 n = a.d.size();
    if (q >= n) return BigInt(0);
    if (q) a.d.erase(a.d.begin(), a.d.begin() + q);
    a.norm();
    return a;
}

friend ostream &operator<<(ostream &os, const BigInt &x) {
    os << x.str();
    return os;
}

friend istream &operator>>(istream &is, BigInt &x) {
    string s;
    is >> s;
    x.read(s);
    return is;
}

BigInt &operator+=(const BigInt &v) {
    *this = *this + v;
    return *this;
}

BigInt &operator-=(const BigInt &v) {
    *this = *this - v;
    return *this;
}

BigInt &operator*=(const BigInt &v) {
    *this = *this * v;
    return *this;
}

BigInt &operator/=(const BigInt &v) {
    *this = *this / v;
    return *this;
}

friend bool operator==(const BigInt &a, const BigInt &b) { return cmp(a, b) == 0; }
friend bool operator!=(const BigInt &a, const BigInt &b) { return cmp(a, b) != 0; }
friend bool operator<(const BigInt &a, const BigInt &b) { return cmp(a, b) < 0; }
friend bool operator<=(const BigInt &a, const BigInt &b) { return cmp(a, b) <= 0; }
friend bool operator>(const BigInt &a, const BigInt &b) { return cmp(a, b) > 0; }
friend bool operator>=(const BigInt &a, const BigInt &b) { return cmp(a, b) >= 0; }
};

```

02A - 定点数 (FixedPoint, 常规) .cpp

311 lines · C++ · 08 - 高精度

```

/*
用途：动态小数位数的高精度定点数；适合精度会变化、需要 round 的常规场景。
依赖：先粘贴 01A 或 01B 中恰好一个 BigInt。01A 足够常用，01B 适合超大乘法。
性能：默认使用宏展开后的 int；若大规模数据因此超时或超空间，可将纯下标、状态或计数局部改为 i32。
*/

struct FixedPoint {
    BigInt man;
    int sgn;
    int scl;
    static int pre;
    static constexpr int LIM = (1LL << 31) - 1;

public:
    FixedPoint() : man(0), sgn(0), scl(0) {}
    FixedPoint(int v) { read(std::to_string(v)); }
    FixedPoint(const string &s) { read(s); }
    FixedPoint(const BigInt &m, int sc, int sgn = 1) : man(m), sgn(m.ask0() ? 0 : sgn), scl(sc) {}

    static void setPre(int d) { pre = d < 0 ? 0 : d; }

    static BigInt pow10(int k) {
        if (k < 0)
            throw runtime_error("negative pow10");
        if (k > LIM)
            throw overflow_error("decimal scale too large");
        return BigInt::mul10(BigInt(1), k);
    }

    static BigInt shift(BigInt a, int k) {
        if (k < 0 || k > LIM)
            throw overflow_error("decimal scale out of range");
        return BigInt::mul10(std::move(a), k);
    }

    void norm() {
        if (man.ask0()) {
            sgn = 0;
            scl = 0;
            return;
        }
        while (scl > 0) {
            auto qr = BigInt::divmod(man, 10);
            if (qr.second != 0)
                break;
            man = qr.first;
            --scl;
        }
        if (man.ask0())
            sgn = 0;
    }

    void round(int kp) {
        if (kp < 0)
            throw runtime_error("negative round");
        if (man.ask0()) {
            sgn = 0;
            scl = 0;
            return;
        }
    }
}

```

```

    if (scl <= kp) {
        norm();
        return;
    }

    int drp = scl - kp;
    BigInt p10 = pow10(drp);
    auto qr = BigInt::divmod(man, p10);
    BigInt q = qr.first;
    BigInt r = qr.second;
    BigInt r2 = BigInt::mulInt(r, 2);
    if (BigInt::cmp(r2, p10) >= 0)
        q = q + BigInt(1);
    man = q;
    scl = kp;
    norm();
}

FixedPoint abs() const {
    if (sgn == 0)
        return FixedPoint();
    FixedPoint r = *this;
    r.sgn = 1;
    return r;
}

void read(const string &s) {
    int n = s.size();
    int i = 0;
    while (i < n && isspace((unsigned char)s[i]))
        ++i;
    int sg = 1;
    if (i < n && (s[i] == '+' || s[i] == '-')) {
        if (s[i] == '-')
            sg = -1;
        ++i;
    }
    string d;
    int fd = 0;
    bool dot = false;
    for (; i < n; ++i) {
        char c = s[i];
        if (c == '.') {
            if (dot)
                break;
            dot = true;
            continue;
        }
        if (c >= '0' && c <= '9') {
            d.push_back(c);
            if (dot)
                ++fd;
        } else
            break;
    }
    int p = 0;
    while (p < d.size() && d[p] == '0')
        ++p;
    if (p == d.size()) {
        sgn = 0;
        scl = 0;
        man = BigInt(0);
        return;
    }
}

```

```

        string cor = d.substr(p);
        man = BigInt(cor);
        sgn = sg;
        scl = fd;
        norm();
    }

    string str(int p = -1) const {
        if (sgn == 0) {
            if (p <= 0)
                return "0";
            string s = "0.";
            s += string(p, '0');
            return s;
        }
        string ms = man.str();
        if (scl == 0) {
            string out = (sgn < 0 ? "-" : "") + ms;
            if (p > 0)
                out += "." + string(p, '0');
            return out;
        }
        if (ms.size() <= scl)
            ms = string(scl + 1 - ms.size(), '0') + ms;
        string ip = ms.substr(0, ms.size() - scl);
        string fra = ms.substr(ms.size() - scl);
        if (p >= 0) {
            if (fra.size() < p)
                fra += string(p - fra.size(), '0');
            else if (fra.size() > p)
                fra = fra.substr(0, p);
        } else {
            int t = fra.size();
            while (t > 0 && fra[t - 1] == '0')
                --t;
            fra.resize(t);
        }
        string out = (sgn < 0 ? "-" : "") + (ip.empty() ? string("0") : ip);
        if (!fra.empty())
            out += "." + fra;
        else if (p > 0)
            out += "." + string(p, '0');
        return out;
    }

    friend FixedPoint operator+(const FixedPoint &a, const FixedPoint &b) {
        if (a.sgn == 0)
            return b;
        if (b.sgn == 0)
            return a;
        FixedPoint _a = a, _b = b;
        if (_a.scl < _b.scl) {
            int dif = _b.scl - _a.scl;
            _a.man = shift(std::move(_a.man), dif);
            _a.scl = _b.scl;
        } else if (_b.scl < _a.scl) {
            int dif = _a.scl - _b.scl;
            _b.man = shift(std::move(_b.man), dif);
            _b.scl = _a.scl;
        }
        FixedPoint res;
        if (_a.sgn == _b.sgn) {
            res.man = _a.man + _b.man;
            res.sgn = _a.sgn;
        }
    }

```

```

        res.scl = _a.scl;
    } else {
        int c = BigInt::cmp(_a.man, _b.man);
        if (c == 0)
            return FixedPoint();
        else if (c > 0) {
            res.man = _a.man - _b.man;
            res.sgn = _a.sgn;
            res.scl = _a.scl;
        } else {
            res.man = _b.man - _a.man;
            res.sgn = _b.sgn;
            res.scl = _a.scl;
        }
    }
    res.round(max<int>(0, pre + 5));
    res.norm();
    return res;
}

friend FixedPoint operator-(const FixedPoint &a, const FixedPoint &b) {
    FixedPoint _b = b;
    _b.sgn = -_b.sgn;
    return a + _b;
}

friend FixedPoint operator*(const FixedPoint &a, const FixedPoint &b) {
    if (a.sgn == 0 || b.sgn == 0)
        return FixedPoint();
    FixedPoint res;
    res.man = a.man * b.man;
    res.sgn = a.sgn * b.sgn;
    if (a.scl > numeric_limits<int>::max() - b.scl)
        throw overflow_error("scale overflow");
    res.scl = a.scl + b.scl;
    res.round(max<int>(0, pre + 5));
    res.norm();
    return res;
}

friend FixedPoint operator/(const FixedPoint &a, const FixedPoint &b) {
    if (b.sgn == 0)
        throw runtime_error("division by zero");
    if (a.sgn == 0)
        return FixedPoint();
    int p = pre, wk = p + 15;
    int e = wk + b.scl - a.scl;
    BigInt num = a.man, den = b.man;
    if (e >= 0) num = shift(std::move(num), e);
    else den = shift(std::move(den), -e);
    auto qr = BigInt::divmod(num, den);
    BigInt q = qr.first;
    FixedPoint res(q, wk, a.sgn * b.sgn);
    res.round(p);
    res.norm();
    return res;
}

FixedPoint &operator+=(const FixedPoint &o) {
    *this = *this + o;
    return *this;
}

FixedPoint &operator-=(const FixedPoint &o) {
    *this = *this - o;
}

```

```

        return *this;
    }
    FixedPoint &operator*=(const FixedPoint &o) {
        *this = *this * o;
        return *this;
    }
    FixedPoint &operator/=(const FixedPoint &o) {
        *this = *this / o;
        return *this;
    }

    friend bool operator==(const FixedPoint &a, const FixedPoint &b) {
        if (a.sgn != b.sgn)
            return false;
        if (a.sgn == 0)
            return true;
        FixedPoint _a = a, _b = b;
        if (_a.scl < _b.scl) {
            _a.man = shift(std::move(_a.man), _b.scl - _a.scl);
            _a.scl = _b.scl;
        } else if (_b.scl < _a.scl) {
            _b.man = shift(std::move(_b.man), _a.scl - _b.scl);
            _b.scl = _a.scl;
        }
        return _a.man == _b.man;
    }

    friend bool operator!=(const FixedPoint &a, const FixedPoint &b) { return !(a == b); }

    friend bool operator<(const FixedPoint &a, const FixedPoint &b) {
        if (a.sgn != b.sgn)
            return a.sgn < b.sgn;
        if (a.sgn == 0)
            return b.sgn > 0;
        FixedPoint _a = a, _b = b;
        if (_a.scl < _b.scl) {
            _a.man = shift(std::move(_a.man), _b.scl - _a.scl);
            _a.scl = _b.scl;
        } else if (_b.scl < _a.scl) {
            _b.man = shift(std::move(_b.man), _a.scl - _b.scl);
            _b.scl = _a.scl;
        }
        if (_a.sgn > 0)
            return _a.man < _b.man;
        else
            return _a.man > _b.man;
    }

    friend bool operator<=(const FixedPoint &a, const FixedPoint &b) { return !(b < a); }
    friend bool operator>(const FixedPoint &a, const FixedPoint &b) { return b < a; }
    friend bool operator>=(const FixedPoint &a, const FixedPoint &b) { return !(a < b); }

    friend ostream &operator<<(ostream &os, const FixedPoint &x) {
        os << x.str();
        return os;
    }
};

int FixedPoint::pre = 50;

```

02B - 定点数 (FastFixedPoint, 极致优化) .cpp

152 lines · C++ · 08 - 高精度

```

/*
用途：编译期固定 P 位小数的高精度定点数；频繁四则运算时避免动态对齐、去零和幂缓存。
依赖：先粘贴 01B (推荐) 或 01A 中恰好一个 BigInt。P 是小数位数，构造和乘除均向 0 截断。
示例：using F = FastFixedPoint<1000>; F a("1.25"), b("3");
性能：默认使用宏展开后的 int；若大规模数据因此超时或超空间，可将纯下标、状态或计数局部改为 i32。
*/

template <int P>
struct FixedPoint {
    static_assert(P >= 0);

    BigInt x;

    FixedPoint() : x(0) {}
    FixedPoint(int a) : x(BigInt::mul10(BigInt(a), P)) {}
    explicit FixedPoint(const string& s) { read(s); }

    static FixedPoint raw(BigInt x) {
        FixedPoint a;
        a.x = std::move(x);
        return a;
    }

    bool ask0() const { return x.ask0(); }

    FixedPoint abs() const {
        return x < BigInt(0) ? raw(BigInt(0) - x) : *this;
    }

    void read(const string& s) {
        int n = s.size(), i = 0, f = 0;
        while (i < n && isspace(static_cast<unsigned char>(s[i]))) ++i;
        int sg = 1;
        if (i < n && (s[i] == '+' || s[i] == '-')) {
            if (s[i] == '-') sg = -1;
            ++i;
        }
        string a;
        bool dot = false;
        for (; i < n; ++i) {
            char c = s[i];
            if (c == '.') {
                if (dot) break;
                dot = true;
            } else if ('0' <= c && c <= '9') {
                a += c;
                if (dot) ++f;
            } else {
                break;
            }
        }
        int p = 0;
        while (p < a.size() && a[p] == '0') ++p;
        if (p == a.size()) {
            x = BigInt(0);
            return;
        }
        BigInt z(a.substr(p));
        if (f < P) z = BigInt::mul10(std::move(z), P - f);
        if (f > P) {

```



```

        BigInt q = BigInt::mul10(BigInt(1), f - P);
        auto [u, v] = BigInt::divmod(z, q);
        if (BigInt::cmp(BigInt::mulInt(v, 2), q) >= 0) u += BigInt(1);
        z = std::move(u);
    }
    x = sg < 0 ? BigInt(0) - z : z;
}

string str(int o = -1) const {
    if (x.ask0()) {
        if (o <= 0) return "0";
        return "0." + string(o, '0');
    }
    string a = x.str();
    bool neg = a[0] == '-';
    if (neg) a.erase(a.begin());
    if constexpr (P == 0) {
        if (o > 0) a += "." + string(o, '0');
        return (neg ? "-" : "") + a;
    }
    if (a.size() <= P)
        a = string(P + 1 - a.size(), '0') + a;
    string ip = a.substr(0, a.size() - P);
    string fp = a.substr(a.size() - P);
    if (o >= 0) {
        if (o < P) fp.resize(o);
        else fp += string(o - P, '0');
    } else {
        while (!fp.empty() && fp.back() == '0') fp.pop_back();
    }
    string r = (neg ? "-" : "") + ip;
    if (!fp.empty()) r += "." + fp;
    else if (o > 0) r += "." + string(o, '0');
    return r;
}

friend FixedPoint operator+(const FixedPoint& a, const FixedPoint& b) {
    return raw(a.x + b.x);
}

friend FixedPoint operator-(const FixedPoint& a, const FixedPoint& b) {
    return raw(a.x - b.x);
}

friend FixedPoint operator*(const FixedPoint& a, const FixedPoint& b) {
    return raw(BigInt::div10(a.x * b.x, P));
}

friend FixedPoint operator/(const FixedPoint& a, const FixedPoint& b) {
    if (b.x.ask0()) throw runtime_error("division by zero");
    return raw(BigInt::mul10(a.x, P) / b.x);
}

friend FixedPoint operator%(const FixedPoint& a, const FixedPoint& b) {
    if (b.x.ask0()) throw runtime_error("division by zero");
    return raw(a.x % b.x);
}

FixedPoint& operator+=(const FixedPoint& a) { return *this = *this + a; }
FixedPoint& operator-=(const FixedPoint& a) { return *this = *this - a; }
FixedPoint& operator*=(const FixedPoint& a) { return *this = *this * a; }
FixedPoint& operator/=(const FixedPoint& a) { return *this = *this / a; }
FixedPoint& operator%=(const FixedPoint& a) { return *this = *this % a; }

```

```
static FixedPoint pow(FixedPoint a, int n) {
    FixedPoint r(1);
    while (n) {
        if (n & 1) r *= a;
        a *= a;
        n >>= 1;
    }
    return r;
}

friend bool operator==(const FixedPoint& a, const FixedPoint& b) { return a.x == b.x; }
friend bool operator!=(const FixedPoint& a, const FixedPoint& b) { return a.x != b.x; }
friend bool operator<(const FixedPoint& a, const FixedPoint& b) { return a.x < b.x; }
friend bool operator<=(const FixedPoint& a, const FixedPoint& b) { return a.x <= b.x; }
friend bool operator>(const FixedPoint& a, const FixedPoint& b) { return a.x > b.x; }
friend bool operator>=(const FixedPoint& a, const FixedPoint& b) { return a.x >= b.x; }

friend ostream& operator<<(ostream& os, const FixedPoint& a) {
    return os << a.str();
}

friend istream& operator>>(istream& is, FixedPoint& a) {
    string s;
    is >> s;
    a.read(s);
    return is;
}
};
```

03A - 浮点数 (Decimal, 常规) .cpp

406 lines · C++ · 08 - 高精度

```

/*
用途：自动按 setPre 截断的高精度十进制数，支持 sqrt、pow、trunc 等常规接口。
依赖：先粘贴 01A 或 01B 中恰好一个 BigInt；高精度乘法建议选择 01B。
性能：默认使用宏展开后的 int；若大规模数据因此超时或超空间，可将纯下标、状态或计数局部改为 i32。
*/

class Decimal {
public:
    static void setPre(int d) {
        if (d < 0)
            d = 0;
        pre = d;
    }
    static void useFFT(bool on) { BigInt::useFFT(on); }

    Decimal() : sgn(0), scl(0) {}
    Decimal(int v) { read(std::to_string(v)); }
    explicit Decimal(const string &s) { read(s); }

    friend std::ostream &operator<<(std::ostream &os, const Decimal &x) {
        if (x.sgn == 0) {
            os << '0';
            return os;
        }
        string m = x.man.str();
        if (x.scl == 0) {
            if (x.sgn < 0)
                os << '-';
            os << m;
            return os;
        }
        if (m.size() <= x.scl) {
            string pad(x.scl + 1 - m.size(), '0');
            m = pad + m;
        }
        string ip = m.substr(0, m.size() - x.scl);
        string fp = m.substr(m.size() - x.scl);
        int trm = fp.size();
        while (trm > 0 && fp[trm - 1] == '0')
            --trm;
        fp.resize(trm);
        if (x.sgn < 0)
            os << '-';
        if (ip.empty())
            os << '0';
        else
            os << ip;
        if (!fp.empty()) {
            os << '.';
            os << fp;
        }
        return os;
    }

    friend std::istream &operator>>(std::istream &is, Decimal &x) {
        string s;
        is >> s;
        x.read(s);
        return is;
    }
}

```

```

string str() const { return strD(); }

friend int cmp(const Decimal &a, const Decimal &b) {
    if (a.sgn != b.sgn)
        return a.sgn < b.sgn ? -1 : 1;
    if (a.sgn == 0)
        return 0;
    if (a.scl == b.scl) {
        int c = BigInt::cmp(a.man, b.man);
        return a.sgn > 0 ? c : -c;
    }
    Decimal _a = a, _b = b;
    if (a.scl < b.scl) {
        int dif = b.scl - a.scl;
        _a.man = shift(std::move(_a.man), dif);
        _a.scl = b.scl;
    } else {
        int dif = a.scl - b.scl;
        _b.man = shift(std::move(_b.man), dif);
        _b.scl = a.scl;
    }
    int c = BigInt::cmp(_a.man, _b.man);
    return a.sgn > 0 ? c : -c;
}

friend bool operator==(const Decimal &a, const Decimal &b) { return cmp(a, b) == 0; }
friend bool operator!=(const Decimal &a, const Decimal &b) { return cmp(a, b) != 0; }
friend bool operator<(const Decimal &a, const Decimal &b) { return cmp(a, b) < 0; }
friend bool operator<=(const Decimal &a, const Decimal &b) { return cmp(a, b) <= 0; }
friend bool operator>(const Decimal &a, const Decimal &b) { return cmp(a, b) > 0; }
friend bool operator>=(const Decimal &a, const Decimal &b) { return cmp(a, b) >= 0; }

friend Decimal operator+(const Decimal &a, const Decimal &b) {
    if (a.sgn == 0)
        return b;
    if (b.sgn == 0)
        return a;
    Decimal _a = a, _b = b;
    if (_a.scl < _b.scl) {
        int dif = _b.scl - _a.scl;
        _a.man = shift(std::move(_a.man), dif);
        _a.scl = _b.scl;
    } else if (_b.scl < _a.scl) {
        int dif = _a.scl - _b.scl;
        _b.man = shift(std::move(_b.man), dif);
        _b.scl = _a.scl;
    }
    Decimal res;
    if (_a.sgn == _b.sgn) {
        res.man = _a.man + _b.man;
        res.sgn = res.man.ask0() ? 0 : _a.sgn;
        res.scl = _a.scl;
    } else {
        int c = BigInt::cmp(_a.man, _b.man);
        if (c == 0) {
            res = Decimal(); // zero
        } else if (c > 0) {
            res.man = _a.man - _b.man;
            res.sgn = _a.sgn;
            res.scl = _a.scl;
        } else {
            res.man = _b.man - _a.man;
            res.sgn = _b.sgn;
            res.scl = _a.scl;
        }
    }
}

```

```

    }
}
res.norm();
return res;
}

friend Decimal operator-(const Decimal &a, const Decimal &b) {
    Decimal _b = b;
    _b.sgn = -_b.sgn;
    return a + _b;
}

friend Decimal operator*(const Decimal &a, const Decimal &b) {
    if (a.sgn == 0 || b.sgn == 0)
        return Decimal();
    Decimal res;
    res.man = a.man * b.man;
    res.sgn = a.sgn * b.sgn;
    res.scl = a.scl + b.scl;
    res.round(pre);
    return res;
}

friend Decimal operator/(const Decimal &a, const Decimal &b) {
    if (b.sgn == 0)
        throw std::runtime_error("Decimal division by zero");
    if (a.sgn == 0)
        return Decimal();
    int prc = pre, wk = prc + 10;
    int e = wk + b.scl - a.scl;
    BigInt num = a.man, den = b.man;
    if (e >= 0) num = shift(std::move(num), e);
    else den = shift(std::move(den), -e);
    auto qr = BigInt::divmod(num, den);
    Decimal res;
    res.man = qr.first;
    res.sgn = a.sgn * b.sgn;
    res.scl = wk;
    res.round(prc);
    return res;
}

friend Decimal operator%(const Decimal &a, const Decimal &b) {
    if (b.sgn == 0)
        throw std::runtime_error("Decimal modulo by zero");
    if (a.sgn == 0)
        return Decimal();
    Decimal div = a / b;
    Decimal q = div.trunc();
    Decimal prd = q * b;
    Decimal rem = a - prd;
    return rem;
}

Decimal& operator+=(const Decimal& a) { return *this = *this + a; }
Decimal& operator-=(const Decimal& a) { return *this = *this - a; }
Decimal& operator*=(const Decimal& a) { return *this = *this * a; }
Decimal& operator/=(const Decimal& a) { return *this = *this / a; }
Decimal& operator%=(const Decimal& a) { return *this = *this % a; }

Decimal abs() const {
    Decimal a = *this;
    if (a.sgn < 0) a.sgn = 1;
    return a;
}

```

```

}

static Decimal pow(Decimal bas, int exp) {
    if (exp == 0)
        return Decimal(1);
    Decimal res(1);
    while (exp) {
        if (exp & 1)
            res = res * bas;
        bas = bas * bas;
        exp >>= 1;
    }
    return res;
}

Decimal sqrt() const {
    if (sgn < 0)
        throw std::runtime_error("sqrt of negative");
    if (sgn == 0)
        return Decimal(0);
    int prc = pre;
    int e = man.str().size() - scl;
    int t = e >= 0 ? e / 2 : -((-e + 1) / 2);
    if (std::abs(t) > LIM)
        throw std::overflow_error("sqrt exponent too large");
    string s;
    if (t >= 0) s = "1" + string(t, '0');
    else s = "0." + string(-t - 1, '0') + "1";
    Decimal x(s);
    for (int it = 0; it < 60; ++it) {
        Decimal nx = (x + (*this) / x) * Decimal("0.5");
        Decimal dif = nx - x;
        dif.norm();
        if (dif.ask0() || dif.small(-prc - 2)) {
            x = nx;
            break;
        }
        x = nx;
    }
    x.round(prc);
    return x;
}

BigInt fac() const {
    if (scl != 0)
        throw std::runtime_error("factorial only for integers");
    if (sgn < 0)
        throw std::runtime_error("factorial only for non-negative ints");
    int n = 0;
    string s = man.str();
    if (s.size() > 20)
        throw std::runtime_error("factorial too large to compute by this helper");
    n = std::stoull(s);
    return BigInt::fac(n);
}

BigInt gcd(const Decimal &o) const {
    if (scl != 0 || o.scl != 0)
        throw std::runtime_error("gcd defined for integers only in this template");
    BigInt a = man, b = o.man;
    return BigInt::gcd(a, b);
}

Decimal trunc() const {

```

```
        if (sgn == 0)
            return Decimal();
        if (scl == 0)
            return *this;
        auto qr = BigInt::divmod(man, pow10(scl));
        Decimal r;
        r.man = qr.first;
        r.sgn = sgn;
        r.scl = 0;
        return r;
    }

    bool ask0() const {
        return sgn == 0;
    }

    bool small(int e) const {
        if (e >= 0)
            return false;
        int k = -e;
        if (scl < k)
            return false;
        int l = scl - k;
        if (l == 0)
            return man.ask0();
        BigInt p10 = pow10(l);
        int c = BigInt::cmp(man, p10);
        return c < 0;
    }

    string strD() const {
        std::ostringstream oss;
        oss << *this;
        string s = oss.str();
        return s;
    }

private:
    int sgn;
    BigInt man;
    int scl;

    static int pre;
    static constexpr int LIM = (1LL << 31) - 1;

    void norm() {
        if (man.ask0()) {
            sgn = 0;
            scl = 0;
            return;
        }
        while (scl > 0) {
            auto qr = BigInt::divmod(man, 10);
            if (qr.second != 0)
                break;
            man = qr.first;
            --scl;
        }
    }

    void round(int kp) {
        if (man.ask0()) {
            sgn = 0;
            scl = 0;
        }
    }
}
```

```

        return;
    }
    if (scl <= kp) {
        norm();
        return;
    }
    int drp = scl - kp;
    BigInt p10 = pow10(drp);
    auto qr = BigInt::divmod(man, p10);
    BigInt q = qr.first;
    BigInt r = qr.second;
    BigInt rr = BigInt::mulInt(r, 2);
    if (BigInt::cmp(rr, p10) >= 0)
        q = q + BigInt(1);
    man = q;
    scl = kp;
    norm();
}

void read(const string &s) {
    string t;
    int n = s.size(), pos = 0;
    while (pos < n && isspace(s[pos]))
        ++pos;
    int sg = 1;
    if (pos < n && (s[pos] == '+' || s[pos] == '-')) {
        if (s[pos] == '-')
            sg = -1;
        ++pos;
    }
    string d;
    int fd = 0;
    bool dot = false;
    while (pos < n) {
        char c = s[pos++];
        if (c == '.') {
            if (dot)
                break;
            dot = true;
            continue;
        }
        if (c >= '0' && c <= '9') {
            d.push_back(c);
            if (dot)
                ++fd;
        } else
            break;
    }
    int p = 0;
    while (p < d.size() && d[p] == '0')
        ++p;
    if (p == d.size()) {
        sgn = 0;
        scl = 0;
        man = BigInt(0);
        return;
    }
    string cor = d.substr(p);
    man = BigInt(cor);
    sgn = sg;
    scl = fd;
    norm();
}

```



```
static BigInt shift(BigInt a, int k) {
    if (k < 0 || k > LIM)
        throw std::overflow_error("decimal scale out of range");
    return BigInt::mul10(std::move(a), k);
}

static BigInt pow10(int k) {
    if (k < 0)
        throw std::runtime_error("negative pow10");
    return shift(BigInt(1), k);
}

static int pow10i(int k) {
    if (k < 0)
        throw std::runtime_error("negative pow10");
    static const int mx = 1'000'000'000'000'000'000LL;
    int r = 1;
    for (int i = 0; i < k; ++i) {
        if (r > mx / 10)
            throw std::overflow_error("pow10Int overflow");
        r *= 10;
    }
    return r;
};

int Decimal::pre = 50;
```

03B - 浮点数（FastDecimal，极致优化）.cpp

8 lines · C++ · 08 - 高精度

```
/*
用途：03A Decimal 的卡常替代；精度改为编译期常量，避免每次运算读取/维护动态 scale。
依赖：先粘贴 01B，再粘贴 02B。FastDecimal<P> 与 FastFixedPoint<P> 语义相同，P 为小数位数。
取舍：保留常用四则、比较、pow、输入输出；需要 sqrt、动态 setPre、round 时使用 03A。
性能：默认使用宏展开后的 int；若大规模数据因此超时或超空间，可将纯下标、状态或计数局部改为 i32。
*/
template <int P>
using Decimal = FixedPoint<P>;
```

04A - 2至62进制转换（常规）.cpp

46 lines · C++ · 08 - 高精度

```

/*
用途：非负/负整数字符串在 2..62 进制间精确转换，不依赖 BigInt。
复杂度：逐次短除，O(输入位数*输出位数)；普通长度优先使用。
字符顺序：0-9A-Za-z。
性能：默认使用宏展开后的 int；若大规模数据因此超时或超空间，可将纯下标、状态或计数局部改为 i32。
*/
struct BaseConv {
    static int val(char c) {
        if (isdigit((unsigned char)c)) return c - '0';
        if (isupper((unsigned char)c)) return c - 'A' + 10;
        return c - 'a' + 36;
    }
    static char chr(int x) {
        if (x < 10) return char('0' + x);
        if (x < 36) return char('A' + x - 10);
        return char('a' + x - 36);
    }

    static string cvt(string s, int a, int b) {
        assert(2 <= a && a <= 62 && 2 <= b && b <= 62);
        bool neg = !s.empty() && s[0] == '-';
        if (neg) s.erase(s.begin());
        vector<int> d;
        for (char c : s) {
            int x = val(c);
            if (x >= a) throw invalid_argument("BaseConv digit");
            if (!d.empty() || x) d.push_back(x);
        }
        if (d.empty()) return "0";
        string z;
        while (!d.empty()) {
            int r = 0;
            for (auto &x : d) {
                int y = r * a + x;
                x = y / b;
                r = y % b;
            }
            z += chr(r);
            auto p = find_if(d.begin(), d.end(), [](int x) { return x != 0; });
            d.erase(d.begin(), p);
        }
        if (neg) z += '-';
        reverse(z.begin(), z.end());
        return z;
    }
};

```

04B - 2至62进制转换（极致优化）.cpp

63 lines · C++ · 08 - 高精度

```

/*
用途：超长 2..62 进制整数转换；依赖已选择的 01A 或 01B `BigInt`。
优化：按不超过 1e9 的大块读入/除出，显著减少高精度乘除次数。
字符顺序：0-9A-Za-z。
*/
struct BaseConv {
    static int val(char c) {
        if (isdigit((unsigned char)c)) return c - '0';
        if (isupper((unsigned char)c)) return c - 'A' + 10;
        return c - 'a' + 36;
    }
    static char chr(int x) {
        if (x < 10) return char('0' + x);
        if (x < 36) return char('A' + x - 10);
        return char('a' + x - 36);
    }
    static pii block(int b) {
        int p = 1, k = 0;
        while (p <= 1000000000 / b) p *= b, ++k;
        return {p, k};
    }

    static string cvt(string s, int a, int b) {
        assert(2 <= a && a <= 62 && 2 <= b && b <= 62);
        bool neg = !s.empty() && s[0] == '-';
        if (neg) s.erase(s.begin());
        int p = 0;
        while (p < s.size() && s[p] == '0') ++p;
        s.erase(0, p);
        if (s.empty()) return "0";

        auto [pa, ka] = block(a);
        BigInt x = 0;
        int f = s.size() % ka;
        if (!f) f = ka;
        for (int l = 0; l < s.size(); ) {
            int z = l ? ka : f, q = 1, v = 0;
            for (int i = 0; i < z; ++i) {
                int d = val(s[l + i]);
                if (d >= a) throw invalid_argument("FastBaseConv digit");
                v = v * a + d;
                q *= a;
            }
            x = x * BigInt(q) + BigInt(v);
            l += z;
        }

        auto [pb, kb] = block(b);
        string z;
        while (!x.ask0()) {
            auto [q, r] = BigInt::divmod(x, pb);
            x = move(q);
            for (int i = 0; i < kb; ++i) {
                z += chr(r % b);
                r /= b;
            }
        }
        while (z.size() > 1 && z.back() == '0') z.pop_back();
        if (neg) z += '-';
        reverse(z.begin(), z.end());
    }
};

```

```
        return z;  
    }  
};
```

01A - 普通莫队.cpp

44 lines · C++ · 09 - 杂类

```

/*
用途：离线处理静态序列区间询问，约  $O((n + q) \sqrt{n})$ 。
约定：下标从 1 开始，询问为闭区间；add/del 接收下标，get 返回当前答案。
性能：默认使用宏展开后的 int；若大规模数据因此超时或超空间，可将纯下标、状态或计数局部改为 i32。
*/
struct Mo {
    struct Q {
        int l, r, id;
    };

    int n;
    vector<Q> q;

    explicit Mo(int n) : n(n) {}

    void ask(int l, int r) {
        q.push_back({l, r, q.size()});
    }

    template <class A, class D, class G>
    auto solve(A add, D del, G get) {
        using R = decay_t<decltype(get())>;
        int m = q.size();
        vector<R> ans(m);
        if (!m) return ans;
        int z = max<int>(1, sqrt(m));
        int b = max<int>(1, n / z);
        sort(q.begin(), q.end(), [b](const Q& x, const Q& y) {
            int a = (x.l - 1) / b, c = (y.l - 1) / b;
            if (a != c) return a < c;
            return a & 1 ? x.r > y.r : x.r < y.r;
        });

        int l = 1, r = 0;
        for (auto [x, y, id] : q) {
            while (l > x) add(--l);
            while (r < y) add(++r);
            while (l < x) del(l++);
            while (r > y) del(r--);
            ans[id] = get();
        }
        return ans;
    }
};

```

01B - 带修改莫队.cpp

50 lines · C++ · 09 - 杂类

```

/*
用途：离线处理区间询问与单点修改， $n, q$  同阶时约  $O(n^{5/3})$ 。
约定：下标从 1 开始。每读到一次修改先调用 modify，再加入后续询问。
apply/undo 接收修改编号和当前  $[l, r]$ ；若修改点在区间内，应先 del，再改值，最后 add。
性能：默认使用宏展开后的 int；若大规模数据因此超时或超空间，可将纯下标、状态或计数局部改为 i32。
*/
struct Mo {
    struct Q {
        int l, r, t, id;
    };

    int n, c = 0;
    vector<Q> q;

    explicit Mo(int n) : n(n) {}

    void modify() { ++c; }

    void ask(int l, int r) {
        q.push_back({l, r, c, q.size()});
    }

    template <class A, class D, class P, class U, class G>
    auto solve(A add, D del, P ap, U ud, G get) {
        using R = decay_t<decltype(get())>;
        int m = q.size();
        vector<R> ans(m);
        if (!m) return ans;
        int b = max<int>(1, pow(max(n, m), 2.0 / 3.0));
        sort(q.begin(), q.end(), [b](const Q& x, const Q& y) {
            int a = (x.l - 1) / b, c = (y.l - 1) / b;
            if (a != c) return a < c;
            int d = (x.r - 1) / b, e = (y.r - 1) / b;
            if (d != e) return a & 1 ? d > e : d < e;
            return (a ^ d) & 1 ? x.t > y.t : x.t < y.t;
        });

        int l = 1, r = 0, t = 0;
        for (auto [x, y, z, id] : q) {
            while (t < z) ap(++t, l, r);
            while (t > z) ud(t--, l, r);
            while (l > x) add(--l);
            while (r < y) add(++r);
            while (l < x) del(l++);
            while (r > y) del(r--);
            ans[id] = get();
        }
        return ans;
    }
};

```

01C - 回滚莫队.cpp

56 lines · C++ · 09 - 杂类

```

/*
用途：add 容易而 del 困难时的静态区间询问，约  $O((n + q) \sqrt{n})$ 。
回调：clear 清空状态；save 返回快照；rollback(s) 回到快照；brute(l, r) 独立暴力同块询问。
约定：下标从 1 开始，add 只加入下标，get 返回当前答案。
性能：默认使用宏展开后的 int；若大规模数据因此超时或超空间，可将纯下标、状态或计数局部改为 i32。
*/
struct Mo {
    struct Q {
        int l, r, id;
    };

    int n;
    vector<Q> q;

    explicit Mo(int n) : n(n) {}

    void ask(int l, int r) {
        q.push_back({l, r, q.size()});
    }

    template <class A, class C, class S, class R, class G, class B>
    auto solve(A add, C clr, S sav, R ud, G get, B bf) {
        using V = decay_t<decltype(get())>;
        int m = q.size();
        vector<V> ans(m);
        if (!m) return ans;
        int b = max<int>(1, sqrt(n));
        sort(q.begin(), q.end(), [b](const Q& x, const Q& y) {
            int a = (x.l - 1) / b, c = (y.l - 1) / b;
            if (a != c) return a < c;
            return x.r < y.r;
        });

        for (int p = 0; p < m; p++) {
            int k = (q[p].l - 1) / b, z = p;
            while (z < m && (q[z].l - 1) / b == k) ++z;
            int e = min<int>(n + 1, (k + 1) * b + 1);
            clr();
            int r = e - 1;
            for (int i = p; i < z; ++i) {
                auto [l, qr, id] = q[i];
                if (qr < e) {
                    ans[id] = bf(l, qr);
                    continue;
                }
                while (r < qr) add(++r);
                auto s = sav();
                for (int j = e - 1; j >= l; --j) add(j);
                ans[id] = get();
                ud(s);
            }
            p = z;
        }
        return ans;
    }
};

```


01D - Hilbert序莫队.cpp

52 lines · C++ · 09 - 杂类

```

/*
用途：用 Hilbert 序替代分块排序的静态莫队，省去块长调参，常数通常更稳定。
约定：下标从 1 开始；add/del 接收下标，get 返回当前答案。
性能：默认使用宏展开后的 int；若大规模数据因此超时或超空间，可将纯下标、状态或计数局部改为 i32。
*/
struct Mo {
    struct Q {
        int l, r, id;
        int o;
    };

    int n, p = 1;
    vector<Q> q;

    explicit Mo(int n) : n(n) {
        int x = max<int>(1, n) - 1;
        p = 0;
        do ++p; while (x >= 1);
    }

    static int ord(int x, int y, int p, int r = 0) {
        if (!p) return 0;
        int h = 1LL << (p - 1);
        int s = x < h ? (y < h ? 0 : 3) : (y < h ? 1 : 2);
        s = (s + r) & 3;
        static constexpr int d[] = {3, 0, 0, 1};
        int nr = (r + d[s]) & 3;
        int z = 1LL << (2 * p - 2);
        int w = ord(x & (h - 1), y & (h - 1), p - 1, nr);
        return s * z + (s == 1 || s == 2 ? w : z - w - 1);
    }

    void ask(int l, int r) {
        q.push_back({l, r, q.size(), ord(l - 1, r - 1, p)});
    }

    template <class A, class D, class G>
    auto solve(A add, D del, G get) {
        using R = decay_t<decltype(get())>;
        vector<R> ans(q.size());
        sort(q.begin(), q.end(), [](const Q& a, const Q& b) { return a.o < b.o; });
        int l = 1, r = 0;
        for (auto [x, y, id, o] : q) {
            while (l > x) add(--l);
            while (r < y) add(++r);
            while (l < x) del(l++);
            while (r > y) del(r--);
            ans[id] = get();
        }
        return ans;
    }
};

```

02 - 笛卡尔树.cpp

40 lines · C++ · 09 - 杂类

```

/*
用途：O(n) 构造笛卡尔树；中序遍历为原下标顺序，同时满足堆性质。
模板：C=less<T> 为小根树，C=greater<T> 为大根树；E=true 时相等元素中右侧在上层。
约定：输入及 rt/lc/rc/fa 均为 0-indexed，空节点为 -1。
性能：默认使用宏展开后的 int；若大规模数据因此超时或超空间，可将纯下标、状态或计数局部改为 i32。
*/
template <class T, class C = less<T>, bool E = false>
struct CartTree {
    int rt = -1;
    vector<int> lc, rc, fa;
    C c;

    CartTree() = default;
    explicit CartTree(const vector<T>& a, C c = {}) : c(c) { build(a); }

    void build(const vector<T>& a) {
        int n = a.size();
        rt = -1;
        lc.assign(n, -1);
        rc.assign(n, -1);
        fa.assign(n, -1);
        vector<int> s;
        s.reserve(n);
        for (int i = 0; i < n; ++i) {
            int x = -1;
            while (!s.empty()) {
                int j = s.back();
                bool f = c(a[i], a[j]);
                if constexpr (E) f = f || !c(a[j], a[i]);
                if (!f) break;
                x = j;
                s.pop_back();
            }
            if (s.empty()) rt = i;
            else fa[i] = s.back(), rc[s.back()] = i;
            if (x != -1) fa[x] = i, lc[i] = x;
            s.push_back(i);
        }
    };
};

```

03 - CDQ分治（三维偏序）.cpp

76 lines · C++ · 09 - 杂类

```

/*
用途：统计三维偏序。返回 ans[k]：恰有 k 个其他点三维坐标均不大于它的点数。
复杂度：O(n log n)，重复点自动合并；坐标类型由 T 适配。
性能：默认使用宏展开后的 int；若大规模数据因此超时或超空间，可将纯下标、状态或计数局部改为 i32。
*/
template <class T>
struct CDQ3 {
    struct P {
        T y;
        int z, c, v;
    };

    vector<P> a, t;
    vector<int> bit;

    void modify(int x, int v) {
        for (++x; x < bit.size(); x += x & -x) bit[x] += v;
    }

    int ask(int x) const {
        int r = 0;
        for (++x; x; x -= x & -x) r += bit[x];
        return r;
    }

    void cdq(int l, int r) {
        if (r - l <= 1) return;
        int m = (l + r) / 2;
        cdq(l, m);
        cdq(m, r);
        int i = l, j = m, k = l;
        while (i < m && j < r) {
            if (a[i].y <= a[j].y) {
                modify(a[i].z, a[i].c);
                t[k++] = a[i++];
            } else {
                a[j].v += ask(a[j].z);
                t[k++] = a[j++];
            }
        }
        int p = i;
        while (j < r) {
            a[j].v += ask(a[j].z);
            t[k++] = a[j++];
        }
        while (i < m) t[k++] = a[i++];
        for (int x = l; x < p; ++x) modify(a[x].z, -a[x].c);
        copy(t.begin() + l, t.begin() + r, a.begin() + l);
    }

    vector<int> solve(vector<array<T, 3>> p) {
        int n = p.size();
        vector<int> ans(n);
        if (!n) return ans;
        sort(p.begin(), p.end());
        vector<T> z;
        z.reserve(n);
        for (auto& x : p) z.push_back(x[2]);
        sort(z.begin(), z.end());
        z.erase(unique(z.begin(), z.end()), z.end());
    }
}

```

```
    a.clear();
    for (int i = 0; i < n; i) {
        int j = i + 1;
        while (j < n && p[j] == p[i]) ++j;
        int k = lower_bound(z.begin(), z.end(), p[i][2]) - z.begin();
        a.push_back({p[i][1], k, j - i, 0});
        i = j;
    }
    t.resize(a.size());
    bit.assign(z.size() + 1, 0);
    cdq(0, a.size());
    for (auto& x : a) ans[x.v + x.c - 1] += x.c;
    return ans;
}
};
```

04 - 整体二分（带修改区间第k小）.cpp

101 lines · C++ · 09 - 杂类

```

/*
用途：离线处理单点赋值与区间第 k 小，复杂度  $O((n+q) \log V \log n)$ 。
约定：初始数组 a 为 1-indexed (a[0] 不使用)；modify/ask 的位置同样从 1 开始。
性能：默认使用宏展开后的 int；若大规模数据因此超时或超空间，可将纯下标、状态或计数局部改为 i32。
*/
template <class T>
struct KthModify {
    struct O {
        int t, p, l, r, k, id;
        T x;
    };
    struct E {
        int t, p, d, v, l, r, k, id;
    };

    vector<T> a, val, ans;
    vector<O> o;
    vector<int> bit;
    int n, qc = 0;

    explicit KthModify(vector<T> a) : a(std::move(a)) {
        assert(this->a.size() >= 2);
        n = this->a.size() - 1;
    }

    void modify(int p, T x) {
        assert(1 <= p && p <= n);
        o.push_back({0, p, 0, 0, 0, -1, x});
    }

    int ask(int l, int r, int k) {
        assert(1 <= l && l <= r && r <= n && 1 <= k && k <= r - l + 1);
        o.push_back({1, 0, l, r, k, qc, T{}});
        return qc++;
    }

    void bitAdd(int x, int v) {
        for (; x <= n; x += x & -x) bit[x] += v;
    }

    int bitAsk(int x) const {
        int r = 0;
        for (; x; x -= x & -x) r += bit[x];
        return r;
    }

    void dfs(int l, int r, vector<E> e) {
        bool h = false;
        for (auto& x : e) h |= x.t;
        if (!h) return;
        if (l == r) {
            for (auto& x : e) if (x.t) ans[x.id] = val[l];
            return;
        }
        int m = (l + r) / 2;
        vector<E> x, y;
        x.reserve(e.size());
        y.reserve(e.size());
        for (auto z : e) {
            if (!z.t) {

```

```

        if (z.v <= m) bitAdd(z.p, z.d), x.push_back(z);
        else y.push_back(z);
    } else {
        int c = bitAsk(z.r) - bitAsk(z.l - 1);
        if (c >= z.k) x.push_back(z);
        else z.k -= c, y.push_back(z);
    }
}
for (auto& z : e) if (!z.t && z.v <= m) bitAdd(z.p, -z.d);
dfs(l, m, std::move(x));
dfs(m + 1, r, std::move(y));
}

vector<T> solve() {
    val.assign(a.begin() + 1, a.end());
    for (auto& x : o) if (!x.t) val.push_back(x.x);
    sort(val.begin(), val.end());
    val.erase(unique(val.begin(), val.end()), val.end());
    auto id = [&](const T& x) -> int {
        return lower_bound(val.begin(), val.end(), x) - val.begin();
    };

    vector<E> e;
    e.reserve(n + 2 * o.size());
    vector<T> cur = a;
    for (int i = 1; i <= n; ++i) e.push_back({0, i, 1, id(cur[i]), 0, 0, 0, -1});
    for (auto& x : o) {
        if (!x.t) {
            e.push_back({0, x.p, -1, id(cur[x.p]), 0, 0, 0, -1});
            e.push_back({0, x.p, 1, id(x.x), 0, 0, 0, -1});
            cur[x.p] = x.x;
        } else {
            e.push_back({1, 0, 0, 0, x.l, x.r, x.k, x.id});
        }
    }
    ans.assign(qc, T{});
    bit.assign(n + 1, 0);
    if (qc) dfs(0, val.size() - 1, std::move(e));
    return ans;
}
};

```

05 - WQS二分（带权二分）.cpp

26 lines · C++ · 09 - 杂类

```
/*
用途：把“恰选 k 个”的凸/凹离散优化转成带惩罚 DP。
wqsMin: f(c) 最小化 base + c*cnt; wqsMax: f(c) 最大化 base - c*cnt。
要求：惩罚 c 增大时 cnt 单调不增，且同值时 f 必须优先返回更大的 cnt。
*/
template <class T, class F>
T wqsMin(T l, T r, int k, F f) {
    while (l < r) {
        T m = l + (r - l + 1) / 2;
        auto p = f(m);
        if (p.second >= k) l = m;
        else r = m - 1;
    }
    return f(l).first - l * static_cast<T>(k);
}

template <class T, class F>
T wqsMax(T l, T r, int k, F f) {
    while (l < r) {
        T m = l + (r - l + 1) / 2;
        auto p = f(m);
        if (p.second >= k) l = m;
        else r = m - 1;
    }
    return f(l).first + l * static_cast<T>(k);
}
```

06 - 舞蹈链 (DLX, 精确覆盖) .cpp

99 lines · C++ · 09 - 杂类

```

/*
用途：求精确覆盖的一组可行行，适合数独、棋盘覆盖等；搜索复杂度取决于实例。
列编号为 1..m，addRow(id, cols) 加入一行；同一行的列不得重复。
性能：默认使用宏展开后的 int；若大规模数据因此超时或超空间，可将纯下标、状态或计数局部改为 i32。
*/
struct DLX {
    int m;
    vector<int> l, r, u, d, row, col, s, cur, ans;

    explicit DLX(int m, int z = 0) : m(m) {
        l.resize(m + 1);
        r.resize(m + 1);
        u.resize(m + 1);
        d.resize(m + 1);
        row.assign(m + 1, 0);
        col.resize(m + 1);
        s.assign(m + 1, 0);
        for (int i = 0; i <= m; ++i) {
            l[i] = i - 1;
            r[i] = i + 1;
            u[i] = d[i] = col[i] = i;
        }
        l[0] = m;
        r[m] = 0;
        if (z) {
            int n = m + 1 + z;
            l.reserve(n), r.reserve(n), u.reserve(n), d.reserve(n);
            row.reserve(n), col.reserve(n);
        }
    }

    void addRow(int id, const vector<int>& a) {
        int h = -1;
        for (int c : a) {
            assert(1 <= c && c <= m);
            int x = l.size();
            l.push_back(x), r.push_back(x), u.push_back(0), d.push_back(0);
            row.push_back(id), col.push_back(c);
            u[x] = u[c], d[x] = c;
            d[u[c]] = x, u[c] = x;
            ++s[c];
            if (h == -1) h = x;
            else {
                l[x] = l[h], r[x] = h;
                r[l[h]] = x, l[h] = x;
            }
        }
    }

    void cover(int c) {
        r[l[c]] = r[c], l[r[c]] = l[c];
        for (int i = d[c]; i != c; i = d[i]) {
            for (int j = r[i]; j != i; j = r[j]) {
                d[u[j]] = d[j], u[d[j]] = u[j];
                --s[col[j]];
            }
        }
    }

    void uncover(int c) {

```



```
        for (int i = u[c]; i != c; i = u[i]) {
            for (int j = l[i]; j != i; j = l[j]) {
                ++s[col[j]];
                d[u[j]] = j, u[d[j]] = j;
            }
        }
        r[l[c]] = c, l[r[c]] = c;
    }

    bool dfs() {
        if (!r[0]) {
            ans = cur;
            return true;
        }
        int c = r[0];
        for (int j = r[c]; j; j = r[j]) if (s[j] < s[c]) c = j;
        if (!s[c]) return false;
        cover(c);
        for (int i = d[c]; i != c; i = d[i]) {
            cur.push_back(row[i]);
            for (int j = r[i]; j != i; j = r[j]) cover(col[j]);
            bool f = dfs();
            for (int j = l[i]; j != i; j = l[j]) uncover(col[j]);
            cur.pop_back();
            if (f) {
                uncover(c);
                return true;
            }
        }
        uncover(c);
        return false;
    }

    bool solve() {
        cur.clear();
        ans.clear();
        return dfs();
    }
};
```

07 - 模拟退火.cpp

27 lines · C++ · 09 - 杂类

```

/*
用途：连续或离散状态的启发式优化；不保证正确，只用于允许随机化搜索的题。
go(s,t,rng) 返回邻居，f(s) 返回可转为 double 的目标值；MAX=true 时改为最大化。
随机数生成器可直接传入基础算法目录中的 Random::rng。
*/
template <bool MAX = false, class S, class F, class G, class R>
S anneal(S s, F f, G go, R& rng, double t = 1e4, double e = 1e-8,
         double a = 0.995, int rep = 1) {
    assert(t > e && 0 < a && a < 1 && rep > 0);
    uniform_real_distribution<double> u(0, 1);
    S st = s, ans = s;
    double bv = static_cast<double>(f(s));
    for (int z = 0; z < rep; ++z) {
        s = st;
        double v = static_cast<double>(f(s));
        for (double c = t; c > e; c *= a) {
            S ns = go(s, c, rng);
            double nv = static_cast<double>(f(ns));
            double d = nv - v;
            if constexpr (MAX) d = -d;
            if (d <= 0 || exp(-d / c) > u(rng)) s = std::move(ns), v = nv;
            bool ok = MAX ? v > bv : v < bv;
            if (ok) ans = s, bv = v;
        }
    }
    return ans;
}

```

08 - 公历日期换算.cpp

43 lines · C++ · 09 - 杂类

```
/*
用途：公历日期与相对 1970-01-01 的天数互转，支持负年份；所有操作 O(1)。
weekday 返回 0..6，分别表示周日到周六。
*/
struct Date {
    int y, m, d;
};

bool leap(int y) {
    return y % 4 == 0 && (y % 100 != 0 || y % 400 == 0);
}

int days(int y, int m) {
    static constexpr int a[] = {0, 31, 28, 31, 30, 31, 30, 31, 31, 30, 31, 30, 31};
    return a[m] + (m == 2 && leap(y));
}

int days(Date a) {
    int y = a.y - (a.m <= 2);
    int e = (y >= 0 ? y : y - 399) / 400;
    int u = y - e * 400;
    int m = a.m + (a.m > 2 ? -3 : 9);
    int d = (153 * m + 2) / 5 + a.d - 1;
    return e * 146097 + u * 365 + u / 4 - u / 100 + d - 719468;
}

Date date(int z) {
    z += 719468;
    int e = (z >= 0 ? z : z - 146096) / 146097;
    int d = z - e * 146097;
    int u = (d - d / 1460 + d / 36524 - d / 146096) / 365;
    int y = u + e * 400;
    d -= 365 * u + u / 4 - u / 100;
    int m = (5 * d + 2) / 153;
    int x = d - (153 * m + 2) / 5 + 1;
    m += m < 10 ? 3 : -9;
    return {y + (m <= 2), m, x};
}

int weekday(Date a) {
    int x = (days(a) + 4) % 7;
    return x < 0 ? x + 7 : x;
}
```

10 - 德州扑克五张牌比较.cpp

46 lines · C++ · 09 - 杂类

```

/*
用途：比较两手五张德州扑克；返回负数/0/正数。
输入牌用两字符表示，如 AS、TD；第二字符花色只需相同可比较。
牌型从高到低：同花顺、四条、葫芦、同花、顺子、三条、两对、一对、高牌。
*/
struct Poker5 {
    static int rk(char c) {
        string s = "--23456789TJQKA";
        return s.find(c);
    }

    static vector<int> val(const array<string, 5> &a) {
        array<int, 15> c{};
        vector<int> r;
        bool f = true;
        for (auto &s : a) {
            r.push_back(rk(s[0]));
            ++c[r.back()];
            f &= s[1] == a[0][1];
        }
        sort(r.begin(), r.end(), greater<>());
        vector<int> q = r;
        q.erase(unique(q.begin(), q.end()), q.end());
        int h = 0;
        if (q.size() == 5 && q[0] - q[4] == 4) h = q[0];
        if (q == vector<int>({14, 5, 4, 3, 2})) h = 5;

        vector<pii> g;
        for (int x = 2; x <= 14; ++x) if (c[x]) g.push_back({c[x], x});
        sort(g.begin(), g.end(), greater<>());
        if (f && h) return {8, h};
        if (g[0][0] == 4) return {7, g[0][1], g[1][1]};
        if (g[0][0] == 3 && g[1][0] == 2) return {6, g[0][1], g[1][1]};
        if (f) return {5, r[0], r[1], r[2], r[3], r[4]};
        if (h) return {4, h};
        if (g[0][0] == 3) return {3, g[0][1], g[1][1], g[2][1]};
        if (g[0][0] == 2 && g[1][0] == 2) return {2, g[0][1], g[1][1], g[2][1]};
        if (g[0][0] == 2) return {1, g[0][1], g[1][1], g[2][1], g[3][1]};
        return {0, r[0], r[1], r[2], r[3], r[4]};
    }

    static int cmp(const array<string, 5> &a, const array<string, 5> &b) {
        auto x = val(a), y = val(b);
        return (x > y) - (x < y);
    }
};

```

11A - 物品装箱 (First Fit线段树) .cpp

26 lines · C++ · 09 - 杂类

```
/*
用途：按输入顺序，把物品放入最靠前且剩余容量足够的箱子 (First Fit) 。
说明：这是指定在线策略，不保证最少箱数； $O(n \log n)$ 。
性能：默认使用宏展开后的 int；若大规模数据因此超时或超空间，可将纯下标、状态或计数局部改为 i32。
*/
template <class T> int binPack(const vector<T> &a, T c) {
    int n = a.size();
    vector<T> t(n * 4, c);
    auto sub = [&](auto go, int p, int l, int r, T x) -> int {
        if (l + 1 == r) {
            t[p] -= x;
            return l;
        }
        int m = (l + r) >> 1, q;
        if (t[p << 1] >= x) q = go(go, p << 1, l, m, x);
        else q = go(go, p << 1 | 1, m, r, x);
        t[p] = max(t[p << 1], t[p << 1 | 1]);
        return q;
    };
    int z = 0;
    for (T x : a) {
        if (x > c) return -1;
        z = max(z, sub(sub, 1, 0, n, x) + 1);
    }
    return z;
}
```

11B - 物品装箱 (Best Fit多重集) .cpp

18 lines · C++ · 09 - 杂类

```
/*
用途：按输入顺序，把物品放入“能容纳它且剩余容量最小”的箱子（Best Fit）。
说明：这是指定在线策略，不保证最少箱数； $O(n \log n)$ 。
*/
template <class T> int binPack(const vector<T> &a, T c) {
    multiset<T> s;
    for (T x : a) {
        if (x > c) return -1;
        auto p = s.lower_bound(x);
        if (p == s.end()) s.insert(c - x);
        else {
            T y = *p;
            s.erase(p);
            s.insert(y - x);
        }
    }
    return s.size();
}
```

12 - 奇阶幻方.cpp

16 lines · C++ · 09 - 杂类

```
/*
用途：Siamese 法构造  $n$  为正奇数的标准幻方，使用  $1..n^2$ 。
性质：每行、每列、两条主对角线之和均为  $n(n^2+1)/2$ ； $O(n^2)$ 。
*/
vector<vector<int>> magic(int n) {
    assert(n > 0 && (n & 1));
    vector a(n, vector<int>(n));
    int x = 0, y = n / 2;
    for (int z = 1; z <= n * n; ++z) {
        a[x][y] = z;
        int nx = (x + n - 1) % n, ny = (y + 1) % n;
        if (a[nx][ny]) x = (x + 1) % n;
        else x = nx, y = ny;
    }
    return a;
}
```

13 - 阿达马矩阵.cpp

17 lines · C++ · 09 - 杂类

```
/*
用途：Sylvester 构造  $2^k$  阶阿达马矩阵；元素为  $+1/-1$ ，任意两行正交。
复杂度： $O(4^k)$ ，输出矩阵本身即需要该规模。
性能：默认使用宏展开后的 int；若大规模数据因此超时或超空间，可将纯下标、状态或计数局部改为 i32。
*/
vector<vector<int>>> hadam(int k) {
    int n = 1LL << k;
    vector a(n, vector<int>(n, 1));
    for (int s = 1; s < n; s <= 1)
        for (int i = 0; i < s; ++i)
            for (int j = 0; j < s; ++j) {
                a[i][j + s] = a[i][j];
                a[i + s][j] = a[i][j];
                a[i + s][j + s] = -a[i][j];
            }
    return a;
}
```


14A - 选数枚举 (DFS) .cpp

19 lines · C++ · 09 - 杂类

/* 用途：枚举 n 个数中恰选 k 个；回调参数为当前元素下标与元素和。

性能：默认使用宏展开后的 `int`；若大规模数据因此超时或超空间，可将纯下标、状态或计数局部改为 `i32`。

*/

```
template <class T, class F> void choose(const vector<T> &a, int k, F f) {
    vector<int> q;
    auto dfs = [&](auto go, int p, T s) -> void {
        if (q.size() == k) {
            f(q, s);
            return;
        }
        int d = k - q.size();
        for (int i = p; i + d <= a.size(); ++i) {
            q.push_back(i);
            go(go, i + 1, s + a[i]);
            q.pop_back();
        }
    };
    dfs(dfs, 0, T{});
}
```

14B - 选数枚举 (Gosper位运算) .cpp

19 lines · C++ · 09 - 杂类

/* 用途：n<=63 时枚举所有恰含 k 个 1 的掩码；回调参数为 u64 掩码。

性能：默认使用宏展开后的 int；若大规模数据因此超时或超空间，可将纯下标、状态或计数局部改为 i32。

*/

using u64 = uint64_t;

```
template <class F> void choose(int n, int k, F f) {
    if (k < 0 || k > n || n > 63) return;
    if (!k) {
        f(u64(0));
        return;
    }
    u64 x = (u64(1) << k) - 1, u = u64(1) << n;
    while (x < u) {
        f(x);
        u64 a = x & -x, b = x + a;
        x = b | ((x ^ b) >> 2) / a;
    }
}
```

01A - 01背包.cpp

32 lines · C++ · 10 - 动态规划

```
/*
用途：01 背包；每件物品至多取一次，求容量不超过 m 的最大价值。
约定：容量下标使用 int；T 是价值类型。返回 f[j] 表示容量上限为 j 的答案。
复杂度：O(nm) 时间、O(m) 空间。
性能：默认使用宏展开后的 int；若大规模数据因此超时或超空间，可将纯下标、状态或计数局部改为 i32。
*/
template <class T>
vector<T> pack01(int m, const vector<pair<int, T>>& a) {
    assert(m >= 0);
    vector<T> f(m + 1);
    for (auto [w, v] : a) {
        assert(w > 0);
        for (int j = m; j >= w; --j) f[j] = max(f[j], f[j - w] + v);
    }
    return f;
}

// 恰好装满。不可达状态为 neg，调用者应保证 neg + v 不溢出。
template <class T>
vector<T> packEq(int m, const vector<pair<int, T>>& a,
                 T neg = numeric_limits<T>::lowest() / 4) {
    assert(m >= 0);
    vector<T> f(m + 1, neg);
    f[0] = T{};
    for (auto [w, v] : a) {
        assert(w > 0);
        for (int j = m; j >= w; --j) {
            if (f[j - w] != neg) f[j] = max(f[j], f[j - w] + v);
        }
    }
    return f;
}
```

01B - 完全背包.cpp

16 lines · C++ · 10 - 动态规划

```
/*
用途：完全背包；每种物品可取任意多次。
约定：容量下标使用 int；返回 f[j] 表示容量不超过 j 的最大价值。
复杂度：O(nm) 时间、O(m) 空间。
性能：默认使用宏展开后的 int；若大规模数据因此超时或超空间，可将纯下标、状态或计数局部改为 i32。
*/
template <class T>
vector<T> packAll(int m, const vector<pair<int, T>>& a) {
    assert(m >= 0);
    vector<T> f(m + 1);
    for (auto [w, v] : a) {
        assert(w > 0);
        for (int j = w; j <= m; ++j) f[j] = max(f[j], f[j - w] + v);
    }
    return f;
}
```

01C - 多重背包（二进制分组）.cpp

27 lines · C++ · 10 - 动态规划

```
/*
用途：多重背包的二进制分组实现；代码短、适合物品数量不太大。
复杂度： $O(m * \sum(\log(c+1)))$ 。
性能：默认使用宏展开后的 int；若大规模数据因此超时或超空间，可将纯下标、状态或计数局部改为 i32。
*/
template <class T> struct Item {
    int w, c;
    T v;
};

template <class T>
vector<T> packMul(int m, const vector<Item<T>>& a) {
    assert(m >= 0);
    vector<T> f(m + 1);
    for (auto [w, c, v] : a) {
        assert(w > 0 && c >= 0);
        for (int k = 1; c > 0; k <= 1) {
            int x = min<int>(k, c);
            c -= x;
            int sw = w * x;
            if (sw > m) continue;
            T sv = v * T(x);
            for (int j = m; j >= sw; --j) f[j] = max(f[j], f[j - sw] + sv);
        }
    }
    return f;
}
```

01D - 多重背包（单调队列）.cpp

33 lines · C++ · 10 - 动态规划

```

/*
用途：多重背包的单调队列实现；物品数量  $c$  很大时不再多一个  $\log c$ 。
复杂度： $O(nm)$  时间、 $O(m)$  空间。
与 01C 同名、同接口，二者择一使用。
性能：默认使用宏展开后的 int；若大规模数据因此超时或超空间，可将纯下标、状态或计数局部改为 i32。
*/
template <class T> struct Item {
    int w, c;
    T v;
};

template <class T>
vector<T> packMul(int m, const vector<Item<T>>& a) {
    assert(m >= 0);
    vector<T> f(m + 1);
    for (auto [w, c, v] : a) {
        assert(w > 0 && c >= 0);
        vector<T> g = f;
        for (int r = 0; r < w && r <= m; ++r) {
            deque<int> q;
            int z = (m - r) / w;
            for (int k = 0; k <= z; ++k) {
                while (!q.empty() && q.front() < k - c) q.pop_front();
                T x = g[r + k * w] - T(k) * v;
                while (!q.empty() && g[r + q.back() * w] - T(q.back()) * v <= x) q.pop_back();
                q.push_back(k);
                int p = q.front();
                f[r + k * w] = g[r + p * w] + T(k - p) * v;
            }
        }
    }
    return f;
}

```

01E - 分组背包.cpp

20 lines · C++ · 10 - 动态规划

```
/*
用途：分组背包；每组至多选择一件物品。
groups[g] 中每个元素为 {体积, 价值}。
复杂度：O(m * 物品总数)。
性能：默认使用宏展开后的 int；若大规模数据因此超时或超空间，可将纯下标、状态或计数局部改为 i32。
*/
template <class T>
vector<T> packGrp(int m, const vector<vector<pair<int, T>>>& a) {
    assert(m >= 0);
    vector<T> f(m + 1);
    for (const auto& g : a) {
        vector<T> h = f;
        for (auto [w, v] : g) {
            assert(w > 0);
            for (int j = w; j <= m; ++j) h[j] = max(h[j], f[j - w] + v);
        }
        f.swap(h);
    }
    return f;
}
```

01F - 二维费用背包.cpp

22 lines · C++ · 10 - 动态规划

```
/*
用途：每件物品至多取一次，同时受二维容量约束。
item = {第一维费用， 第二维费用， 价值}。
复杂度：O(nab) 时间、O(ab) 空间。
性能：默认使用宏展开后的 int；若大规模数据因此超时或超空间，可将纯下标、状态或计数局部改为 i32。
*/
template <class T> struct Item2D {
    int x, y;
    T v;
};

template <class T>
vector<vector<T>> pack2D(int a, int b, const vector<Item2D<T>>& c) {
    assert(a >= 0 && b >= 0);
    vector f(a + 1, vector<T>(b + 1));
    for (auto [x, y, v] : c) {
        assert(x > 0 || y > 0);
        for (int i = a; i >= x; --i)
            for (int j = b; j >= y; --j) f[i][j] = max(f[i][j], f[i - x][j - y] + v);
    }
    return f;
}
```


01G - 01背包最优方案数（自动取模）.cpp

26 lines · C++ · 10 - 动态规划

```
/*
用途：01 背包恰好装满时，同时求最大价值和达到最大值的方案数。
Z 可取 int（精确计数）或 MInt（自动取模）。每件物品按下标区分。
性能：默认使用宏展开后的 int；若大规模数据因此超时或超空间，可将纯下标、状态或计数局部改为 i32。
*/
template <class T, class Z> struct PackCount {
    T val;
    Z cnt;
};

template <class T, class Z>
vector<PackCount<T, Z>> packCnt(int m, const vector<pair<int, T>>& a,
                                T neg = numeric_limits<T>::lowest() / 4) {
    vector<PackCount<T, Z>> f(m + 1, {neg, Z{}});
    f[0] = {T{}, Z{1}};
    for (auto [w, v] : a) {
        assert(w > 0);
        for (int j = m; j >= w; --j) {
            if (f[j - w].val == neg) continue;
            T x = f[j - w].val + v;
            if (x > f[j].val) f[j] = {x, f[j - w].cnt};
            else if (x == f[j].val) f[j].cnt += f[j - w].cnt;
        }
    }
    return f;
}
```

01H - 混合背包.cpp

32 lines · C++ · 10 - 动态规划

```

/*
用途：同一组物品混合 01、完全、多重三种数量限制，求容量不超过 m 的最大价值。
约定：Item.c=-1 表示至多一次，0 表示无限次，正数表示至多 c 次；w>0。
复杂度：二进制拆分多重物品，O(m * 拆分后的物品数)。
性能：默认使用宏展开后的 int；若大规模数据因此超时或超空间，可将纯下标、状态或计数局部改为 i32。
*/
template <class T> struct MixedItem {
    int w, c;
    T v;
};

template <class T> vector<T> packMix(int m, const vector<MixedItem<T>>& a) {
    assert(m >= 0);
    vector<T> f(m + 1, T{});
    auto one = [&](int w, T v) {
        for (int j = m; j >= w; --j) f[j] = max(f[j], f[j - w] + v);
    };
    for (auto [w, c, v] : a) {
        assert(w > 0 && c >= -1);
        if (c < 0) one(w, v);
        else if (!c) for (int j = w; j <= m; ++j) f[j] = max(f[j], f[j - w] + v);
        else {
            for (int k = 1; c;) {
                int x = min(k, c);
                one(w * x, v * x);
                c -= x;
                k = k <= c ? k << 1 : c;
            }
        }
    }
    return f;
}

```

011- 有依赖的背包（树形）.cpp

38 lines · C++ · 10 - 动态规划

```

/*
用途：有依赖背包；选择节点前必须选择父节点，多个 parent=0 的根可独立选择。
输入：w/v/parent 均为 1-indexed 等长数组，容量 m；返回每个“恰好容量”的最优值。
复杂度： $O(nm^2)$ ，适合树形依赖与中小容量；不可达状态为 NEG。
性能：默认使用宏展开后的 int；若大规模数据因此超时或超空间，可将纯下标、状态或计数局部改为 i32。
*/
template <class T>
vector<T> packDep(int m, const vector<int>& w, const vector<T>& v,
                  const vector<int>& fa) {
    static_assert(is_signed_v<T>);
    assert(m >= 0 && w.size() == v.size() && v.size() == fa.size());
    int n = w.size() - 1;
    vector<vector<int>>> g(n + 1);
    for (int i = 1; i <= n; ++i) {
        assert(0 <= fa[i] && fa[i] <= n && fa[i] != i && w[i] >= 0);
        g[fa[i]].push_back(i);
    }
    const T NEG = numeric_limits<T>::lowest() / 4;
    int vis = 0;
    auto dfs = [&](auto&& go, int u) -> vector<T> {
        vis += u != 0;
        vector<T> f(m + 1, NEG);
        int s = u ? w[u] : 0;
        if (s <= m) f[s] = u ? v[u] : T{};
        for (int x : g[u]) {
            auto d = go(go, x);
            vector<T> h = f;
            for (int i = 0; i <= m; ++i) if (f[i] != NEG)
                for (int j = 0; i + j <= m; ++j) if (d[j] != NEG)
                    h[i + j] = max(h[i + j], f[i] + d[j]);
            f.swap(h);
        }
        return f;
    };
    auto f = dfs(dfs, 0);
    assert(vis == n); // 否则 parent 中存在不连向虚根的环。
    return f;
}

```

01J - 01背包具体方案.cpp

24 lines · C++ · 10 - 动态规划

/*

用途：01 背包求最大价值并恢复一组方案；能保持最优时优先选择编号更小的物品。

输入：w/v 为 0-indexed 等长数组，容量 m；返回 {最大值, 选中下标}。

复杂度：O(nm) 时间与空间。只求最优值时使用 01A 的一维版本。

性能：默认使用宏展开后的 int；若大规模数据因此超时或超空间，可将纯下标、状态或计数局部改为 i32。

*/

```
template <class T>
pair<T, vector<int>> packSol(int m, const vector<int>& w, const vector<T>& v) {
    assert(m >= 0 && w.size() == v.size());
    int n = w.size();
    vector<vector<T>> f(n + 1, vector<T>(m + 1));
    for (int i = n; i-- > 0;) {
        assert(w[i] > 0);
        for (int j = 0; j <= m; ++j) {
            f[i][j] = f[i + 1][j];
            if (w[i] <= j) f[i][j] = max(f[i][j], f[i + 1][j - w[i]] + v[i]);
        }
    }
    vector<int> a;
    int c = m;
    for (int i = 0; i < n; ++i) if (w[i] <= c && f[i][c] == f[i + 1][c - w[i]] + v[i])
        a.push_back(i), c -= w[i];
    return {f[0][m], a};
}
```

02 - 数位DP (DFA) .cpp

43 lines · C++ · 10 - 动态规划

```

/*
用途：统计 [0,n] 中十进制表示被有限状态自动机接受的整数个数。
go(state, digit) 返回下一状态；若需要区分前导零，把 started 一并编码进状态。
good(state) 判断读完固定宽度（含前导零）后的状态是否接受。
T 可取 int、高精整数或 MInt，后者会自动取模。
复杂度：O(位数 * 状态数 * 10)。
性能：默认使用宏展开后的 int；若大规模数据因此超时或超空间，可将纯下标、状态或计数局部改为 i32。
*/
template <class T, class Go, class G>
T digitDP(int n, int sc, int st, Go go, G ok) {
    assert(sc > 0 && 0 <= st && st < sc);
    vector<int> d;
    do {
        d.push_back(n % 10);
        n /= 10;
    } while (n);
    reverse(d.begin(), d.end());

    int m = d.size();
    vector f(m + 1, vector<T>(sc));
    vector vis(m + 1, vector<char>(sc));
    auto dfs = [&](auto&& rec, int p, int s, bool lim) -> T {
        if (p == m) return T(ok(s));
        if (!lim && vis[p][s]) return f[p][s];
        int up = lim ? d[p] : 9;
        T r{};
        for (int x = 0; x <= up; ++x) {
            int t = go(s, x);
            assert(0 <= t && t < sc);
            r += rec(rec, p + 1, t, lim && x == up);
        }
        if (!lim) vis[p][s] = 1, f[p][s] = r;
        return r;
    };
    return dfs(dfs, 0, st, true);
}

template <class T, class Go, class G>
T digitDP(int l, int r, int sc, int st, Go go, G ok) {
    assert(l <= r);
    T x = digitDP<T>(r, sc, st, go, ok);
    return l ? x - digitDP<T>(l - 1, sc, st, go, ok) : x;
}

```

03 - 状压DP (TSP) .cpp

32 lines · C++ · 10 - 动态规划

```

/*
用途：Held-Karp 状压 DP，求从 s 出发、恰好访问每个点一次并回到 s 的最短 Hamilton 回路。
w[u][v] 为边权；不可达写 INF。n 通常不超过 20。
复杂度： $O(2^n \cdot n^2)$  时间、 $O(2^n \cdot n)$  空间。
性能：默认使用宏展开后的 int；若大规模数据因此超时或超空间，可将纯下标、状态或计数局部改为 i32。
*/
template <class T>
optional<T> tsp(const vector<vector<T>>& w, int s = 0,
               T inf = numeric_limits<T>::max() / 4) {
    int n = w.size();
    assert(n > 0 && n <= 22 && 0 <= s && s < n);
    for (const auto& a : w) assert(a.size() == n);
    int z = 1LL << n;
    vector f(z, vector<T>(n, inf));
    f[1LL << s][s] = T{};
    for (int m = 0; m < z; ++m) {
        if (!(m >> s & 1)) continue;
        for (int u = 0; u < n; ++u) {
            if (!(m >> u & 1) || f[m][u] == inf) continue;
            for (int v = 0; v < n; ++v) {
                if (m >> v & 1 || w[u][v] == inf) continue;
                int t = m | (1LL << v);
                f[t][v] = min(f[t][v], f[m][u] + w[u][v]);
            }
        }
    }
    T r = inf;
    for (int u = 0; u < n; ++u)
        if (u != s && f[z - 1][u] != inf && w[u][s] != inf) r = min(r, f[z - 1][u] + w[u][s]);
    if (n == 1) r = T{};
    return r == inf ? nullopt : optional<T>(r);
}

```

04 - 轮廓线DP（多米诺骨牌）.cpp

30 lines · C++ · 10 - 动态规划

```

/*
用途：用 1x2 多米诺骨牌铺满 n*m 网格的方案数。
T 可取 int、高精整数或 MInt（自动取模）。宽度取 min(n,m)，通常不超过 15。
复杂度：逐格转移  $O(n * m * 2^m)$ 。
性能：默认使用宏展开后的 int；若大规模数据因此超时或超空间，可将纯下标、状态或计数局部改为 i32。
*/
template <class T>
T domino(int n, int m) {
    assert(n >= 0 && m >= 0);
    if (n < m) swap(n, m);
    if (!m) return T(1);
    assert(m < 31);
    int z = 1LL << m;
    vector<T> f(z), g(z);
    f[0] = T(1);
    for (int r = 0; r < n; ++r) {
        for (int c = 0; c < m; ++c) {
            fill(g.begin(), g.end(), T{});
            for (int s = 0; s < z; ++s) {
                if (s & 1) g[s >> 1] += f[s];
                else {
                    if (r + 1 < n) g[(s >> 1) | (1LL << (m - 1))] += f[s];
                    if (c + 1 < m && !(s & 2)) g[(s >> 1) | 1] += f[s];
                }
            }
            f.swap(g);
        }
    }
    return f[0];
}

```

05A - 分治DP优化.cpp

26 lines · C++ · 10 - 动态规划

```

/*
用途：计算  $g[j] = \min_{\{0 \leq k < j\}} (f[k] + \text{cost}(k+1, j))$ 。
前提：最优决策点随  $j$  单调；单调性不成立时结果错误。
调用：divide(1, n, 0, n-1, f, g, p, cst, inf)。
复杂度：每一层 DP  $O(n \log n)$  次转移计算，常见进一步可证为  $O(n \log n)$ 。
性能：默认使用宏展开后的 int；若大规模数据因此超时或超空间，可将纯下标、状态或计数局部改为 i32。
*/
template <class T, class C>
void divide(int l, int r, int ql, int qr,
            const vector<T>& f, vector<T>& g, vector<int>& p,
            C cst, T inf) {
    if (l > r) return;
    int m = (l + r) >> 1;
    pair<T, int> z{inf, -1};
    int up = min(qr, m - 1);
    for (int k = ql; k <= up; ++k) {
        if (f[k] == inf) continue;
        z = min(z, pair<T, int>{f[k] + cst(k + 1, m), k});
    }
    g[m] = z.first;
    p[m] = z.second;
    if (l == r) return;
    int q = z.second == -1 ? ql : z.second;
    divide(l, m - 1, ql, q, f, g, p, cst, inf);
    divide(m + 1, r, q, qr, f, g, p, cst, inf);
}

```


05B - Knuth优化.cpp

25 lines · C++ · 10 - 动态规划

```

/*
用途：区间 DP：f[l][r]=min(f[l][k]+f[k+1][r])+cost(l,r)。
前提：cost 满足四边形不等式且区间包含单调，因而 opt[l][r-1]<=opt[l][r]<=opt[l+1][r]。
下标：0..n-1。复杂度 O(n^2) 时间、O(n^2) 空间。
性能：默认使用宏展开后的 int；若大规模数据因此超时或超空间，可将纯下标、状态或计数局部改为 i32。
*/
template <class T, class C>
pair<vector<vector<T>>, vector<vector<int>>> knuthDP(int n, C cst) {
    assert(n >= 0);
    vector f(n, vector<T>(n));
    vector p(n, vector<int>(n));
    for (int i = 0; i < n; ++i) p[i][i] = i;
    for (int d = 1; d < n; ++d) {
        for (int l = 0, r = d; r < n; ++l, ++r) {
            int a = p[l][r - 1];
            int b = min<int>(r - 1, p[l + 1][r]);
            f[l][r] = numeric_limits<T>::max() / 4;
            for (int k = a; k <= b; ++k) {
                T x = f[l][k] + f[k + 1][r] + cst(l, r);
                if (x < f[l][r]) f[l][r] = x, p[l][r] = k;
            }
        }
    }
    return {move(f), move(p)};
}

```

06A - 最长公共子序列（常规）.cpp

24 lines · C++ · 10 - 动态规划

```
/*
用途：任意可比较序列的 LCS；返回一组最长公共子序列。
复杂度：O(nm) 时间、O(nm) 空间。只求字符串长度且 m 很大时使用 06B。
性能：默认使用宏展开后的 int；若大规模数据因此超时或超空间，可将纯下标、状态或计数局部改为 i32。
*/
template <class Seq>
vector<typename Seq::value_type> lcs(const Seq& a, const Seq& b) {
    int n = a.size(), m = b.size();
    vector f(n + 1, vector<int>(m + 1));
    for (int i = 1; i <= n; ++i) {
        for (int j = 1; j <= m; ++j) {
            if (a[i - 1] == b[j - 1]) f[i][j] = f[i - 1][j - 1] + 1;
            else f[i][j] = max(f[i - 1][j], f[i][j - 1]);
        }
    }
    vector<typename Seq::value_type> r;
    for (int i = n, j = m; i && j;) {
        if (a[i - 1] == b[j - 1]) r.push_back(a[--i]), --j;
        else if (f[i - 1][j] >= f[i][j - 1]) --i;
        else --j;
    }
    reverse(r.begin(), r.end());
    return r;
}
```

06B - 最长公共子序列 (Bitset) .cpp

35 lines · C++ · 10 - 动态规划

```

/*
用途：只求两个字节串的 LCS 长度；利用 64 位并行优化。
复杂度： $O(|a||b|/64 + |b|)$ ，空间  $O(256|b|/64)$ 。
大位集必须使用 u64；若被 #define int long long 替换会显著增大内存和降低速度。
性能：默认使用宏展开后的 int；若大规模数据因此超时或超空间，可将纯下标、状态或计数局部改为 i32。
*/

using u64 = uint64_t;

int lcs(const string& a, const string& b) {
    int n = b.size(), z = (n + 63) >> 6;
    vector<vector<u64>> p(256, vector<u64>(z));
    for (int i = 0; i < n; ++i) p[(unsigned char)b[i]][i >> 6] |= u64(1) << (i & 63);
    vector<u64> s(z), x(z), y(z), d(z);
    for (unsigned char c : a) {
        u64 q = 1;
        for (int i = 0; i < z; ++i) {
            x[i] = p[c][i] | s[i];
            u64 nq = s[i] >> 63;
            y[i] = (s[i] << 1) | q;
            q = nq;
        }
        u64 br = 0;
        for (int i = 0; i < z; ++i) {
            u64 t = y[i] + br;
            u64 ov = t < y[i];
            d[i] = x[i] - t;
            br = ov || x[i] < t;
            s[i] = x[i] & ~d[i];
        }
    }
    int r = 0;
    for (u64 x : s) r += __builtin_popcountll(x);
    return r;
}

```

X - FastIO.cpp

214 lines · C++ · .

```
#ifdef __linux__
#include <fcntl.h>
#include <sys/mman.h>
#include <sys/stat.h>
#include <unistd.h>
#endif

namespace fastio {
using i32 = std::int32_t;
using u32 = std::uint32_t;
using i64 = std::int64_t;
using u64 = std::uint64_t;
using u8 = std::uint8_t;

struct Tables {
    alignas(64) std::int8_t r2[1 << 16];
    alignas(64) u32 p4[10000];

    Tables() {
        std::memset(r2, -1, sizeof(r2));

        for (int hi = '0'; hi <= '9'; ++hi)
            for (int lo = '0'; lo <= '9'; ++lo) r2[(hi << 8) | lo] = (lo & 15) * 10 + (hi & 15);

        for (u32 x = 0; x < 10000; ++x) {
            p4[x] = u32('0' + x / 1000) | (u32('0' + x / 100 % 10) << 8) | (u32('0' + x / 10 % 10)
                << 16) | (u32('0' + x % 10) << 24);
        }
    }
};

static Tables tab;

static const char *cur = [] {
    struct stat st;
    fstat(STDIN_FILENO, &st);

    void *p = mmap(nullptr, st.st_size, PROT_READ, MAP_PRIVATE, STDIN_FILENO, 0);
    madvise(p, st.st_size, MADV_SEQUENTIAL);
    return reinterpret_cast<const char *>(p);
}();

#define READ2(N, LABEL)
do {
    \
    const int d = tab.r2[*reinterpret_cast<const std::uint16_t *>(cur)];
    \
    if (__builtin_expect(d < 0, 0)) goto LABEL;
    \
    (N) = (N) * 100u + static_cast<unsigned>(d);
    \
    cur += 2;
    \
} while (0)

#define IS_DIGIT(C) (static_cast<unsigned>(static_cast<unsigned char>(C) - '0') < 10u)

#define LAST_AND_NEXT(N)
\
```

```

do {
    \
    if (IS_DIGIT(*cur)) (N) = (N) * 10u + static_cast<unsigned>(*cur++ & 15);
    \
    while (__builtin_expect(static_cast<unsigned>(static_cast<unsigned char>(*++cur)) - 1u < 32u,
        0)) {
        \
        }
        \
        return (N);
    \
} while (0)

static inline u32 ru32_body() {
    u32 n = 0;

    READ2(n, ru32_done);
    READ2(n, ru32_done);
    READ2(n, ru32_done);
    READ2(n, ru32_done);
    READ2(n, ru32_done);

ru32_done:
    LAST_AND_NEXT(n);
}

static inline u64 ru64_body() {
    u64 n = 0;
    READ2(n, ru64_done);
    READ2(n, ru64_done);
    READ2(n, ru64_done);
    READ2(n, ru64_done);
    READ2(n, ru64_done);
    READ2(n, ru64_done);
    READ2(n, ru64_done);
    READ2(n, ru64_done);
    READ2(n, ru64_done);
    READ2(n, ru64_done);

ru64_done:
    LAST_AND_NEXT(n);
}

#undef READ2
#undef IS_DIGIT
#undef LAST_AND_NEXT

static inline u32 ru32() {
    if (__builtin_expect(*cur == '+', 0)) ++cur;
    return ru32_body();
}

static inline u64 ru64() {
    if (__builtin_expect(*cur == '+', 0)) ++cur;
    return ru64_body();
}

static inline i32 r32() {
    const bool neg = (*cur == '-');
    cur += neg;
    const u32 x = ru32_body();

    return static_cast<i32>(neg ? 0u - x : x);
}

```

```

static inline i64 r64() {
    const bool neg = (*cur == '-');
    cur += neg;
    const u64 x = ru64_body();

    return static_cast<i64>(neg ? 0ull - x : x);
}

static constexpr std::size_t OUT_BYTES = 1u << 22;
alignas(64) static u8 obuf[OUT_BYTES + 32];
static u8 *now = obuf;

static inline void flush() {
    if (now != obuf) {
        std::fwrite(obuf, 1, now - obuf, stdout);
        now = obuf;
    }
}

static inline void reserve(std::size_t n) {
    if (__builtin_expect(now + n > obuf + OUT_BYTES, 0)) flush();
}

template <class U, int K> static inline void pu_unchecked(U x) {
    u8 *const end = now + K;
    u8 *p = end;

    while (x > 9999) {
        p -= 4;
        *reinterpret_cast<u32 *>(p) = tab.p4[static_cast<u32>(x % 10000)];
        x /= 10000;
    }

    const u32 w = tab.p4[static_cast<u32>(x)];

    if (x < 10) {
        *--p = static_cast<u8>('0' + x);
    } else if (x < 100) {
        p -= 2;
        *reinterpret_cast<std::uint16_t *>(p) = static_cast<std::uint16_t>(w >> 16);
    } else if (x < 1000) {
        p -= 4;
        *reinterpret_cast<u32 *>(p) = w;
        ++p;
    } else {
        p -= 4;
        *reinterpret_cast<u32 *>(p) = w;
    }

    std::memmove(now, p, end - p);
    now += end - p;
}

static inline void p32(u32 x) {
    reserve(10);
    pu_unchecked<u32, 10>(x);
}

static inline void p64(u64 x) {
    reserve(20);
    pu_unchecked<u64, 20>(x);
}

```

```
static inline void p32(i32 x) {
    reserve(11);
    if (x < 0) {
        *now++ = '-';
        pu_unchecked<u32, 10>(0u - static_cast<u32>(x));
    } else {
        pu_unchecked<u32, 10>(static_cast<u32>(x));
    }
}

static inline void p64(i64 x) {
    reserve(21);
    if (x < 0) {
        *now++ = '-';
        pu_unchecked<u64, 20>(0ull - static_cast<u64>(x));
    } else {
        pu_unchecked<u64, 20>(static_cast<u64>(x));
    }
}

static inline void pc(char c) {
    reserve(1);
    *now++ = static_cast<u8>(c);
}

template <class T> static inline void write(T x) {
    if constexpr (sizeof(T) <= 4) {
        if constexpr (std::is_signed_v<T>)
            p32(static_cast<i32>(x));
        else
            p32(static_cast<u32>(x));
    } else {
        if constexpr (std::is_signed_v<T>)
            p64(static_cast<i64>(x));
        else
            p64(static_cast<u64>(x));
    }
}

} // namespace fastio
```